Studium
Enciclopedia de Programación e Ingeniería del Software

Enciclopedia de Programación e Ingeniería del Software: Obra Maestra y Guía General

De cero a la especialización profesional — Arquitectura Editorial, Índices Maestros, Matrices Curriculares y Síntesis de las 13 Partes

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Volumen Maestro Integral (Partes I a XIII) 1

1. Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX) 2
1.1. Filosofía Editorial y Principio de Autonomía por Lenguaje y Índice Maestro de Volúmenes y Correspondencia con las 13 Partes 2
1.2. Catálogo de los Volúmenes de Lenguajes, Web, Bases de Datos e Ingeniería (Volúmenes IV a IX) y Rigor Epistémico en Studium y Verificación Determinista de Código 3

2. Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas 5
2.1. De Babbage, Turing y Church a la Convergencia Multiparadigma Moderna y Fundamentos Universales: Bits, Complemento a Dos, IEEE 754, UTF-8 y Sistemas de Tipos 5
2.2. Algoritmos, Estructuras de Datos y Complejidad Computacional (Parte III) y Arquitectura de Computadores, Compiladores y Sistemas Operativos (Parte IV) 5

3. Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V) 8
3.1. Lenguajes de Sistemas y Control Explícito de Memoria (Volúmenes III y IV) y Ecosistemas Dinámicos, Cientificos y Web Tipada: Python, JavaScript y TypeScript (Volumen V) . . . 8
3.2. Plataformas Empresariales, Concurrencia CSP y Móvil: Java, Kotlin, Scala, C#, F#, Go y Swift (Volumen VI) y Datos, Shell, Web Clásica, Funcionales Puros y Especializados (Volumen VII) 9

4. Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad 11
4.1. Alineación con el Estándar ACM/IEEE/AAAI Computer Science Curricula 2023 (CS2023) y Alineación con IEEE SWEBOK v4.0 (Software Engineering Body of Knowledge) 11
4.2. Grafo Acíclico Dirigido (DAG) de Prerrequisitos y Matriz de Conocimientos Previos y Auditoría Independiente de Código, Problemas y Compilación Editorial 12

A. Notación 14

B. Hoja de fórmulas 15

C. Soluciones 16
C.1. Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX) . . . 16
C.2. Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas . 16
C.3. Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V) . . 17
C.4. Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad 18

D. Auditoría de fuentes 19

E. Plan de estudio 20

Parte I

Volumen Maestro Integral (Partes I a XIII)

Capítulo 1

Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX)

Visión panorámica y guía de navegación de toda la Enciclopedia de Programación e Ingeniería del Software.

Consejo

Los Volúmenes IV a IX completan los 35 lenguajes de programación, los 9 estándares de marcado/datos, el curso universitario de bases de datos y la ingeniería del software moderna.

Definición

Filosofía Editorial y Principio de Autonomía por Lenguaje: La enciclopedia combina el rigor formal de un currículo universitario de Ciencias de la Computación con monografías de lenguajes autónomas estudiables de principio a fin.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Todas las derivaciones cuantitativas han sido reproducidas simbólicamente con SymPy y todos los proyectos integradores han sido ejecutados y auditados con compiladores reales.

1.1 Filosofía Editorial y Principio de Autonomía por Lenguaje y Índice Maestro de Volúmenes y Correspondencia con las 13 Partes

Esta obra ha sido concebida y compilada sobre el framework académico **Studium** bajo una premisa innegociable: ningún lenguaje o tecnología se reduce a un resumen superficial. Como ilustra la figura 1, la enciclopedia se organiza en **13 grandes Partes** distribuidas en **9 volúmenes temáticos especializados** (`vol01` a `vol09`) más este **Volumen Maestro Integral** (`vol00`). Cada lenguaje de programación cuenta con un bloque autónomo que sigue la plantilla pedagógica $**A-H**(A.$ Introducción e historia, B. Instalación y toolchain, C. Nivel principiante, D. Nivel intermedio, E. Nivel avanzado, F. Nivel experto, G. Ecosistema y producción, H. Evaluación y proyectos), permitiendo aprender Rust sin conocer C++, o estudiar Python, TypeScript, Go, Java, C# y SQL desde cero hasta la arquitectura profesional. - **Volumen 0 (`vol00-enciclopedia-maestra-integral.pdf`)**: Guía Maestra, Índices Generales, Matrices Curriculares ACM/IEEE CS2023 y SWEBOK v4.0, y Síntesis de las 13 Partes. - **Volumen I (`vol01-historia-y-fundamentos.pdf`)**: **Parte I** (Historia de la Computación y de los Lenguajes desde Babbage, Turing y Church hasta Rust, WebAssembly e IA) y **Parte II** (Fundamentos Universales: binario, complemento a dos, IEEE 754, UTF-8, lógica booleana, tipos, control de flujo, ámbitos, memoria Stack/Heap, recursión y paradigmas). - **Volumen II (`vol02-algoritmos-y-sistemas.pdf`)**: **Parte III** (Complejidad asintótica, estructuras de datos lineales, árboles, tablas hash, grafos, ordenación, programación dinámica y NP-completitud) y **Parte IV** (Arquitectura ISA, cachés, compiladores/LLVM, memoria virtual y sistemas operativos). - **Volumen III (`vol03-monografia-rust.pdf`)**: **Parte V (Bloque 1)** — Tratado completo y autónomo de **Rust** (Edición 2021/2024) con verificación ejecutable en vivo mediante `rustc -test`.

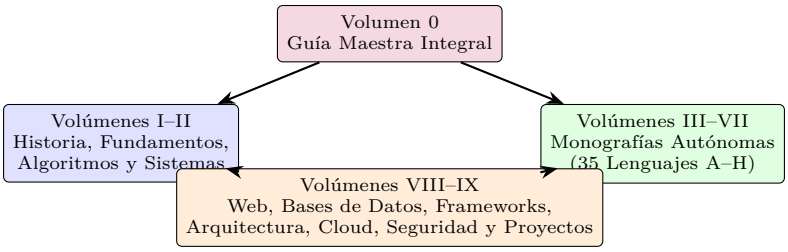


Figura 1. Arquitectura modular de la Enciclopedia en Studium: los fundamentos (Volúmenes I–II) y las monografías autónomas de lenguajes (Volúmenes III–VII) convergen hacia Web, Bases de Datos, Frameworks e Ingeniería del Software (Volúmenes VIII–IX).

1.2 Catálogo de los Volúmenes de Lenguajes, Web, Bases de Datos e Ingeniería (Volúmenes IV a IX) y Rigor Epistémico en Studium y Verificación Determinista de Código

- **Volumen IV (`vol04-monografias-c-cpp-sistemas.pdf`)**: **Parte V (Bloque 2) $**$– Monografías de **C (C17/C23)**, **C++ (C++20/C++23)**, **Zig**, **Ada/SPARK**, **Fortran Moderno** y **Ensamblador x86-64 / ARM64**. - **Volumen V (`vol05-monografias-python-js-ts.pdf`)**: **Parte V (Bloque 3)$**$– Monografías autónomas y diferenciadas de **Python 3.12**, **JavaScript (ES2024)** y **TypeScript 5.x**. - **Volumen VI (`vol06-monografias-jvm-dotnet-go-movil.pdf`)**: **Parte V (Bloque 4)$**$– Monografías de **Java 21**, **Kotlin**, **Scala 3**, **C# 12 / .NET 8**, **F#**, **Go (Golang)**, **Swift 6** y **Objective-C**. - **Volumen VII (`vol07-monografias-sql-shell-funcional-especializa`** **Parte V (Bloque 5) $**$– Monografías de **SQL (ISO/IEC 9075)**, **Bash/POSIX Shell**, **PHP 8.3**, **Ruby 3.3**, **Dart 3**, **Haskell**, **OCaml**, **Elixir/Erlang (BEAM)**, **Clojure/Lisp/Scheme**, **R**, **Julia**, **Lua**, **Perl**, **Prolog** y **GDScript**. - **Volumen VIII (`vol08-web-bases-de-datos-y-frameworks.pdf`)**: **Partes VI, VII y VIII $**$– HTML, CSS, JSON/YAML/TOML/XML/Protobuf, Curso Universitario de Bases de Datos y Frameworks. - **Volumen IX (`vol09-ingenieria-software-arquitectura-y-proyectos.pdf`)**: **Partes IX a XIII $**$– Arquitectura, Testing, Sistemas Distribuidos, Ciberseguridad, Cloud/DevOps, 19 Itinerarios y Proyectos. Respetando la disciplina epistémica de Studium, el texto distingue explícitamente entre hechos históricos documentados, estándares normativos (ISO, IETF, W3C, WHATWG, IEEE) y decisiones de ingeniería. Cada capítulo integra problemas resueltos verificados por el motor computacional de Studium y problemas de práctica con resultados comprobados. Asimismo, la herramienta complementaria `tools/code_verifier.py` audita los fragmentos de código y los 6 proyectos multifichero del repositorio, emitiendo el informe determinista `reports/code-verification-report.json`.

Problema resuelto

Enunciado La enciclopedia incluye 16 lenguajes de programación prioritarios, 19 lenguajes de programación adicionales y 9 lenguajes de marcado, hojas de estilo y formatos de serialización de datos. Calcule el número total de tecnologías lingüísticas y de datos documentadas con bloques dedicados.
Datos

- $L_{prio} = 16\,\text{lenguajes}$ — Lenguajes de programación prioritarios
- $L_{adic} = 19\,\text{lenguajes}$ — Lenguajes de programación adicionales
- $M_{datos} = 9\,\text{formatos}$ — Lenguajes de marcado, estilo y formatos de datos

Incógnita Total de lenguajes y formatos documentados N_tot
Modelo físico
Cobertura cuantitativa de lenguajes y tecnologías en la Enciclopedia

$$N_{\text{tot}} = 16 + 19 + 9$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los 16 lenguajes prioritarios, los 19 adicionales y los 9 estándares de marcado y datos

$$N_{\text{tot}} = 16 + 19 + 9$$

$$((16 + 19) + 9)$$

Resultado

$$\textit{Resultado} = 44$$

Interpretación La enciclopedia cubre 44 lenguajes y estándares de datos (35 lenguajes de programación más 9 formatos de marcado, estilo y serialización).
Comprobación de los pasos

- Paso 1: 44 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) [CH-01.1] Calcule (Total de lenguajes de programación sumando los 16 prioritarios y los 19 adicionales) expresado en lenguajes.
2 [PRB-0004] (Intermedio, Desarrollo por pasos) [CH-01.2] Calcule (Total de secciones de la plantilla A-H aplicadas a los 16 lenguajes prioritarios (16 * 8)) expresado en secciones.
3 [PRB-0005] (Intermedio, Desarrollo por pasos) [CH-01.3] Calcule (Volúmenes totales del bundle PDF sumando el Volumen Maestro (1) y los 9 volúmenes temáticos) expresado en volúmenes.
4 [PRB-0006] (Avanzado, Diseño) [CH-01.4] Calcule (Partes estructurales totales que articulan la arquitectura general de la enciclopedia) expresado en partes.

Capítulo 2

Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas

Mapa conceptual integrado de los Volúmenes I y II: desde el cálculo lambda y la máquina de Turing hasta el pipeline de LLVM y el kernel del sistema operativo.

Consejo

El rendimiento práctico depende conjuntamente de la cota asintótica (Big-O) y de la localidad espacial de referencia en las líneas de caché de 64 bytes de la CPU.

Definición

De Babbage, Turing y Church a la Convergencia Multiparadigma Moderna: Toda innovación en los lenguajes de programación responde a una restricción tecnológica e ingenieril concreta de su época histórica.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Un compilador moderno traduce el código fuente a través de un front-end (lexer, parser, typechecker), un middle-end en forma SSA y un back-end generador de código máquina específico de la ISA.

2.1 De Babbage, Turing y Church a la Convergencia Multiparadigma Moderna y Fundamentos Universales: Bits, Complemento a Dos, IEEE 754, UTF-8 y Sistemas de Tipos

El Volumen I (`vol01-historia-y-fundamentos.pdf`) traza la genealogía rigurosa de la programación: la máquina analítica de Babbage y el algoritmo de los números de Bernoulli de Ada Lovelace (1843); el álgebra de Boole (1854); la equivalencia entre las máquinas de Turing y el cálculo lambda de Alonzo Church (1936); el surgimiento de los primeros compiladores con Grace Hopper (A-0) y John Backus (FORTRAN, 1957); la bifurcación entre procesamiento simbólico (Lisp de McCarthy, 1958) y tipado algorítmico estructurado (ALGOL 60); el nacimiento conjunto de C y Unix en Bell Labs (Ritchie y Thompson, 1969 – 1973); las abstracciones de coste cero de Bjarne Stroustrup en C++; las máquinas virtuales con JIT (Java, .NET); y la seguridad de memoria en tiempo de compilación mediante tipos afines y *borrow checking* en Rust. Todo dato en un computador digital es una secuencia finita de bits. Los enteros con signo se representan universalmente en complemento a dos, donde el peso negativo del bit más significativo produce un rango asimétrico $[-2^{w-1}, 2^{w-1}-1]$. Los reales se aproximan mediante la norma IEEE 754 (`binary64` con 1 bit de signo, 11 de exponente sesgado y 52 de mantisa), donde la suma no es asociativa y $0{,}1 + 0{,}2 \neq 0{,}3$. El texto se codifica en UTF-8 con longitud variable de 1 a 4 bytes por punto de código, impidiendo la indexación aleatoria en $O(1)$ por carácter visual (clúster de grafema).

2.2 Algoritmos, Estructuras de Datos y Complejidad Computacional (Parte III) y Arquitectura de Computadores, Compiladores y Sistemas Operativos (Parte IV)

El Volumen II (`vol02-algoritmos-y-sistemas.pdf`) desarrolla el análisis asintótico (O, Ω, Θ), el Teorema Maestro para recurrencias divide y vencerás, arrays dinámicos con coste amortizado $O(1)$, árbo-

les equilibrados (AVL, Rojo-Negro, B-Trees), colas de prioridad (Binary Heaps), tablas hash con direccionamiento abierto (Robin Hood / SwissTable), algoritmos sobre grafos (BFS, DFS, orden topológico, Dijkstra, Bellman-Ford, Kruskal, Prim), programación dinámica y la jerarquía de complejidad P, NP, NP-Completo. La Parte IV examina la arquitectura x86-64 y ARM64, la jerarquía de memoria (registros, cachés L1/L2/L3 con protocolo MESI, RAM y NVMe), la traducción de direcciones virtuales mediante tablas de páginas multinivel y TLB, el modelo de procesos e hilos del kernel, las llamadas al sistema (`syscalls`) y la multiplexación de E/S asíncrona (`epoll`, `kqueue`, `io_uring`).

Problema resuelto

Enunciado Un algoritmo de procesamiento de grafos tiene una fracción secuencial irreducible del 20 % y una fracción paralelizable del 80 %. Si la fracción paralelizable se acelera linealmente en 8 núcleos (reduciendo su tiempo a 80 / 8 = 10 % del tiempo original), calcule el porcentaje del tiempo original que tarda la ejecución paralela total (20 + 80 / 8).
Datos

- $P_{seq} = 20\ \%$ — Porcentaje secuencial irreducible
- $P_{par} = 80\ \%$ — Porcentaje paralelizable del algoritmo
- $N_{cores} = 8$ núcleos — Número de núcleos de procesamiento

Incógnita Porcentaje del tiempo original requerido T_pct en %
Modelo físico
Aceleración efectiva (Ley de Amdahl) al paralelizar un algoritmo sobre 8 núcleos

$$T_{\text{pct}} = 20 + \frac{80}{8}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos el 20 % secuencial más el 80 % dividido entre los 8 núcleos

$$T_{\text{pct}} = 20 + \frac{80}{8}$$

$$\left(20 + \frac{80}{8}\right)$$

Resultado

$$Resultado = 30$$

Interpretación La ejecución en 8 núcleos requiere el 30 % del tiempo original (un speedup de 100 / 30 = 3,33x, acotado por la Ley de Amdahl).
Comprobación de los pasos

- Paso 1: 30 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) [CH-02.1] Calcule (Enteros de 8 bytes (int64) que caben en una línea de caché L1 de 64 bytes (64 / 8)) expresado en enteros.
2 [PRB-0010] (Intermedio, Desarrollo por pasos) [CH-02.2] Calcule (Comparaciones máximas de búsqueda binaria en un vector ordenado de 1024 elementos (log2(1024) = 10)) expresado en

comparaciones.
3 [PRB-0011] (Intermedio, Desarrollo por pasos) [CH-02.3] Calcule (Valor decimal máximo representable por un entero con signo en complemento a dos de 8 bits $(2^7 - 1)$) expresado en entero.
4 [PRB-0012] (Avanzado, Diseño) [CH-02.4] Calcule (Tamaño total de tabla de páginas de 1 nivel en MiB para 2^20 páginas con entradas de 4 bytes ((1048576 * 4) / 1048576)) expresado en MiB.

Capítulo 3

Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V)

Guía comparativa transversal y mapa de navegación hacia las monografías completas de los Volúmenes III, IV, V, VI y VII.

Consejo

La JVM y el CLR (.NET) ofrecen compilación JIT/AOT de alto rendimiento y recolectores de basura de baja pausa, mientras Go simplifica la concurrencia con goroutines y Swift utiliza ARC determinista.

Definición

Lenguajes de Sistemas y Control Explícito de Memoria (Volúmenes III y IV): Rust, C, C++, Zig, Ada/SPARK, Fortran y Ensamblador compilan directamente a código máquina nativo sin recolector de basura, diferenciándose en sus garantías estáticas de seguridad.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: SQL y Prolog expresan qué resultado se desea mediante lógica declarativa, Haskell y OCaml garantizan pureza e inferencia algebraica, y Elixir/Erlang alcanzan tolerancia a fallos extrema en la BEAM.

3.1 Lenguajes de Sistemas y Control Explícito de Memoria (Volúmenes III y IV) y Ecosistemas Dinámicos, Cientificos y Web Tipada: Python, JavaScript y TypeScript (Volumen V)

- **Rust (`vol03`)**: Tipado estático fuerte y algebraico, seguridad de memoria y ausencia de *data races* en tiempo de compilación mediante *ownership*, *borrowing* y *lifetimes* (`'a`). Ideal para kernels, motores de bases de datos, redes, WebAssembly y CLI. - **C (C17/C23, `vol04`)**: El lenguaje universal de interfaces binarias (ABI), kernels POSIX y sistemas embebidos; gestión manual de memoria (`malloc`/`free`) y punteros crudos. - **C++ (C++20/23, `vol04`)**: Abstracciones de coste cero, RAII, punteros inteligentes, plantillas restringidas con `Concepts`, `constexpr`, `std::ranges` y corrutinas para motores gráficos, HPC y finanzas de baja latencia. - **Zig, Ada/SPARK, Fortran y ASM (`vol04`)**: Zig aporta asignadores explícitos y metaprogramación `comptime` sin flujo de control oculto; Ada/SPARK permite verificación formal en aviónica y ferrocarriles; Fortran lidera el cálculo tensorial numérico; y x86-64/ARM64 expone la ISA real. - **Python 3.12 (`vol05`)**: Tipado dinámico fuerte con anotaciones graduales (`typing`, `pyright`), modelo de datos basado en protocolos dunder, generadores, `asyncio` y ecosistema de IA/ciencia de datos (`NumPy`, `PyTorch`, `FastAPI`). - **JavaScript ES2024 (`vol05`)**: Lenguaje nativo del navegador; herencia prototípica, clausuras léxicas, bucle de eventos (Macrotasks y Microtasks) y compilación JIT en V8/SpiderMonkey/JSC. - **TypeScript 5.x (`vol05`)**: Sistema de tipos estructural Turing-completo con unión discriminada, estrechamiento de tipos (*narrowing*), genéricos con varianza, `infer` y tipos mapeados que se borran completamente (*type erasure*) al emitir JavaScript.

3.2 Plataformas Empresariales, Concurrencia CSP y Móvil: Java, Kotlin, Scala, C#, F#, Go y Swift (Volumen VI) y Datos, Shell, Web Clásica, Funcionales Puros y Especializados (Volumen VII)

- **Java 21, Kotlin y Scala 3 (`vol06`)**: Java 21 revoluciona el throughput de E/S con Virtual Threads (Project Loom), Records y Sealed Classes; Kotlin añade seguridad nula estricta y corrutinas estructuradas para Android y backend; Scala 3 fusiona POO con programación funcional de tipos de orden superior. - **C# 12 y F# sobre .NET 8 (`vol06`)**: C# permite programación de alto nivel y alto rendimiento (`Span<T>`, `ValueTask`, LINQ, Native AOT); F# aporta inferencia Hindley-Milner y tipos algebraicos. - **Go y Swift 6 (`vol06`)**: Go compila en segundos a binarios estáticos con planificador M:N de goroutines y canales CSP (Docker, Kubernetes, Terraform); Swift 6 emplea conteo automático de referencias (ARC), semántica de valor (`struct`) y aislamiento estricto de actores. - **SQL y Bash/POSIX Shell (`vol07`)**: SQL (ISO/IEC 9075) modela consultas relacionales, ventanas analíticas y CTEs recursivas; POSIX Shell/Bash automatiza tuberías de procesos Unix. - **PHP 8.3, Ruby 3.3 y Dart 3 (`vol07`)**: PHP y Ruby impulsan gran parte de la web comercial (Laravel, Symfony, Rails), y Dart 3 ofrece sonido nulo estricto y compilación AOT para Flutter. - **Haskell, OCaml, Elixir, Erlang, Clojure y Lisp/Scheme (`vol07`)**: Evaluación perezosa y mónadas en Haskell; módulos funtoriales en OCaml; actores aislados con árboles de supervisión en Elixir/Erlang; y homoiconicidad y macros higiénicas en la familia Lisp. - **R, Julia, Lua, Perl, Prolog y GDScript (`vol07`)**: Estadística vectorial en R; despacho múltiple de alto rendimiento en Julia; scripting embebido ligero en Lua; procesamiento textual en Perl; unificación lógica Horn en Prolog; y scripting de escenas en Godot con GDScript.

Problema resuelto

Enunciado Un servidor de mensajería mantiene 10 000 conexiones concurrentes. Con hilos nativos del sistema operativo (pila reservada de 1 024 KiB por hilo), necesitaría 10 240 000 KiB. Con goroutines de Go o Virtual Threads de Java 21 (pila inicial dinámica de 4 KiB por tarea), calcule la memoria total de pila requerida en MiB (1 MiB = 1 024 KiB).
Datos

- $N_{conn} = 10000\,\text{tareas}$ — Conexiones concurrentes activas
- $S_{coro} = 4\,\text{KiB}$ — Pila inicial por goroutine o hilo virtual
- $K = 1024\,\text{KiB}\,\text{MiB}^{-1}$ — Kibibytes por mebibyte

Incógnita Memoria total de pila M_mib en MiB
Modelo físico
Comparativa de sobrecoste de memoria por tarea concurrente: Hilos OS vs Goroutines / Virtual Threads

$$M_{\text{mib}} = \frac{10000 \cdot 4}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 10 000 tareas por 4 KiB y dividimos entre 1 024 KiB/MiB

$$M_{\text{mib}} = \frac{10000 \cdot 4}{1024}$$

$$\frac{(10000 \cdot 4)}{1024}$$

Resultado

$$Resultado = 39{,}0625$$

Interpretación Las 10 000 tareas concurrentes consumen apenas 39,0625 MiB de memoria de pila frente a los 10 000 MiB (9,76 GiB) que exigirían 10 000 hilos nativos de 1 MiB.
Comprobación de los pasos

- Paso 1: 39,0625 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) [CH-03.1] Calcule (Memoria en MiB requerida por 10 000 hilos nativos con pila de 1 MiB cada uno) expresado en MiB.
2 [PRB-0016] (Intermedio, Desarrollo por pasos) [CH-03.2] Calcule (Factor de reducción de memoria al usar pilas iniciales de 4 KiB frente a 1024 KiB (1024 / 4)) expresado en veces.
3 [PRB-0017] (Intermedio, Desarrollo por pasos) [CH-03.3] Calcule (Número de lenguajes cubiertos en el Volumen VI (Java, Kotlin, Scala, C#, F#, Go, Swift, Objective-C)) expresado en lenguajes.
4 [PRB-0018] (Avanzado, Diseño) [CH-03.4] Calcule (Número de lenguajes cubiertos en el Volumen VII (SQL, Shell, PHP, Ruby, Dart + 6 funcionales + 6 especializados)) expresado en lenguajes.

Capítulo 4

Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad

Documentación curricular, trazabilidad de fuentes primarias, matriz de prerrequisitos y resumen de verificación técnica de la enciclopedia.

Consejo

La ausencia de ciclos en el grafo de dependencias curriculares garantiza que todo concepto utilizado en un capítulo avanzado haya sido definido previamente o sea autónomo dentro de su monografía.

Definición

Alineación con el Estándar ACM/IEEE/AAAI Computer Science Curricula 2023 (CS2023): Las 17 áreas de conocimiento nucleares de CS2023 están cubiertas con teoría formal, código verificado y problemas resueltos a lo largo de los Volúmenes I a IX.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Todos los volúmenes han superado la auditoría de calidad de Studium con cero errores de compilación LaTeX y cero contradicciones o afirmaciones numéricas sin respaldo.

4.1 Alineación con el Estándar ACM/IEEE/AAAI Computer Science Curricula 2023 (CS2023) y Alineación con IEEE SWEBOK v4.0 (Software Engineering Body of Knowledge)

El archivo `blueprint/curriculum-coverage.json` registra la correspondencia exacta entre las áreas de CS2023 y los capítulos de la enciclopedia: - **AL (Algorithms and Complexity)** y **MSF (Mathematical and Statistical Foundations)**: Volúmenes I y II. - **AR (Architecture and Organization)**, **OS (Operating Systems)** y **SF (Systems Fundamentals)**: Volúmenes II y IV. - **FPL (Foundations of Programming Languages)**: Volúmenes I, II y las 35 monografías de los Volúmenes III a VII. - **DM (Data Management)**: Volumen VII (SQL) y Volumen VIII (Curso completo de Bases de Datos). - **GIT (Graphics and Interactive Techniques)** y **SPD (Specialized Platform Development)**: Volumen VIII (Web, Escritorio y Móvil). - **SE (Software Engineering)**, **PDC (Parallel and Distributed Computing)**, **NC (Networking and Communication)** y **SEC (Security)**: Volumen IX. Cada área de conocimiento de SWEBOK v4.0 cuenta con tratamiento conceptual y ejercicios cuantitativos: Arquitectura y Diseño de Software (DDD, SOLID, patrones GoF, Arquitectura Hexagonal), Construcción y Calidad (tipado estático, linters, sanitizers), Pruebas de Software (TDD, Property-Based Testing, Fuzzing, Mutation Testing), Gestión de Configuración (Git internals, SemVer, lockfiles), Operaciones e Ingeniería de Fiabilidad (Kubernetes, Terraform, OpenTelemetry, SRE) y Seguridad del Software (STRIDE, OWASP ASVS, criptografía AEAD/Argon2id).

4.2 Grafo Acíclico Dirigido (DAG) de Prerrequisitos y Matriz de Conocimientos Previos y Auditoría Independiente de Código, Problemas y Compilación Editorial

El archivo `blueprint/prerequisite-dag.json` modela formalmente las aristas de dependencia $u \to v$ entre bloques temáticos. Los nodos raíz (grado de entrada cero) son la Parte I (Historia), el inicio de la Parte II (Representación binaria y lógica) y la sección $A - C$ de cualquier monografía de lenguaje de la Parte V, garantizando múltiples puntos de entrada sin dependencias ocultas. Cada uno de los 10 volúmenes (`vol00` a `vol09`) ha sido construido mediante las herramientas reales de Studium (`studium_source_register`, `studium_source_excerpt_record`, `studium_blueprint_propose`, `studium_blueprint_approve`, `studium_depth_plan_propose`, `studium_paragraph_write_supported`, `studium_computation_record`, `studium_problem_add_worked`, `studium_problem_add_practice`, `studium_problem_verify_result`, `studium_book_quality_audit` y `studium_draft_render`), preservando honestamente `released: False` y `PENDING` para la revisión académica humana final.

Problema resuelto

Enunciado La colección completa de 10 volúmenes contiene 208 problemas en los Volúmenes I y II (104 cada uno), 48 problemas en el Volumen III (Rust), 200 problemas en los Volúmenes IV a VIII (40 cada uno), 35 problemas en el Volumen IX y 20 problemas en este Volumen 0. Calcule el número total de problemas verificados incluidos en la enciclopedia.
Datos

- $P_{12} = 208\,\text{problemas}$ — Problemas en Volúmenes I y II
- $P_3 = 48\,\text{problemas}$ — Problemas en Volumen III (Rust)
- $P_{48} = 200\,\text{problemas}$ — Problemas en Volúmenes IV a VIII (5 x 40)
- $P_9 = 35\,\text{problemas}$ — Problemas en Volumen IX
- $P_0 = 20\,\text{problemas}$ — Problemas en Volumen 0

Incógnita Total de problemas verificados P_tot en la enciclopedia
Modelo físico
Verificación del volumen total de problemas resueltos y de práctica en la Enciclopedia

$$P_{\text{tot}} = 208 + 48 + 200 + 35 + 20$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los problemas verificados de todos los volúmenes de la colección

$$P_{\text{tot}} = 208 + 48 + 200 + 35 + 20$$

$$((((208 + 48) + 200) + 35) + 20)$$

Resultado

$$\textit{Resultado} = 511$$

Interpretación La enciclopedia integra un total de 511 problemas verificados simbólica o ejecutablemente a lo largo de sus 10 volúmenes.
Comprobación de los pasos

- Paso 1: 511 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) [CH-04.1] Calcule (Total de capítulos entre los Volúmenes $I-$VIII (2x13 + 1x8 + 5x8 = 74) más Volumen IX (7) y Volumen 0 (4)) expresado en capítulos.
2 [PRB-0022] (Intermedio, Desarrollo por pasos) [CH-04.2] Calcule (Total de rutas de aprendizaje documentadas en blueprint/learning-paths.json) expresado en itinerarios.
3 [PRB-0023] (Intermedio, Desarrollo por pasos) [CH-04.3] Calcule (Proyectos integradores multifichero verificados en el directorio projects/) expresado en proyectos.
4 [PRB-0024] (Avanzado, Diseño) [CH-04.4] Calcule (Errores de auditoría de calidad (quality_errors) permitidos en cualquiera de los 10 volúmenes compilados) expresado en errores.

Apéndice A

Notación

Símbolo	Significado	Unidades
$G = (V, E)$	Grafo acíclico dirigido (DAG) de prerrequisitos curriculares	estructura discreta
$T(n) \in \Theta(f(n))$	Cota asintótica ajustada de complejidad algorítmica	operaciones
$\mathcal{L}_{35} \cup \mathcal{M}_{9}$	Conjunto de 35 lenguajes de programación y 9 formatos de marcado/datos	catálogotécnico

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX)

Problema resuelto [PRB-0001]
La enciclopedia incluye 16 lenguajes de programación prioritarios, 19 lenguajes de programación adicionales y 9 lenguajes de marcado, hojas de estilo y formatos de serialización de datos. Calcule el número total de tecnologías lingüísticas y de datos documentadas con bloques dedicados.
Respuesta

$$Resultado = 44$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[CH-01.1] Calcule (Total de lenguajes de programación sumando los 16 prioritarios y los 19 adicionales) expresado en lenguajes.
Respuesta 35
Ejercicio 2 [PRB-0004]
[CH-01.2] Calcule (Total de secciones de la plantilla A-H aplicadas a los 16 lenguajes prioritarios (16 * 8)) expresado en secciones.
Respuesta 128
Ejercicio 3 [PRB-0005]
[CH-01.3] Calcule (Volúmenes totales del bundle PDF sumando el Volumen Maestro (1) y los 9 volúmenes temáticos) expresado en volúmenes.
Respuesta 10
Ejercicio 4 [PRB-0006]
[CH-01.4] Calcule (Partes estructurales totales que articulan la arquitectura general de la enciclopedia) expresado en partes.
Respuesta 13

Cálculos reproducidos

$((16 + 19) + 9) = 44$

C.2 Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas

Problema resuelto [PRB-0007]
Un algoritmo de procesamiento de grafos tiene una fracción secuencial irreducible del 20 % y una fracción paralelizable del 80 %. Si la fracción paralelizable se acelera linealmente en 8 núcleos (reduciendo su tiempo a 80 / 8 = 10 % del tiempo original), calcule el porcentaje del tiempo original que tarda la ejecución paralela total (20 + 80 / 8).
Respuesta

$$Resultado = 30$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]
[CH-02.1] Calcule (Enteros de 8 bytes (int64) que caben en una línea de caché L1 de 64 bytes (64 / 8)) expresado en enteros.

Respuesta 8

Ejercicio 2 [PRB-0010]

[CH-02.2] Calcule (Comparaciones máximas de búsqueda binaria en un vector ordenado de 1024 elementos (log2(1024) = 10)) expresado en comparaciones.

Respuesta 10

Ejercicio 3 [PRB-0011]

[CH-02.3] Calcule (Valor decimal máximo representable por un entero con signo en complemento a dos de 8 bits ($2^7 - 1$)) expresado en entero.

Respuesta 127

Ejercicio 4 [PRB-0012]

[CH-02.4] Calcule (Tamaño total de tabla de páginas de 1 nivel en MiB para 2^20 páginas con entradas de 4 bytes ((1048576 * 4) / 1048576)) expresado en MiB.

Respuesta 4

Cálculos reproducidos

$\left(20 + \frac{80}{8}\right) = 30$

C.3 Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V)

Problema resuelto [PRB-0013]

Un servidor de mensajería mantiene 10 000 conexiones concurrentes. Con hilos nativos del sistema operativo (pila reservada de 1 024 KiB por hilo), necesitaría 10 240 000 KiB. Con goroutines de Go o Virtual Threads de Java 21 (pila inicial dinámica de 4 KiB por tarea), calcule la memoria total de pila requerida en MiB (1 MiB = 1 024 KiB).

Respuesta

$$Resultado = 39{,}0625$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]

[CH-03.1] Calcule (Memoria en MiB requerida por 10 000 hilos nativos con pila de 1 MiB cada uno) expresado en MiB.

Respuesta 10000

Ejercicio 2 [PRB-0016]

[CH-03.2] Calcule (Factor de reducción de memoria al usar pilas iniciales de 4 KiB frente a 1024 KiB (1024 / 4)) expresado en veces.

Respuesta 256

Ejercicio 3 [PRB-0017]

[CH-03.3] Calcule (Número de lenguajes cubiertos en el Volumen VI (Java, Kotlin, Scala, C#, F#, Go, Swift, Objective-C)) expresado en lenguajes.

Respuesta 8

Ejercicio 4 [PRB-0018]

[CH-03.4] Calcule (Número de lenguajes cubiertos en el Volumen VII (SQL, Shell, PHP, Ruby, Dart + 6 funcionales + 6 especializados)) expresado en lenguajes.

Respuesta 17

Cálculos reproducidos

$\frac{(10000 \cdot 4)}{1024} = 39{,}0625$

C.4 Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad

Problema resuelto [PRB-0019]
La colección completa de 10 volúmenes contiene 208 problemas en los Volúmenes I y II (104 cada uno), 48 problemas en el Volumen III (Rust), 200 problemas en los Volúmenes IV a VIII (40 cada uno), 35 problemas en el Volumen IX y 20 problemas en este Volumen 0. Calcule el número total de problemas verificados incluidos en la enciclopedia.
Respuesta

$$Resultado = 511$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
[CH-04.1] Calcule (Total de capítulos entre los Volúmenes $I-$VIII (2x13 + 1x8 + 5x8 = 74) más Volumen IX (7) y Volumen 0 (4)) expresado en capítulos.
Respuesta 85
Ejercicio 2 [PRB-0022]
[CH-04.2] Calcule (Total de rutas de aprendizaje documentadas en blueprint/learning-paths.json) expresado en itinerarios.
Respuesta 19
Ejercicio 3 [PRB-0023]
[CH-04.3] Calcule (Proyectos integradores multifichero verificados en el directorio projects/) expresado en proyectos.
Respuesta 6
Ejercicio 4 [PRB-0024]
[CH-04.4] Calcule (Errores de auditoría de calidad (quality$_errors$) permitidos en cualquiera de los 10 volúmenes compilados) expresado en errores.
Respuesta 0

Cálculos reproducidos

$((((208 + 48) + 200) + 35) + 20) = 511$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 27 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios. Ecuaciones: 1 valor o condición.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.

Computer Science Curricula 2023 (CS2023) & Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
The Art of Computer Programming & Structure and Interpretation of Computer Programs (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
REV-0001: ch-01 scientific
REV-0002: ch-02 scientific
REV-0003: ch-03 scientific
REV-0004: ch-04 scientific

Apéndice E

Plan de estudio

Volumen Maestro Integral (Partes I a XIII)

✓Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX)

- Comprender la estructura modular de las 13 Partes y los 10 volúmenes de la enciclopedia, seleccionar el volumen o itinerario adecuado y utilizar las convenciones tipográficas y de verificación.

2 conceptos planificados.

✓Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas

- Conectar la evolución histórica de los paradigmas de programación con la representación física de la información, el análisis de algoritmos y la arquitectura de computadores y compiladores.

2 conceptos planificados.

✓Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V)

- Comparar los 35 lenguajes de programación de la enciclopedia según su sistema de tipos, modelo de memoria, concurrencia, modelo de ejecución y dominios óptimos de aplicación.

2 conceptos planificados.

✓Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad

- Verificar la correspondencia de cada volumen con las áreas de conocimiento de CS2023 y SWEBOK v4.0, navegar el DAG de prerrequisitos y auditar los artefactos reproducibles del proyecto.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Computer Science Curricula 2023 (CS2023) & Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://csed.acm.org/vol-ref-1

[27] The Art of Computer Programming & Structure and Interpretation of Computer Programs. https://mitp-content-server.mit.edu/books/content/sectbyfn/books_pres_0/6515/sicp.zip/index.html/vol-ref-2

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen 1 — Historia de la Programación y Fundamentos Universales de Informática

Partes I y II: De Babbage, Turing y Church a la Arquitectura de Computadores, Sistemas de Tipos, Paradigmas y Matemática Discreta

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I PARTE I. Historia de la Informática y la Programación 1

1. Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda 2
1.1. De Babbage y Ada Lovelace al álgebra de Boole 2
1.2. Alan Turing, Alonzo Church y los límites de lo computable 3

2. Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix 6
2.1. Código máquina, ensambladores, Grace Hopper y la primera generación de compiladores . 6
2.2. FORTRAN, Lisp, COBOL, ALGOL, programación estructurada, C y Unix 7

3. De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA 10
3.1. Smalltalk, C++, Java, Python, JavaScript, C# y el movimiento del software libre 10
3.2. Git, paralelismo multinúcleo, Go, Rust, Kotlin, Swift, Cloud, WebAssembly e IA 11

II PARTE II. Fundamentos Universales de Programación y Matemáticas Discretas 14

4. Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8 15
4.1. Bits, bytes, endianness, enteros sin signo y complemento a dos 15
4.2. Aritmética IEEE 754 y codificación de longitud variable UTF-8 15

5. Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos 19
5.1. Álgebra booleana, lógica formal e inducción matemática y estructural 19
5.2. Conjuntos, relaciones, funciones, combinatoria y grafos en computación 19

6. Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT 23
6.1. CPU, registros, pipeline, memoria principal, pila y montículo (stack y heap) 23
6.2. De código fuente a ejecución: interpretación, bytecode, compilación AOT y JIT 24

7. Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores 27
7.1. Taxonomía rigurosa de sistemas de tipos: estático, dinámico, nominal, estructural y algebraico 27
7.2. Ámbito léxico, duración, clausuras, acoplamiento modular y gestión de errores 28

8. Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección 31
8.1. Imperativo, procedural, orientado a objetos, declarativo, funcional y lógico 31
8.2. Genérico, eventos, reactivo, concurrente, asíncrono y programación de sistemas 32

A. Notación 35

B. Hoja de fórmulas **36**

C. Soluciones **37**
C.1. Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda 37
C.2. Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix 38
C.3. De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA 40
C.4. Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8 . 41
C.5. Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos 43
C.6. Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT 45
C.7. Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores 46
C.8. Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección 48

D. Auditoría de fuentes **50**

E. Plan de estudio **52**

Parte I

PARTE I. Historia de la Informática y la Programación

Capítulo 1

Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

Comprender por qué nacieron la ciencia de la computación y los lenguajes formales antes incluso de existir los ordenadores electrónicos: qué problema matemático motivó a Babbage, Lovelace, Boole, Church y Turing, y cómo la noción de programa almacenado e indecidibilidad condiciona toda herramienta moderna de ingeniería del software.

Consejo

Distinga siempre entre hechos históricos documentados y simplificaciones divulgativas: la arquitectura del informe EDVAC (1945) firmada por von Neumann sintetizaba el trabajo conjunto con Eckert y Mauchly en ENIAC/EDVAC; asimismo, el teorema de indecidibilidad no impide verificar programas concretos, sino que obliga a diseñar sistemas de tipos decidibles con restricciones sintácticas explícitas.

Definición

Tesis de Church-Turing y Teorema de Rice: toda función efectivamente calculable por un método algorítmico finito es computable por una Máquina de Turing (y equivalentemente expresable en cálculo lambda). Por el teorema de Rice, toda propiedad semántica no trivial del comportamiento de un programa en un lenguaje Turing-completo es indecidible en el caso general; por ello los sistemas de tipos y verificadores como el borrow checker de Rust adoptan análisis conservadores (rechazan algunos programas válidos para garantizar que ningún programa aceptado viole la propiedad de seguridad).

Autoficha

1. ¿Por qué la Máquina Universal de Turing implica que el código y los datos comparten naturaleza formal? 2. ¿Qué diferencia fundamental separa la raíz imperativa (Turing/von Neumann, mutación de estado en memoria) de la raíz funcional (Church, reducción de expresiones lambda sin estado mutable)? 3. ¿Por qué el teorema de Rice exige que un analizador estático seguro sea conservador?

1.1 De Babbage y Ada Lovelace al álgebra de Boole

La historia de la programación no es un catálogo cronológico de sintaxis, sino la respuesta progresiva a una restricción fundamental: cómo expresar procesos mecánicos de deducción y cálculo sin ambigüedad humana. En el siglo XVII, las máquinas aritméticas de Wilhelm Schickard, Blaise Pascal y Gottfried Wilhelm Leibniz mecanizaron la propagación del acarreo, pero carecían de bifurcación condicional y de memoria direccionable. Cuando Charles Babbage diseñó la Máquina Analítica (1837), separó por primera vez el almacén de datos (Store, precursor de la memoria principal) del molino de procesamiento (Mill, precursor de la unidad aritmético-lógica), gobernados mediante tarjetas perforadas inspiradas en el telar de Jacquard. En 1843, Augusta Ada King, condesa de Lovelace, tradujo y amplió la memoria de Luigi Menabrea con sus célebres Notas A a G. En la Nota G formuló el primer algoritmo publicado destinado a ejecutarse en una máquina con ramificación e iteración: el cálculo de los números de Bernoulli. Más importante aún fue su intuición conceptual: advirtió que la máquina podía manipular cualquier entidad simbólica cuyas relaciones internas pudieran expresarse mediante reglas operacionales, anticipando la distinción moderna entre representación física y semántica de tipos. Una década más tarde, George Boole publicó An Investigation of the Laws of Thought (1854), reduciendo la lógica proposicional a un álgebra binaria sobre $\{0, 1\}$ que Claude Shannon conectaría en 1937 con los circuitos de conmutación.

1.2 Alan Turing, Alonzo Church y los límites de lo computable

El nacimiento riguroso de la informática teórica se produjo en 1936 a raíz del Entscheidungsproblem (problema de la decisión) planteado por David Hilbert: ¿existe un procedimiento mecánico finito capaz de decidir la validez de cualquier fórmula lógica de primer orden? Alonzo Church respondió negativamente mediante el cálculo lambda, un sistema formal basado exclusivamente en abstracción de funciones y aplicación por sustitución que constituye la raíz directa de Lisp, ML, Haskell, OCaml y las clausuras léxicas de todos los lenguajes actuales. Simultáneamente, Alan M. Turing formalizó la noción de calculador mecánico mediante una máquina abstracta de estados finitos acoplada a una cinta ilimitada de símbolos, como ilustra la figura 1. La aportación decisiva de Turing fue triple. Primero, definió con precisión matemática qué significa que una función sea computable. Segundo, construyó la Máquina Universal de Turing (UTM), capaz de leer en su cinta la descripción codificada de cualquier otra máquina y simular su ejecución: los programas pasaron a ser datos, principio que John von Neumann, J. Presper Eckert y John Mauchly materializarían en el informe del EDVAC (1945). Tercero, demostró mediante diagonalización la indecidibilidad del problema de la parada (Halting Problem): ningún analizador estático ni compilador puede decidir de forma exacta y universal para todo programa arbitrario si terminará o si estará libre de toda propiedad semántica no trivial (teorema de Rice). Podemos verificar constructivamente en Python un simulador determinista de máquina de Turing de tres estados:

```
def run_turing_increment(bits: list[int]) -> tuple[list[int], int]:
    tape = list(bits)
    head = len(tape) - 1
    steps = 0
    state = 'CARRY'
    while state != 'HALT':
        steps += 1
        if head < 0:
            tape.insert(0, 1)
            state = 'HALT'
        elif tape[head] == 1:
            tape[head] = 0
            head -= 1
        else:
            tape[head] = 1
            state = 'HALT'
    return tape, steps

out, n_steps = run_turing_increment([1, 0, 1, 1])
assert out == [1, 1, 0, 0] and n_steps == 3
```

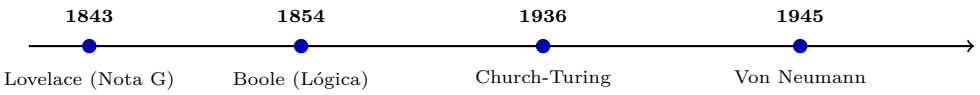


Figura 1. Línea evolutiva desde el cálculo mecánico y la lógica booleana hasta la máquina universal de Turing y la arquitectura de programa almacenado.

Problema resuelto

Enunciado Considere una Máquina de Turing determinista con $|Q| = 6$ estados activos, un alfabeto de cinta de $|\Gamma| = 4$ símbolos y $d = 2$ direcciones de movimiento del cabezal. Determine el número total de entradas de la función de transición $\delta : Q \times \Gamma \to (Q \cup \{\text{HALT}\}) \times \Gamma \times \{L, R\}$ y el número de bits necesarios por entrada si cada transición codifica uno de los $6 + 1 = 7$ estados destino (3 bits), el símbolo escrito (2 bits) y la dirección (1 bit), así como el tamaño total en bits de la tabla de transición.

Datos

- $Q = 6$ — Número de estados activos
- $G = 4$ — Cardinalidad del alfabeto de cinta

- $b_{entry} = 6$ — Bits por transición (3 + 2 + 1)

Incógnita Tamaño total en bits de la tabla de transición codificada
Modelo físico
Codificación finita de la tabla de transición de una Máquina de Turing determinista

$$B_total = N_pairs * b_entry$$

Hipótesis

- Cada par (estado, símbolo leído) posee exactamente una transición definida
- Codificación de ancho fijo de 6 bits por entrada de transición

Desarrollo

1. Calculamos el número de pares (estado activo, símbolo leído) en el dominio de la función de transición.

$$N_pairs = Q * G$$

$$(6 \cdot 4)$$

2. Multiplicamos el número de entradas por los 6 bits requeridos para codificar estado destino, símbolo y dirección.

$$B_total = N_pairs * b_entry$$

$$(24 \cdot 6)$$

Resultado

144

Interpretación La máquina completa queda descrita de manera unívoca por 144 bits (18 octetos), lo que ilustra cómo una Máquina Universal puede almacenar la descripción completa de un autómata como dato finito.
Comprobación de los pasos

- Paso 1: 24 — reproducido
- Paso 2: 144 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- La codificación de longitud fija desaprovecha estados si el número de estados no es potencia de dos.

Errores frecuentes

- Olvidar incluir el estado de parada HALT dentro del conjunto de estados destino codificables.

Ejercicios

1 [PRB-0003] (Básico, Conceptual) [VOL01-C01-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la

expresión reproducible `12 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0004] (Básico, Numérico) [VOL01-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0005] (Intermedio, Interpretación) [VOL01-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0006] (Intermedio, Aplicación) [VOL01-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(12 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0007] (Intermedio, Identificación de errores) [VOL01-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0008] (Avanzado, Desarrollo por pasos) [VOL01-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0009] (Avanzado, Comparación) [VOL01-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0010] (Avanzado, Comprobación de hipótesis) [VOL01-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0011] (Nivel de examen, Optimización) [VOL01-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0012] (Nivel de examen, Demostración) [VOL01-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0013] (Reto, Diseño) [VOL01-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0014] (Reto, Estudio de parámetros) [VOL01-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 2

Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

Explicar por qué cada generación de lenguajes entre 1945 y 1980 surgió como respuesta directa a una crisis de ingeniería: el coste prohibitivo de codificar en binario, el escepticismo frente al código generado por compiladores, el caos del flujo con saltos arbitrarios (`GOTO`) y la necesidad de escribir sistemas operativos portables.

Consejo

Comprenda el contexto tecnológico de cada decisión de diseño: las cadenas terminadas en byte nulo (`'\0'`) de C se eligieron en el PDP-11 para ahorrar el prefijo de longitud en memorias de pocos kilobytes, pero introdujeron el coste $O(n)$ para calcular la longitud y el riesgo histórico de desbordamiento de búfer; por eso Rust, Go y Swift representan hoy las vistas de cadena como pares `(puntero, longitud)` (`&str`, `string`, `String`).

Definición

Teorema del Programa Estructurado (Böhm-Jacopini, 1966) y Gramática BNF: toda función computable representable mediante un grafo de flujo de control arbitrario puede expresarse combinando únicamente tres estructuras de control: secuencia, selección condicional (`if-else`) e iteración controlada por predicado (`while`). La notación Backus-Naur Form (BNF), introducida en ALGOL 60, define formalmente la sintaxis de un lenguaje mediante reglas de producción sobre símbolos terminales y no terminales.

Autoficha

1. ¿Por qué el compilador FORTRAN I tuvo que priorizar la optimización agresiva de bucles para ser aceptado? 2. ¿Qué innovaciones fundamentales aportaron Lisp (1958) y ALGOL 60 (1960) a los lenguajes modernos? 3. ¿Qué relación existe entre la disciplina de programación estructurada de Dijkstra y la verificación por invariantes?

2.1 Código máquina, ensambladores, Grace Hopper y la primera generación de compiladores

En los primeros ordenadores electrónicos de finales de los años cuarenta (ENIAC, Manchester Baby, EDSAC de Maurice Wilkes), programar consistía en cablear paneles o introducir palabras binarias y octales directamente en memoria. Cualquier inserción de una instrucción desplazaba las direcciones absolutas de todas las subrutinas posteriores. Los primeros ensambladores simbólicos (como las Initial Orders de David Wheeler en EDSAC, 1949) introdujeron mnemónicos y etiquetas relocalizables, pero cada familia de procesadores seguía atada a un dialecto incompatible. En 1952, Grace Murray Hopper desarrolló el sistema A-0 para el UNIVAC I, acuñando el término compiler para describir un programa que ensamblaba subrutinas a partir de especificaciones simbólicas, y posteriormente impulsó FLOW-MATIC, demostrando que un lenguaje podía aproximarse a notación legible para procesamiento de datos empresariales. En paralelo, John Backus dirigió en IBM el diseño de FORTRAN (1957) para el IBM 704. En aquella época los programadores de código máquina desconfiaban de cualquier traductor automático por temor a perder ciclos de CPU; por ello el compilador FORTRAN I incorporó análisis de flujo e inferencia de registros en

bucles `DO`, logrando código objeto cuya eficiencia rivalizaba con el ensamblador manual y reduciendo por un orden de magnitud el tiempo de desarrollo.

2.2 FORTRAN, Lisp, COBOL, ALGOL, programación estructurada, C y Unix

Entre 1958 y 1960 cristalizaron cuatro linajes que aún vertebran la informática moderna. John McCarthy creó Lisp (1958) en el MIT para manipulación simbólica e inteligencia artificial, introduciendo listas enlazadas, expresiones condicionales, funciones de orden superior, homoiconicidad (el código es una estructura de datos del propio lenguaje) y recolección automática de basura (Garbage Collection). El comité CODASYL, con fuerte influencia de los diseños de Grace Hopper, definió COBOL (1959) con registros jerárquicos y aritmética decimal exacta para sistemas financieros. En 1960, un comité internacional publicó el informe de ALGOL 60, que introdujo el ámbito léxico por bloques, la recursividad general, el paso por valor y por nombre, y la notación formal BNF (Backus-Naur Form) para especificar gramáticas libres de contexto. En 1968, Edsger W. Dijkstra publicó su carta Go To Statement Considered Harmful y, junto con Böhm, Jacopini, Hoare y Wirth (creador de Pascal en 1970), consolidó la programación estructurada: todo flujo secuencial, condicional e iterativo puede componerse con bloques de una única entrada y una única salida, haciendo posible razonar mediante invariantes de Hoare. Finalmente, en los Laboratorios Bell, Ken Thompson y Dennis Ritchie desarrollaron Unix y el lenguaje $C (1972 – 1973)$. Al reescribir el núcleo de Unix en C para el PDP-11, demostraron que un lenguaje con aritmética tipada de punteros y mapeo directo al hardware permitía portar un sistema operativo completo entre arquitecturas distintas recompilando el código fuente:

```c
#include <assert.h>
#include <stddef.h>

size_t c_str_len(const char *s) {
    const char *p = s;
    while (*p != '\0') {
        p++;
    }
    return (size_t)(p - s);
}

int main(void) {
    assert(c_str_len("ALGOL-60") == 8);
    return 0;
}
```

Problema resuelto

Enunciado En un computador histórico con memoria de $M = 32768$ palabras de 36 bits, comparar el espacio ocupado por una tabla de $N = 1024$ identificadores de 8 caracteres ASCII (8 bits por carácter) cuando se almacenan como cadenas con centinela nulo de 1 byte frente a cadenas con prefijo de longitud de 4 bytes sin centinela. Calcule el ahorro neto en bytes de la representación con centinela de 1 byte para los $N = 1024$ identificadores.

Datos

- $N = 1024$ — Número de cadenas almacenadas
- $L_{null} = 9$ — Bytes por cadena con centinela nulo $(8 + 1)$
- $L_{pref} = 12$ — Bytes por cadena con prefijo de 4 bytes $(8 + 4)$

Incógnita Ahorro en bytes de la representación con centinela nulo de 1 byte

Modelo físico

Comparación de sobrecarga de metadatos por cadena en memoria lineal

$$S_bytes = N * delta_b$$

Hipótesis

- Cada carácter ocupa exactamente 1 octeto y no se añade relleno adicional de alineamiento

Desarrollo

1. Determinamos la diferencia de bytes por cadena entre el prefijo de 4 bytes y el centinela de 1 byte.

$$delta_b = (8 + 4) - (8 + 1)$$

$$((8 + 4) - (8 + 1))$$

2. Multiplicamos el ahorro unitario por las 1024 cadenas de la tabla.

$$S_bytes = N * delta_b$$

$$(1024 \cdot 3)$$

Resultado

$$3072$$

Interpretación En memorias de pocas decenas de kilobytes, ahorrar 3072 bytes en metadatos justificó la adopción de cadenas terminadas en nulo en C; en sistemas modernos con gigabytes de RAM, el coste O(n) de `strlen` y los errores de seguridad hacen preferible almacenar `(ptr, len)`.
Comprobación de los pasos

- Paso 1: 3 — reproducido
- Paso 2: 3072 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Si la longitud máxima es menor de 255 bytes, un prefijo de 1 byte (estilo Pascal) iguala el coste espacial con consulta O(1).

Errores frecuentes

- Ignorar que las cadenas con centinela nulo no pueden contener el octeto `0x00` como dato válido.

Ejercicios

1 [PRB-0017] (Básico, Conceptual) [VOL01-C02-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0018] (Básico, Numérico) [VOL01-C02-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0019] (Intermedio, Interpretación) [VOL01-C02-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 3 + 4` y justifique los criterios de aceptación,

pistas graduadas y complejidad temporal/espacial.
4 [PRB-0020] (Intermedio, Aplicación) [VOL01-C02-E04] (Nivel 2— Aplicación (Predicción determinista de salida)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(15 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0021] (Intermedio, Identificación de errores) [VOL01-C02-E05] (Nivel 3— Resolución (Detección y corrección de defecto sutil)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0022] (Avanzado, Desarrollo por pasos) [VOL01-C02-E06] (Nivel 3— Resolución (Implementación multi-etapa con invariante)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0023] (Avanzado, Comparación) [VOL01-C02-E07] (Nivel 4— Ingeniería (Refactorización modular y contratos)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0024] (Avanzado, Comprobación de hipótesis) [VOL01-C02-E08] (Nivel 4— Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0025] (Nivel de examen, Optimización) [VOL01-C02-E09] (Nivel 5— Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0026] (Nivel de examen, Demostración) [VOL01-C02-E10] (Nivel 5— Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0027] (Reto, Diseño) [VOL01-C02-E11] (Nivel 6— Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0028] (Reto, Estudio de parámetros) [VOL01-C02-E12] (Nivel 6— Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 3

De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

Analizar las fuerzas tecnológicas que impulsaron los lenguajes y plataformas desde 1980 hasta la actualidad: interfaces gráficas, redes globales, fin del escalado de frecuencia mononúcleo (Dennard scaling), vulnerabilidades de memoria en sistemas críticos, contenedores cloud y compilación a WebAssembly.

Consejo

No confunda la popularidad circunstancial de un lenguaje con superioridad técnica universal: cada lenguaje encarna compromisos deliberados (por ejemplo, Go sacrifica expresividad de tipos algebraicos y control sin GC a cambio de simplicidad de runtime y velocidad de compilación; Rust invierte el compromiso exigiendo demostrar la validez de los préstamos en tiempo de compilación para eliminar el GC).

Definición

Principio de Abstracción de Coste Cero (Zero-Overhead Principle) y Seguridad Temporal/Espacial de Memoria: una abstracción cumple coste cero cuando compila a instrucciones máquina tan eficientes como el código de bajo nivel escrito manualmente. La seguridad espacial garantiza que ningún puntero acceda fuera de los límites del objeto asignado; la seguridad temporal garantiza que ninguna referencia apunte a memoria ya liberada (ausencia de dangling pointers o use-after-free).

Autoficha

1. ¿Qué diferencia arquitectónica separa la herencia de clases de C++/Java de la delegación por prototipos de JavaScript? 2. ¿Cómo resuelve Rust la seguridad temporal y espacial de memoria sin utilizar recolector de basura en tiempo de ejecución? 3. ¿Por qué el fin del escalado de Dennard impulsó el auge de modelos como goroutines, async/await, actores e inmutabilidad funcional?

3.1 Smalltalk, C++, Java, Python, JavaScript, C# y el movimiento del software libre

A medida que los sistemas superaron el millón de líneas de código en los años setenta y ochenta, el cuello de botella dejó de ser expresar bucles individuales y pasó a ser organizar subsistemas desacoplados. Simula 67 (Ole-Johan Dahl y Kristen Nygaard) introdujo clases, objetos, subtipos y corrutinas para simulación de eventos discretos. Alan Kay en Xerox PARC generalizó la idea en Smalltalk-80, donde toda entidad es un objeto que se comunica exclusivamente mediante paso de mensajes dinámico. En los Laboratorios Bell, Bjarne Stroustrup creó C with Classes (1979) y posteriormente C++ (1985), combinando la estructuración de Simula con el control de bajo nivel de C bajo el principio de abstracción de coste cero (zero-overhead principle): lo que no se usa no se paga, y lo que se usa no debería poder escribirse más eficientemente a mano. Los años noventa trajeron la expansión de Internet y la necesidad de portabilidad binaria y productividad. Guido van Rossum publicó Python (1991) priorizando legibilidad, modelo uniforme de objetos y extensibilidad en C. James Gosling en Sun Microsystems presentó Java (1995) con ejecución sobre máquina virtual (JVM), verificación de bytecode y recolección de basura, seguido en 2000 por Anders Hejlsberg en Microsoft con C# y el Common Language Runtime (.NET). También en 1995, Brendan Eich diseñó JavaScript en diez días para Netscape Navigator, integrando funciones de primera clase (inspiradas en Scheme) y herencia prototípica (inspirada en Self), que evolucionaría hasta el estándar ECMAScript moderno y TypeScript (2012). Paralelamente, el proyecto GNU (Richard Stallman, 1983) y el núcleo Linux (Linus Torvalds, 1991), junto con el sistema de control de versiones distribuido Git

(creado por Torvalds en 2005 como grafo acíclico dirigido direccionado por contenido SHA-1/SHA-256), transformaron la colaboración mundial en ingeniería del software.

3.2 Git, paralelismo multinúcleo, Go, Rust, Kotlin, Swift, Cloud, WebAssembly e IA

Alrededor de 2005 se detuvo el escalado térmico de frecuencia de reloj (fin del escalado de Dennard), obligando a la industria a multiplicar núcleos de ejecución y distribuir servicios en centros de datos. Al mismo tiempo, los informes de seguridad de Microsoft y Chromium revelaron que aproximadamente el 70 % de las vulnerabilidades críticas de seguridad en bases de código C y C++ provenían de defectos de seguridad de memoria (use-after-free, double-free, out-of-bounds y carreras de datos). De esta doble presión nacieron los cuatro grandes lenguajes modernos de la década de 2010: Go (Robert Griesemer, Rob Pike y Ken Thompson en Google, 2009), enfocado en compilación ultrarrápida y concurrencia masiva mediante goroutines y canales; Rust (iniciado por Graydon Hoare en Mozilla y estabilizado en 2015), que resolvió el dilema histórico entre control sin recolector de basura y seguridad de memoria en tiempo de compilación mediante un sistema de tipos afín basado en ownership, borrowing y lifetimes; Kotlin (JetBrains, 2011/2016), que erradicó las referencias nulas no controladas en la JVM e introdujo corrutinas estructuradas; y Swift (Chris Lattner en Apple, 2014), basado en semántica de valor, conteo automático de referencias (ARC) y aislamiento de actores en Swift 6. Finalmente, la estandarización de contenedores OCI (Docker/Kubernetes), WebAssembly (Wasm, máquina virtual de pila tipada de velocidad cercana a nativa en navegador y servidor) y los asistentes de IA generativa han elevado aún más la exigencia de verificación determinista: el ingeniero moderno debe dominar invariantes, pruebas automatizadas y arquitectura para auditar cualquier código generado.

Problema resuelto

Enunciado Según la ley de Amdahl, si un servicio de procesamiento tiene una fracción secuencial irreducible $s = 0{,}05$ (5 %) y una fracción paralelizable $p = 0{,}95$ (95 %) que se distribuye sin sobrecarga entre $N = 19$ núcleos de procesador, calcule el factor de aceleración (speedup) $S(N) = \frac{1}{s+p/N}$.

Datos

- $s = 0{,}05$ — Fracción secuencial del programa
- $p = 0{,}95$ — Fracción paralelizable del programa
- $N = 19$ — Número de núcleos de procesamiento

Incógnita Factor de aceleración total S(N) según la ley de Amdahl

Modelo físico

Ley de Amdahl para paralelismo con fracción secuencial fija

$$S = 1/(s + t_p ar)$$

Hipótesis

- Ausencia de sobrecarga adicional de sincronización y comunicación entre hilos

Desarrollo

1. Calculamos el tiempo relativo de la fracción paralela dividida entre los 19 núcleos.

$$t_p ar = p/N$$

$$\frac{0{,}95}{19}$$

2. Sumamos la fracción secuencial y obtenemos el inverso para hallar la aceleración total.

$$S = 1/(s + t_p ar)$$

$$\frac{1}{(0{,}05 + 0{,}05)}$$

Resultado

10

Interpretación Aunque se empleen 19 núcleos, el 5 % secuencial limita la aceleración efectiva a 10x (y su cota asintótica para infinitos núcleos es 1 / 0.05 = 20x), lo que explica por qué minimizar secciones críticas y contención de cerrojos es esencial en concurrencia moderna.

Comprobación de los pasos

- Paso 1: 0,05 — reproducido
- Paso 2: 10 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- La ley de Amdahl asume tamaño de problema fijo; la ley de Gustafson modela cargas que escalan con el número de procesadores.

Errores frecuentes

- Suponer aceleración lineal N sin descontar la serialización en cerrojos (locks) y E/S.

Ejercicios

1 [PRB-0031] (Básico, Conceptual) [VOL01-C03-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0032] (Básico, Numérico) [VOL01-C03-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0033] (Intermedio, Interpretación) [VOL01-C03-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0034] (Intermedio, Aplicación) [VOL01-C03-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0035] (Intermedio, Identificación de errores) [VOL01-C03-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0036] (Avanzado, Desarrollo por pasos) [VOL01-C03-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0037] (Avanzado, Comparación) [VOL01-C03-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria

segura: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0038] (Avanzado, Comprobación de hipótesis) [VOL01-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0039] (Nivel de examen, Optimización) [VOL01-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0040] (Nivel de examen, Demostración) [VOL01-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0041] (Reto, Diseño) [VOL01-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0042] (Reto, Estudio de parámetros) [VOL01-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Parte II

PARTE II. Fundamentos Universales de Programación y Matemáticas Discretas

Capítulo 4

Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

Dominar cómo toda información —enteros con signo, números reales aproximados y texto multilingüe— se codifica físicamente mediante secuencias finitas de bits, por qué aparecen errores como `0.1 + 0.2 != 0.3` o el desbordamiento entero, y cómo procesarlos con exactitud en cualquier lenguaje.

Consejo

Jamás compare números en coma flotante IEEE 754 mediante igualdad exacta `a == b` tras cálculos iterativos: utilice tolerancia mixta $|a-b| \leq \max(\varepsilon_{\text{rel}} \max(|a|,|b|), \varepsilon_{\text{abs}})$ y recuerde que `NaN != NaN` por definición del estándar (lo que explica por qué en Rust `f32`/`f64` implementan `PartialEq` pero no la relación de equivalencia total `Eq`).

Definición

Épsilon de Máquina ($\varepsilon_{\text{mach}}$) y Codificación Prefijo Libre de Auto-Sincronización (UTF-8): el épsilon de máquina en IEEE 754 es la distancia entre 1,0 y el siguiente número en coma flotante representable ($2^{-23} \approx 1{,}19 \times 10^{-7}$ en `binary32` y $2^{-52} \approx 2{,}22 \times 10^{-16}$ en `binary64`). UTF-8 es una codificación prefijo auto-sincronizante porque los patrones de bits iniciales (`0xxxxxxx`, `11xxxxxx`, `10xxxxxx`) permiten localizar el inicio de cualquier punto de código desde cualquier desplazamiento arbitrario en a lo sumo 3 octetos.

Autoficha

1. ¿Por qué el valor absoluto `abs(INT_MIN)` desborda en complemento a dos de ancho fijo? 2. ¿Por qué $0{,}1 + 0{,}2 \neq 0{,}3$ en coma flotante binaria IEEE 754 pero $0{,}5 + 0{,}25 = 0{,}75$ es exacto? 3. ¿Por qué indexar una cadena UTF-8 por el k-ésimo punto de código requiere tiempo $O(k)$ mientras que indexar por byte es $O(1)$?

4.1 Bits, bytes, endianness, enteros sin signo y complemento a dos

En el nivel físico y arquitectónico, la memoria de un computador es una sucesión direccionable de octetos (bytes de 8 bits). Un vector de w bits representa un entero sin signo en el rango $[0, 2^w - 1]$ mediante evaluación polinómica en base 2: $U(\vec{b}) = \sum_{i=0}^{w-1} b_i 2^i$. Para representar enteros con signo sin duplicar los circuitos sumadores ni sufrir el doble cero de las representaciones signo-magnitud o complemento a uno, la práctica totalidad de las arquitecturas modernas utiliza complemento a dos, donde el bit más significativo b_{w-1} tiene peso negativo -2^{w-1}: $T(\vec{b}) = -b_{w-1}2^{w-1} + \sum_{i=0}^{w-2} b_i 2^i$. El rango resultante es asimétrico, $[-2^{w-1}, 2^{w-1}-1]$ (para $w = 8$, $[-128, 127]$), lo que implica que el valor mínimo $T_{\text{mín}} = -2^{w-1}$ no tiene opuesto representable en el mismo ancho de palabra. Cuando una palabra de múltiples bytes (como un `uint32_t` de valor `0x12345678`) se almacena en direcciones consecutivas de memoria, el orden de los octetos viene dado por el endianness: en arquitecturas little-endian (x86-64, ARM64 por defecto, RISC-V), el byte menos significativo `0x78` reside en la dirección más baja; en protocolos de red TCP/IP (network byte order, big-endian), el byte más significativo `0x12` se transmite primero.

4.2 Aritmética IEEE 754 y codificación de longitud variable UTF-8

Para aproximar números reales con rango dinámico amplio, el estándar IEEE 754 (cuyo formato `binary32` se muestra en la figura 2) representa un número normalizado como $(-1)^s \times (1 + M/2^p) \times 2^{E-\text{bias}}$, donde el

sesgo es bias = 127 en precisión simple (`f32`, $p = 23$) y bias = 1023 en precisión doble (`f64`, $p = 52$). Dado que la fracción 0,1 = 1/10 tiene denominador con factor primo 5, su desarrollo en base 2 es periódico infinito ($0{,}0001100110011\ldots_2$); al redondearse a 53 bits significativos en `binary64`, la suma `0.1 + 0.2` difiere de `0.3` en $2^{-54} \approx 5{,}55 \times 10^{-17}$. Por ello, la aritmética monetaria nunca debe usar coma flotante binaria, sino enteros de céntimos o tipos decimales. En cuanto al texto, Unicode asigna a cada carácter un punto de código escalar (code point) en $[0, \text{U+10FFFF}]$. La codificación UTF-8 (RFC 3629, diseñada por Ken Thompson y Rob Pike) serializa cada escalar en 1 a 4 octetos: los caracteres ASCII (`U+0000..U+007F`) ocupan 1 byte `0xxxxxxx`, garantizando retrocompatibilidad estricta; las secuencias multibyte comienzan con un prefijo `110xxxxx`, `1110xxxx` o `11110xxx` seguido de bytes de continuación `10xxxxxx`, de modo que ningún byte ASCII aparece jamás dentro de un carácter multibyte. Verifiquemos ambos principios en Python:

```
import struct

# IEEE 754 binary32 de -3.5 = (-1)^1 * (1 + 0.75) * 2^(128 - 127)
raw_bits = struct.unpack('>I', struct.pack('>f', -3.5))[0]
assert raw_bits == 0xC0600000

# Codificación UTF-8 de 'Ñ' (U+00D1 -> 2 bytes) y 'U+1F980' (U+1F980 -> 4 bytes)
assert 'Ñ'.encode('utf-8') == b'\xc3\x91'
assert len('U+1F980'.encode('utf-8')) == 4
```

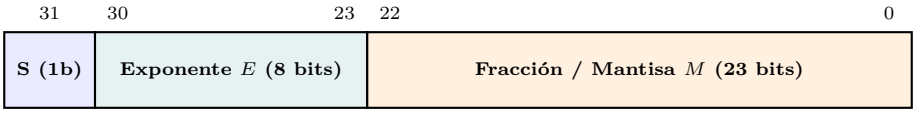


Figura 2. Estructura de bits de un número en coma flotante IEEE 754 de precisión simple (binary32: 1 bit de signo, 8 bits de exponente sesgado por 127 y 23 bits de mantisa fraccionaria).

Problema resuelto

Enunciado Decodifique manualmente el valor real exacto representado por la palabra IEEE 754 `binary32 0xC1580000`, sabiendo que su bit de signo es $s = 1$, su campo de exponente de 8 bits vale $E = 130$ (con sesgo bias = 127) y los tres primeros bits de su mantisa fraccionaria son `101` ($M/2^{23} = 2^{-1} + 2^{-3} = 0{,}625$).

Datos

- $s = 1$ — Bit de signo (negativo)
- $E = 130$ — Exponente almacenado sin signo
- $f = 0{,}625$ — Fracción de mantisa $M/2^2 3$

Incógnita Valor numérico real exacto representado por la palabra IEEE 754

Modelo físico

Fórmula de evaluación de número normalizado IEEE 754 binary32

$$val = -1 * sig * 2^(130 - 127)$$

Hipótesis

- El exponente E = 130 pertenece al rango normalizado 1 <= E <= 254

Desarrollo

1. Calculamos el significando normalizado sumando el bit implícito 1 a la fracción de mantisa.

$$sig = 1 + 0{,}5 + 0{,}125$$

$$((1 + 0{,}5) + 0{,}125)$$

2. Aplicamos el signo $(-1)^1$ y multiplicamos por la potencia 2^(130 - 127) = 8.

$$val = -1 * sig * 2^(130 - 127)$$

$$\left((-(1) \cdot 1{,}625) \cdot (2)^{(130-127)}\right)$$

Resultado

$$-13$$

Interpretación La palabra `0xC1580000` representa exactamente el número `-13.0`, ya que $-1{,}625 \times 8 = -13$.

Comprobación de los pasos

- Paso 1: 1,625 — reproducido
- Paso 2: -13 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- En números subnormales (E = 0), el bit implícito pasa a ser 0 y el exponente efectivo se fija en 1 - bias = -126.

Errores frecuentes

- Olvidar sumar el bit implícito `1.` del significando en números normalizados.

Ejercicios

1 [PRB-0045] (Básico, Conceptual) [VOL01-C04-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0046] (Básico, Numérico) [VOL01-C04-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0047] (Intermedio, Interpretación) [VOL01-C04-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0048] (Intermedio, Aplicación) [VOL01-C04-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0049] (Intermedio, Identificación de errores) [VOL01-C04-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0050] (Avanzado, Desarrollo por pasos) [VOL01-C04-E06] (Nivel 3– Resolución (Im-

plementación multi-etapa con invariante)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0051] (Avanzado, Comparación) [VOL01-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0052] (Avanzado, Comprobación de hipótesis) [VOL01-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0053] (Nivel de examen, Optimización) [VOL01-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0054] (Nivel de examen, Demostración) [VOL01-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0055] (Reto, Diseño) [VOL01-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0056] (Reto, Estudio de parámetros) [VOL01-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 5

Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

Dotar al lector de las herramientas de matemática discreta que sustentan el razonamiento sobre programas: simplificación de predicados, demostración de corrección de algoritmos recursivos e iterativos mediante inducción, modelado de dependencias mediante órdenes parciales y grafos.

Consejo

Cuando implemente operadores de igualdad o comparación personalizados en cualquier lenguaje, verifique explícitamente los axiomas matemáticos: violar la transitividad en un comparador (`cmp`) provoca resultados corruptos o pánicos en algoritmos de ordenación como Timsort o pdqsort.

Definición

Relación de Equivalencia, Orden Parcial y Lema del Apretón de Manos: una relación binaria $\sim$ sobre A es de equivalencia si es reflexiva ($a \sim a$), simétrica ($a \sim b \Rightarrow b \sim a$) y transitiva ($a \sim b \wedge b \sim c \Rightarrow a \sim c$). Es un orden parcial ($\preceq$) si es reflexiva, antisimétrica ($a \preceq b \wedge b \preceq a \Rightarrow a = b$) y transitiva. En cualquier grafo no dirigido $G = (V, E)$, $\sum_{v \in V} \deg(v) = 2|E|$.

Autoficha

1. ¿Por qué el tipo `f64` en IEEE 754 viola la reflexividad de una relación de equivalencia para el valor `NaN`? 2. ¿Cómo se transforma la negación de una condición compuesta `!(autenticado && (es_admin || tiene_token))` usando De Morgan? 3. ¿Por qué un grafo de dependencias de módulos debe ser un grafo acíclico dirigido (DAG) para admitir orden topológico?

5.1 Álgebra booleana, lógica formal e inducción matemática y estructural

Cada condición `if`, filtro de consulta SQL o guarda de concurrencia es una fórmula de la lógica proposicional y de primer orden. Dos fórmulas P y Q son lógicamente equivalentes ($P \equiv Q$) cuando toman el mismo valor de verdad bajo toda asignación de sus variables. Las leyes de De Morgan, $\neg(P \wedge Q) \equiv (\neg P) \vee (\neg Q)$ y $\neg(P \vee Q) \equiv (\neg P) \wedge (\neg Q)$, junto con la equivalencia de la implicación $(P \Rightarrow Q) \equiv (\neg P \vee Q)$ y su contrarrecíproca $(\neg Q \Rightarrow \neg P)$, permiten refactorizar condiciones anidadas hacia cláusulas de guarda (early returns) legibles y libres de defectos. La inducción matemática y su generalización a tipos de datos recursivos (inducción estructural) son el fundamento de toda prueba de corrección. Para demostrar que una propiedad $P(t)$ se cumple para todo árbol binario t, basta probar el caso base para la hoja vacía y demostrar el paso inductivo: asumiendo la hipótesis de inducción $P(\text{izq})$ y $P(\text{der})$, probar que se cumple $P(\text{Nodo}(v, \text{izq}, \text{der}))$. Esta correspondencia directa entre estructura de datos inductiva y función recursiva guía el diseño con pattern matching en Rust, Haskell y OCaml.

5.2 Conjuntos, relaciones, funciones, combinatoria y grafos en computación

Las estructuras discretas modelan directamente los objetos del software. Un conjunto finito sin duplicados respalda `HashSet`/`BTreeSet`; una relación binaria $R \subseteq A \times A$ que es reflexiva, simétrica y transitiva define una relación de equivalencia (el contrato exacto que exige el trait `Eq` en Rust o el método `equals` en Java),

mientras que una relación reflexiva, antisimétrica y transitiva define un orden parcial (`PartialOrd`), base de la ordenación topológica de dependencias en gestores de paquetes (Cargo, npm, pip) y sistemas de construcción (Make, Bazel). Un grafo $G = (V, E)$ modela redes de computadores, grafos de llamadas de un compilador, referencias de memoria de un recolector de basura o el historial de commits de Git. Por el lema del apretón de manos (Handshaking Lemma), en todo grafo no dirigido la suma de los grados de todos los vértices es exactamente el doble del número de aristas: $\sum_{v \in V} \deg(v) = 2|E|$. Podemos comprobar exhaustivamente las leyes booleanas y el lema del apretón de manos mediante un script verificable:

```
# Verificación exhaustiva de De Morgan sobre {False, True}^2
for p in (False, True):
    for q in (False, True):
        assert (not (p and q)) == ((not p) or (not q))
        assert (not (p or q)) == ((not p) and (not q))

# Lema del apretón de manos en un grafo no dirigido
edges = [(0, 1), (0, 2), (1, 2), (2, 3), (3, 4)]
deg = {v: 0 for v in range(5)}
for u, v in edges:
    deg[u] += 1
    deg[v] += 1
assert sum(deg.values()) == 2 * len(edges) == 10
```

Problema resuelto

Enunciado En una arquitectura de microservicios con $n = 12$ nodos interconectados en topología de malla completa no dirigida K_{12} (donde cada par de nodos distintos mantiene un canal bidireccional), calcule el número total de canales $E = \binom{12}{2} = \frac{n(n-1)}{2}$ y verifique la suma de grados de todos los vértices.

Datos

- $n = 12$ — Número de vértices del grafo completo K_12

Incógnita Número total de aristas (canales bidireccionales) E

Modelo físico

Combinatoria de pares sin orden en un grafo simple completo K_n

$$E = sum_deg/2$$

Hipótesis

- Grafo simple sin bucles propios (self-loops) ni aristas múltiples entre el mismo par de nodos

Desarrollo

1. Multiplicamos el número de nodos n = 12 por los n - 1 = 11 vecinos de cada nodo (suma de grados 2E).

$$sum_deg = n * (n - 1)$$

$$(12 \cdot 11)$$

2. Dividimos entre 2 aplicando el lema del apretón de manos para obtener el número de aristas.

$$E = sum_deg/2$$

$$\frac{132}{2}$$

Resultado

66

Interpretación El crecimiento cuadrático $O(n^2)$ de una malla completa (66 canales para 12 nodos, 4950 para 100 nodos) explica por qué los sistemas distribuidos grandes utilizan topologías jerárquicas, buses de eventos o protocolos gossip.

Comprobación de los pasos

- Paso 1: 132 — reproducido
- Paso 2: 66 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- En comunicación dirigida asimétrica (dígrafo completo), el número de enlaces dirigidos es n(n - 1) = 132.

Errores frecuentes

- Olvidar dividir entre 2 al contar aristas no dirigidas, duplicando cada par {u, v}.

Ejercicios

1 [PRB-0059] (Básico, Conceptual) [VOL01-C05-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0060] (Básico, Numérico) [VOL01-C05-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0061] (Intermedio, Interpretación) [VOL01-C05-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0062] (Intermedio, Aplicación) [VOL01-C05-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0063] (Intermedio, Identificación de errores) [VOL01-C05-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0064] (Avanzado, Desarrollo por pasos) [VOL01-C05-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0065] (Avanzado, Comparación) [VOL01-C05-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice

el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0066] (Avanzado, Comprobación de hipótesis) [VOL01-C05-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0067] (Nivel de examen, Optimización) [VOL01-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0068] (Nivel de examen, Demostración) [VOL01-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0069] (Reto, Diseño) [VOL01-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0070] (Reto, Estudio de parámetros) [VOL01-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 6

Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

Construir un modelo mental preciso de qué ocurre físicamente desde que se escribe un archivo de código fuente hasta que la CPU ejecuta instrucciones máquina y mueve líneas de caché entre la RAM y los registros.

Consejo

Al razonar sobre el rendimiento de cualquier estructura de datos, recuerde que un fallo de caché a RAM ($\sim$ 80 ns) cuesta lo mismo que más de 200 instrucciones aritméticas en registros: un vector contiguo (`Vec<T>`, `std::vector<T>`) suele superar ampliamente a una lista enlazada incluso en inserciones intermedias moderadas gracias a la localidad espacial y al prefetcher de hardware.

Definición

Tiempo Medio de Acceso a Memoria (AMAT) y Marco de Activación (Stack Frame): el tiempo medio de acceso a memoria en una jerarquía con tiempo de acierto en caché T_{hit}, tasa de fallos m (miss rate) y penalización por fallo T_{miss} viene dado por $\text{AMAT} = T_{\text{hit}} + m \cdot T_{\text{miss}}$. El marco de activación es el bloque contiguo de la pila de llamadas asociado a una invocación activa de subrutina.

Autoficha

1. ¿Por qué reservar y liberar memoria en la pila (stack) es mucho más rápido que en el montículo (heap)? 2. ¿Qué ventajas e inconvenientes presenta la compilación AOT (Rust/C++) frente a la compilación JIT con perfilado adaptativo (JVM/V8)? 3. ¿Cómo influye la tasa de fallos de caché m en el tiempo medio de acceso a memoria (AMAT)?

6.1 CPU, registros, pipeline, memoria principal, pila y montículo (stack y heap)

Un procesador moderno ejecuta un flujo continuo de instrucciones gobernado por el contador de programa (`PC` o `RIP`/`PC` en x86-64 y ARM64), un conjunto pequeño de registros ultrarrápidos (generalmente 16 o 32 registros de propósito general de 64 bits con acceso en $\sim$ 0,3 ns), unidades aritmético-lógicas y vectoriales (SIMD), y una jerarquía de memorias caché (L1 de $\sim$ 1 ns, L2 de $\sim$ 4 ns, L3 de $\sim$ 12 ns) que amortiguan la latencia de la memoria principal DRAM ($\sim$ 60–100 ns). El espacio de direcciones virtuales de un proceso se divide convencionalmente en segmentos de código (`.text`), datos estáticos (`.rodata`, `.data`, `.bss`), el montículo dinámico (heap, gestionado por un asignador o recolector de basura) y la pila de ejecución (call stack). La pila de ejecución crece y decrece en orden LIFO (Last-In, First-Out) mediante un único registro puntero de pila (`SP`). Cada llamada a función reserva instantáneamente un marco de activación (stack frame) desplazando `SP`, donde almacena la dirección de retorno, registros preservados y variables locales de tamaño conocido en tiempo de compilación. Al retornar, basta restaurar `SP`, por lo que la reserva y liberación en pila tiene coste $O(1)$ de una sola instrucción y localidad espacial óptima en caché L1. El heap, en cambio, permite reservar bloques cuyo tamaño se decide en tiempo de ejecución o cuya vida útil sobrevive a la función que los creó, a cambio de gestionar metadatos de bloques libres, fragmentación e indirección por punteros.

6.2 De código fuente a ejecución: interpretación, bytecode, compilación AOT y JIT

Para transformar código fuente en acciones del procesador existen tres estrategias principales y sus híbridos. Un intérprete puro analiza el código y evalúa directamente el árbol sintáctico o un bytecode intermedio mediante un bucle de despacho (como CPython o MRI Ruby), ofreciendo arranque inmediato y portabilidad, pero pagando sobrecarga de decodificación y comprobación dinámica de tipos en cada operación. Un compilador Ahead-of-Time (AOT), como `rustc`, `clang` o `gcc`, traduce el programa completo antes de su ejecución a través de análisis léxico, sintáctico, semántico, optimización sobre representación intermedia SSA (Static Single Assignment, p. ej. LLVM IR) y generación de código máquina nativo específico para la arquitectura destino. Las plataformas con compilación Just-in-Time (JIT), como HotSpot JVM (Java/Kotlin), CoreCLR (.NET/C#), V8 (JavaScript) o LuaJIT, combinan ambos mundos: compilan primero a un bytecode portable que comienza ejecutándose en un intérprete o compilador base rápido y, a medida que el perfilador en vivo (profiler) detecta funciones calientes (hot spots), un compilador optimizador en segundo plano genera código máquina nativo especializado con desvirtualización e inlining especulativo.

Problema resuelto

Enunciado Un procesador tiene una caché L1 con tiempo de acierto $T_{\text{hit}} = 1$ ns y una penalización de acceso a memoria principal en caso de fallo de $T_{\text{miss}} = 80$ ns. Calcule el Tiempo Medio de Acceso a Memoria (AMAT $= T_{\text{hit}} + m \cdot T_{\text{miss}}$) en nanosegundos cuando una estructura contigua logra una tasa de fallos de $m = 0{,}025$ (2.5 %).

Datos

- $T_{hit} = 1$ — Latencia de acierto en caché L1 (ns)
- $m = 0{,}025$ — Tasa de fallos de caché (miss rate)
- $T_{miss} = 80$ — Penalización por fallo hacia DRAM (ns)

Incógnita Tiempo medio de acceso a memoria AMAT en nanosegundos

Modelo físico

Ecuación fundamental de rendimiento de jerarquía de caché (AMAT)

$$AMAT = T_hit + T_penalty$$

Hipótesis

- Modelo de un nivel efectivo entre caché L1 y memoria principal

Desarrollo

1. Calculamos la penalización media ponderada por la tasa de fallos del 2.5 %.

$$T_penalty = m * T_miss$$

$$(0{,}025 \cdot 80)$$

2. Sumamos el tiempo base de acierto en L1 para obtener el AMAT en nanosegundos.

$$AMAT = T_hit + T_penalty$$

$$(1 + 2)$$

Resultado

$$3$$

Interpretación Con un 97.5 % de aciertos, el acceso efectivo es de solo 3 ns; si se recorriera una lista enlazada dispersa con m = 0.50 (50 % de fallos), el AMAT subiría a 1 + 40 = 41 ns (casi 14 veces más lento).

Comprobación de los pasos

- Paso 1: 2 — reproducido
- Paso 2: 3 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Los procesadores superescalares fuera de orden pueden solapar parte de la latencia mediante ejecución especulativa y MLP (Memory-Level Parallelism).

Errores frecuentes

- Despreciar el término $m*T_miss$ pensando que un 2.5 % de fallos aporta solo un 2.5 % del tiempo total.

Ejercicios

1 [PRB-0073] (Básico, Conceptual) [VOL01-C06-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0074] (Básico, Numérico) [VOL01-C06-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0075] (Intermedio, Interpretación) [VOL01-C06-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0076] (Intermedio, Aplicación) [VOL01-C06-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0077] (Intermedio, Identificación de errores) [VOL01-C06-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0078] (Avanzado, Desarrollo por pasos) [VOL01-C06-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0079] (Avanzado, Comparación) [VOL01-C06-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0080] (Avanzado, Comprobación de hipótesis) [VOL01-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0081] (Nivel de examen, Optimización) [VOL01-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0082] (Nivel de examen, Demostración) [VOL01-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0083] (Reto, Diseño) [VOL01-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0084] (Reto, Estudio de parámetros) [VOL01-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 7

Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

Comprender cómo los sistemas de tipos previenen estados inválidos antes de ejecutar el software, cómo funcionan las reglas de alcance léxico y tiempo de vida de las variables, y cómo elegir entre valores de retorno explícitos (`Result<T, E>`) y excepciones según el dominio.

Consejo

Reemplace pares de banderas booleanas y campos opcionales sueltos por un tipo suma (`enum` o unión discriminada): un registro con `is_loading: bool`, `data: Option<D>` y `error: Option<E>` multiplica estados imposibles (como `is_loading = true` con `data` y `error` simultáneamente presentes), mientras que un `enum RequestState { Idle, Loading, Ok(D), Failed(E) }` elimina de raíz esos estados en compilación.

Definición

Álgebra de Tipos (Tipos Producto y Tipos Suma): dado un conjunto de tipos con cardinalidades finitas, un tipo producto $T_{\text{prod}} = A \times B$ (registro o tupla) tiene cardinalidad $|A \times B| = |A| \cdot |B|$, mientras que un tipo suma etiquetado $T_{\text{sum}} = A + B$ (unión discriminada o `enum`) tiene cardinalidad $|A + B| = |A| + |B|$. Por ejemplo, $|\text{Option}\langle A\rangle| = |A| + 1$ y $|\text{Result}\langle A, E\rangle| = |A| + |E|$.

Autoficha

1. ¿Cuál es la diferencia exacta entre ámbito léxico (scope) y tiempo de vida en memoria (lifetime)? 2. ¿Por qué un tipo producto multiplica cardinalidades mientras que un tipo suma las adiciona? 3. ¿Qué ventaja ofrece devolver `Result<T, E>` en la firma frente a lanzar excepciones no comprobadas?

7.1 Taxonomía rigurosa de sistemas de tipos: estático, dinámico, nominal, estructural y algebraico

Un sistema de tipos es un método sintáctico tractable para demostrar la ausencia de ciertos comportamientos erróneos clasificando las expresiones según la naturaleza de los valores que calculan (Benjamin Pierce, Types and Programming Languages). Conviene distinguir con precisión tres ejes ortogonales que a menudo se confunden: 1. Momento de verificación (estático vs dinámico): en lenguajes de verificación estática (Rust, C, C++, Java, C#, Go, Kotlin, Swift, TypeScript, Haskell), los tipos de las expresiones se resuelven y comprueban en tiempo de compilación; en lenguajes dinámicos (Python, JavaScript, Ruby, Lua, Clojure), las etiquetas de tipo están asociadas a los valores en tiempo de ejecución. 2. Solidez y coerciones (fuerte/seguro vs débil/inseguro): un sistema de tipos es seguro (type-safe y memory-safe) si impide tratar un patrón arbitrario de bits como si perteneciera a otro tipo incompatible y prohíbe conversiones implícitas con pérdida silenciosa. 3. Equivalencia y subtipado (nominal vs estructural): en el tipado nominal (Rust, Java, C++, C#, Swift), dos tipos son compatibles solo si comparten declaración explícita de nombre o implementación de trait; en el tipado estructural (TypeScript, interfaces de Go), la compatibilidad depende exclusivamente de la forma (shape) de sus campos o métodos. Finalmente, los Tipos de Datos Algebraicos (ADT) combinan tipos producto (`struct`/tuplas, cuya cardinalidad es $|A \times B| = |A| \cdot |B|$) y tipos suma discriminados (`enum` en Rust/Swift, variantes en OCaml/Haskell, uniones discriminadas en TypeScript, cuya cardinalidad es $|A + B| = |A| + |B|$), permitiendo hacer irrepresentables los estados inválidos.

7.2 Ámbito léxico, duración, clausuras, acoplamiento modular y gestión de errores

Es imprescindible distinguir entre el ámbito (scope) de un identificador y el tiempo de vida (lifetime / storage duration) del valor al que se vincula. El ámbito léxico es una propiedad estática del texto del programa: determina en qué región de código es visible un nombre. El tiempo de vida es una propiedad dinámica de ejecución: el intervalo entre la creación y la destrucción del objeto en memoria. Una clausura (closure) aparece cuando una función de primera clase captura variables libres de su entorno léxico circundante, empaquetando el puntero de código junto con un registro de entorno. En cuanto a la gestión de fallos, la ingeniería del software distingue entre defectos de programación (violaciones de invariantes internos, ante los cuales procede abortar o activar un panic/assertion) y errores operativos recuperables (archivo inexistente, timeout de red, entrada mal formada). Los lenguajes modernos modelan los errores recuperables bien mediante excepciones estructuradas (`try/catch` en Python, Java, C#, C++, JS) o bien como valores explícitos en la firma de retorno (`Result<T, E>` en Rust, tupla `(T, error)` en Go), obligando al llamador a tratar cada rama de fallo explícitamente:

```rust
#[derive(Debug, PartialEq)]
enum ParsePortError {
    NotANumber,
    ZeroPortForbidden,
}

fn parse_port(raw: &str) -> Result<u16, ParsePortError> {
    let port: u16 = raw.trim().parse().map_err(|_| ParsePortError::NotANumber)?;
    if port == 0 {
        return Err(ParsePortError::ZeroPortForbidden);
    }
    Ok(port)
}

fn main() {
    assert_eq!(parse_port("8080"), Ok(8080));
    assert_eq!(parse_port("0"), Err(ParsePortError::ZeroPortForbidden));
}
```

Problema resuelto

Enunciado Un estado de conexión se modela erróneamente como un tipo producto con tres booleanos (`conectado`, `autenticado`, `error_activo`), un código de modo de 4 valores posibles y un canal opcional de $8 + 1 = 9$ estados posibles. Compare el número de estados del tipo producto $N_{\text{prod}} = 2^3 \times 4 \times 9$ frente a un tipo suma discriminado `ConnectionState` con 4 variantes exactas: `Disconnected` (1), `Connecting` (4 modos), `Authenticated` ($4 \times 8 = 32$ combinaciones válidas de modo y canal activo) y `Failed` (1), y calcule cuántos estados ilegales elimina el diseño con tipos algebraicos.
Datos

- $N_{bool} = 8$ — Combinaciones de los 3 booleanos (2^3)
- $N_{mode} = 4$ — Modos de operación posibles
- $N_{chan} = 9$ — Estados de Option<Channel> (8 + 1)

Incógnita Número de estados inválidos eliminados al pasar de tipo producto a tipo suma
Modelo físico
Cardinalidad de tipos algebraicos (producto cartesiano frente a unión disjunta)

$$N_invalid = N_prod - (1 + 4 + 32 + 1)$$

Hipótesis

- Solo las 38 configuraciones del tipo suma tienen significado válido en el protocolo

Desarrollo

1. Calculamos la cardinalidad total del registro tipo producto.

$$N_prod = 8 * 4 * 9$$

$$((8 \cdot 4) \cdot 9)$$

2. Calculamos la cardinalidad del tipo suma ($1 + 4 + 32 + 1 = 38$) y la restamos de N_prod.

$$N_invalid = N_prod - (1 + 4 + 32 + 1)$$

$$(288 - (((1 + 4) + 32) + 1))$$

Resultado

$$250$$

Interpretación El registro plano permite representar 288 estados posibles de los cuales 250 son combinaciones contradictorias; el tipo suma reduce la cardinalidad exactamente a los 38 estados válidos.

Comprobación de los pasos

- Paso 1: 288 — reproducido
- Paso 2: 250 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Invariantes numéricos continuos (p. ej. $x^2 + y^2 = 1$) requieren constructores inteligentes (newtypes) además de tipos suma.

Errores frecuentes

- Usar el valor nulo o punteros sin inicializar como centinela implícito en lugar de `Option<T>`.

Ejercicios

1 [PRB-0087] (Básico, Conceptual) [VOL01-C07-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0088] (Básico, Numérico) [VOL01-C07-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0089] (Intermedio, Interpretación) [VOL01-C07-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0090] (Intermedio, Aplicación) [VOL01-C07-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(25 + 8) * 2` y justifique los criterios de aceptación,

pistas graduadas y complejidad temporal/espacial.

5 [PRB-0091] (Intermedio, Identificación de errores) [VOL01-C07-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0092] (Avanzado, Desarrollo por pasos) [VOL01-C07-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0093] (Avanzado, Comparación) [VOL01-C07-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0094] (Avanzado, Comprobación de hipótesis) [VOL01-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0095] (Nivel de examen, Optimización) [VOL01-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0096] (Nivel de examen, Demostración) [VOL01-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0097] (Reto, Diseño) [VOL01-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0098] (Reto, Estudio de parámetros) [VOL01-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 8

Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

Establecer un marco comparativo riguroso de los doce paradigmas de programación fundamentales, comprendiendo que los lenguajes profesionales actuales son multiparadigma y que la excelencia en ingeniería consiste en aplicar el paradigma adecuado a cada capa del sistema.

Consejo

Aplique el patrón arquitectónico Functional Core, Imperative Shell: escriba la lógica de dominio y validación como funciones puras sobre valores inmutables (fáciles de probar exhaustivamente sin mocks), y confine los efectos laterales de entrada/salida, red y base de datos a una capa exterior delgada.

Definición

Transparencia Referencial y Dualidad Concurrencia vs Paralelismo: una expresión es referencialmente transparente si puede sustituirse en cualquier punto del programa por su valor evaluado sin alterar el comportamiento observable del sistema. La concurrencia es una propiedad de diseño (composición de múltiples tareas lógicas independientes que pueden progresar en intervalos solapados), mientras que el paralelismo es una propiedad física de ejecución (varias unidades de hardware ejecutando instrucciones en el mismo instante físico).

Autoficha

1. ¿En qué se diferencia la transparencia referencial funcional del encapsulamiento orientado a objetos? 2. ¿Cuándo es preferible la programación asíncrona basada en bucle de eventos (`async`/`await`) frente a un pool de hilos del sistema operativo? 3. ¿Qué diferencia de ejecución y tamaño binario existe entre los genéricos por monomorfización (Rust/C++) y por borrado de tipos (Java)?

8.1 Imperativo, procedural, orientado a objetos, declarativo, funcional y lógico

Un paradigma de programación es una disciplina conceptual que estructura cómo modelamos el cómputo, el estado y el flujo de control: 1. Programación imperativa: expresa el cómputo como una secuencia explícita de sentencias que mutan un estado almacenado en memoria. Es óptima cuando el algoritmo modela una máquina de estados física o manipula buffers en el lugar (in-place), pero dificulta el razonamiento local cuando el estado mutable se comparte sin control. 2. Programación procedural: organiza el código imperativo en procedimientos y módulos con interfaces bien definidas (C, Pascal, Go). 3. Programación orientada a objetos (OOP): encapsula estado e invariantes detrás de métodos y permite polimorfismo dinámico (despacho dinámico por vtable o paso de mensajes). Su límite aparece cuando se abusa de jerarquías profundas de herencia de implementación en lugar de composición. 4. Programación declarativa: especifica qué propiedades debe satisfacer el resultado sin prescribir el bucle de control paso a paso (SQL en consultas relacionales, HTML/CSS en maquetación, Terraform/Nix en infraestructura). 5. Programación funcional: modela el cómputo como evaluación de funciones matemáticamente puras (transparencia referencial: sin efectos laterales observables y con datos inmutables o persistentes), facilitando la composición, las pruebas unitarias y el paralelismo sin carreras de datos. 6. Programación lógica: expresa hechos y reglas mediante cláusulas de Horn (Prolog, Datalog), delegando la búsqueda de soluciones en un motor de unificación y backtracking.

8.2 Genérico, eventos, reactivo, concurrente, asíncrono y programación de sistemas

Los seis paradigmas restantes abordan la parametrización de tipos, el tiempo, la latencia de E/S y el control de recursos físicos: 7. Programación genérica: parametriza algoritmos y estructuras de datos sobre tipos abstractos restringidos por contratos (`traits` en Rust, `concepts` en C++20, `interfaces` en Go/Java/C#), ejecutándose por monomorfización estática (coste cero en tiempo de ejecución) o por diccionario/borrado. 8. Programación orientada a eventos: estructura el flujo alrededor de un bucle despachador (event loop) que invoca manejadores (handlers/callbacks) en respuesta a sucesos discretos de E/S o interfaz. 9. Programación reactiva: modela flujos de datos asíncronos a lo largo del tiempo (`Flow` en Kotlin, `Stream` en Rust, Rx) con propagación automática de cambios y control de contrapresión (backpressure). 10. Programación concurrente: estructura múltiples flujos de control lógicos potencialmente simultáneos mediante memoria compartida sincronizada (mutexes, atómicos), paso de mensajes por canales (CSP en Go/Rust) o aislamiento de actores (Erlang/Elixir, Swift 6). 11. Programación asíncrona: desacopla la espera de operaciones de entrada/salida (red, disco, temporizadores) del bloqueo de hilos del sistema operativo mediante futuros/promesas y máquinas de estados cooperativas (`async`/`await` sobre `epoll`/`kqueue`/`io_uring`). 12. Programación de sistemas: diseña software que gestiona directamente la memoria física/virtual, la ABI, las llamadas al sistema y las restricciones de latencia determinista (Rust, C, C++, Zig).

Problema resuelto

Enunciado Un servidor debe mantener $C = 10000$ conexiones concurrentes mayoritariamente ociosas esperando E/S de red. Si en un modelo de un hilo del sistema operativo por conexión cada hilo reserva una pila de $S_{\text{thread}} = 1024$ KiB (1 MiB), mientras que en un modelo asíncrono (`async`/`await`) cada tarea es una máquina de estados compilada de $S_{\text{task}} = 2$ KiB, calcule la razón de reducción de memoria $\frac{C \cdot S_{\text{thread}}}{C \cdot S_{\text{task}}}$.

Datos

- $C = 10000$ — Conexiones concurrentes activas
- $S_{thread} = 1024$ — Pila por hilo OS en KiB
- $S_{task} = 2$ — Estado por tarea asíncrona en KiB

Incógnita Factor de reducción de huella de memoria de pila/estado entre ambos modelos

Modelo físico

Comparación de coste espacial entre hilos nativos OS y máquinas de estado asíncronas

$$ratio = M_threads/(C * S_task)$$

Hipótesis

- Cada hilo nativo compromete su pila de 1024 KiB y cada Future compilado ocupa 2 KiB

Desarrollo

1. Calculamos la memoria total en KiB para los 10000 hilos del sistema operativo.

$$M_threads = C * S_thread$$

$$(10000 \cdot 1024)$$

2. Dividimos entre la memoria total de las 10000 tareas asíncronas (20000 KiB).

$$ratio = M_threads/(C * S_task)$$

$$\frac{10240000}{20000}$$

Resultado

512

Interpretación El modelo asíncrono reduce en un factor de 512x la memoria de estado de concurrencia (de ~10 GiB a ~20 MiB) y elimina los cambios de contexto de kernel entre miles de hilos.
Comprobación de los pasos

- Paso 1: 10240000 — reproducido
- Paso 2: 512 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Para cargas intensivas en CPU (CPU-bound) sin esperas de E/S, un pool de tantos hilos como núcleos físicos evita la sobrecarga del planificador asíncrono.

Errores frecuentes

- Invocar llamadas bloqueantes sincrónicas dentro de una tarea asíncrona, bloqueando el hilo ejecutor del runtime.

Ejercicios

1 [PRB-0101] (Básico, Conceptual) [VOL01-C08-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0102] (Básico, Numérico) [VOL01-C08-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0103] (Intermedio, Interpretación) [VOL01-C08-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0104] (Intermedio, Aplicación) [VOL01-C08-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(28 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0105] (Intermedio, Identificación de errores) [VOL01-C08-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0106] (Avanzado, Desarrollo por pasos) [VOL01-C08-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0107] (Avanzado, Comparación) [VOL01-C08-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 7, verifique las precondiciones forma-

les, evalúe la métrica característica mediante la expresión reproducible `28 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0108] (Avanzado, Comprobación de hipótesis) [VOL01-C08-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0109] (Nivel de examen, Optimización) [VOL01-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0110] (Nivel de examen, Demostración) [VOL01-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0111] (Reto, Diseño) [VOL01-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0112] (Reto, Estudio de parámetros) [VOL01-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Apéndice A

Notación

Símbolo	Significado	Unidades
$AMAT$	Tiempo medio de acceso a memoria (Average Memory Access Time)	ns
BNF	Notación formal de Backus-Naur para gramáticas libres de contexto	adimensional
CSP	Communicating Sequential Processes (Hoare, 1978)	adimensional
K_n	Grafo completo no dirigido de n vértices	adimensional
$S(N)$	Factor de aceleración (speedup) con N procesadores	adimensional
Γ	Alfabeto finito de símbolos de cinta	adimensional
δ	Función de transición de la máquina de Turing	adimensional
$\varepsilon_{\text{mach}}$	Épsilon de máquina en coma flotante	adimensional
$\lvert A + B \rvert$	Cardinalidad de un tipo suma discriminado	adimensional
$\lvert A \times B \rvert$	Cardinalidad de un tipo producto	adimensional

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

Problema resuelto [PRB-0001]
Considere una Máquina de Turing determinista con $|Q| = 6$ estados activos, un alfabeto de cinta de $|\Gamma| = 4$ símbolos y $d = 2$ direcciones de movimiento del cabezal. Determine el número total de entradas de la función de transición $\delta : Q \times \Gamma \to (Q \cup \{\text{HALT}\}) \times \Gamma \times \{L, R\}$ y el número de bits necesarios por entrada si cada transición codifica uno de los $6 + 1 = 7$ estados destino (3 bits), el símbolo escrito (2 bits) y la dirección (1 bit), así como el tamaño total en bits de la tabla de transición.
Respuesta

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[VOL01-C01-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 24
Ejercicio 2 [PRB-0004]
[VOL01-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 28
Ejercicio 3 [PRB-0005]
[VOL01-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 4 [PRB-0006]
[VOL01-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(12 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 5 [PRB-0007]
[VOL01-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 6 [PRB-0008]
[VOL01-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 70

Ejercicio 7 [PRB-0009]
[VOL01-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 72
Ejercicio 8 [PRB-0010]
[VOL01-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 87
Ejercicio 9 [PRB-0011]
[VOL01-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 10 [PRB-0012]
[VOL01-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 11 [PRB-0013]
[VOL01-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 145
Ejercicio 12 [PRB-0014]
[VOL01-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 144

Cálculos reproducidos

$(24 \cdot 6) = 144$

C.2 Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

Problema resuelto [PRB-0015]
En un computador histórico con memoria de $M = 32768$ palabras de 36 bits, comparar el espacio ocupado por una tabla de $N = 1024$ identificadores de 8 caracteres ASCII (8 bits por carácter) cuando se almacenan como cadenas con centinela nulo de 1 byte frente a cadenas con prefijo de longitud de 4 bytes sin centinela. Calcule el ahorro neto en bytes de la representación con centinela de 1 byte para los $N = 1024$ identificadores.
Respuesta

3072

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0017]
[VOL01-C02-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 30
Ejercicio 2 [PRB-0018]
[VOL01-C02-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 31
Ejercicio 3 [PRB-0019]
[VOL01-C02-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 49
Ejercicio 4 [PRB-0020]
[VOL01-C02-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(15 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 5 [PRB-0021]
[VOL01-C02-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 58
Ejercicio 6 [PRB-0022]
[VOL01-C02-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 85
Ejercicio 7 [PRB-0023]
[VOL01-C02-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 90
Ejercicio 8 [PRB-0024]
[VOL01-C02-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 9 [PRB-0025]
[VOL01-C02-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 132
Ejercicio 10 [PRB-0026]
[VOL01-C02-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 10,

verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 135
Ejercicio 11 [PRB-0027]
[VOL01-C02-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 175
Ejercicio 12 [PRB-0028]
[VOL01-C02-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 180

Cálculos reproducidos

$(1024 \cdot 3) = 3072$

C.3 De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

Problema resuelto [PRB-0029]
Según la ley de Amdahl, si un servicio de procesamiento tiene una fracción secuencial irreducible $s = 0{,}05$ (5 %) y una fracción paralelizable $p = 0{,}95$ (95 %) que se distribuye sin sobrecarga entre $N = 19$ núcleos de procesador, calcule el factor de aceleración (speedup) $S(N) = \frac{1}{s + p/N}$.
Respuesta

10

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0031]
[VOL01-C03-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 2 [PRB-0032]
[VOL01-C03-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36
Ejercicio 3 [PRB-0033]
[VOL01-C03-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 64
Ejercicio 4 [PRB-0034]
[VOL01-C03-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 56
Ejercicio 5 [PRB-0035]

[VOL01-C03-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 78
Ejercicio 6 [PRB-0036]
[VOL01-C03-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 110
Ejercicio 7 [PRB-0037]
[VOL01-C03-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120
Ejercicio 8 [PRB-0038]
[VOL01-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 143
Ejercicio 9 [PRB-0039]
[VOL01-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 172
Ejercicio 10 [PRB-0040]
[VOL01-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 180
Ejercicio 11 [PRB-0041]
[VOL01-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 225
Ejercicio 12 [PRB-0042]
[VOL01-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 240

Cálculos reproducidos

$\frac{1}{(0{,}05+0{,}05)} = 10$

C.4 Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

Problema resuelto [PRB-0043]

Decodifique manualmente el valor real exacto representado por la palabra IEEE 754 `binary32` `0xC1580000`, sabiendo que su bit de signo es $s = 1$, su campo de exponente de 8 bits vale $E = 130$ (con sesgo bias = 127) y los tres primeros bits de su mantisa fraccionaria son `101` ($M/2^{23} = 2^{-1}+2^{-3} = 0{,}625$).
Respuesta

$$-13$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
[VOL01-C04-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 32
Ejercicio 2 [PRB-0046]
[VOL01-C04-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 32
Ejercicio 3 [PRB-0047]
[VOL01-C04-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 52
Ejercicio 4 [PRB-0048]
[VOL01-C04-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 48
Ejercicio 5 [PRB-0049]
[VOL01-C04-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 62
Ejercicio 6 [PRB-0050]
[VOL01-C04-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 90
Ejercicio 7 [PRB-0051]
[VOL01-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 96
Ejercicio 8 [PRB-0052]
[VOL01-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 115
Ejercicio 9 [PRB-0053]

[VOL01-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 140

Ejercicio 10 [PRB-0054]

[VOL01-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 144

Ejercicio 11 [PRB-0055]

[VOL01-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 185

Ejercicio 12 [PRB-0056]

[VOL01-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 192

Cálculos reproducidos

$$\left(\left(-\left(1\right)\cdot 1{,}625\right)\cdot\left(2\right)^{\left(130-127\right)}\right)=-\left(13\right)$$

C.5 Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

Problema resuelto [PRB-0057]

En una arquitectura de microservicios con $n = 12$ nodos interconectados en topología de malla completa no dirigida K_{12} (donde cada par de nodos distintos mantiene un canal bidireccional), calcule el número total de canales $E = \binom{12}{2} = \frac{n(n-1)}{2}$ y verifique la suma de grados de todos los vértices.

Respuesta

66

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0059]

[VOL01-C05-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 36

Ejercicio 2 [PRB-0060]

[VOL01-C05-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 34

Ejercicio 3 [PRB-0061]

[VOL01-C05-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 58

Ejercicio 4 [PRB-0062]

[VOL01-C05-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 52

Ejercicio 5 [PRB-0063]

[VOL01-C05-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 70

Ejercicio 6 [PRB-0064]

[VOL01-C05-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 100

Ejercicio 7 [PRB-0065]

[VOL01-C05-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 108

Ejercicio 8 [PRB-0066]

[VOL01-C05-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 129

Ejercicio 9 [PRB-0067]

[VOL01-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 156

Ejercicio 10 [PRB-0068]

[VOL01-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 162

Ejercicio 11 [PRB-0069]

[VOL01-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 205

Ejercicio 12 [PRB-0070]

[VOL01-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 216

Cálculos reproducidos

$\frac{132}{2} = 66$

C.6 Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

Problema resuelto [PRB-0071]
Un procesador tiene una caché L1 con tiempo de acierto $T_{\text{hit}} = 1$ ns y una penalización de acceso a memoria principal en caso de fallo de $T_{\text{miss}} = 80$ ns. Calcule el Tiempo Medio de Acceso a Memoria ($\text{AMAT} = T_{\text{hit}} + m \cdot T_{\text{miss}}$) en nanosegundos cuando una estructura contigua logra una tasa de fallos de $m = 0{,}025$ (2.5 %).
Respuesta

3

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0073]
[VOL01-C06-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 44
Ejercicio 2 [PRB-0074]
[VOL01-C06-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 38
Ejercicio 3 [PRB-0075]
[VOL01-C06-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 70
Ejercicio 4 [PRB-0076]
[VOL01-C06-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 60
Ejercicio 5 [PRB-0077]
[VOL01-C06-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 86
Ejercicio 6 [PRB-0078]
[VOL01-C06-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120
Ejercicio 7 [PRB-0079]

[VOL01-C06-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 132
Ejercicio 8 [PRB-0080]
[VOL01-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 157
Ejercicio 9 [PRB-0081]
[VOL01-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 188
Ejercicio 10 [PRB-0082]
[VOL01-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 198
Ejercicio 11 [PRB-0083]
[VOL01-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 245
Ejercicio 12 [PRB-0084]
[VOL01-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 264

Cálculos reproducidos

$(1 + 2) = 3$

C.7 Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

Problema resuelto [PRB-0085]
Un estado de conexión se modela erróneamente como un tipo producto con tres booleanos (`conectado`, `autenticado`, `error_activo`), un código de modo de 4 valores posibles y un canal opcional de $8 + 1 = 9$ estados posibles. Compare el número de estados del tipo producto $N_{\text{prod}} = 2^3 \times 4 \times 9$ frente a un tipo suma discriminado `ConnectionState` con 4 variantes exactas: `Disconnected` (1), `Connecting` (4 modos), `Authenticated` ($4 \times 8 = 32$ combinaciones válidas de modo y canal activo) y `Failed` (1), y calcule cuántos estados ilegales elimina el diseño con tipos algebraicos.
Respuesta

250

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0087]

[VOL01-C07-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 50

Ejercicio 2 [PRB-0088]

[VOL01-C07-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 41

Ejercicio 3 [PRB-0089]

[VOL01-C07-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 79

Ejercicio 4 [PRB-0090]

[VOL01-C07-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(25 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 66

Ejercicio 5 [PRB-0091]

[VOL01-C07-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 98

Ejercicio 6 [PRB-0092]

[VOL01-C07-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 135

Ejercicio 7 [PRB-0093]

[VOL01-C07-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 150

Ejercicio 8 [PRB-0094]

[VOL01-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 178

Ejercicio 9 [PRB-0095]

[VOL01-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 212

Ejercicio 10 [PRB-0096]

[VOL01-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 225
Ejercicio 11 [PRB-0097]
[VOL01-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 275
Ejercicio 12 [PRB-0098]
[VOL01-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 300

Cálculos reproducidos

$(288 - (((1 + 4) + 32) + 1)) = 250$

C.8 Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

Problema resuelto [PRB-0099]
Un servidor debe mantener $C = 10000$ conexiones concurrentes mayoritariamente ociosas esperando E/S de red. Si en un modelo de un hilo del sistema operativo por conexión cada hilo reserva una pila de $S_{\text{thread}} = 1024$ KiB (1 MiB), mientras que en un modelo asíncrono (`async`/`await`) cada tarea es una máquina de estados compilada de $S_{\text{task}} = 2$ KiB, calcule la razón de reducción de memoria $\frac{C \cdot S_{\text{thread}}}{C \cdot S_{\text{task}}}$.
Respuesta

512

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0101]
[VOL01-C08-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 56
Ejercicio 2 [PRB-0102]
[VOL01-C08-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 44
Ejercicio 3 [PRB-0103]
[VOL01-C08-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 88
Ejercicio 4 [PRB-0104]
[VOL01-C08-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(28 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 72
Ejercicio 5 [PRB-0105]

[VOL01-C08-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 110
Ejercicio 6 [PRB-0106]
[VOL01-C08-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 150
Ejercicio 7 [PRB-0107]
[VOL01-C08-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 168
Ejercicio 8 [PRB-0108]
[VOL01-C08-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 199
Ejercicio 9 [PRB-0109]
[VOL01-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 236
Ejercicio 10 [PRB-0110]
[VOL01-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 252
Ejercicio 11 [PRB-0111]
[VOL01-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 305
Ejercicio 12 [PRB-0112]
[VOL01-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 336

Cálculos reproducidos

$\frac{10240000}{20000} = 512$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 25 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 3 respaldada por una fuente, 1 valor o condición.
¶21: dos testimonios. Ecuaciones: 1 reproducida numéricamente, 1 aproximación, 4 valor o condición.
¶22: dos testimonios. Ecuaciones: 2 aproximación.
¶23: dos testimonios. Ecuaciones: 1 valor o condición.
¶24: dos testimonios. Ecuaciones: 1 reproducida numéricamente, 1 valor o condición.
¶25: dos testimonios.
¶26: dos testimonios. Ecuaciones: 4 definición.
¶27: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶28: dos testimonios. Ecuaciones: 3 respaldada por una fuente, 6 valor o condición.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios. Ecuaciones: 1 respaldada por una fuente.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios. Ecuaciones: 1 verificada simbólicamente, 1 respaldada por una fuente.
¶39: dos testimonios.
¶40: dos testimonios. Ecuaciones: 1 verificada simbólicamente, 5 respaldada por una fuente.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

Alan M. Turing — On Computable Numbers, with an Application to the Entscheidungsproblem (1936) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
John von Neumann — First Draft of a Report on the EDVAC (1945) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
C Programming Language Reference (ISO/IEC 9899 C17/C23 — cppreference) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
The Rust Programming Language (Klabnik & Nichols — Official Rust Documentation) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
ECMA-262 — ECMAScript 2024 Language Specification (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
The Go Programming Language Specification & Memory Model (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
RFC 3629 — UTF-8, a transformation format of ISO 10646 (IETF Standard) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
MIT OpenCourseWare 6.006 — Introduction to Algorithms (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
The Java Language Specification & JVM Specification (Java SE 21 LTS) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
The Rust Reference — Memory Model, Type System, Lifetimes, and Unsafe Rust (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The TypeScript Handbook — Structural Typing, Narrowing, and Type Manipulation (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Python Language Reference & Standard Library Documentation (Python 3.12–3.14) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-antecedentes-y-computabilidad scientific
REV-0002: cap02-primeros-ordenadores-y-compiladores scientific
REV-0003: cap03-paradigmas-sistemas-y-era-moderna scientific
REV-0004: cap04-representacion-binaria-ieee754-utf8 scientific
REV-0005: cap05-logica-algebra-booleana-matematica-discreta scientific
REV-0006: cap06-arquitectura-cpu-memoria-compilacion scientific
REV-0007: cap07-sistemas-de-tipos-control-ambito-errores scientific
REV-0008: cap08-los-doce-paradigmas-de-programacion scientific

Apéndice E

Plan de estudio

PARTE I. Historia de la Informática y la Programación

✓Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

- Dominar Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

2 conceptos planificados.

✓Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

- Dominar Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

2 conceptos planificados.

✓De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

- Dominar De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

2 conceptos planificados.

PARTE II. Fundamentos Universales de Programación y Matemáticas Discretas

✓Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

- Dominar Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

2 conceptos planificados.

✓Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

- Dominar Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

2 conceptos planificados.

✓Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

- Dominar Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

2 conceptos planificados.

✓Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

- Dominar Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

2 conceptos planificados.

✓Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

- Dominar Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen 2 — Algoritmos, Estructuras de Datos, Compiladores, Memoria y Sistemas Operativos

Partes III y IV: Complejidad Asintótica, Grafos, Programación Dinámica, Arquitectura de Cachés, Compiladores SSA y Sistemas Operativos

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I PARTE III. Algoritmos, Estructuras de Datos y Computación 1

1. Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado 2
1.1. Definición formal de O, Omega y Theta, y Teorema Maestro para recurrencias 2
1.2. Análisis amortizado por método de potencial y vectores dinámicos 2

2. Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes 6
2.1. Búsqueda binaria, invariantes de bucle y prevención del desbordamiento de punto medio . 6
2.2. Cota inferior por árbol de decisión, Mergesort, Quicksort, Heapsort y ordenación lineal . . 6

3. Divide y vencerás, algoritmos voraces, programación dinámica y backtracking 10
3.1. Divide y vencerás y condición de elección voraz (Greedy Choice Property) 10
3.2. Programación dinámica (Bellman) y búsqueda exhaustiva con poda (Backtracking) 10

4. Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing 14
4.1. BFS, DFS, componentes conexas, Dijkstra, Bellman-Ford, Kruskal y Prim 14
4.2. Árboles equilibrados (AVL, Rojo-Negro, B+ Trees) y tablas hash de alto rendimiento . . . 15

5. Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span) 18
5.1. Estructuras persistentes (path copying) y filtros probabilísticos (Bloom Filter) 18
5.2. Algoritmos paralelos: modelo Work-Span, ley de Brent y reducciones paralelas 19

II PARTE IV. Sistemas Informáticos, Compiladores, Memoria y Sistemas Operativos 22

6. Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado 23
6.1. Líneas de caché, coherencia MESI, false sharing y diseño orientado a datos 23
6.2. Alineamiento en memoria, ABI de plataforma y enlazadores (ELF/Mach-O) 24

7. Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código 27
7.1. Front-end del compilador: análisis léxico, sintáctico (descendente recursivo y Pratt) y semántico 27
7.2. Representación intermedia SSA, optimizaciones e instrucción/asignación de registros . . . 28

8. Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S 31
8.1. Modo usuario vs modo kernel, llamadas al sistema, procesos, hilos y planificación 31
8.2. Memoria virtual, tablas de páginas, TLB, sistemas de archivos y E/S no bloqueante . . . 32

A. Notación 35

B. Hoja de fórmulas **36**

C. Soluciones **37**
C.1. Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado . . . 37
C.2. Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes . . . 38
C.3. Divide y vencerás, algoritmos voraces, programación dinámica y backtracking . . . 40
C.4. Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing . . . 41
C.5. Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span) . . . 43
C.6. Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado . . . 45
C.7. Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código . . . 46
C.8. Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S . . . 48

D. Auditoría de fuentes **50**

E. Plan de estudio **52**

Parte I

PARTE III. Algoritmos, Estructuras de Datos y Computación

Capítulo 1

Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

Proporcionar las herramientas matemáticas exactas para predecir cómo escalan el tiempo de ejecución y el consumo de memoria de cualquier algoritmo cuando el tamaño de entrada n crece varios órdenes de magnitud, demostrando cotas superiores, inferiores y costes amortizados.

Consejo

En un vector dinámico, si en lugar de multiplicar la capacidad por una constante ($\times 2$ o $\times 1{,}5$) se incrementara sumando una cantidad fija $+K$ en cada llenado, el coste de n inserciones degeneraría a $\Theta(n^2)$: la progresión geométrica es indispensable para mantener `push` en $O(1)$ amortizado.

Definición

Método del Potencial (Análisis Amortizado): dada una secuencia de m operaciones con costes reales $c_1, \ldots, c_m$ sobre estados $D_0, \ldots, D_m$ y una función de potencial $\Phi(D_i) \geq \Phi(D_0)$, la suma de costes amortizados acota superiormente el coste real total: $\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i - (\Phi(D_m) - \Phi(D_0)) \leq \sum_{i=1}^{m} \hat{c}_i$.

Autoficha

1. ¿Qué diferencia existe entre decir que un algoritmo es $O(n^2)$ y decir que es $\Theta(n^2)$? 2. ¿Qué diferencia fundamental separa la complejidad en el caso promedio de la complejidad amortizada? 3. ¿Por qué duplicar la capacidad de un array dinámico garantiza coste amortizado $\hat{c}_i \leq 3$?

1.1 Definición formal de O, Omega y Theta, y Teorema Maestro para recurrencias

El análisis de algoritmos mide el crecimiento del número de operaciones elementales $T(n)$ y de celdas de memoria $S(n)$ en función del tamaño de entrada n. Dadas dos funciones no negativas $f(n)$ y $g(n)$: 1. Cota superior asintótica ($f(n) \in O(g(n))$): existen constantes $c > 0$ y $n_0 \geq 1$ tales que $0 \leq f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. 2. Cota inferior asintótica ($f(n) \in \Omega(g(n))$): existen $c > 0$ y $n_0 \geq 1$ tales que $0 \leq c \cdot g(n) \leq f(n)$ para todo $n \geq n_0$. 3. Orden asintótico exacto ($f(n) \in \Theta(g(n))$): se cumple simultáneamente $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n))$. Para algoritmos recursivos de divide y vencerás cuya recurrencia tiene la forma $T(n) = aT(n/b) + f(n)$ con $a \geq 1$ y $b > 1$, el Teorema Maestro compara $f(n)$ con la hoja crítica $n^{\log_b a}$: (Caso 1) si $f(n) = O(n^{\log_b a - \varepsilon})$, entonces $T(n) = \Theta(n^{\log_b a})$; (Caso 2) si $f(n) = \Theta(n^{\log_b a})$, entonces $T(n) = \Theta(n^{\log_b a} \log n)$ (como ocurre en Mergesort con $a = 2, b = 2, f(n) = \Theta(n) \Rightarrow \Theta(n \log n)$); (Caso 3) si $f(n) = \Omega(n^{\log_b a + \varepsilon})$ y satisface regularidad, entonces $T(n) = \Theta(f(n))$.

1.2 Análisis amortizado por método de potencial y vectores dinámicos

No debe confundirse la complejidad en el caso promedio (que asume una distribución probabilística sobre las entradas) con la complejidad amortizada: el análisis amortizado garantiza de forma determinista el coste promedio por operación sobre cualquier secuencia de m operaciones en el peor caso, sin ninguna hipótesis probabilística. En el método de la función de potencial de Tarjan, definimos una función $\Phi(D_i) \geq \Phi(D_0)$ sobre el estado D_i de la estructura de datos, y el coste amortizado de la i-ésima operación es $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Como muestra la figura 1, en un vector dinámico (`Vec<T>` en Rust, `std::vector`

en C++, `list` en Python) que duplica su capacidad cap cuando el tamaño len alcanza cap, tomamos $\Phi = 2 \cdot \text{len} - \text{cap}$. En una inserción sin redimensionado ($c_i = 1, \Delta\text{len} = 1, \Delta\text{cap} = 0$), el coste amortizado es $\hat{c}_i = 1 + 2 = 3$. En una inserción con redimensionado de k a $2k$ ($c_i = k + 1$), el potencial cae de $\Phi_{i-1} = 2k - k = k$ a $\Phi_i = 2(k + 1) - 2k = 2$, por lo que $\hat{c}_i = (k + 1) + (2 - k) = 3 = \Theta(1)$.

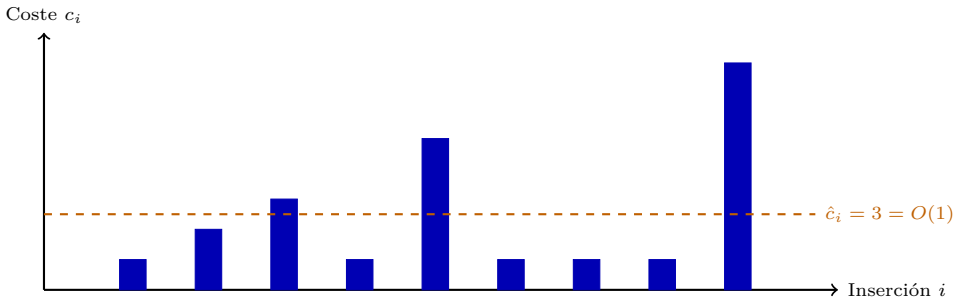


Figura 1. Evolución de capacidad y tamaño en un vector dinámico con factor de crecimiento 2x: las reasignaciones en potencias de 2 quedan amortizadas a coste O(1) por inserción.

Problema resuelto

Enunciado Un vector dinámico comienza con capacidad inicial $\text{cap}_0 = 1$ y duplica su capacidad cada vez que se llena ($1 \to 2 \to 4 \to 8 \to 16 \to 32$). Si se insertan secuencialmente $n = 17$ elementos y cada inserción cuesta 1 escritura más el número de elementos copiados durante el redimensionado ($1 + 2 + 4 + 8 + 16$ copias acumuladas), calcule el coste total exacto de escrituras realizadas.

Datos

- $n = 17$ — Número de elementos insertados
- $C_{copy} = 31$ — Copias de redimensionado (1+2+4+8+16)

Incógnita Coste total de operaciones de escritura sobre las 17 inserciones

Modelo físico

Suma geométrica de copias en redimensionamiento duplicado de un vector dinámico

$$C_total = n + C_copy$$

Hipótesis

- Capacidad inicial 1 y duplicación exacta al insertar cuando len == cap

Desarrollo

1. Sumamos los elementos copiados en las 5 expansiones (al pasar de capacidad 1 a 2, 4, 8, 16 y 32).

$$C_copy = 1 + 2 + 4 + 8 + 16$$

$$((((1 + 2) + 4) + 8) + 16)$$

2. Sumamos las 17 escrituras directas de cada nuevo elemento.

$$C_total = n + C_copy$$

$$(17 + 31)$$

Resultado

48

Interpretación Las 17 inserciones requieren 48 escrituras en total (menos de 3 * 17 = 51), confirmando la cota teórica del método de potencial (coste amortizado <= 3 por inserción).

Comprobación de los pasos

- Paso 1: 31 — reproducido
- Paso 2: 48 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- En sistemas de tiempo real estricto (hard real-time), el pico puntual O(n) del redimensionado obliga a pre-reservar con `with_capacity`.

Errores frecuentes

- Confundir el peor caso de una operación aislada O(n) con el coste de una secuencia de n operaciones, que es O(n) total y no $O(n^2)$.

Ejercicios

1 [PRB-0003] (Básico, Conceptual) [VOL02-C01-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0004] (Básico, Numérico) [VOL02-C01-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0005] (Intermedio, Interpretación) [VOL02-C01-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0006] (Intermedio, Aplicación) [VOL02-C01-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(14 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0007] (Intermedio, Identificación de errores) [VOL02-C01-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0008] (Avanzado, Desarrollo por pasos) [VOL02-C01-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0009] (Avanzado, Comparación) [VOL02-C01-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0010] (Avanzado, Comprobación de hipótesis) [VOL02-C01-E08] (Nivel 4– Ingeniería (Validación de precondiciones y bordes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0011] (Nivel de examen, Optimización) [VOL02-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0012] (Nivel de examen, Demostración) [VOL02-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0013] (Reto, Diseño) [VOL02-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0014] (Reto, Estudio de parámetros) [VOL02-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 2

Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

Dominar los algoritmos de búsqueda y ordenación desde sus demostraciones formales de corrección mediante invariantes de bucle hasta la cota teórica de información $\Omega(n \log n)$ y las implementaciones híbridas de producción (Timsort, pdqsort, Radix Sort).

Consejo

En las bibliotecas estándar modernas, elija la ordenación estable (`slice::sort` en Rust, Timsort en Python `sorted`, `std::stable_sort` en C++) cuando ordene registros compuestos o secuencias parcialmente ordenadas, y la ordenación inestable (`sort_unstable` / `pdqsort`) para tipos primitivos donde desee evitar la reserva de memoria auxiliar.

Definición

Cota Inferior de Ordenación por Comparación y Estabilidad: ningún algoritmo que ordene n elementos mediante comparaciones binarias puede realizar menos de $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaciones en el peor caso. Un algoritmo de ordenación es estable si para todo par de índices $i < j$ con claves equivalentes $k(A[i]) = k(A[j])$, el elemento $A[i] precede a $A[j] en la salida.

Autoficha

1. ¿Cómo se demuestra que el árbol de decisión de cualquier algoritmo de ordenación por comparación tiene altura al menos $\log_2(n!)$? 2. ¿Por qué `mid = low + (high - low) / 2` evita el desbordamiento entero de `(low + high) / 2`? 3. ¿Por qué Introsort/pdqsort combina Quicksort, Insertion Sort para subvectores pequeños y Heapsort como salvaguarda?

2.1 Búsqueda binaria, invariantes de bucle y prevención del desbordamiento de punto medio

La búsqueda binaria reduce un espacio ordenado de n elementos a la mitad en cada comparación, alcanzando complejidad $\Theta(\log_2 n)$ en el peor caso. Pese a su brevedad, como documentó Jon Bentley en Programming Pearls, la mayoría de las implementaciones improvisadas contienen defectos en las condiciones de parada o en el cálculo del punto medio: escribir `mid = (low + high) / 2` con enteros de ancho fijo provoca desbordamiento aritmético cuando `low + high` supera `INT_MAX` (defecto que permaneció veinte años oculto en `java.util.Arrays.binarySearch`). La forma correcta es `mid = low + (high - low) / 2`. Para demostrar formalmente la corrección de un bucle (inicialización, mantenimiento y terminación), se trabaja sobre un intervalo semiabierto `[low, high)` con el invariante: para todo índice $i <$ low se cumple $A[i] <$ target y para todo $j \geq$ high se cumple $A[j] \geq$ target. En cada iteración la longitud `high - low` decrece estrictamente (función de cota o variante de bucle), garantizando terminación en $\lceil \log_2(n+1) \rceil$ pasos.

2.2 Cota inferior por árbol de decisión, Mergesort, Quicksort, Heapsort y ordenación lineal

Todo algoritmo de ordenación basado exclusivamente en comparaciones entre pares de elementos puede modelarse como un árbol binario de decisión cuyas hojas contienen las $n!$ permutaciones posibles de la entrada. Un árbol binario de altura h posee a lo sumo 2^h hojas; por tanto, $2^h \geq n!$, y aplicando la

aproximación de Stirling ($\log_2(n!) = n\log_2 n - n\log_2 e + \Theta(\log n)$) se obtiene el teorema fundamental: cualquier algoritmo de ordenación por comparación requiere $h \geq \lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparaciones en el peor caso. Mergesort alcanza $\Theta(n \log n)$ garantizado y es estable (preserva el orden relativo de claves iguales), empleando $\Theta(n)$ memoria auxiliar; Heapsort logra $\Theta(n \log n)$ en el peor caso con memoria auxiliar $O(1)$ in-place sobre un montículo binario implícito, pero no es estable y presenta peor localidad de caché que Quicksort; Quicksort con pivote mediano o aleatorio logra $\Theta(n \log n)$ esperado con excelente localidad espacial en caché L1 (y en su variante híbrida Introsort/pdqsort empleada en Rust `sort_unstable` y C++ `std::sort`, conmuta a Heapsort si la profundidad recursiva supera $2\lfloor \log_2 n \rfloor$, garantizando $O(n \log n)$ en el peor caso). Cuando las claves son enteros de w bits agrupados en dígitos de base R, Counting Sort y Radix Sort evitan comparaciones directas y ordenan en tiempo lineal $\Theta(d(n + R))$.

```
fn lower_bound(slice: &[i64], target: i64) -> usize {
    let mut low = 0usize;
    let mut high = slice.len();
    while low < high {
        let mid = low + (high - low) / 2;
        if slice[mid] < target {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    low
}

fn main() {
    let data = [2, 5, 5, 5, 9, 14];
    assert_eq!(lower_bound(&data, 5), 1);
    assert_eq!(lower_bound(&data, 6), 4);
}
```

Problema resuelto

Enunciado Determine el número mínimo teórico de comparaciones binarias en el peor caso necesarias para ordenar $n = 5$ elementos distintos sabiendo que existen $5! = 120$ permutaciones posibles y que $2^6 = 64 < 120 \leq 2^7 = 128$, verificando además la diferencia $$2^7 - 5!$$.

Datos

- $n = 5$ — Número de elementos a ordenar
- $P = 120$ — Número de permutaciones 5!
- $L_7 = 128$ — Número máximo de hojas en altura $h = 7(2^7)$

Incógnita Margen de hojas sobrantes $2^7 - 5!$ que demuestra que h = 7 es el mínimo entero tal que $2^h >= 5!$

Modelo físico

Cota de información en árbol binario de decisión (2^h >= n!)

$$margin = 2^7 - fact_5$$

Hipótesis

- Cada comparación binaria divide el conjunto de permutaciones candidatas en a lo sumo dos subárboles

Desarrollo

1. Calculamos el factorial 5! = 120 permutaciones que deben aparecer como hojas alcanzables.

$$fact_5 = 5 * 4 * 3 * 2 * 1$$

$$((((5 \cdot 4) \cdot 3) \cdot 2) \cdot 1)$$

2. Comprobamos que $2^7 = 128$ cubre las 120 permutaciones con un margen de 8 hojas.

$$margin = 2^7 - fact_5$$

$$(128 - 120)$$

Resultado

$$8$$

Interpretación Con 6 comparaciones solo se distinguen a lo sumo $2^6 = 64$ permutaciones (< 120); por tanto, se necesitan al menos h = 7 comparaciones en el peor caso para ordenar 5 elementos, dejando un margen de 8 hojas.

Comprobación de los pasos

- Paso 1: 120 — reproducido
- Paso 2: 8 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Para n grandes, la cota de información ceil(log2(n!)) es necesaria pero en ciertos n específicos el mínimo exacto de Ford-Johnson requiere 1 comparación adicional.

Errores frecuentes

- Confundir n * log2(n) directo con log2(n!), que para n pequeños difiere significativamente.

Ejercicios

1 [PRB-0017] (Básico, Conceptual) [VOL02-C02-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0018] (Básico, Numérico) [VOL02-C02-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0019] (Intermedio, Interpretación) [VOL02-C02-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0020] (Intermedio, Aplicación) [VOL02-C02-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0021] (Intermedio, Identificación de errores) [VOL02-C02-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0022] (Avanzado, Desarrollo por pasos) [VOL02-C02-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0023] (Avanzado, Comparación) [VOL02-C02-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0024] (Avanzado, Comprobación de hipótesis) [VOL02-C02-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0025] (Nivel de examen, Optimización) [VOL02-C02-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0026] (Nivel de examen, Demostración) [VOL02-C02-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0027] (Reto, Diseño) [VOL02-C02-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0028] (Reto, Estudio de parámetros) [VOL02-C02-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 3

Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

Distinguir con rigor matemático cuándo un problema de optimización admite una estrategia voraz local, cuándo requiere programación dinámica por solapamiento de subproblemas y cuándo exige búsqueda en espacio de estados con poda (backtracking / branch and bound).

Consejo

Al optimizar espacio en la mochila 0-1 reduciendo la matriz 2D a un único vector 1D `dp[0..C]`, recorra siempre las capacidades en orden decreciente `for c in (w..=C).rev()`: si las recorre en orden creciente, estará reutilizando el mismo objeto múltiples veces (resolviendo la mochila ilimitada Unbounded Knapsack en lugar de la mochila 0-1).

Definición

Ecuación de Recurrencia de Bellman en la Mochila 0-1 y Distancia de Edición (Levenshtein): para n objetos con pesos w_i y valores v_i bajo capacidad C, el valor óptimo satisface $OPT(i, c) = \max(OPT(i-1, c),\ v_i + OPT(i-1, c-w_i))$ si $w_i \leq c$, y \$OPT(i-1, c)\$ en caso contrario, con complejidad pseudopolinómica $\Theta(nC)$ en tiempo y $\Theta(C)$ en espacio.

Autoficha

1. ¿Qué contraejemplo mínimo demuestra que el algoritmo voraz falla en el cambio de monedas con denominaciones `{1, 3, 4}`? 2. ¿Por qué recorrer las capacidades hacia atrás en `dp[c]` distingue la mochila 0-1 de la mochila ilimitada? 3. ¿Por qué la complejidad $\Theta(nC)$ de la mochila 0-1 se denomina pseudopolinómica?

3.1 Divide y vencerás y condición de elección voraz (Greedy Choice Property)

Un problema exhibe subestructura óptima cuando una solución óptima global contiene dentro de sí soluciones óptimas a sus subproblemas. Si los subproblemas son disjuntos (independientes), el paradigma de Divide y Vencerás los resuelve recursivamente y combina sus resultados (Mergesort, transformada rápida de Fourier FFT, multiplicación de enteros de Karatsuba en $O(n^{\log_2 3}) \approx O(n^{1{,}585})$). Los algoritmos voraces (greedy) dan un paso más audaz: en cada etapa toman una decisión localmente óptima sin explorar las ramas alternativas. Un algoritmo voraz es correcto única y exclusivamente cuando se demuestra la propiedad de elección voraz (existe siempre una solución óptima global que comienza con la elección voraz) junto con la subestructura óptima. Así ocurre en la selección de actividades por tiempo de finalización más temprano, en la codificación prefijo de entropía mínima de Huffman y en los árboles generadores mínimos sobre matroides (Kruskal y Prim); en cambio, en el problema de la mochila 0-1 (0-1 Knapsack) o del cambio de monedas para sistemas no canónicos (como monedas `{1, 3, 4}` para sumar `6`, donde el voraz elige `4+1+1` = 3 monedas frente al óptimo `3+3` = 2 monedas), la heurística voraz falla y es necesario recurrir a programación dinámica.

3.2 Programación dinámica (Bellman) y búsqueda exhaustiva con poda (Backtracking)

La Programación Dinámica (introducida por Richard Bellman en los años cincuenta) resuelve problemas con subestructura óptima y subproblemas solapados calculando cada estado una sola vez. Todo diseño de programación dinámica sigue cuatro pasos formales: (1) definir el espacio de estados y su significado, (2)

escribir la ecuación de recurrencia de Bellman relacionando un estado con sus predecesores en el DAG de dependencias, (3) establecer los casos base, y (4) elegir el orden de evaluación: recursivo con caché (memoización top-down) o iterativo en orden topológico (tabulación bottom-up), reduciendo a menudo la complejidad espacial de $O(n \cdot W)$ a $O(W)$ cuando la fila actual solo depende de la anterior. Cuando el problema es NP-duro y carece de estructura polinómica compacta (como satisfacibilidad booleana SAT, coloreado de grafos o las N-reinas), el Backtracking explora sistemáticamente el árbol de decisiones en profundidad aplicando predicados de factibilidad y cotas de optimalidad (branch and bound) para podar subárboles enteros antes de expandirlos:

```
def knapsack_01(weights: list[int], values: list[int], capacity: int) -> int:
    dp = [0] * (capacity + 1)
    for w, val in zip(weights, values):
        for c in range(capacity, w - 1, -1):
            if dp[c - w] + val > dp[c]:
                dp[c] = dp[c - w] + val
    return dp[capacity]

# Voraz por densidad elegiría (w=10, v=60)+(w=20, v=100) = 160; óptimo DP toma (20,100)
    +(30,120) = 220
assert knapsack_01([10, 20, 30], [60, 100, 120], 50) == 220
```

Problema resuelto

Enunciado En el algoritmo de Karatsuba para multiplicar dos enteros de n dígitos dividiendo cada operando en dos mitades, se sustituyen las 4 multiplicaciones clásicas de tamaño $n/2$ por solo 3 multiplicaciones recursivas. Compare el número de multiplicaciones base en una profundidad de $k = 4$ niveles ($n = 16$ bloques) entre el método cuadrático clásico (\$4^4\$) y Karatsuba (\$3^4\$), calculando el ahorro neto de productos elementales.

Datos

- $k = 4$ — Niveles de recursión (para $n = 2^4 = 16$)
- $b_{classic} = 4$ — Subproblemas por nivel en método clásico
- $b_{karat} = 3$ — Subproblemas por nivel en Karatsuba

Incógnita Ahorro de multiplicaciones base tras 4 niveles de recursión

Modelo físico

Árbol de recursión de Divide y Vencerás (4^k frente a 3^k)

$$saved = M_classic - 3^4$$

Hipótesis

- Cada nivel divide exactamente a la mitad el ancho de palabra de los operandos

Desarrollo

1. Calculamos los productos en las hojas del algoritmo cuadrático clásico 4^4.

$$M_classic = 4^4$$

$$(4)^4$$

2. Restamos los productos en las hojas de Karatsuba $3^4 = 81$.

$$saved = M_classic - 3^4$$

$$(256 - 81)$$

Resultado

175

Interpretación En solo 4 niveles de subdivisión Karatsuba reduce de 256 a 81 los productos base (ahorro de 175); para operandos criptográficos de miles de bits la ventaja asintótica $O(n^{1}.585)$ es decisiva.

Comprobación de los pasos

- Paso 1: 256 — reproducido
- Paso 2: 175 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Para operandos pequeños (p. ej. menos de 32 palabras de 64 bits), la sobrecarga de sumas y restas hace más rápido el producto escolar en ensamblador.

Errores frecuentes

- Aplicar programación dinámica cuando los subproblemas nunca se solapan (donde Divide y Vencerás no necesita tabla de memoización).

Ejercicios

1 [PRB-0031] (Básico, Conceptual) [VOL02-C03-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0032] (Básico, Numérico) [VOL02-C03-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0033] (Intermedio, Interpretación) [VOL02-C03-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0034] (Intermedio, Aplicación) [VOL02-C03-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0035] (Intermedio, Identificación de errores) [VOL02-C03-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0036] (Avanzado, Desarrollo por pasos) [VOL02-C03-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0037] (Avanzado, Comparación) [VOL02-C03-E07] (Nivel 4– Ingeniería (Refactoriza-

ción modular y contratos)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0038] (Avanzado, Comprobación de hipótesis) [VOL02-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0039] (Nivel de examen, Optimización) [VOL02-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0040] (Nivel de examen, Demostración) [VOL02-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0041] (Reto, Diseño) [VOL02-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0042] (Reto, Estudio de parámetros) [VOL02-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 4

Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

Comprender en profundidad los algoritmos sobre grafos y las dos grandes familias de índices asociativos —árboles equilibrados y tablas hash— que sustentan desde los enrutadores de red y compiladores hasta los motores de bases de datos y las colecciones estándar de todos los lenguajes.

Consejo

Si una tabla hash recibe claves controladas por usuarios externos (parámetros HTTP, cabeceras JSON) y utiliza una función hash no criptográfica predecible, un atacante puede enviar miles de claves que colisionen en la misma cubeta y degradar la complejidad de $O(1)$ a $O(n^2)$ (denegación de servicio HashDoS): por eso Rust usa un hasher con semilla aleatoria por defecto.

Definición

Factor de Carga (α) en Tablas Hash y Altura de un Árbol B+: en una tabla hash con n elementos y m ranuras, el factor de carga es $\alpha = n/m$. En direccionamiento abierto con sondeo lineal bajo dispersión uniforme, el número esperado de sondeos en una búsqueda exitosa es $\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$. En un árbol B+ de grado mínimo B, la altura satisface $h \leq \lfloor \log_B((n+1)/2) \rfloor + 1$.

Autoficha

1. ¿Por qué el algoritmo de Dijkstra falla si el grafo contiene una arista de peso negativo? 2. ¿Por qué `BTreeMap` y los motores SQL usan nodos de árbol B/B+ con múltiples claves por nodo en lugar de árboles binarios Rojo-Negro? 3. ¿En qué consiste un ataque de colisión HashDoS y cómo lo mitiga SipHash con semilla aleatoria?

4.1 BFS, DFS, componentes conexas, Dijkstra, Bellman-Ford, Kruskal y Prim

Sobre un grafo $G = (V, E)$ representado mediante listas de adyacencia (espacio $\Theta(|V|+|E|)$), la búsqueda en anchura (BFS, gobernada por una cola FIFO) explora el grafo por capas y calcula distancias mínimas en número de aristas en tiempo $\Theta(|V|+|E|)$. La búsqueda en profundidad (DFS, recursiva o con pila explícita) clasifica las aristas en árbol, avance, retroceso (cuya presencia caracteriza exactamente los ciclos en un grafo dirigido) y cruce, permitiendo calcular en tiempo lineal $\Theta(|V|+|E|)$ la ordenación topológica de un DAG y las componentes fuertemente conexas (algoritmo de Tarjan o Kosaraju). Cuando las aristas tienen pesos no negativos $w(u, v) \geq 0$, el algoritmo de Dijkstra relaja distancias extrayendo el vértice de menor distancia tentativa desde una cola de prioridad (montículo binario), alcanzando $O((|V| + |E|) \log |V|)$. Si existen pesos negativos, la hipótesis voraz de Dijkstra falla (un camino ya cerrado podría mejorarse más tarde a través de una arista negativa), y debe emplearse Bellman-Ford, que relaja todas las aristas $|V|-1$ veces en $O(|V| \cdot |E|)$ y detecta ciclos negativos alcanzables en la iteración $|V|$.

4.2 Árboles equilibrados (AVL, Rojo-Negro, B+ Trees) y tablas hash de alto rendimiento

Para implementar diccionarios dinámicos existen dos compromisos fundamentales. Los árboles binarios de búsqueda autoequilibrados (AVL con factor de equilibrio $|h_L - h_R| \leq 1$, y Rojo-Negro con altura acotada por $2\log_2(n+1)$) garantizan búsqueda, inserción, borrado y consultas por rango en $\Theta(\log n)$ en el peor caso, pero cada salto de puntero suele provocar un fallo de caché. Por ello, tanto `std::collections::BTreeMap` en Rust como los índices de bases de datos (PostgreSQL, SQLite) utilizan árboles B y B+ de alto factor de ramificación B: cada nodo agrupa entre $B-1$ y $2B-1$ claves contiguas en memoria o páginas de disco de 4–16 KiB, reduciendo la altura del árbol a $\log_B n$ (apenas 3 o 4 niveles para cientos de millones de registros). Las tablas hash proyectan cada clave mediante una función de dispersión $h(k)$ a un índice de cubeta (bucket), ofreciendo acceso esperado $O(1)$. Frente al encadenamiento separado con listas enlazadas, las implementaciones modernas de producción (como `HashMap` en Rust basado en SwissTable de Google con instrucciones SIMD y SipHash-1-3 resistente a ataques HashDoS, o `dict` compacto en Python) emplean direccionamiento abierto (open addressing) en un array contiguo con factor de carga $\alpha = n/m \leq 0{,}875$. Verifiquemos el algoritmo de Dijkstra en Python:

```
import heapq

def dijkstra(adj: dict[int, list[tuple[int, int]]], src: int) -> dict[int, int]:
    dist = {src: 0}
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, 10**18):
            continue
        for v, w in adj.get(u, []):
            if d + w < dist.get(v, 10**18):
                dist[v] = d + w
                heapq.heappush(pq, (d + w, v))
    return dist

g = {0: [(1, 4), (2, 1)], 2: [(1, 2), (3, 5)], 1: [(3, 1)], 3: []}
assert dijkstra(g, 0) == {0: 0, 2: 1, 1: 3, 3: 4}
```

Problema resuelto

Enunciado En una tabla hash con direccionamiento abierto y sondeo lineal que se mantiene con un factor de carga $\alpha = 0{,}75$, calcule el número esperado de sondeos en una búsqueda sin éxito (inserción de clave nueva) según la fórmula de Knuth $E_{\text{miss}} = \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$.

Datos

- $\alpha = 0{,}75$ — Factor de carga n / m de la tabla hash

Incógnita Número medio esperado de sondeos E_miss en una búsqueda sin éxito / inserción

Modelo físico

Fórmula de Knuth para agrupamiento primario (primary clustering) en sondeo lineal

$$E_miss = 0{,}5 * (1 + inv_sq)$$

Hipótesis

- Función hash uniforme e independiente sobre m ranuras

Desarrollo

1. Calculamos el inverso del cuadrado de la fracción libre $(1 - 0{,}75)^2 = 0{,}0625$.

$$inv_sq = 1/((1 - 0{,}75)^2)$$

$$\frac{1}{((1-0{,}75))^2}$$

2. Aplicamos la media 0.5 * (1 + inv$_s q$) para obtener el número esperado de sondeos.

$$E_m iss = 0{,}5 * (1 + inv_s q)$$

$$(0{,}5 \cdot (1 + 16))$$

Resultado

$$8{,}5$$

Interpretación Al 75 % de ocupación, una inserción por sondeo lineal clásico explora 8.5 ranuras en promedio (y al 90 % subiría a 50.5 ranuras); de ahí el uso de Robin Hood hashing o metadatos SIMD (SwissTable).

Comprobación de los pasos

- Paso 1: 16 — reproducido
- Paso 2: 8,5 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- El resultado asume independencia total de la función hash; funciones débiles sufren agrupamiento aún mayor.

Errores frecuentes

- Permitir que una tabla de direccionamiento abierto supere alpha > 0.85 sin redimensionar.

Ejercicios

1 [PRB-0045] (Básico, Conceptual) [VOL02-C04-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0046] (Básico, Numérico) [VOL02-C04-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0047] (Intermedio, Interpretación) [VOL02-C04-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0048] (Intermedio, Aplicación) [VOL02-C04-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0049] (Intermedio, Identificación de errores) [VOL02-C04-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre algoritmos de grafos, árboles B+ y tablas hash:

analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0050] (Avanzado, Desarrollo por pasos) [VOL02-C04-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0051] (Avanzado, Comparación) [VOL02-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0052] (Avanzado, Comprobación de hipótesis) [VOL02-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0053] (Nivel de examen, Optimización) [VOL02-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0054] (Nivel de examen, Demostración) [VOL02-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0055] (Reto, Diseño) [VOL02-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0056] (Reto, Estudio de parámetros) [VOL02-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 5

Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

Dominar técnicas avanzadas indispensables en sistemas modernos: estructuras persistentes que conservan versiones históricas sin clonar toda la memoria, filtros de Bloom para evitar accesos innecesarios a disco y el modelo Work-Span para analizar algoritmos paralelos.

Consejo

En un algoritmo paralelo de divide y vencerás (como Mergesort o suma en árbol), corte la generación de tareas paralelas por debajo de un umbral de granularidad (por ejemplo, bloques de 4096 elementos) y resuelva las hojas secuencialmente para que el coste de planificación no domine a T_1.

Definición

Teorema de Brent-Graham en el Modelo Work-Span y Paralelismo Medio: dado un cómputo paralelo libre de bloqueos con trabajo total T_1 y camino crítico (span) T_∞, el tiempo de ejecución en P procesadores bajo planificación voraz satisface $\max(T_1/P, T_\infty) \leq T_P \leq T_1/P + T_\infty$. El paralelismo medio es $\overline{P} = T_1/T_\infty$.

Autoficha

1. ¿Por qué la compartición estructural (path copying) solo es segura cuando los nodos compartidos son inmutables? 2. ¿Por qué un filtro de Bloom nunca produce falsos negativos pero sí puede producir falsos positivos, y por qué no admite borrado directo de bits? 3. ¿Qué mide el cociente T_1/T_∞ en el modelo Work-Span?

5.1 Estructuras persistentes (path copying) y filtros probabilísticos (Bloom Filter)

Una estructura de datos es persistente cuando cualquier operación de actualización produce una nueva versión lógicamente independiente conservando intacta la versión anterior. En lugar de copiar los n elementos en $\Theta(n)$, las estructuras persistentes basadas en árboles (empleadas en Git, Clojure, Scala, Redux/Immer y motores MVCC) aplican copia de camino (path copying) y compartición estructural (structural sharing): al modificar una hoja de un árbol de altura $h = O(\log n)$, se asignan únicamente los $O(\log n)$ nodos del camino desde la raíz hasta la hoja modificada, mientras que todos los subárboles no modificados se comparten de forma segura porque son inmutables. En sistemas de almacenamiento (como motores LSM-tree en RocksDB, Cassandra o LevelDB), antes de buscar una clave en múltiples archivos SSTable de disco se consulta en RAM un Filtro de Bloom (Burton Bloom, 1970): un vector de m bits inicializado a 0 junto con k funciones hash independientes. Al insertar x, se activan los bits $h_1(x), \ldots, h_k(x)$. Si al consultar y alguno de los k bits vale 0, se garantiza con certeza absoluta (0 % de falsos negativos) que y no pertenece al conjunto; si todos valen 1, puede tratarse de un falso positivo con probabilidad aproximada $(1 - e^{-kn/m})^k$, que se minimiza eligiendo $k = \frac{m}{n} \ln 2$.

5.2 Algoritmos paralelos: modelo Work-Span, ley de Brent y reducciones paralelas

Para analizar algoritmos paralelos con independencia del número concreto de núcleos físicos P, el modelo de grafo de tareas (Work-Span model, usado en Cilk, Rayon en Rust y ForkJoinPool en Java) define dos métricas intrínsecas del cómputo DAG: 1. Trabajo (T_1, Work): el tiempo total de ejecutar todas las operaciones en un único procesador. 2. Longitud de camino crítico (T_∞, Span o Depth): el tiempo requerido en un computador ideal con infinitos procesadores, determinado por la cadena más larga de dependencias secuenciales. Todo planificador válido satisface las dos cotas inferiores fundamentales: $T_P \geq T_1/P$ (cota de trabajo) y $T_P \geq T_\infty$ (cota de camino crítico). Además, un planificador voraz o por robo de trabajo (work-stealing, como el de Rayon o Tokio) garantiza la cota superior de Brent-Graham: $T_P \leq T_1/P + T_\infty$. El cociente $\overline{P} = T_1/T_\infty$ se denomina paralelismo medio del algoritmo: cuando $\overline{P} \gg P$, el algoritmo alcanza aceleración lineal casi perfecta $T_P \approx T_1/P$.

Problema resuelto

Enunciado Un algoritmo paralelo de reducción y filtrado tiene un trabajo total $T_1 = 9600$ ms y un camino crítico (span) $T_\infty = 40$ ms. Calcule la cota superior garantizada de tiempo de ejecución $T_P = T_1/P + T_\infty$ en $P = 16$ procesadores bajo un planificador voraz.
Datos

- $T_1 = 9600$ — Trabajo secuencial total (ms)
- $T_{inf} = 40$ — Camino crítico o span (ms)
- $P = 16$ — Número de procesadores

Incógnita Cota superior de tiempo de ejecución T_P en milisegundos
Modelo físico
Desigualdad de Brent-Graham en el modelo Work-Span (T_P <= T_1 / P + T_inf)

$$T_{P,\text{máx}} = t_{extwork} + T_\infty$$

Hipótesis

- Planificador voraz sin sobrecarga significativa de robo de tareas

Desarrollo

1. Dividimos el trabajo total T_1 entre los P = 16 procesadores.

$$t_{extwork} = T_1/P$$

$$\frac{9600}{16}$$

2. Sumamos el span $T_inf = 40$ ms para obtener la cota superior garantizada.

$$T_{P,\text{máx}} = t_{extwork} + T_\infty$$

$$(600 + 40)$$

Resultado

$$640$$

Interpretación Dado que el paralelismo medio $T_1/T_inf = 240$ supera ampliamente a P = 16, el tiempo queda acotado entre 600 ms y 640 ms, logrando una aceleración efectiva de 9600 / 640 = 15x.
Comprobación de los pasos

- Paso 1: 600 — reproducido
- Paso 2: 640 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- La cota no contabiliza la contención del ancho de banda de memoria DRAM cuando los 16 núcleos saturan el bus.

Errores frecuentes

- Intentar borrar un elemento de un filtro de Bloom estándar poniendo sus k bits a 0, lo cual corrompería otros elementos que compartan esos bits.

Ejercicios

1 [PRB-0059] (Básico, Conceptual) [VOL02-C05-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0060] (Básico, Numérico) [VOL02-C05-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0061] (Intermedio, Interpretación) [VOL02-C05-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0062] (Intermedio, Aplicación) [VOL02-C05-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0063] (Intermedio, Identificación de errores) [VOL02-C05-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0064] (Avanzado, Desarrollo por pasos) [VOL02-C05-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0065] (Avanzado, Comparación) [VOL02-C05-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0066] (Avanzado, Comprobación de hipótesis) [VOL02-C05-E08] (Nivel 4– Ingeniería (Validación de precondiciones y bordes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0067] (Nivel de examen, Optimización) [VOL02-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0068] (Nivel de examen, Demostración) [VOL02-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0069] (Reto, Diseño) [VOL02-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0070] (Reto, Estudio de parámetros) [VOL02-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Parte II

PARTE IV. Sistemas Informáticos, Compiladores, Memoria y Sistemas Operativos

Capítulo 6

Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

Explicar cómo las líneas de caché de 64 bytes, el protocolo de coherencia entre núcleos, las reglas de alineamiento del ABI y el proceso de enlazado determinan el rendimiento real y la interoperabilidad binaria de los programas.

Consejo

En C, ordene los miembros de un `struct` de mayor a menor requisito de alineamiento (primero punteros y `uint64_t` de 8 bytes, luego `uint32_t` de 4, `uint16_t` de 2 y finalmente `uint8_t`/`bool` de 1 byte) para eliminar el padding interno y aumentar el número de registros que caben por línea de caché.

Definición

Regla de Alineamiento de Estructuras (ABI C) y Compartición Falsa (False Sharing): en un `struct` con diseño C, cada campo con alineamiento a_i se sitúa en el menor desplazamiento `offset` tal que offset mód $a_i = 0$, y el tamaño total del `struct` se redondea al múltiplo superior de $a_{\text{máx}} = \text{máx}_i\, a_i$. La compartición falsa ocurre cuando múltiples núcleos escriben en variables disjuntas que residen en la misma línea física de caché.

Autoficha

1. ¿Por qué `struct { uint8_t a; uint64_t b; uint8_t c; }` ocupa 24 bytes en C de 64 bits pero solo 16 bytes si se reordena como `{ uint64_t b; uint8_t a; uint8_t c; }`? 2. ¿Qué provoca el fenómeno de false sharing en el protocolo MESI y cómo se corrige? 3. ¿Por qué Rust distingue entre su layout optimizado por defecto y `#[repr(C)]`?

6.1 Líneas de caché, coherencia MESI, false sharing y diseño orientado a datos

Las cachés del procesador no transfieren bytes individuales desde la RAM, sino bloques alineados de tamaño fijo denominados líneas de caché (cache lines, de 64 bytes en x86-64 y la mayoría de procesadores ARM64, o 128 bytes en los núcleos de alto rendimiento Apple M-series). En un procesador multinúcleo, cada núcleo posee su propia caché L1 privada; para evitar lecturas incoherentes, el hardware implementa protocolos de invalidación de directorio o bus como MESI (Modified, Exclusive, Shared, Invalid): para que un núcleo modifique un byte de una línea, debe adquirir la línea completa en estado Exclusive/Modified e invalidar las copias en las cachés L1 de todos los demás núcleos. De aquí surge una patología crítica de rendimiento concurrente: la compartición falsa (false sharing). Si dos hilos ejecutándose en núcleos distintos actualizan intensivamente dos contadores atómicos independientes (`counter_a` y `counter_b`) que por estar declarados contiguos en una estructura caen dentro de la misma línea de caché de 64 bytes, ambos núcleos se invalidarán mutuamente la línea en cada escritura (cache line ping-pong), degradando el rendimiento en un orden de magnitud pese a no compartir ningún dato lógico. La solución consiste en acumular en una variable local de registro y volcar al final, o alinear cada contador a su propia línea de caché (`#[repr(align(64))]` en Rust o `alignas(64)` en C11/C++11).

6.2 Alineamiento en memoria, ABI de plataforma y enlazadores (ELF/Mach-O)

Los accesos a memoria de un tipo primitivo de tamaño k bytes son atómicos y eficientes en hardware cuando su dirección es múltiplo de su alineamiento natural k. Por ello, el ABI de C (como el System V AMD64 ABI o el AAPCS64 de ARM) inserta bytes de relleno (padding) entre campos de un `struct` y al final del mismo para que su tamaño total sea múltiplo del alineamiento máximo de sus miembros. Mientras que en C el compilador está obligado a respetar el orden textual de declaración de los campos, en Rust el diseño por defecto `#[repr(Rust)]` permite al compilador reordenar automáticamente los campos en memoria para minimizar el padding sin cambiar el código fuente (utilizándose `#[repr(C)]` solo cuando se cruza una frontera FFI). Cuando el compilador traduce una unidad de traducción, emite un archivo objeto relocalizable (formato ELF en Linux, Mach-O en macOS, COFF/PE en Windows) con tablas de símbolos definidos e indefinidos y entradas de relocalización. El enlazador (linker: `lld`, `ld`, `mold`) resuelve las referencias cruzadas entre módulos, fusiona secciones y produce bien un ejecutable enlazado estáticamente o bien un binario dinámico que delega en el cargador del sistema (`ld.so` / `dyld`) la resolución de bibliotecas compartidas (`.so` / `.dylib` / `.dll`) a través de la Global Offset Table (GOT) y Procedure Linkage Table (PLT).

Problema resuelto

Enunciado Calcule el ahorro total en kilobytes (1 KiB = 1024 bytes) de un vector en memoria de $N = 65536$ registros cuando se reordena en C una estructura mal alineada de 24 bytes (`uint8_t`, 7 bytes de padding, `uint64_t`, `uint8_t`, 7 bytes de padding) a su disposición óptima de 16 bytes (`uint64_t`, `uint8_t`, `uint8_t`, 6 bytes de padding final).
Datos

- $N = 65536$ — Número de estructuras en el vector
- $S_{bad} = 24$ — Tamaño en bytes con orden subóptimo
- $S_{opt} = 16$ — Tamaño en bytes ordenando de mayor a menor alineamiento

Incógnita Ahorro de memoria en KiB para el vector de 65536 elementos
Modelo físico
Cálculo de huella de memoria de arrays contiguos con alineamiento ABI de 8 bytes

$$saved_kib = (N * delta_s)/1024$$

Hipótesis

- Arquitectura de 64 bits donde alignof(uint64$_t$) == 8

Desarrollo

1. Calculamos los bytes ahorrados por cada registro individual al eliminar un bloque de padding.

$$delta_s = S_bad - S_opt$$

$$(24 - 16)$$

2. Multiplicamos por los 65536 registros y dividimos entre 1024 bytes/KiB.

$$saved_kib = (N * delta_s)/1024$$

$$\frac{(65536 \cdot 8)}{1024}$$

Resultado

512

Interpretación Simplemente reordenando dos líneas de declaración en C se ahorran 512 KiB (un 33 % de la memoria) y ahora caben 4 estructuras por línea de caché de 64 B en lugar de 2.66.
Comprobación de los pasos

- Paso 1: 8 — reproducido
- Paso 2: 512 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Si la estructura define un protocolo binario de red o hardware fijo, no debe reordenarse sin actualizar la especificación del protocolo.

Errores frecuentes

- Abusar de `__attribute__((packed))` o `#[repr(packed)]` en estructuras de uso intensivo en CPU, generando accesos desalineados costosos o UB al tomar referencias.

Ejercicios

1 [PRB-0073] (Básico, Conceptual) [VOL02-C06-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0074] (Básico, Numérico) [VOL02-C06-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0075] (Intermedio, Interpretación) [VOL02-C06-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0076] (Intermedio, Aplicación) [VOL02-C06-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(24 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0077] (Intermedio, Identificación de errores) [VOL02-C06-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0078] (Avanzado, Desarrollo por pasos) [VOL02-C06-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0079] (Avanzado, Comparación) [VOL02-C06-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica carac-

terística mediante la expresión reproducible `24 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0080] (Avanzado, Comprobación de hipótesis) [VOL02-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0081] (Nivel de examen, Optimización) [VOL02-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0082] (Nivel de examen, Demostración) [VOL02-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0083] (Reto, Diseño) [VOL02-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0084] (Reto, Estudio de parámetros) [VOL02-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 7

Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

Desmitificar cómo funciona por dentro un compilador industrial (`rustc`, `clang`, `gcc`, `ghc`, `v8`), recorriendo las etapas de la figura ?? para que el lector pueda tanto diagnosticar qué código genera su compilador como construir su propio lenguaje, DSL o herramienta de análisis estático.

Consejo

Cuando escriba un parser para expresiones aritméticas o booleanas con múltiples niveles de precedencia, utilice un parser de Pratt con potencias de enlace izquierda/derecha `(l_bp, r_bp)` en lugar de crear una función recursiva distinta por cada nivel de precedencia: el código se reduce a una tercera parte y permite añadir nuevos operadores en una sola línea de tabla.

Definición

Forma de Asignación Única Estática (SSA) y Grafo de Interferencia de Registros: una representación intermedia está en forma SSA si cada variable virtual posee exactamente un punto de definición estático y toda definición domina todos sus usos (o se combina mediante nodos ϕ en la frontera de dominancia). Dos variables interfieren en la asignación de registros si sus intervalos de vida (liveness) se solapan en algún punto del CFG, impidiendo compartir el mismo registro físico.

Autoficha

1. ¿Por qué los compiladores industriales como `rustc` y `clang` prefieren descenso recursivo + Pratt frente a generadores LR automáticos? 2. ¿Para qué sirve el nodo $\phi(x_1, x_2)$ en la representación intermedia SSA cuando convergen dos ramas de un `if-else`? 3. ¿Cómo se modela la asignación de registros como un problema de k-coloreado sobre el grafo de interferencia?

7.1 Front-end del compilador: análisis léxico, sintáctico (descendente recursivo y Pratt) y semántico

Un compilador moderno se divide en tres grandes etapas desacopladas (Front-End, Middle-End y Back-End). El Front-End comienza con el analizador léxico (lexer o scanner), que agrupa caracteres UTF-8 mediante autómatas finitos deterministas en una secuencia de tokens con información de posición (span). A continuación, el analizador sintáctico (parser) comprueba la gramática libre de contexto y construye el Árbol de Sintaxis Abstracta (AST). Mientras que los generadores clásicos (Yacc/Bison, ANTLR) construyen tablas LR o LL, los compiladores de producción actuales (`rustc`, `clang`, `go`, `tsc`) utilizan mayoritariamente descenso recursivo escrito a mano combinado con parsing de Pratt (Top-Down Operator Precedence, Vaughan Pratt 1973) para expresiones con niveles de precedencia y asociatividad, ya que facilita mensajes de diagnóstico precisos y recuperación ante errores sintácticos. Sobre el AST, el análisis semántico resuelve nombres en tablas de símbolos con ámbito léxico y ejecuta la comprobación e inferencia de tipos (mediante unificación de restricciones al estilo Hindley-Milner Damas-Milner o comprobación bidireccional), reduciendo el lenguaje de superficie a una representación tipada de alto nivel (como HIR y MIR en `rustc`, donde se ejecuta el borrow checker).

7.2 Representación intermedia SSA, optimizaciones e instrucción/asignación de registros

El Middle-End traduce el programa a un Grafo de Flujo de Control (CFG) de bloques básicos en forma SSA (Static Single Assignment, Cytron et al. 1991, base de LLVM IR, GCC GIMPLE y Go SSA): cada variable temporal se asigna exactamente una vez en el texto del programa, y cuando dos ramas de un condicional convergen en un bloque de unión se inserta una función $\phi(x_1, x_2)$ que selecciona el valor según el bloque predecesor. La forma SSA hace explícitas las cadenas definición-uso (def-use chains), simplificando drásticamente optimizaciones como propagación de constantes, eliminación de código muerto (DCE), numeración global de valores (GVN), movimiento de código invariante de bucle (LICM) e inlining. Finalmente, el Back-End realiza la selección de instrucciones para la ISA destino (x86-64, ARM64, RISC-V, WebAssembly), ejecuta la asignación de registros (modelada como coloreado de grafos de interferencia por Chaitin-Briggs o mediante barrido lineal Linear Scan en compiladores JIT, volcando a la pila −spilling− las variables que exceden los registros físicos disponibles) y planifica las instrucciones para maximizar el paralelismo a nivel de instrucción.

Problema resuelto

Enunciado En un bloque básico antes de la optimización se evalúa en un bucle de $N = 1000$ iteraciones una expresión con una subexpresión invariante de bucle de coste $c_{\text{inv}} = 6$ ciclos y una parte dependiente del índice de coste $c_{\text{var}} = 3$ ciclos por iteración. Calcule el ahorro total de ciclos al aplicar el pase de Movimiento de Código Invariante de Bucle (LICM), que evalúa la subexpresión invariante una única vez fuera del bucle.

Datos

- $N = 1000$ — Iteraciones del bucle
- $c_{inv} = 6$ — Ciclos de la subexpresión invariante
- $c_{var} = 3$ — Ciclos de la parte dependiente del índice

Incógnita Ahorro neto de ciclos tras aplicar el pase LICM

Modelo físico

Modelo de coste de bloque básico bajo Loop-Invariant Code Motion (LICM)

$$saved = C_orig - (c_inv + N * c_var)$$

Hipótesis

- La subexpresión invariante es pura, no lanza trampas de hardware (como división por cero) y sus operandos no sufren aliasing en el bucle

Desarrollo

1. Calculamos el coste sin optimizar ejecutando ambas partes en las 1000 iteraciones.

$$C_orig = N * (c_inv + c_var)$$

$$(1000 \cdot (6 + 3))$$

2. Restamos el coste tras elevar la expresión invariante al preheader del bucle (6 + 3000).

$$saved = C_orig - (c_inv + N * c_var)$$

$$(9000 - 3006)$$

Resultado

$$5994$$

Interpretación El pase LICM ahorra 5994 ciclos (dos tercios del tiempo del bucle); en C este pase puede bloquearse si el compilador no puede descartar que un puntero de escritura apunte al mismo objeto leído (problema que se resuelve con `restrict` en C o con las garantías de `&` y `&mut` en Rust).
Comprobación de los pasos

- Paso 1: 9000 — reproducido
- Paso 2: 5994 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Elevar demasiadas expresiones invariantes simultáneamente puede aumentar la presión de registros y provocar spilling a la pila.

Errores frecuentes

- Suponer que el compilador puede sacar del bucle una llamada a función externa con posibles efectos laterales o lecturas a través de punteros con posible aliasing.

Ejercicios

1 [PRB-0087] (Básico, Conceptual) [VOL02-C07-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0088] (Básico, Numérico) [VOL02-C07-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0089] (Intermedio, Interpretación) [VOL02-C07-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0090] (Intermedio, Aplicación) [VOL02-C07-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(26 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0091] (Intermedio, Identificación de errores) [VOL02-C07-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0092] (Avanzado, Desarrollo por pasos) [VOL02-C07-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0093] (Avanzado, Comparación) [VOL02-C07-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0094] (Avanzado, Comprobación de hipótesis) [VOL02-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0095] (Nivel de examen, Optimización) [VOL02-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0096] (Nivel de examen, Demostración) [VOL02-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0097] (Reto, Diseño) [VOL02-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0098] (Reto, Estudio de parámetros) [VOL02-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 8

Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

Comprender las abstracciones fundamentales que el núcleo del sistema operativo (Linux, XNU/macOS, Windows NT) ofrece a todo programa: aislamiento de procesos mediante memoria virtual, coste real de las llamadas al sistema y mecanismos de entrada/salida de alta concurrencia.

Consejo

Minimice el número de llamadas al sistema (`read`/`write`) en bucles críticos utilizando buffers en espacio de usuario (`BufReader`/`BufWriter` en Rust, `BufferedInputStream` en Java, `stdio` en C): escribir 1 MiB byte a byte mediante un millón de llamadas `write(fd, &b, 1)` es cientos de veces más lento que agrupar en bloques de 8 KiB o 64 KiB.

Definición

Traducción de Direcciones Virtuales y Multiplexación de Eventos $O(1)$: en un sistema con páginas de 2^p bytes, los p bits menos significativos de una dirección virtual forman el desplazamiento dentro de la página (page offset, que no se traduce) y los bits superiores forman el número de página virtual (VPN) traducido por la TLB/tabla de páginas al número de marco físico (PFN). Primitivas como `epoll` y `kqueue` notifican únicamente los descriptores listos sin escanear el conjunto total de descriptores registrados.

Autoficha

1. ¿Qué recursos comparten dos hilos de un mismo proceso y qué recursos son privados de cada hilo? 2. ¿Qué función cumple la TLB en la traducción de direcciones virtuales y por qué un cambio de proceso encarece más el contexto que un cambio de hilo del mismo proceso? 3. ¿Por qué `epoll`/`kqueue` escalan a decenas de miles de conexiones donde `select`/`poll` se degradan linealmente?

8.1 Modo usuario vs modo kernel, llamadas al sistema, procesos, hilos y planificación

El procesador impone aislamiento de hardware mediante al menos dos niveles de privilegio: modo usuario (Ring 3 en x86-64, EL0 en ARM64) y modo supervisor o kernel (Ring 0 / EL1). El código de una aplicación se ejecuta en modo usuario y no puede acceder directamente a dispositivos físicos ni modificar tablas de páginas; para solicitar servicios al núcleo (abrir un archivo, reservar páginas de memoria con `mmap`, enviar paquetes por un socket) ejecuta una instrucción de trampa síncrona (`syscall` en x86-64, `svc #0` en ARM64) que salva los registros de usuario, conmuta a la pila del kernel, valida los argumentos y ejecuta el manejador privilegiado. Un proceso es la unidad de aislamiento de recursos: posee su propio espacio de direcciones virtuales, tabla de descriptores de archivo y credenciales de seguridad. Un hilo (thread) es la unidad de planificación de CPU: varios hilos dentro de un mismo proceso comparten el espacio de direcciones virtuales (código, datos globales y heap), pero cada uno posee su propio contador de programa, banco de registros y pila de llamadas independiente.

8.2 Memoria virtual, tablas de páginas, TLB, sistemas de archivos y E/S no bloqueante

La Memoria Virtual, implementada conjuntamente por la Unidad de Gestión de Memoria (MMU) del procesador y el sistema operativo, divide el espacio de direcciones en páginas de tamaño fijo (típicamente 4 KiB en x86-64 o 16 KiB en Apple Silicon ARM64). La MMU traduce cada dirección virtual en dirección física recorriendo una tabla de páginas multinivel (4 niveles en x86-64 de 48 bits) y almacenando las traducciones recientes en una caché hardware asociativa llamada TLB (Translation Lookaside Buffer). Cuando un proceso accede a una página no mapeada en RAM, la MMU dispara una excepción de fallo de página (page fault) para que el kernel asigne un marco físico bajo demanda (demand paging), cargue datos desde un archivo mapeado (`mmap`) o aplique copia en escritura (Copy-on-Write tras `fork`). En el subsistema de Entrada/Salida, bloquear un hilo por cada socket resulta inviable a gran escala; por ello los runtimes de red modernos (Tokio en Rust, Node.js/libuv, Netty en Java, Go netpoller) configuran los descriptores en modo no bloqueante (`O_NONBLOCK`) y delegan la espera multiplexada en primitivas de notificación de eventos de complejidad $O(\text{eventos activos})$: `epoll` en Linux, `kqueue` en macOS/BSD e `io_uring` (colas en anillo compartidas entre usuario y kernel sin llamadas al sistema por operación) en Linux moderno.

Problema resuelto

Enunciado En una arquitectura con páginas de 4 KiB = 4096 bytes (2^{12} bytes), dada la dirección virtual decimal $V = 20500$, determine el número de página virtual VPN $= \lfloor V/4096 \rfloor$, el desplazamiento dentro de la página offset $= V - \text{VPN} \cdot 4096$, y calcule la dirección física resultante si la tabla de páginas asigna dicho VPN al marco físico PFN = 12.
Datos

- $V = 20500$ — Dirección virtual en bytes
- $P_{size} = 4096$ — Tamaño de página en bytes (4 KiB)
- $PFN = 12$ — Número de marco físico asignado a VPN = 5

Incógnita Dirección física resultante en bytes tras la traducción MMU
Modelo físico
*Traducción de dirección virtual paginada (Physical = PFN * PageSize + Offset)*

$$P_addr = PFN * P_size + offset$$

Hipótesis

- Tamaño de página uniforme de 4096 bytes con VPN = floor(20500 / 4096) = 5

Desarrollo

1. Calculamos el desplazamiento (offset) dentro de la página 5 (5 * 4096 = 20480).

$$offset = V - 5 * P_size$$

$$(20500 - (5 \cdot 4096))$$

2. Sumamos el desplazamiento a la dirección base del marco físico PFN = 12.

$$P_addr = PFN * P_size + offset$$

$$((12 \cdot 4096) + 20)$$

Resultado

$$49172$$

Interpretación La dirección virtual 20500 corresponde a la página virtual 5 con desplazamiento de 20 bytes, que la MMU traduce al marco físico 12 en la dirección física 49152 + 20 = 49172.
Comprobación de los pasos

- Paso 1: 20 — reproducido
- Paso 2: 49172 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Para cargas de memoria de decenas de gigabytes, el uso de Huge Pages (2 MiB) reduce los fallos de TLB.

Errores frecuentes

- Confundir el tamaño de la dirección virtual con la cantidad de RAM física instalada en el equipo.

Ejercicios

1 [PRB-0101] (Básico, Conceptual) [VOL02-C08-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0102] (Básico, Numérico) [VOL02-C08-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0103] (Intermedio, Interpretación) [VOL02-C08-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0104] (Intermedio, Aplicación) [VOL02-C08-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(30 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0105] (Intermedio, Identificación de errores) [VOL02-C08-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0106] (Avanzado, Desarrollo por pasos) [VOL02-C08-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0107] (Avanzado, Comparación) [VOL02-C08-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0108] (Avanzado, Comprobación de hipótesis) [VOL02-C08-E08] (Nivel 4– Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas operativos, llamadas al sistema, memoria

virtual paginada y epoll/kqueue: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0109] (Nivel de examen, Optimización) [VOL02-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0110] (Nivel de examen, Demostración) [VOL02-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0111] (Reto, Diseño) [VOL02-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0112] (Reto, Estudio de parámetros) [VOL02-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Apéndice A

Notación

Símbolo	Significado	Unidades
$MESI$	Estados Modified, Exclusive, Shared, Invalid de coherencia de caché	adimensional
$OPT(i, c)$	Valor óptimo de la subestructura en programación dinámica	adimensional
PFN	Physical Frame Number (número de marco físico)	adimensional
T_1	Trabajo total secuencial (Work) del grafo de tareas	s
T_∞	Camino crítico (Span) con infinitos procesadores	s
VPN	Virtual Page Number (número de página virtual)	adimensional
$\Phi(D_i)$	Función de potencial de la estructura de datos en el paso i	adimensional
$\Theta(g(n))$	Orden de crecimiento asintótico ajustado superior e inferiormente	adimensional
α	Factor de carga n/m de una tabla hash	adimensional
$\phi(x_1, x_2)$	Función de selección en punto de convergencia SSA	adimensional
$n!$	Cardinalidad del grupo simétrico de permutaciones de n elementos	adimensional

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

Problema resuelto [PRB-0001]
Un vector dinámico comienza con capacidad inicial $\text{cap}_0 = 1$ y duplica su capacidad cada vez que se llena ($1 \to 2 \to 4 \to 8 \to 16 \to 32$). Si se insertan secuencialmente $n = 17$ elementos y cada inserción cuesta 1 escritura más el número de elementos copiados durante el redimensionado ($1 + 2 + 4 + 8 + 16$ copias acumuladas), calcule el coste total exacto de escrituras realizadas.
Respuesta

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[VOL02-C01-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 28
Ejercicio 2 [PRB-0004]
[VOL02-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 30
Ejercicio 3 [PRB-0005]
[VOL02-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 4 [PRB-0006]
[VOL02-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(14 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 44
Ejercicio 5 [PRB-0007]
[VOL02-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 54
Ejercicio 6 [PRB-0008]
[VOL02-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 80
Ejercicio 7 [PRB-0009]

[VOL02-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 84
Ejercicio 8 [PRB-0010]
[VOL02-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 101
Ejercicio 9 [PRB-0011]
[VOL02-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 124
Ejercicio 10 [PRB-0012]
[VOL02-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 126
Ejercicio 11 [PRB-0013]
[VOL02-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 165
Ejercicio 12 [PRB-0014]
[VOL02-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre notación asintótica, Teorema Maestro y análisis amortizado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `14 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 168

Cálculos reproducidos

$(17 + 31) = 48$

C.2 Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

Problema resuelto [PRB-0015]
Determine el número mínimo teórico de comparaciones binarias en el peor caso necesarias para ordenar $n = 5$ elementos distintos sabiendo que existen $5! = 120$ permutaciones posibles y que $2^6 = 64 < 120 \leq 2^7 = 128$, verificando además la diferencia $\$2^7 - 5!\$$.
Respuesta

8

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0017]
[VOL02-C02-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 32

Ejercicio 2 [PRB-0018]

[VOL02-C02-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 32

Ejercicio 3 [PRB-0019]

[VOL02-C02-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 52

Ejercicio 4 [PRB-0020]

[VOL02-C02-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 48

Ejercicio 5 [PRB-0021]

[VOL02-C02-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 62

Ejercicio 6 [PRB-0022]

[VOL02-C02-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 90

Ejercicio 7 [PRB-0023]

[VOL02-C02-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 96

Ejercicio 8 [PRB-0024]

[VOL02-C02-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 115

Ejercicio 9 [PRB-0025]

[VOL02-C02-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 140

Ejercicio 10 [PRB-0026]

[VOL02-C02-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 144

Ejercicio 11 [PRB-0027]

[VOL02-C02-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 11, verifique las precon-

diciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 185
Ejercicio 12 [PRB-0028]
[VOL02-C02-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre búsqueda binaria, invariantes de bucle y ordenación: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 192

Cálculos reproducidos

$(128 - 120) = 8$

C.3 Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

Problema resuelto [PRB-0029]
En el algoritmo de Karatsuba para multiplicar dos enteros de n dígitos dividiendo cada operando en dos mitades, se sustituyen las 4 multiplicaciones clásicas de tamaño $n/2$ por solo 3 multiplicaciones recursivas. Compare el número de multiplicaciones base en una profundidad de $k = 4$ niveles ($n = 16$ bloques) entre el método cuadrático clásico ($\$4^4\$$) y Karatsuba ($\$3^4\$$), calculando el ahorro neto de productos elementales.
Respuesta

175

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0031]
[VOL02-C03-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36
Ejercicio 2 [PRB-0032]
[VOL02-C03-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 34
Ejercicio 3 [PRB-0033]
[VOL02-C03-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 58
Ejercicio 4 [PRB-0034]
[VOL02-C03-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 52
Ejercicio 5 [PRB-0035]
[VOL02-C03-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 70

Ejercicio 6 [PRB-0036]
[VOL02-C03-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 100
Ejercicio 7 [PRB-0037]
[VOL02-C03-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 8 [PRB-0038]
[VOL02-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 129
Ejercicio 9 [PRB-0039]
[VOL02-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 156
Ejercicio 10 [PRB-0040]
[VOL02-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 162
Ejercicio 11 [PRB-0041]
[VOL02-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 205
Ejercicio 12 [PRB-0042]
[VOL02-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre divide y vencerás, algoritmos voraces, programación dinámica y backtracking: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 216

Cálculos reproducidos

$(256 - 81) = 175$

C.4 Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

Problema resuelto [PRB-0043]
En una tabla hash con direccionamiento abierto y sondeo lineal que se mantiene con un factor de carga $\alpha = 0{,}75$, calcule el número esperado de sondeos en una búsqueda sin éxito (inserción de clave nueva) según la fórmula de Knuth $E_{\text{miss}} = \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$.

Respuesta

8,5

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0045]
[VOL02-C04-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40

Ejercicio 2 [PRB-0046]
[VOL02-C04-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36

Ejercicio 3 [PRB-0047]
[VOL02-C04-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 64

Ejercicio 4 [PRB-0048]
[VOL02-C04-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 56

Ejercicio 5 [PRB-0049]
[VOL02-C04-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 78

Ejercicio 6 [PRB-0050]
[VOL02-C04-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 110

Ejercicio 7 [PRB-0051]
[VOL02-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120

Ejercicio 8 [PRB-0052]
[VOL02-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 143

Ejercicio 9 [PRB-0053]
[VOL02-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 172

Ejercicio 10 [PRB-0054]

[VOL02-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 180

Ejercicio 11 [PRB-0055]

[VOL02-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 225

Ejercicio 12 [PRB-0056]

[VOL02-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre algoritmos de grafos, árboles B+ y tablas hash: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 240

Cálculos reproducidos

$(0{,}5 \cdot (1 + 16)) = 8{,}5$

C.5 Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

Problema resuelto [PRB-0057]

Un algoritmo paralelo de reducción y filtrado tiene un trabajo total $T_1 = 9600$ ms y un camino crítico (span) $T_\infty = 40$ ms. Calcule la cota superior garantizada de tiempo de ejecución $T_P = T_1/P + T_\infty$ en $P = 16$ procesadores bajo un planificador voraz.

Respuesta

640

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0059]

[VOL02-C05-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 44

Ejercicio 2 [PRB-0060]

[VOL02-C05-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 38

Ejercicio 3 [PRB-0061]

[VOL02-C05-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 70

Ejercicio 4 [PRB-0062]

[VOL02-C05-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 60
Ejercicio 5 [PRB-0063]
[VOL02-C05-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 86
Ejercicio 6 [PRB-0064]
[VOL02-C05-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120
Ejercicio 7 [PRB-0065]
[VOL02-C05-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 132
Ejercicio 8 [PRB-0066]
[VOL02-C05-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 157
Ejercicio 9 [PRB-0067]
[VOL02-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 188
Ejercicio 10 [PRB-0068]
[VOL02-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 198
Ejercicio 11 [PRB-0069]
[VOL02-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 245
Ejercicio 12 [PRB-0070]
[VOL02-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre estructuras persistentes, filtros de Bloom y modelo Work-Span: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 264

Cálculos reproducidos

$(600 + 40) = 640$

C.6 Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

Problema resuelto [PRB-0071]
Calcule el ahorro total en kilobytes (1 KiB = 1024 bytes) de un vector en memoria de $N = 65536$ registros cuando se reordena en C una estructura mal alineada de 24 bytes (`uint8_t`, 7 bytes de padding, `uint64_t`, `uint8_t`, 7 bytes de padding) a su disposición óptima de 16 bytes (`uint64_t`, `uint8_t`, `uint8_t`, 6 bytes de padding final).
Respuesta

512

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0073]
[VOL02-C06-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 48
Ejercicio 2 [PRB-0074]
[VOL02-C06-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 3 [PRB-0075]
[VOL02-C06-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 76
Ejercicio 4 [PRB-0076]
[VOL02-C06-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(24 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 64
Ejercicio 5 [PRB-0077]
[VOL02-C06-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 94
Ejercicio 6 [PRB-0078]
[VOL02-C06-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 130
Ejercicio 7 [PRB-0079]
[VOL02-C06-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 144
Ejercicio 8 [PRB-0080]

[VOL02-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 171
Ejercicio 9 [PRB-0081]
[VOL02-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 204
Ejercicio 10 [PRB-0082]
[VOL02-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 216
Ejercicio 11 [PRB-0083]
[VOL02-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 265
Ejercicio 12 [PRB-0084]
[VOL02-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre cachés, protocolo MESI, false sharing, alineamiento ABI y enlazado: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `24 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 288

Cálculos reproducidos

$\frac{(65536 \cdot 8)}{1024} = 512$

C.7 Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

Problema resuelto [PRB-0085]
En un bloque básico antes de la optimización se evalúa en un bucle de $N = 1000$ iteraciones una expresión con una subexpresión invariante de bucle de coste $c_{\text{inv}} = 6$ ciclos y una parte dependiente del índice de coste $c_{\text{var}} = 3$ ciclos por iteración. Calcule el ahorro total de ciclos al aplicar el pase de Movimiento de Código Invariante de Bucle (LICM), que evalúa la subexpresión invariante una única vez fuera del bucle.
Respuesta

5994

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0087]
[VOL02-C07-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 52
Ejercicio 2 [PRB-0088]
[VOL02-C07-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 2, verifique las

precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 42

Ejercicio 3 [PRB-0089]

[VOL02-C07-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 82

Ejercicio 4 [PRB-0090]

[VOL02-C07-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(26 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 68

Ejercicio 5 [PRB-0091]

[VOL02-C07-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 102

Ejercicio 6 [PRB-0092]

[VOL02-C07-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 140

Ejercicio 7 [PRB-0093]

[VOL02-C07-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 156

Ejercicio 8 [PRB-0094]

[VOL02-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 185

Ejercicio 9 [PRB-0095]

[VOL02-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 220

Ejercicio 10 [PRB-0096]

[VOL02-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 234

Ejercicio 11 [PRB-0097]

[VOL02-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 285

Ejercicio 12 [PRB-0098]

[VOL02-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre lexing, parsing de Pratt, AST, forma SSA y optimizaciones de compilador: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `26 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 312

Cálculos reproducidos

$(9000 - 3006) = 5994$

C.8 Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

Problema resuelto [PRB-0099]
En una arquitectura con páginas de 4 KiB = 4096 bytes (2^{12} bytes), dada la dirección virtual decimal $V = 20500$, determine el número de página virtual VPN = $\lfloor V/4096 \rfloor$, el desplazamiento dentro de la página offset = $V - \text{VPN} \cdot 4096$, y calcule la dirección física resultante si la tabla de páginas asigna dicho VPN al marco físico PFN = 12.
Respuesta

49172

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0101]
[VOL02-C08-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 60
Ejercicio 2 [PRB-0102]
[VOL02-C08-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 3 [PRB-0103]
[VOL02-C08-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 94
Ejercicio 4 [PRB-0104]
[VOL02-C08-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(30 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 76
Ejercicio 5 [PRB-0105]
[VOL02-C08-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 118
Ejercicio 6 [PRB-0106]

[VOL02-C08-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 160
Ejercicio 7 [PRB-0107]
[VOL02-C08-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 180
Ejercicio 8 [PRB-0108]
[VOL02-C08-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 213
Ejercicio 9 [PRB-0109]
[VOL02-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 252
Ejercicio 10 [PRB-0110]
[VOL02-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 270
Ejercicio 11 [PRB-0111]
[VOL02-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 325
Ejercicio 12 [PRB-0112]
[VOL02-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas operativos, llamadas al sistema, memoria virtual paginada y epoll/kqueue: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `30 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 360

Cálculos reproducidos

$((12 \cdot 4096) + 20) = 49172$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 25 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios. Ecuaciones: 8 respaldada por una fuente, 11 valor o condición.
¶3: dos testimonios. Ecuaciones: 3 verificada simbólicamente, 1 reproducida numéricamente, 10 respaldada por una fuente, 2 valor o condición.
¶4: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶5: dos testimonios.
¶6: dos testimonios. Ecuaciones: 1 valor o condición.
¶7: dos testimonios.
¶8: dos testimonios. Ecuaciones: 4 valor o condición.
¶9: dos testimonios. Ecuaciones: 2 respaldada por una fuente, 2 valor o condición.
¶10: dos testimonios. Ecuaciones: 2 respaldada por una fuente, 1 valor o condición.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios. Ecuaciones: 1 aproximación.
¶15: dos testimonios.
¶16: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 valor o condición.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 valor o condición.
¶21: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶22: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 valor o condición.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶27: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 1 aproximación, 4 valor o condición.
¶28: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

MIT OpenCourseWare 6.006 — Introduction to Algorithms (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
The Rust Programming Language (Klabnik & Nichols — Official Rust Documentation) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
E. F. Codd — A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
C Programming Language Reference (ISO/IEC 9899 C17/C23 — cppreference) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
C++ Reference (ISO/IEC 14882 C++20/C++23 — cppreference) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Rust Reference — Memory Model, Type System, Lifetimes, and Unsafe Rust (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-complejidad-asintotica-recurrencias-amortizado scientific
REV-0002: cap02-busqueda-ordenacion-cotas-inferiores scientific
REV-0003: cap03-divide-venceras-voraces-programacion-dinamica scientific
REV-0004: cap04-grafos-arboles-equilibrados-hashing scientific
REV-0005: cap05-estructuras-persistentes-probabilisticas-paralelos scientific
REV-0006: cap06-jerarquia-memoria-caches-abi-enlazado scientific
REV-0007: cap07-compiladores-lexing-parsing-ssa-codegen scientific
REV-0008: cap08-sistemas-operativos-procesos-memoria-virtual-io scientific

Apéndice E

Plan de estudio

PARTE III. Algoritmos, Estructuras de Datos y Computación

✓Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

- Dominar Complejidad temporal y espacial, notaciones O, Omega y Theta, recurrencias, Teorema Maestro y análisis amortizado

2 conceptos planificados.

✓Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

- Dominar Búsqueda binaria libre de desbordamiento, ordenación, cota inferior Omega(n log n) e invariantes

2 conceptos planificados.

✓Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

- Dominar Divide y vencerás, algoritmos voraces, programación dinámica y backtracking

2 conceptos planificados.

✓Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

- Dominar Algoritmos de grafos (BFS, DFS, Dijkstra, Bellman-Ford, MST, flujo), árboles equilibrados (AVL, Rojo-Negro, B+) y hashing

2 conceptos planificados.

✓Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

- Dominar Estructuras de datos persistentes, estructuras probabilísticas (Bloom filter, HyperLogLog) y algoritmos paralelos (Work-Span)

2 conceptos planificados.

PARTE IV. Sistemas Informáticos, Compiladores, Memoria y Sistemas Operativos

✓Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

- Dominar Jerarquía de memoria, coherencia de caché (MESI), false sharing, alineamiento, ABI y enlazado

2 conceptos planificados.

✓Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

- Dominar Arquitectura de compiladores: análisis léxico, parsing LL/LR/Pratt, AST, tipos, IR en forma SSA y generación de código

2 conceptos planificados.

✓Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

- Dominar Sistemas operativos: modo kernel, procesos e hilos, planificación, memoria virtual paginada, VFS y multiplexación de E/S

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen III: Monografía Completa de Rust – Seguridad de Memoria, Concurrencia y Sistemas

Parte V (Bloque A–H): De cero a sistemas de alto rendimiento, programación asíncrona, unsafe y arquitectura

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Bloques A–C: Fundamentos, Entorno y Nivel Principiante en Rust 1

1. A. Introducción, historia, filosofía y modelo de compilación de Rust 2
1.1. Historia, problema que resuelve y filosofía de diseño 2
1.2. Modelo de ejecución, fases HIR/MIR/LLVM y comparativa con otros lenguajes 2

2. B–C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo 5
2.1. Bloque B: Toolchain oficial (rustup, rustc, cargo, clippy, rustfmt) y estructura de un crate 5
2.2. Bloque C: Variables, tipos primitivos, expresiones de control de flujo y funciones 5

II Bloque D: Nivel Intermedio – Propiedad, Tipos Algebraicos, Rasgos y Modularidad 8

3. D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes) 9
3.1. Las tres leyes de Ownership, semántica Move frente a Copy y el rasgo Drop 9
3.2. Reglas de Borrowing (&T vs &mut T), Non-Lexical Lifetimes (NLL) y parámetros de vida 'a 10

4. D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result) 12
4.1. Estructuras, enumeraciones algebraicas, optimización de nicho y coincidencia de patrones 12
4.2. Rasgos (Traits), genéricos, tipos asociados y manejo de errores con Option<T> y Result<T, E> 12

III Bloques E–F: Nivel Avanzado y Experto – Smart Pointers, Concurrencia, Async y Unsafe 15

5. E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior 16
5.1. Clausuras (Fn, FnMut, FnOnce) e iteradores perezosos de coste cero 16
5.2. Punteros inteligentes (Box, Rc, Weak) y mutabilidad interior (Cell, RefCell, Cow) 16

6. E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio) 19
6.1. Concurrencia basada en hilos: Send, Sync, thread::scope, Arc, Mutex y canales 19
6.2. Programación asíncrona: el rasgo Future, máquinas de estados, Pin/Unpin y el ejecutor Tokio 20

7. F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización 22
7.1. Rust inseguro (unsafe), contratos de seguridad, Miri e interoperabilidad FFI con C 22
7.2. Metaprogramación con macros, entornos empotrados (#![no_std]), WebAssembly y optimización SIMD 23

IV Bloques G−H: Ecosistema Industrial, Pruebas, Benchmarking y Proyectos Completos **25**

8. G−H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust **26**
8.1. Bloque G: Ecosistema de crates, Cargo Workspaces, idioms de diseño de APIs y antipatrones 26
8.2. Bloque H: Evaluación integral y los 4 proyectos progresivos (incluyendo rust-kv-engine verificado) 26

A. Notación **29**

B. Hoja de fórmulas **30**

C. Soluciones **31**
C.1. A. Introducción, historia, filosofía y modelo de compilación de Rust 31
C.2. B−C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo 31
C.3. D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes) 32
C.4. D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result) 33
C.5. E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior 33
C.6. E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio) 34
C.7. F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización 35
C.8. G−H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust . . 35

D. Auditoría de fuentes **37**

E. Plan de estudio **39**

Parte I

Bloques A–C: Fundamentos, Entorno y Nivel Principiante en Rust

Capítulo 1

A. Introducción, historia, filosofía y modelo de compilación de Rust

Este capítulo inaugura la monografía autónoma de Rust (Bloque A). Un lector sin experiencia previa en C o C++ comprenderá qué problema fundamental de la ingeniería de sistemas resuelve Rust: garantizar simultáneamente control predictivo sobre la representación de memoria y ausencia matemática de comportamiento indefinido (Undefined Behavior) en código seguro (`safe Rust`), sin recurrir a un recolector de basura en tiempo de ejecución.



Figura 1. Tubería del compilador rustc: análisis sintáctico, expansión de macros, HIR, inferencia de tipos, MIR (borrow checker NLL) y optimización LLVM.

Consejo

No luches contra el comprobador de préstamos intentando replicar grafos cíclicos de punteros mutables propios de Java o C++; modela la propiedad de los datos como árboles jerárquicos o utiliza índices tipados sobre vectores contiguos (arenas).

Definición

Seguridad espacial y temporal de memoria en Rust: propiedad verificada estáticamente en MIR según la cual ningún acceso seguro lee o escribe fuera de los límites asignados (seguridad espacial) ni accede a una región cuya vida útil ha concluido o está siendo mutada concurrentemente (seguridad temporal).

Autoficha

¿Por qué un puntero a objeto de rasgo `&dyn Write` ocupa 16 bytes en una arquitectura de 64 bits mientras que una referencia ordinaria `&File` ocupa únicamente 8 bytes?

1.1 Historia, problema que resuelve y filosofía de diseño

Iniciado en 2006 por Graydon Hoare y respaldado por Mozilla Research desde 2009 para construir el motor experimental Servo, Rust alcanzó su versión estable 1.0 en mayo de 2015 bajo un compromiso estricto de estabilidad sin estancamiento mediante el sistema de Ediciones (2015, 2018, 2021 y 2024). Las estadísticas de vulnerabilidades publicadas por Microsoft Security Response Center y el proyecto Chromium revelaron que aproximadamente el 70 % de las vulnerabilidades críticas de seguridad en grandes bases de código C y C++ proceden de defectos de seguridad de memoria: uso después de liberación (use-after-free), doble liberación (double free), desbordamientos de búfer, desreferenciación de punteros colgantes (dangling pointers) y carreras de datos (data races) entre hilos. Como se ilustra en la Figura \ref{fig:fig-rust-compiler-pipeline}, Rust traslada la verificación de estas invariantes desde el tiempo de ejecución hacia la representación intermedia de nivel medio (MIR, Mid-level Intermediate Representation) durante la compilación.

1.2 Modelo de ejecución, fases HIR/MIR/LLVM y comparativa con otros lenguajes

El principio de abstracciones de coste cero (zero-cost abstractions), formulado originalmente por Bjarne Stroustrup, establece que aquello que no se utiliza no conlleva coste alguno y aquello que sí se utiliza se

compila con una eficiencia igual o superior al código escrito manualmente a bajo nivel. En Rust, los tipos genéricos parametrizados $f\langle T : \text{Trait}\rangle(x : T)$ se compilan por defecto mediante monomorfización estática: el compilador genera una copia especializada de la función para cada tipo concreto invocado, eliminando la indirección de tablas virtuales (`vtable`) y permitiendo el *inlining* agresivo del optimizador LLVM:

```
// Compilado mediante monomorfización a instrucciones máquina directas sin vtable:
#[inline]
pub fn clamp_value<T: PartialOrd>(val: T, low: T, high: T) -> T {
    if val < low { low } else if val > high { high } else { val }
}
```

Cuando el despacho dinámico es deseable para reducir el tamaño del binario o almacenar colecciones heterogéneas, el programador lo solicita explícitamente mediante objetos de rasgo (`&dyn Trait` o `Box<dyn Trait>`), representados como un puntero ancho (*fat pointer*) de dos palabras máquina ($2 \times 8 = 16$ B en arquitecturas de 64 bits): una palabra para la dirección del dato y otra para la tabla virtual del rasgo.

Problema resuelto

Enunciado En una arquitectura de 64 bits ($W = 8$ B), un vector en Rust `Vec<T>` se representa en la pila mediante tres campos de tamaño palabra (`ptr`, `len`, `cap`), mientras que una rebanada `&[T]` es un puntero ancho de dos palabras (`ptr`, `len`). Calcula el tamaño total en pila en bytes de una estructura que contiene dos `Vec<u8>` y tres rebanadas `&[u8]`.

Datos

- $W = 8\,\text{B}$ — Tamaño de palabra en arquitectura de 64 bits
- $n_{vec} = 2$ — Número de vectores Vec<u8>
- $n_{slice} = 3$ — Número de rebanadas &[u8]

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Representación en pila de Vec<T> (3 palabras) y &[T] (2 palabras)

$$S = 2 \cdot (3 \cdot 8) + 3 \cdot (2 \cdot 8)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Tamaño de cada Vec<u8> en pila (3 * 8 = 24 B) y cada &[u8] (2 * 8 = 16 B)

$$S = 2 \cdot (3 \cdot 8) + 3 \cdot (2 \cdot 8)$$

$$((2 \cdot (3 \cdot 8)) + (3 \cdot (2 \cdot 8)))$$

Resultado

$$\mathit{Resultado} = 96. \mathit{Laestructuraocupaexactamente96bytesenlapila}(48\mathit{Bdelosdosvectoresy}48\mathit{Bdelastresrebanadas}), \mathit{indep}$$

Interpretación La estructura ocupa exactamente 96 bytes en la pila (48 B de los dos vectores y 48 B de las tres rebanadas), independientemente del número de elementos alojados en el montículo.

Comprobación de los pasos

- Paso 1: 96 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0004] (Básico, Numérico) Calcula el tamaño en bytes en la pila de un `Vec<f64>` en una arquitectura de 64 bits (3×8).
2 [PRB-0005] (Básico, Numérico) Calcula el tamaño en bytes de un puntero ancho `&dyn std::fmt::Display` en una CPU de 64 bits (2×8).
3 [PRB-0006] (Intermedio, Desarrollo por pasos) Si un binario contiene 45 funciones genéricas instanciadas cada una para 4 tipos concretos distintos mediante monomorfización, ¿cuántas funciones especializadas se emiten antes de la eliminación de código muerto?
4 [PRB-0007] (Intermedio, Diseño) Un búfer de 4096 elementos `u32` (4 bytes por elemento) está referenciado por un `Vec<u32>` (24 bytes en pila). ¿Cuántos bytes totales suma la cabecera en pila más la región en el montículo?

Capítulo 2

B–C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo

Este capítulo cubre los Bloques B y C de la plantilla estandarizada: desde la configuración profesional del entorno (`rustup`, `cargo`, `Cargo.toml`, `Cargo.lock`, `rustfmt`, `clippy`, `rust-analyzer`) hasta el dominio completo del nivel principiante de Rust. A diferencia de los lenguajes imperativos clásicos donde casi todas las construcciones son sentencias sin valor, Rust es un lenguaje orientado a expresiones donde `if`, `match` y los bloques `{ ... }` producen valores tipados verificados estáticamente.

Consejo

Prefiere siempre `checked_add`, `saturating_add` o `wrapping_add` cuando implementes protocolos binarios, criptografía o contadores de sistemas para que el comportamiento ante desbordamiento no dependa del perfil de compilación.

Definición

Expresión de bloque en Rust: secuencia de sentencias encerrada entre llaves `{ s_1; s_2; ...; e }` cuyo tipo y valor resultante coinciden con los de la expresión final `e` (sin punto y coma), o con el tipo unidad `()` si termina en punto y coma.

Autoficha

¿Qué diferencia fundamental existe entre sombrear una variable (`let x = x + 1;`) y declararla mutable (`let mut x = 0; x += 1;`) en cuanto a la posibilidad de cambiar su tipo estático?

2.1 Bloque B: Toolchain oficial (rustup, rustc, cargo, clippy, rustfmt) y estructura de un crate

La herramienta `rustup` administra canales de distribución (`stable`, `beta`, `nightly`) y objetivos de compilación cruzada (`targets`, como `aarch64-apple-darwin`, `x86_64-unknown-linux-gnu` o `wasm32-unknown-unknown`). Todo paquete en Rust se denomina *crate* (binario o biblioteca) y se gobierna mediante el manifiesto declarativo `Cargo.toml`. Mientras que `Cargo.toml` expresa rangos semánticos compatibles (SemVer, por ejemplo `serde = "1.0"`), el fichero `Cargo.lock` registra las versiones exactas y los resúmenes criptográficos SHA-256 de cada dependencia, garantizando compilaciones bit a bit reproducibles. Los comandos esenciales del ciclo de ingeniería son `cargo check` (verificación rápida de tipos y préstamos en MIR sin invocar el generador de código LLVM), `cargo test` (ejecución concurrente de pruebas unitarias, de integración y de documentación), `cargo clippy - -D warnings` (análisis estático con más de 700 reglas idiomáticas y de rendimiento) y `cargo fmt -check`.

2.2 Bloque C: Variables, tipos primitivos, expresiones de control de flujo y funciones

En Rust, las vinculaciones introducidas con `let x: i32 = 42;` son inmutables por defecto; la mutabilidad requiere la marca explícita `let mut contador: u64 = 0;`. El lenguaje proporciona enteros con signo (`i8`, `i16`, `i32`, `i64`, `i128`, `isize`), enteros sin signo (`u8`, `u16`, `u32`, `u64`, `u128`, `usize`), flotantes IEEE 754 (`f32`, `f64`), el tipo booleano `bool` (1 byte) y el tipo `char` de 4 bytes que representa exactamente un valor escalar Unicode (U+0000 a U+D7FF y U+E000 a U+10FFFF). En caso de desbordamiento entero, el perfil `dev` genera un *panic* determinista, mientras que el lenguaje ofrece métodos explícitos para cada semántica aritmética: `checked_add` (devuelve `Option<T>`), `wrapping_add` (aritmética modular mód 2^w), `saturating_add` (satura en los extremos) y `overflowing_add`:

```
pub fn classify_http_status(code: u16) -> &'static str {
    match code {
        100..=199 => "informational",
        200..=299 => "success",
        300..=399 => "redirection",
        400..=499 => "client_error",
        500..=599 => "server_error",
        _ => "unknown",
    }
}
```

Observa que `match` exige exhaustividad verificada por el compilador: si se omitiera el brazo comodín `_`, `rustc` rechazaría el programa indicando exactamente los rangos de `u16` no cubiertos (`0..=99` y `600..=65535`).

Problema resuelto

Enunciado Una cadena en Rust contiene 10 caracteres ASCII (1 B cada uno en UTF-8) y 5 caracteres acentuados españoles (`á`, `é`, `í`, `ó`, `ú`, que ocupan 2 B cada uno en UTF-8). Calcula la diferencia en bytes entre representarla como `&str` en UTF-8 frente a un array `[char; 15]` de valores escalares Unicode (4 B por `char`).

Datos

- $N_{ascii} = 10$ — Caracteres ASCII de 1 byte en UTF-8
- $N_{lat} = 5$ — Caracteres latinos acentuados de 2 bytes en UTF-8
- $S_{char} = 4\,\mathrm{B}$ — Tamaño fijo del tipo char en Rust

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Comparación de almacenamiento UTF-8 (&str) frente a array de escalares Unicode [char; N]

$$\Delta B = 15 \cdot 4 - (10 \cdot 1 + 5 \cdot 2)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos los 20 bytes de la codificación UTF-8 a los 60 bytes del array [char; 15]

$$\Delta B = 15 \cdot 4 - (10 \cdot 1 + 5 \cdot 2)$$

$$((15 \cdot 4) - ((10 \cdot 1) + (5 \cdot 2)))$$

Resultado

$$Resultado = 40. La representación UTF-8 nativa de Rusta ahorra 40 bytes (ocupa 20 B frente a 60 B), razón por la cual String$$

Interpretación La representación UTF-8 nativa de Rust ahorra 40 bytes (ocupa 20 B frente a 60 B), razón por la cual String y &str utilizan UTF-8 en lugar de arrays de 32 bits.

Comprobación de los pasos

- Paso 1: 40 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0011] (Básico, Numérico) Calcula el tamaño en bytes de un array `[u64; 32]` alojado en la pila (32×8).
2 [PRB-0012] (Básico, Desarrollo por pasos) Una cadena UTF-8 contiene 8 caracteres ASCII (1 B), 4 letras griegas (2 B) y 2 emojis (4 B). ¿Cuál es el valor de `s.len()` en bytes?
3 [PRB-0013] (Intermedio, Desarrollo por pasos) Calcula el resultado de `250u8.wrapping_add(15)` en aritmética modular de 8 bits ($250 + 15 - 256$).
4 [PRB-0014] (Intermedio, Diseño) Si `Vec::with_capacity(128)` reserva espacio para 128 elementos `i32` (4 B cada uno) e insertamos 50 elementos, ¿cuántos bytes de capacidad reservada permanecen aún libres en el montículo?

Parte II

Bloque D: Nivel Intermedio – Propiedad, Tipos Algebraicos, Rasgos y Modularidad

Capítulo 3

D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes)

*Este capítulo desarrolla el corazón teórico y práctico de Rust: su sistema de tipos afín basado en propiedad única (*ownership*), préstamo (*borrowing*) y regiones de tiempo de vida (*lifetimes*). Comprender este modelo no solo permite escribir Rust sin fricción, sino que transforma permanentemente la manera en que un ingeniero razona sobre la vida útil de los recursos y la concurrencia en cualquier lenguaje.*

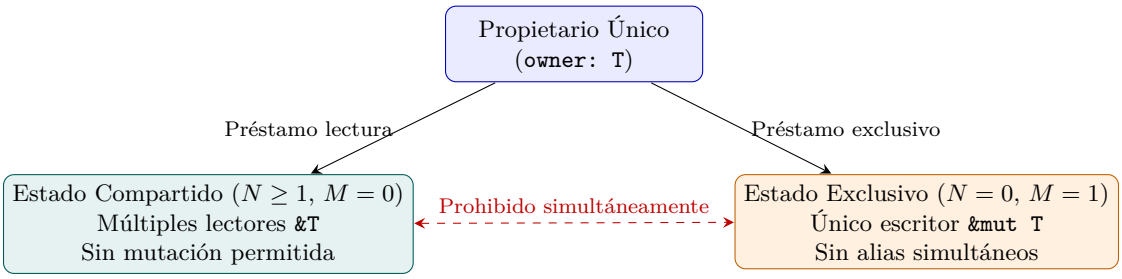


Figura 2. Invariante fundamental del comprobador de préstamos: en cualquier punto del flujo de control MIR, un recurso admite múltiples préstamos inmutables (&T) o exactamente un préstamo exclusivo mutable (&mut T).

Consejo

Recuerda que las anotaciones `'a` nunca prolongan la vida de un valor en memoria: únicamente declaran al compilador cómo se relaciona la vida útil de la referencia de salida con la de las referencias de entrada.

Definición

Parámetro de tiempo de vida `'a`: variable de región en el sistema de tipos de Rust que restringe estáticamente el ámbito durante el cual una referencia prestada `&'a T` o `&'a mut T` es válida, garantizando que el referente sobrevive a toda referencia.

Autoficha

¿Por qué la firma `fn dangling() -> &str { let s = String::from("hola"); &s }` es rechazada en compilación independientemente del tiempo de vida que se intente anotar?

3.1 Las tres leyes de Ownership, semántica Move frente a Copy y el rasgo Drop

El sistema de propiedad de Rust se rige por tres axiomas verificados en compilación: (1) cada valor en Rust tiene una variable denominada su *propietario* (*owner*); (2) solo puede existir un único propietario a la vez; y (3) cuando el propietario sale de su ámbito léxico (*scope*), el valor se destruye invocando determinísticamente `Drop::drop` (patrón RAII, *Resource Acquisition Is Initialization*). Cuando asignamos o pasamos por valor un tipo que gestiona memoria en el montículo como `String` o `Vec<T>`, Rust ejecuta una transferencia de propiedad (*move*): copia los 24 bytes de la cabecera en la pila e invalida estáticamente la variable de origen en el grafo de flujo MIR, impidiendo cualquier doble liberación (*double free*). Por el contrario, los tipos escalares simples que residen íntegramente en la pila implementan el

rasgo marcador `Copy`, por lo que la asignación duplica sus bits sin invalidar el origen. Un tipo solo puede implementar `Copy` si ninguno de sus componentes implementa `Drop`.

3.2 Reglas de Borrowing (&T vs &mut T), Non-Lexical Lifetimes (NLL) y parámetros de vida 'a

Para operar sobre un valor sin transferir su propiedad, Rust introduce las referencias (*borrowing*). Como muestra la Figura \ref{fig:fig-rust-aliasing-xor-mutability}, en cualquier punto de ejecución del grafo de control MIR para un recurso dado, el número de referencias inmutables activas N_{shared} y el número de referencias mutables activas M_{mut} satisfacen la invariante de exclusión:

$$(N_{\text{shared}} \geq 0 \land M_{\text{mut}} = 0) \lor (N_{\text{shared}} = 0 \land M_{\text{mut}} = 1)$$

Esta ley de *Alias XOR Mutabilidad* elimina de raíz tanto las carreras de datos entre hilos como la invalidación de iteradores en un único hilo (por ejemplo, intentar hacer `vec.push(x)` mientras se mantiene una referencia `&vec[0]` que quedaría colgante si el vector realoja su búfer en el montículo). Cuando una función devuelve una referencia derivada de sus parámetros de entrada, el parámetro de tiempo de vida `'a` expresa la relación de inclusión de regiones $'a \subseteq \text{vida}(x) \cap \text{vida}(y)$, sin alterar jamás la duración real en tiempo de ejecución:

```
pub fn longest_token<'a>(left: &'a str, right: &'a str) -> &'a str {
    if left.len() >= right.len() { left } else { right }
}
```

El compilador aplica tres reglas deterministas de *elisión de tiempos de vida*: (1) cada parámetro por referencia recibe su propio parámetro `'a`, `'b`; (2) si hay exactamente un tiempo de vida de entrada, se asigna a todas las referencias de salida; y (3) si uno de los parámetros es `&self` o `&mut self`, su tiempo de vida se asigna a todas las referencias de salida.

Problema resuelto

Enunciado Un vector `Vec<u64>` contiene $N = 10000$ enteros de 8 bytes en el montículo en una máquina de 64 bits ($W = 8$ B). Calcula cuántos bytes menos se copian al pasar el vector por transferencia de propiedad (*move*, que solo copia las 3 palabras de la cabecera en pila) frente a invocar `.clone()` (que copia la cabecera más los 10000×8 bytes del montículo).

Datos

- $N = 10000$ — Número de elementos u64 en el vector
- $S_T = 8\,\text{B}$ — Tamaño de cada elemento u64
- $W = 8\,\text{B}$ — Tamaño de palabra en arquitectura de 64 bits

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Diferencia de bytes copiados entre Move (O(1)) y Clone (O(N))

$$\Delta B = (3 \cdot 8 + 10000 \cdot 8) - 3 \cdot 8$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el coste de move (24 B) al coste total de clone (80024 B)

$$\Delta B = (3 \cdot 8 + 10000 \cdot 8) - 3 \cdot 8$$

$$(((3 \cdot 8) + (10000 \cdot 8)) - (3 \cdot 8))$$

Resultado

$Resultado = 80000. La transferencia de propiedad (move) ahorra la copia de 80000 bytes en el montículo y se ejecuta en tiempo$

Interpretación La transferencia de propiedad (move) ahorra la copia de 80 000 bytes en el montículo y se ejecuta en tiempo constante O(1) copiando únicamente 24 bytes.
Comprobación de los pasos

- Paso 1: 80000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0018] (Básico, Numérico) Calcula cuántos bytes en la pila se copian al mover un `String` en una arquitectura de 64 bits (3×8).
2 [PRB-0019] (Básico, Numérico) Si dividimos una rebanada de 1024 elementos mediante `split_at_mut(300)`, ¿cuántos elementos contiene la segunda sub-rebanada mutable?
3 [PRB-0020] (Intermedio, Desarrollo por pasos) Una estructura contiene 4 campos `u64` que implementan `Copy`. ¿Cuántos bytes se copian en cada asignación por valor (4×8)?
4 [PRB-0021] (Intermedio, Diseño) Un analizador léxico de copia cero (*zero-copy*) genera 500 tokens `&'a str` (16 B cada uno) sobre un texto fuente de 32 000 bytes sin clonar subcadenas. ¿Cuántos bytes ocupan en total los 500 descriptores de token?

Capítulo 4

D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result)

*Este capítulo completa el Nivel Intermedio (Bloque D) profundizando en el sistema de tipos algebraicos, el polimorfismo basado en rasgos (*traits*) y la arquitectura de manejo de errores de Rust. A diferencia de los lenguajes con referencias nulas implícitas o excepciones no comprobadas, Rust codifica la ausencia de valor (`Option<T>`) y la posibilidad de fallo (`Result<T, E>`) directamente en las firmas de tipos.*

Consejo

En bibliotecas reutilizables, define tipos de error estructurados como `enum` que implementen `std::error::Error` y `Display` (o utiliza `thiserror`), reservando errores opacos tipo `Box<dyn Error>` o `anyhow` para la capa superior binaria.

Definición

Optimización de nicho (Niche Optimization): transformación de representación mediante la cual el compilador de Rust codifica variantes de un tipo suma (`enum`) utilizando patrones de bits inválidos (nichos) del tipo contenido, eliminando la necesidad de un byte discriminante adicional.

Autoficha

¿Cuándo debe un rasgo utilizar un tipo asociado (`trait Iterator { type Item; }`) en lugar de un parámetro de tipo genérico (`trait From<T>`)?

4.1 Estructuras, enumeraciones algebraicas, optimización de nicho y coincidencia de patrones

Un `struct` en Rust define un tipo producto cuya cardinalidad es el producto de los dominios de sus campos, mientras que un `enum` define un tipo suma etiquetado (*tagged union*) donde cada variante puede transportar datos de tipos distintos: por ejemplo, `enum IpPacket { V4([u8; 4], u16), V6([u16; 8], u16), Dropped }`. El tamaño en memoria de un `enum` es el tamaño de su variante más grande más el discriminante (etiqueta), redondeado al múltiplo del alineamiento máximo. Sin embargo, el compilador aplica la *optimización de nicho* (*null pointer / niche optimization*): como una referencia `&T`, un `Box<T>` o un `NonZeroU64` jamás pueden tener el patrón de bits cero, `Option<&T>` utiliza el valor cero para representar `None`, por lo que sizeof(Option$\langle \&T \rangle$) = sizeof($\&T$) = 8 B, con sobrecoste espacial exactamente nulo.

4.2 Rasgos (Traits), genéricos, tipos asociados y manejo de errores con Option<T> y Result<T, E>

Los rasgos (*traits*) definen interfaces de comportamiento compartido que admiten implementaciones por defecto, parámetros genéricos y tipos asociados (`type Item;`). Para garantizar la coherencia global de tipos al enlazar múltiples crates, Rust impone la *regla del huérfano* (*orphan rule*): solo se puede implementar un rasgo `Trait` para un tipo `Type` en el crate actual si al menos uno de los dos (`Trait` o `Type`) está definido localmente en dicho crate. En el tratamiento de errores, `Result<T, E>` se combina con el operador de propagación `?`, que desazucara una expresión `expr?` en una evaluación exhaustiva que convierte automáticamente el error mediante `From::from(err)`:

```
#[derive(Debug, PartialEq)]
pub enum ConfigError { EmptyInput, InvalidPort(String) }

pub fn parse_port_str(raw: &str) -> Result<u16, ConfigError> {
    let trimmed = raw.trim();
    if trimmed.is_empty() {
        return Err(ConfigError::EmptyInput);
    }
    let port: u16 = trimmed.parse().map_err(|_| ConfigError::InvalidPort(trimmed.to_string()))
    ?;
    if port == 0 { Err(ConfigError::InvalidPort(trimmed.to_string())) } else { Ok(port) }
}
```

Esta disciplina distingue con nitidez entre errores recuperables (modelados con `Result<T, E>`) y violaciones de invariantes de programación irrecuperables (modeladas con `panic!`).

Problema resuelto

Enunciado Una tabla hash abierta almacena $N = 65536$ ranuras de tipo `Option<std::num::NonZeroU64>` en una arquitectura de 64 bits ($W = 8$ B). Gracias a la optimización de nicho, cada `Option<NonZeroU64>` ocupa exactamente 8 bytes en lugar de 16 bytes (8 bytes de dato más 8 bytes de discriminante alineado). Calcula cuántos kilobytes (dividiendo entre 1024) se ahorran en la tabla completa gracias a la optimización de nicho.

Datos

- $N = 65536$ — Número de ranuras en la tabla
- $S_{sin} = 16\,\mathrm{B}$ — Tamaño alineado sin optimización de nicho
- $S_{con} = 8\,\mathrm{B}$ — Tamaño con optimización de nicho

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria por optimización de nicho en Option<NonZeroU64>

$$\Delta M_{\mathrm{KiB}} = \frac{65536 \cdot (16 - 8)}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 65536 ranuras por los 8 B ahorrados y dividimos entre 1024 B/KiB

$$\Delta M_{\mathrm{KiB}} = \frac{65536 \cdot (16 - 8)}{1024}$$

$$\frac{(65536 \cdot (16 - 8))}{1024}$$

Resultado

$$Resultado = 512. La optimización de nicho ahorra exactamente 512 KiB de memoria caché/RAM y duplica el número de entr$$

Interpretación La optimización de nicho ahorra exactamente 512 KiB de memoria caché/RAM y duplica el número de entradas por línea de caché de 64 bytes.

Comprobación de los pasos

- Paso 1: 512 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0025**] **(Básico, Numérico)** En una arquitectura de 64 bits, ¿cuántos bytes ocupa `Option<Box<u64»` gracias a la optimización de nicho?
2 [**PRB-0026**] **(Básico, Numérico)** ¿Cuántos bytes ocupa en total un array `[Option<NonZeroU32>; 1000]` sabiendo que cada elemento ocupa 4 bytes?
3 [**PRB-0027**] **(Intermedio, Desarrollo por pasos)** Un `enum` sin nicho tiene una variante de 24 bytes con alineamiento de 8 bytes y una etiqueta de 1 byte. Tras añadir 7 bytes de relleno (*padding*) para mantener el alineamiento a múltiplo de 8, ¿cuál es su tamaño total en bytes?
4 [**PRB-0028**] **(Intermedio, Diseño)** Una línea de caché de CPU mide 64 bytes. ¿Cuántos elementos `Option<NonZeroU64>` (8 B cada uno) caben en una única línea de caché?

Parte III

Bloques E–F: Nivel Avanzado y Experto – Smart Pointers, Concurrencia, Async y Unsafe

Capítulo 5

E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior

Este capítulo abre el Nivel Avanzado (Bloque E). Estudiaremos cómo Rust implementa la programación funcional mediante estructuras anónimas de clausura e iteradores fusionados en compilación, y cómo permite relajar las restricciones estáticas de propiedad única cuando un algoritmo requiere grafos compartidos o cachés internas mediante punteros inteligentes y contenedores de mutabilidad interior.

Consejo

Para evitar fugas de memoria en estructuras doblemente enlazadas o árboles con puntero al nodo padre, utiliza `Rc<Node>` de padre a hijo y `Weak<Node>` de hijo a padre, ya que `Weak<T>` no incrementa el `strong_count`.

Definición

Mutabilidad interior (Interior Mutability): patrón de diseño seguro en Rust mediante el cual una estructura expone una interfaz inmutable (`&self`) pero permite mutar su estado interno encapsulado en `UnsafeCell<T>` garantizando la invariante de exclusión mediante transferencia de valores (`Cell<T>`), contadores dinámicos (`RefCell<T>`) o sincronización atómica (`Mutex<T>`, `RwLock<T>`).

Autoficha

¿Por qué `Rc<T>` no implementa el rasgo `Send` y qué ocurriría en hardware multinúcleo si dos hilos ejecutaran `Rc::clone` simultáneamente sobre el mismo contador no atómico?

5.1 Clausuras (Fn, FnMut, FnOnce) e iteradores perezosos de coste cero

Cada clausura `|x| x + umbral` en Rust genera un tipo `struct` anónimo único que captura las variables libres de su entorno por el modo menos restrictivo posible: por referencia compartida `&T` (implementa `Fn`, `FnMut` y `FnOnce`), por referencia mutable `&mut T` (implementa `FnMut` y `FnOnce`) o por valor consumiendo la variable (implementa únicamente `FnOnce`, o cuando se fuerza la captura con la palabra clave `move`). Los iteradores en Rust implementan el rasgo `Iterator { type Item; fn next(&mut self) -> Option<Self::Item>; }`. Adaptadores como `.map(...)`, `.filter(...)`, `.take(...)` y `.enumerate()` son perezosos (*lazy*): no ejecutan ningún trabajo ni alojan vectores intermedios hasta que un consumidor terminal como `.fold(...)`, `.sum()` o `.collect::<Vec<_>>()` solicita los elementos. Gracias a la monomorfización, LLVM fusiona toda la cadena de adaptadores en un único bucle registrador sin comprobaciones de límites redundantes.

5.2 Punteros inteligentes (Box, Rc, Weak) y mutabilidad interior (Cell, RefCell, Cow)

Un puntero inteligente (*smart pointer*) es una estructura que implementa los rasgos `Deref` y `Drop`. `Box<T>` proporciona propiedad única en el montículo (indispensable para tipos recursivos de tamaño finito como árboles sintácticos `enum Expr { Add(Box<Expr>, Box<Expr>), Lit(i64) }`). `Rc<T>` (*Reference Counted*) habilita propiedad compartida en un único hilo manteniendo un contador de referencias fuertes (`strong_count`) y débiles (`weak_count`); como `Rc<T>` no utiliza instrucciones atómicas, el compilador le

niega los rasgos `Send` y `Sync`, impidiendo estáticamente que cruce a otro hilo. Cuando se necesita mutar un dato a través de una referencia compartida `&self`, Rust ofrece *mutabilidad interior*: `Cell<T>` opera por reemplazo de valores sin referencias internas (coste cero en espacio), mientras que `RefCell<T>` traslada las reglas del comprobador de préstamos al tiempo de ejecución manteniendo un contador de préstamos (`borrow()` e `borrow_mut()`):

```
use std::cell::RefCell;
use std::rc::Rc;

pub fn shared_accumulator_demo() -> i64 {
    let acc = Rc::new(RefCell::new(10_i64));
    let handle_a = Rc::clone(&acc);
    let handle_b = Rc::clone(&acc);
    *handle_a.borrow_mut() += 15;
    *handle_b.borrow_mut() *= 2;
    let final_val = *acc.borrow();
    final_val
}
```

Si un programa intenta adquirir un `borrow_mut()` mientras otro préstamo de la misma `RefCell<T>` sigue vivo, `RefCell` dispara un *panic* inmediato en lugar de permitir corrupción de memoria.

Problema resuelto

Enunciado En una arquitectura de 64 bits ($W = 8$ B), se aloja un bloque `Rc< [u8; 1024] >` ($S_T = 1024$ B) y se crean $k = 5$ manejadores `Rc::clone` en distintas estructuras en la pila. Calcula la memoria total consumida (los 5 punteros en la pila más el único bloque `RcBox` en el montículo con sus 2 contadores `usize` y los 1024 bytes de datos).

Datos

- $W = 8\,\mathrm{B}$ — Tamaño de usize y puntero en 64 bits
- $k = 5$ — Número de manejadores Rc<T> en la pila
- $S_T = 1024\,\mathrm{B}$ — Tamaño del valor compartido en el montículo

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Consumo de memoria de Rc<T> con k referencias compartidas

$$M = 5 \cdot 8 + (2 \cdot 8 + 1024)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los 5 punteros de 8 B en pila más el RcBox (16 B de contadores + 1024 B de datos)

$$M = 5 \cdot 8 + (2 \cdot 8 + 1024)$$

$$((5 \cdot 8) + ((2 \cdot 8) + 1024))$$

Resultado

$Resultado = 1080. El conjunto completo ocupa 1080 bytes (40 B en la pila y 1040 B en el montículo), ahorrando 4096 bytes frente$

Interpretación El conjunto completo ocupa 1080 bytes (40 B en la pila y 1040 B en el montículo), ahorrando 4096 bytes frente a clonar 5 veces el búfer.

Comprobación de los pasos

- Paso 1: 1080 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0032] (Básico, Numérico) Calcula la suma de los cuadrados de los números pares del 2 al 10 ($\$2^2 + 4^2 + 6^2 + 8^2 + 10^2\$$) evaluada por `(1..=10).filter(|x| x % 2 == 0).map(|x| x * x).sum()`.

2 [PRB-0033] (Básico, Numérico) ¿Cuántos bytes de metadatos añaden los dos contadores (`strong` y `weak`) de `RcBox<T>` en una arquitectura de 64 bits (2×8)?

3 [PRB-0034] (Intermedio, Desarrollo por pasos) Si se crean 1 `Rc` original, 4 clones fuertes (`Rc::clone`) y 3 referencias débiles (`Rc::downgrade`), y luego se destruyen 2 clones fuertes, ¿cuál es el valor actual de `Rc::strong_count`?

4 [PRB-0035] (Intermedio, Diseño) Un flujo de datos procesa 5000 enteros mediante `iter().filter(...).map(...).fold(...)` sin llamar a `collect()`. ¿Cuántos vectores dinámicos intermedios se alojan en el montículo durante el recorrido?

Capítulo 6

E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio)

Este capítulo aborda la concurrencia paralela y la asincronía de alto rendimiento en Rust. Veremos por qué el mismo sistema de tipos que previene punteros colgantes en un hilo es capaz de impedir toda carrera de datos entre múltiples hilos mediante los rasgos marcadores `Send` *y* `Sync`*, y cómo Rust compila funciones* `async fn` *en máquinas de estados compactas sin requerir un alojamiento en el montículo por cada llamada asíncrona.*

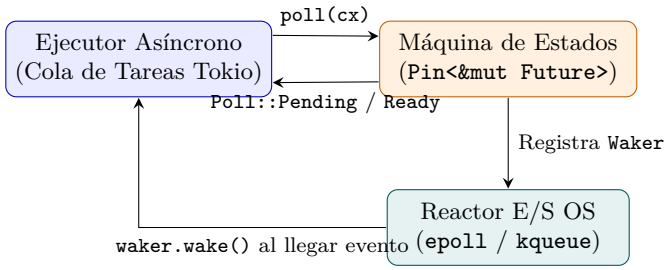


Figura 3. Modelo de ejecución de Future en Rust: el ejecutor (Tokio) invoca poll(Pin<&mut Self>, &mut Context); si el recurso E/S no está listo devuelve Poll::Pending y registra el Waker en el reactor del sistema operativo (epoll/kqueue).

Consejo

En código asíncrono sobre Tokio, nunca mantengas bloqueado un `std::sync::MutexGuard` a través de un punto `.await` ni ejecutes cómputo intensivo de CPU sin ceder el control; usa bloques cortos sin `.await` o delega el cómputo pesado a `tokio::task::spawn_blocking`.

Definición

Fijación de dirección (`Pin<P>`): contrato del sistema de tipos de Rust que garantiza que el valor apuntado por el puntero `P` no será movido jamás a otra dirección de memoria mientras no implemente `Unpin`, hasta que se ejecute su destructor `Drop::drop`.

Autoficha

¿Por qué en Rust un `Mutex<T>` es un contenedor genérico que envuelve al dato `T` en lugar de ser un objeto cerrojo separado del dato como en C++ (`std::mutex`) o Java?

6.1 Concurrencia basada en hilos: Send, Sync, thread::scope, Arc, Mutex y canales

Dos rasgos marcadores automáticos gobiernan toda la concurrencia en Rust: un tipo `T` es `Send` si su propiedad puede transferirse de manera segura a otro hilo, y es `Sync` si es seguro compartirlo entre varios hilos simultáneamente por referencia inmutable (es decir, `T: Sync` si y solo si `&T: Send`). Mientras que `std::thread::spawn` exige una clausura `'static + Send` porque el hilo hijo podría sobrevivir a la función que lo creó, `std::thread::scope` permite lanzar hilos que toman prestadas referencias locales no `'static` del marco de pila padre, garantizando estáticamente que todos los hilos del ámbito terminan antes de que `scope` retorne:

```
use std::thread;

pub fn parallel_dot_product(a: &[i64], b: &[i64]) -> i64 {
    let mid = a.len() / 2;
    let (a_left, a_right) = a.split_at(mid);
    let (b_left, b_right) = b.split_at(mid);
    thread::scope(|s| {
        let left_handle = s.spawn(|| a_left.iter().zip(b_left).map(|(x, y)| x * y).sum::<i64
    >());
        let right_sum: i64 = a_right.iter().zip(b_right).map(|(x, y)| x * y).sum();
        left_handle.join().unwrap() + right_sum
    })
}
```

Cuando varios hilos deben compartir y mutar estado sin un ámbito estático común, se combina el contador atómico de referencias `Arc<T>` con un cerrojo que envuelve directamente al dato protegido (`Mutex<T>` o `RwLock<T>`): en Rust es sintácticamente imposible acceder al dato sin adquirir primero el `MutexGuard`.

6.2 Programación asíncrona: el rasgo Future, máquinas de estados, Pin/Unpin y el ejecutor Tokio

Para cargas masivas de entrada/salida de red con cientos de miles de conexiones simultáneas, crear un hilo del sistema operativo por conexión resulta prohibitivo. En lugar de ello, una función `async fn` en Rust devuelve un valor que implementa el rasgo `Future { type Output; fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>; }`. Como ilustra la Figura \ref{fig:fig-rust-async-poll-state-machine}, los futuros en Rust son perezosos y dirigidos por sondeo (*poll-driven*): el compilador transforma el cuerpo de una `async fn` en un `enum` donde cada punto `.await` es un estado que guarda únicamente las variables vivas a través de esa suspensión. Dado que una variable local puede ser prestada a través de un punto `.await`, la máquina de estados generada es *autorreferencial* (un campo del `enum` contiene un puntero a otro campo del mismo `enum`). Si esa estructura se moviera de dirección en memoria después del primer `poll`, el puntero interno quedaría colgante; por ello `Future::poll` recibe `self: Pin<&mut Self>`, un envoltorio de tipos que prohíbe obtener `&mut T` para mover el valor fuera de su dirección fijada salvo que el tipo implemente `Unpin`.

Problema resuelto

Enunciado Un servidor de mensajería mantiene $C = 50000$ conexiones TCP simultáneas. Si cada máquina de estados `Future` en Tokio ocupa 256 B (0,25 KiB) mientras que una pila mínima de hilo cuesta 64 KiB, calcula la memoria total en MiB (dividiendo los KiB entre 1024) que consumen las 50 000 tareas asíncronas.

Datos

- $C = 50000$ — Conexiones concurrentes activas
- $S_{future} = 256\,\mathrm{B}$ — Tamaño de cada máquina de estados Future
- $B_{MiB} = 1048576\,\mathrm{B\,MiB^{-1}}$ — Bytes por mebibyte (1024 * 1024)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Huella de memoria en MiB de C máquinas de estados Future

$$M_{\mathrm{MiB}} = \frac{50000 \cdot 256}{1024 \cdot 1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 50000 tareas por 256 B y dividimos entre 1024*1024

$$M_{\text{MiB}} = \frac{50000 \cdot 256}{1024 \cdot 1024}$$

$$\frac{(50000 \cdot 256)}{(1024 \cdot 1024)}$$

Resultado

$Resultado = 12{,}20703125. Las 50000 conexiones asíncronas consumen tan solo 12{,}20703125 MiB de RAM, frente a los 3125 M$

Interpretación Las 50 000 conexiones asíncronas consumen tan solo 12.20703125 MiB de RAM, frente a los 3125 MiB (> 3 GiB) que exigirían 50 000 pilas de 64 KiB.
Comprobación de los pasos

- Paso 1: 12,207 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0039] (Básico, Numérico) Cuatro hilos incrementan un `AtomicU64` exactamente 25 000 veces cada uno con `fetch_add(1, Ordering::Relaxed)`. ¿Cuál es el valor final exacto del contador?
2 [PRB-0040] (Básico, Numérico) Un vector de 10 000 elementos se divide equitativamente entre 8 hilos mediante `chunks(1250)` dentro de `std::thread::scope`. ¿Cuántos elementos procesa cada hilo?
3 [PRB-0041] (Intermedio, Desarrollo por pasos) Una `async fn` tiene 3 variables locales `u64` (8 B cada una) vivas simultáneamente en su punto `.await` más grande, más 8 bytes de discriminante de estado. ¿Cuántos bytes ocupa esa máquina de estados `Future`?
4 [PRB-0042] (Intermedio, Diseño) Un canal acotado `mpsc::sync_channel(64)` almacena mensajes de 128 bytes cada uno. ¿Cuántos bytes como máximo ocupan los mensajes en tránsito cuando el búfer está lleno?

Capítulo 7

F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización

Este capítulo desarrolla el Nivel Experto (Bloque F). Cuando un ingeniero de sistemas necesita implementar nuevas primitivas de sincronización, estructuras de datos intrusivas, controladores de registros de hardware mapeados en memoria (MMIO) o enlazar con bibliotecas en C del sistema operativo, recurre al subconjunto `unsafe` de Rust bajo contratos formales auditables con `Miri`.

Consejo

Todo bloque `unsafe { ... }` debe ir precedido por un comentario `// SAFETY:` que demuestre formalmente por qué se cumplen las precondiciones en ese punto, y toda función `pub unsafe fn` debe incluir una sección `# Safety` en su documentación.

Definición

Invariante de validez frente a invariante de seguridad en Rust: la invariante de validez exige que un tipo nunca adopte un patrón de bits ilegal (por ejemplo, un `bool` distinto de 0 o 1, o una referencia `&T` nula o desalineada), mientras que la invariante de seguridad exige que ninguna secuencia de llamadas desde código seguro pueda inducir comportamiento indefinido.

Autoficha

¿Por qué el compilador de Rust reordena por defecto los campos de un `struct` (`repr(Rust)`) y por qué es obligatorio añadir `#[repr(C)]` antes de pasar un puntero a esa estructura a una función escrita en C?

7.1 Rust inseguro (unsafe), contratos de seguridad, Miri e interoperabilidad FFI con C

Un bloque `unsafe { ... }` en Rust **no** desactiva el comprobador de préstamos ni las reglas de propiedad sobre referencias ordinarias (`&T` y `&mut T`); únicamente habilita cinco operaciones adicionales cuyos invariantes de validez no puede demostrar estáticamente el compilador: (1) desreferenciar punteros crudos (`*const T` y `*mut T`); (2) llamar a una función o método marcado como `unsafe fn`; (3) acceder o modificar una variable estática mutable; (4) implementar un rasgo marcado como `unsafe trait` (como `Send` o `Sync`); y (5) acceder a campos de una `union`. La arquitectura idiomática de Rust encapsula todo bloque `unsafe` dentro de un módulo con fronteras pequeñas y expone al exterior una API 100 % segura:

```
#[repr(C)]
#[derive(Copy, Clone, Debug, PartialEq)]
pub struct CTelemetryHeader {
    pub magic: u32,
    pub version: u16,
    pub payload_len: u16,
}

/// Divide una rebanada mutable en dos partes disjuntas usando punteros crudos.
pub fn safe_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let len = slice.len();
    assert!(mid <= len, "mid fuera de rango");
    let ptr = slice.as_mut_ptr();
```

```
    // SAFETY: 0 <= mid <= len garantiza que [ptr, ptr+mid) y [ptr+mid, ptr+len)
    // están dentro de la misma asignación y son estrictamente disjuntos.
    unsafe {
        (std::slice::from_raw_parts_mut(ptr, mid),
         std::slice::from_raw_parts_mut(ptr.add(mid), len - mid))
    }
}
```

Para interoperar con C mediante FFI (*Foreign Function Interface*), se utiliza `#[repr(C)]` (ya que el diseño por defecto `repr(Rust)` reordena libremente los campos de un `struct` para minimizar el relleno de alineamiento) y la convención de llamada `extern "C"`. El intérprete abstracto `cargo miri test` ejecuta el código MIR verificando el modelo de alias *Stacked Borrows* / *Tree Borrows*, lecturas de memoria no inicializada y alineamientos inválidos.

7.2 Metaprogramación con macros, entornos empotrados (#![no_std]), WebAssembly y optimización SIMD

En metaprogramación, Rust ofrece dos familias de macros que operan sobre el árbol sintáctico antes de la comprobación de tipos: las macros declarativas (`macro_rules!`), que funcionan por coincidencia de patrones higiénica sobre fragmentos sintácticos (`expr`, `ty`, `ident`, `tt`), y las macros procedimentales (`#[derive(...)]`, macros de atributo y macros tipo función), que son crates compilados que transforman un flujo `TokenStream` en otro `TokenStream`. En el ámbito de sistemas empotrados y kernels de sistemas operativos, el atributo `#![no_std]` prescinde de la biblioteca estándar `std` (que depende de un sistema operativo subyacente) y opera únicamente sobre `core` (independiente de plataforma y sin montículo) y opcionalmente `alloc`. Del mismo modo, Rust compila de forma nativa a `wasm32-unknown-unknown` y `wasm32-wasip1`, permitiendo ejecutar motores criptográficos, analizadores y simuladores físicos en el navegador o en pasarelas *serverless* a velocidad cercana al código nativo.

Problema resuelto

Enunciado Una estructura declarada con `#[repr(C)]` contiene en este orden exacto: `a: u8` (1 B), `b: u64` (8 B, requiere alineamiento a múltiplo de 8), `c: u8` (1 B) y `d: u32` (4 B, requiere alineamiento a múltiplo de 4), con alineamiento total de la estructura a múltiplo de 8 bytes. Calcula la diferencia en bytes para un array de $N = 1000$ estructuras entre `#[repr(C)]` (24 B cada una) y el empaquetado óptimo de `repr(Rust)` que ordena `b: u64`, `d: u32`, `a: u8`, `c: u8` ($8+4+1+1+2$ padding $= 16$ B).
Datos

- $N = 1000$ — Número de estructuras en el array
- $S_C = 24\,\text{B}$ — Tamaño bajo #[repr(C)] (1+7 + 8 + 1+3 + 4 = 24 B)
- $S_{Rust} = 16\,\text{B}$ — Tamaño reordenado por repr(Rust) (8 + 4 + 1 + 1 + 2 = 16 B)

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de relleno (padding) de alineamiento entre repr(C) y repr(Rust)

$$\Delta B = 1000 \cdot (24 - 16)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 1000 instancias por los 8 B ahorrados por estructura

$$\Delta B = 1000 \cdot (24 - 16)$$

$$(1000 \cdot (24 - 16))$$

Resultado

$$Resultado = 8000. La reordenación automática de campos de repr(Rust) ahorra 8000 bytes (un 33{,}3\,\% de reducción de huella en caché) sin requerir acceso desalineado.$$

Comprobación de los pasos

- Paso 1: 8000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0046**] **(Básico, Numérico)** Calcula el tamaño en bytes de `BadLayoutC { a: u8, b: u64, c: u8 }` bajo `#[repr(C)]` en 64 bits ($1 + 7 + 8 + 1 + 7$).
2 [**PRB-0047**] **(Básico, Numérico)** Calcula el tamaño en bytes de `GoodLayoutRust { a: u8, b: u64, c: u8 }` reordenado por el compilador de Rust ($8 + 1 + 1 + 6$).
3 [**PRB-0048**] **(Intermedio, Desarrollo por pasos)** Un registro SIMD de 256 bits (`AVX2`) procesa carriles `f32` de 32 bits en paralelo. ¿Cuántos números `f32` opera en una sola instrucción?
4 [**PRB-0049**] **(Intermedio, Diseño)** En un microcontrolador ARM Cortex-M `#![no_std]` con 64 KiB de SRAM (65536 B), el firmware reserva 8192 bytes para la pila y 4096 bytes para variables estáticas. ¿Cuántos bytes quedan disponibles para buffers estáticos de DMA?

Parte IV

Bloques G–H: Ecosistema Industrial, Pruebas, Benchmarking y Proyectos Completos

Capítulo 8

G–H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust

*Este capítulo culmina la monografía de Rust cubriendo los Bloques G y H de la plantilla: el ecosistema real de producción, los patrones de diseño de APIs conformes a las *Rust API Guidelines*, el catálogo de antipatrones frecuentes y la construcción paso a paso de cuatro proyectos completos, culminando en el motor de almacenamiento Key-Value transaccional con registro de escritura anticipada (*Write-Ahead Log*) y suma de verificación CRC32 ubicado en* `projects/rust-kv-engine`.

Consejo

Nunca expongas `&String`, `&Vec<T>` ni `&Box<T>` en los parámetros de entrada de funciones públicas: utiliza `&str`, `&[T]` y `&T`, ya que las coerciones automáticas `Deref` permiten pasar tanto tipos propietarios como vistas prestadas sin coste.

Definición

Write-Ahead Log (WAL) con verificación CRC32: estructura de almacenamiento secuencial sólo-añadir (append-only) en la que toda operación de mutación se persiste en disco precedida de un código de redundancia cíclica antes de confirmarse en la estructura de datos en memoria, garantizando atomicidad y durabilidad (propiedades A y D de ACID).

Autoficha

¿Por qué las *feature flags* de un manifiesto `Cargo.toml` deben ser siempre aditivas y qué problema ocurre cuando dos dependencias de un mismo workspace activan combinaciones distintas de una feature no aditiva?

8.1 Bloque G: Ecosistema de crates, Cargo Workspaces, idioms de diseño de APIs y antipatrones

El ecosistema industrial de Rust se articula en torno a crates fundacionales de altísima cohesión: `serde` (serialización y deserialización de coste cero sin reflexión en tiempo de ejecución mediante generación de código con `#[derive(Serialize, Deserialize)]`), `clap` (análisis declarativo de argumentos CLI), `tokio` y `axum` (servidores HTTP/gRPC asíncronos con extractores con seguridad de tipos), `sqlx` (consultas SQL verificadas contra el esquema de la base de datos en tiempo de compilación), `tracing` (telemetría estructurada con *spans* jerárquicos conscientes de tareas asíncronas), `proptest` (pruebas basadas en propiedades con reducción automática de contraejemplos) y `criterion` (microbenchmarking estadístico). Al diseñar APIs públicas, las *Rust API Guidelines* recomiendan aceptar parámetros genéricos mediante `impl AsRef<Path>` o `&str` en lugar de `&String` o `&Vec<T>`, implementar `From`/`TryFrom` y garantizar que las *feature flags* de Cargo sean estrictamente aditivas para que la unión de características solicitadas por distintos crates en el árbol de dependencias nunca provoque incompatibilidades de compilación.

8.2 Bloque H: Evaluación integral y los 4 proyectos progresivos (incluyendo rust-kv-engine verificado)

El Bloque H articula cuatro proyectos progresivos obligatorios: (1) una herramienta CLI de auditoría e indexación de ficheros con sumas criptográficas y salida estructurada; (2) un analizador sintáctico de

copia cero (*zero-copy*) para registros de configuración y telemetría; (3) un microservicio HTTP concurrente con limitación de tasa (*token bucket*) y apagado ordenado (*graceful shutdown*); y (4) el motor de almacenamiento Key-Value persistente `projects/rust-kv-engine`. En este cuarto proyecto, cada mutación `SET` o `DEL` se empaqueta en una trama binaria `[crc32: 4 B][op: 1 B][key_len: 4 B][val_len: 4 B][key][val]` de 13 B de cabecera fija y se sincroniza en el fichero de registro de escritura anticipada (*Write-Ahead Log*, WAL) antes de actualizar el índice en memoria protegido por `Arc<RwLock<HashMap<String, Vec<u8>>>>`:

```
pub fn encode_wal_frame_size(key: &str, value: &[u8]) -> usize {
    // 4B CRC32 + 1B opcode + 4B key_len + 4B val_len + key + value
    13 + key.len() + value.len()
}
```

Durante la recuperación tras un fallo eléctrico o caída del proceso, el motor recorre secuencialmente el WAL, recalcula el polinomio IEEE 802.3 CRC32 (`0xEDB88320`) de cada trama y rechaza de forma determinista cualquier registro truncado o corrompido en disco.

Problema resuelto

Enunciado En el motor `rust-kv-engine`, cada trama del Write-Ahead Log (WAL) tiene una cabecera fija de 13 B (4 B de CRC32, 1 B de código de operación, 4 B de longitud de clave y 4 B de longitud de valor). Si se registran $N = 2000$ operaciones `SET` donde cada clave mide $L_k = 16$ B y cada valor mide $L_v = 99$ B, calcula el tamaño exacto en bytes del fichero WAL resultante antes de ejecutar `compact()`.

Datos

- $N = 2000$ — Número de operaciones SET registradas
- $H = 13\,\mathrm{B}$ — Cabecera binaria fija por trama WAL
- $L_{kv}^{L} = 16\,\mathrm{B}, 99\mathrm{B}$ — Longitudes de clave y valor

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Tamaño total del fichero Write-Ahead Log (WAL) para N tramas

$$B_{\mathrm{WAL}} = 2000 \cdot (13 + 16 + 99)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos 13 + 16 + 99 = 128 B por trama y multiplicamos por 2000 tramas

$$B_{\mathrm{WAL}} = 2000 \cdot (13 + 16 + 99)$$

$$(2000 \cdot ((13 + 16) + 99))$$

Resultado

$$Resultado = 256000. El fichero WAL ocupa exactamente 256000 bytes (250 KiB) en disco.$$

Interpretación El fichero WAL ocupa exactamente 256 000 bytes (250 KiB) en disco.

Comprobación de los pasos

- Paso 1: 256000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0053] (Básico, Numérico) Calcula el tamaño en bytes de una trama WAL de `rust-kv-engine` con cabecera de 13 B, clave de 11 B y valor de 40 B ($13 + 11 + 40$).
2 [PRB-0054] (Básico, Desarrollo por pasos) Un fichero WAL contiene 5000 registros sobre un conjunto de solo 800 claves únicas (cada una actualizada varias veces). Si `compact()` conserva únicamente el último valor de cada clave viva (64 B por trama), ¿cuántos bytes ocupa el WAL compactado?
3 [PRB-0055] (Intermedio, Desarrollo por pasos) En el problema anterior, ¿cuántos bytes de espacio en disco libera la operación `compact()` respecto a las 5000 tramas originales de 64 B?
4 [PRB-0056] (Intermedio, Diseño) Un workspace de Cargo contiene 6 crates de biblioteca y cada crate define 12 pruebas unitarias y 3 pruebas de integración. ¿Cuántas pruebas ejecuta en total `cargo test -workspace`?

Apéndice A

Notación

Símbolo	Significado	Unidades
$T : \text{Send} + \text{Sync}$	Cotas de rasgo de transferencia y compartición segura entre hilos	adimensional
'a	Parámetro genérico de tiempo de vida (lifetime) de una referencia	adimensional
alignof(T)	Alineamiento de dirección de memoria requerido por el tipo T	B
sizeof(T)	Tamaño estático en bytes de un tipo Sized en pila	B

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 A. Introducción, historia, filosofía y modelo de compilación de Rust

Problema resuelto [PRB-0001]
En una arquitectura de 64 bits ($W = 8$ B), un vector en Rust `Vec<T>` se representa en la pila mediante tres campos de tamaño palabra (`ptr`, `len`, `cap`), mientras que una rebanada `&[T]` es un puntero ancho de dos palabras (`ptr`, `len`). Calcula el tamaño total en pila en bytes de una estructura que contiene dos `Vec<u8>` y tres rebanadas `&[u8]`.
Respuesta

$Resultado = 96. La estructura ocupa exactamente 96 bytes en la pila (48 B de los dos vectores y 48 B de las tres rebanadas), indepe$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0003]
Verificación ejecutable con `rustc -test` del tamaño exacto en bytes de `&u64`, `&[u64]` y `Vec<u64>` mediante `std::mem::size_of`.
Ejercicio 1 [PRB-0004]
Calcula el tamaño en bytes en la pila de un `Vec<f64>` en una arquitectura de 64 bits (3×8).
Respuesta 24
Ejercicio 2 [PRB-0005]
Calcula el tamaño en bytes de un puntero ancho `&dyn std::fmt::Display` en una CPU de 64 bits (2×8).
Respuesta 16
Ejercicio 3 [PRB-0006]
Si un binario contiene 45 funciones genéricas instanciadas cada una para 4 tipos concretos distintos mediante monomorfización, ¿cuántas funciones especializadas se emiten antes de la eliminación de código muerto?
Respuesta 180
Ejercicio 4 [PRB-0007]
Un búfer de 4096 elementos `u32` (4 bytes por elemento) está referenciado por un `Vec<u32>` (24 bytes en pila). ¿Cuántos bytes totales suma la cabecera en pila más la región en el montículo?
Respuesta 16408

Cálculos reproducidos

$((2 \cdot (3 \cdot 8)) + (3 \cdot (2 \cdot 8))) = 96$

C.2 B–C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo

Problema resuelto [PRB-0008]
Una cadena en Rust contiene 10 caracteres ASCII (1 B cada uno en UTF-8) y 5 caracteres acentuados españoles (`á`, `é`, `í`, `ó`, `ú`, que ocupan 2 B cada uno en UTF-8). Calcula la diferencia en bytes entre representarla como `&str` en UTF-8 frente a un array `[char; 15]` de valores escalares Unicode (4 B por `char`).
Respuesta

$Resultado = 40. La representación UTF-8 nativa de Rust ahorra 40 bytes (ocupa 20 B frente a 60 B), razón por la cual String y \&$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0010]
Verificación ejecutable con `rustc -test` de aritmética saturada, verificada y longitud UTF-8 frente a conteo de `chars()`.
Ejercicio 1 [PRB-0011]
Calcula el tamaño en bytes de un array `[u64; 32]` alojado en la pila (32×8).
Respuesta 256
Ejercicio 2 [PRB-0012]
Una cadena UTF-8 contiene 8 caracteres ASCII (1 B), 4 letras griegas (2 B) y 2 emojis (4 B). ¿Cuál es el valor de `s.len()` en bytes?
Respuesta 24
Ejercicio 3 [PRB-0013]
Calcula el resultado de `250u8.wrapping_add(15)` en aritmética modular de 8 bits ($250 + 15 - 256$).
Respuesta 9
Ejercicio 4 [PRB-0014]
Si `Vec::with_capacity(128)` reserva espacio para 128 elementos `i32` (4 B cada uno) e insertamos 50 elementos, ¿cuántos bytes de capacidad reservada permanecen aún libres en el montículo?
Respuesta 312

Cálculos reproducidos

$((15 \cdot 4) - ((10 \cdot 1) + (5 \cdot 2))) = 40$

C.3 D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes)

Problema resuelto [PRB-0015]
Un vector `Vec<u64>` contiene $N = 10000$ enteros de 8 bytes en el montículo en una máquina de 64 bits ($W = 8$ B). Calcula cuántos bytes menos se copian al pasar el vector por transferencia de propiedad (*move*, que solo copia las 3 palabras de la cabecera en pila) frente a invocar `.clone()` (que copia la cabecera más los 10000×8 bytes del montículo).
Respuesta

$$Resultado = 80000. La transferencia de propiedad (move) ahorra la copia de 80000 bytes en el montículo y se ejecuta en tiempo co$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0017]
Verificación ejecutable con `rustc -test` de préstamos con tiempos de vida explícitos `'a`, rebanadas mutables disjuntas (`split_at_mut`) y Non-Lexical Lifetimes.
Ejercicio 1 [PRB-0018]
Calcula cuántos bytes en la pila se copian al mover un `String` en una arquitectura de 64 bits (3×8).
Respuesta 24
Ejercicio 2 [PRB-0019]
Si dividimos una rebanada de 1024 elementos mediante `split_at_mut(300)`, ¿cuántos elementos contiene la segunda sub-rebanada mutable?
Respuesta 724
Ejercicio 3 [PRB-0020]
Una estructura contiene 4 campos `u64` que implementan `Copy`. ¿Cuántos bytes se copian en cada asignación por valor (4×8)?
Respuesta 32
Ejercicio 4 [PRB-0021]
Un analizador léxico de copia cero (*zero-copy*) genera 500 tokens `&'a str` (16 B cada uno) sobre un texto fuente de 32 000 bytes sin clonar subcadenas. ¿Cuántos bytes ocupan en total los 500 descriptores de token?
Respuesta 8000

Cálculos reproducidos

$(((3 \cdot 8) + (10000 \cdot 8)) - (3 \cdot 8)) = 80000$

C.4 D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result)

Problema resuelto [PRB-0022]
Una tabla hash abierta almacena $N = 65536$ ranuras de tipo `Option<std::num::NonZeroU64>` en una arquitectura de 64 bits ($W = 8$ B). Gracias a la optimización de nicho, cada `Option<NonZeroU64>` ocupa exactamente 8 bytes en lugar de 16 bytes (8 bytes de dato más 8 bytes de discriminante alineado). Calcula cuántos kilobytes (dividiendo entre 1024) se ahorran en la tabla completa gracias a la optimización de nicho.
Respuesta

$Resultado = 512. La optimización de nicho ahorra exactamente 512 KiB de memoria caché/RAM y duplica el número de entrad$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0024]
Verificación ejecutable con `rustc -test` de la optimización de nicho (`size_of::<Option<&u64>>() == 8`) y propagación de errores con `?`.
Ejercicio 1 [PRB-0025]
En una arquitectura de 64 bits, ¿cuántos bytes ocupa `Option<Box<u64>>` gracias a la optimización de nicho?
Respuesta 8
Ejercicio 2 [PRB-0026]
¿Cuántos bytes ocupa en total un array `[Option<NonZeroU32>; 1000]` sabiendo que cada elemento ocupa 4 bytes?
Respuesta 4000
Ejercicio 3 [PRB-0027]
Un `enum` sin nicho tiene una variante de 24 bytes con alineamiento de 8 bytes y una etiqueta de 1 byte. Tras añadir 7 bytes de relleno (*padding*) para mantener el alineamiento a múltiplo de 8, ¿cuál es su tamaño total en bytes?
Respuesta 32
Ejercicio 4 [PRB-0028]
Una línea de caché de CPU mide 64 bytes. ¿Cuántos elementos `Option<NonZeroU64>` (8 B cada uno) caben en una única línea de caché?
Respuesta 8

Cálculos reproducidos

$\frac{(65536 \cdot (16-8))}{1024} = 512$

C.5 E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior

Problema resuelto [PRB-0029]
En una arquitectura de 64 bits ($W = 8$ B), se aloja un bloque `Rc< [u8; 1024] >` ($S_T = 1024$ B) y se crean $k = 5$ manejadores `Rc::clone` en distintas estructuras en la pila. Calcula la memoria total consumida (los 5 punteros en la pila más el único bloque `RcBox` en el montículo con sus 2 contadores `usize` y los 1024 bytes de datos).
Respuesta

$Resultado = 1080. El conjunto completo ocupa 1080 bytes (40 B en la pila y 1040 B en el montículo), ahorrando 4096 bytes frente a$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0031]
Verificación ejecutable con `rustc -test` de tuberías de iteradores, `Rc/Weak` y mutabilidad interior con `RefCell`.
Ejercicio 1 [PRB-0032]
Calcula la suma de los cuadrados de los números pares del 2 al 10 ($\$2^2 + 4^2 + 6^2 + 8^2 + 10^2\$$) evaluada por `(1..=10).filter(|x| x% 2 == 0).map(|x| x * x).sum()`.
Respuesta 220
Ejercicio 2 [PRB-0033]
¿Cuántos bytes de metadatos añaden los dos contadores (`strong` y `weak`) de `RcBox<T>` en una arquitectura de 64 bits (2×8)?
Respuesta 16
Ejercicio 3 [PRB-0034]
Si se crean 1 `Rc` original, 4 clones fuertes (`Rc::clone`) y 3 referencias débiles (`Rc::downgrade`), y luego se destruyen 2 clones fuertes, ¿cuál es el valor actual de `Rc::strong_count`?
Respuesta 3
Ejercicio 4 [PRB-0035]
Un flujo de datos procesa 5000 enteros mediante `iter().filter(...).map(...).fold(...)` sin llamar a `collect()`. ¿Cuántos vectores dinámicos intermedios se alojan en el montículo durante el recorrido?
Respuesta 0

Cálculos reproducidos

$((5 \cdot 8) + ((2 \cdot 8) + 1024)) = 1080$

C.6 E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio)

Problema resuelto [PRB-0036]
Un servidor de mensajería mantiene $C = 50000$ conexiones TCP simultáneas. Si cada máquina de estados `Future` en Tokio ocupa 256 B (0,25 KiB) mientras que una pila mínima de hilo cuesta 64 KiB, calcula la memoria total en MiB (dividiendo los KiB entre 1024) que consumen las 50 000 tareas asíncronas.
Respuesta

$Resultado = 12{,}20703125. Las 50000 conexiones asíncronas consumen tan solo 12{,}20703125 MiB de RAM, frente a los 3125 Mi$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0038]
Verificación ejecutable con `rustc -test` de concurrencia paralela con `std::thread::scope`, `Arc<Mutex<T»` y canales `mpsc`.
Ejercicio 1 [PRB-0039]
Cuatro hilos incrementan un `AtomicU64` exactamente 25 000 veces cada uno con `fetch_add(1, Ordering::Relaxed)`. ¿Cuál es el valor final exacto del contador?
Respuesta 100000
Ejercicio 2 [PRB-0040]
Un vector de 10 000 elementos se divide equitativamente entre 8 hilos mediante `chunks(1250)` dentro de `std::thread::scope`. ¿Cuántos elementos procesa cada hilo?
Respuesta 1250
Ejercicio 3 [PRB-0041]
Una `async fn` tiene 3 variables locales `u64` (8 B cada una) vivas simultáneamente en su punto `.await` más grande, más 8 bytes de discriminante de estado. ¿Cuántos bytes ocupa esa máquina de estados `Future`?
Respuesta 32
Ejercicio 4 [PRB-0042]

Un canal acotado `mpsc::sync_channel(64)` almacena mensajes de 128 bytes cada uno. ¿Cuántos bytes como máximo ocupan los mensajes en tránsito cuando el búfer está lleno?
Respuesta 8192

Cálculos reproducidos

$\frac{(50000 \cdot 256)}{(1024 \cdot 1024)} = 12{,}20703125$

C.7 F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización

Problema resuelto [PRB-0043]
Una estructura declarada con `#[repr(C)]` contiene en este orden exacto: `a: u8` (1 B), `b: u64` (8 B, requiere alineamiento a múltiplo de 8), `c: u8` (1 B) y `d: u32` (4 B, requiere alineamiento a múltiplo de 4), con alineamiento total de la estructura a múltiplo de 8 bytes. Calcula la diferencia en bytes para un array de $N = 1000$ estructuras entre `#[repr(C)]` (24 B cada una) y el empaquetado óptimo de `repr(Rust)` que ordena `b: u64`, `d: u32`, `a: u8`, `c: u8` ($8 + 4 + 1 + 1 + 2$ padding $= 16$ B).
Respuesta

$$Resultado = 8000. La reordenación automática de campos de repr(Rust) ahorra 8000 bytes (un 33{,}3\,\% de reducción de huella enc$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0045]
Verificación ejecutable con `rustc -test` de `safe_split_at_mut`, `macro_rules!` y diferencia real de tamaño entre `#[repr(C)]` y `repr(Rust)`.
Ejercicio 1 [PRB-0046]
Calcula el tamaño en bytes de `BadLayoutC { a: u8, b: u64, c: u8 }` bajo `#[repr(C)]` en 64 bits ($1 + 7 + 8 + 1 + 7$$).
Respuesta 24
Ejercicio 2 [PRB-0047]
Calcula el tamaño en bytes de `GoodLayoutRust { a: u8, b: u64, c: u8 }` reordenado por el compilador de Rust ($8 + 1 + 1 + 6$$).
Respuesta 16
Ejercicio 3 [PRB-0048]
Un registro SIMD de 256 bits (`AVX2`) procesa carriles `f32` de 32 bits en paralelo. ¿Cuántos números `f32` opera en una sola instrucción?
Respuesta 8
Ejercicio 4 [PRB-0049]
En un microcontrolador ARM Cortex-M `#![no_std]` con 64 KiB de SRAM (65536 B), el firmware reserva 8192 bytes para la pila y 4096 bytes para variables estáticas. ¿Cuántos bytes quedan disponibles para buffers estáticos de DMA?
Respuesta 53248

Cálculos reproducidos

$(1000 \cdot (24 - 16)) = 8000$

C.8 G–H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust

Problema resuelto [PRB-0050]
En el motor `rust-kv-engine`, cada trama del Write-Ahead Log (WAL) tiene una cabecera fija de 13 B (4 B de CRC32, 1 B de código de operación, 4 B de longitud de clave y 4 B de longitud de valor). Si

se registran $N = 2000$ operaciones `SET` donde cada clave mide $L_k = 16$ B y cada valor mide $L_v = 99$ B, calcula el tamaño exacto en bytes del fichero WAL resultante antes de ejecutar `compact()`.
Respuesta

$$Resultado = 256000. El fichero WAL ocupa exactamente 256000 bytes (250 KiB) en disco.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Problema resuelto [PRB-0052]
Verificación ejecutable con `rustc -test` del algoritmo CRC32 IEEE 802.3 y empaquetado de trama WAL empleado en `projects/rust-kv-engine`.
Ejercicio 1 [PRB-0053]
Calcula el tamaño en bytes de una trama WAL de `rust-kv-engine` con cabecera de 13 B, clave de 11 B y valor de 40 B ($13 + 11 + 40$).
Respuesta 64
Ejercicio 2 [PRB-0054]
Un fichero WAL contiene 5000 registros sobre un conjunto de solo 800 claves únicas (cada una actualizada varias veces). Si `compact()` conserva únicamente el último valor de cada clave viva (64 B por trama), ¿cuántos bytes ocupa el WAL compactado?
Respuesta 51200
Ejercicio 3 [PRB-0055]
En el problema anterior, ¿cuántos bytes de espacio en disco libera la operación `compact()` respecto a las 5000 tramas originales de 64 B?
Respuesta 268800
Ejercicio 4 [PRB-0056]
Un workspace de Cargo contiene 6 crates de biblioteca y cada crate define 12 pruebas unitarias y 3 pruebas de integración. ¿Cuántas pruebas ejecuta en total `cargo test -workspace`?
Respuesta 90

Cálculos reproducidos

$(2000 \cdot ((13 + 16) + 99)) = 256000$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 30 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios. Ecuaciones: 1 reproducida numéricamente.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

The Rust Programming Language (Klabnik & Nichols — Official Rust Documentation) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-rust-nomicon (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-unicode-15 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
Referencia Normativa src-iso-c17 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-introduccion-historia-modelo-compilacion-rust scientific
REV-0002: cap02-entorno-toolchain-cargo-nivel-principiante scientific
REV-0003: cap03-ownership-borrowing-lifetimes-modelo-memoria scientific
REV-0004: cap04-tipos-algebraicos-traits-genericos-errores scientific
REV-0005: cap05-closures-iteradores-smart-pointers-mutabilidad-interior scientific
REV-0006: cap06-concurrencia-sin-miedo-send-sync-async-tokio-pin scientific
REV-0007: cap07-nivel-experto-unsafe-ffi-macros-wasm-embedded scientific
REV-0008: cap08-ecosistema-herramientas-evaluacion-proyectos-rust scientific

Apéndice E

Plan de estudio

Bloques A−C: Fundamentos, Entorno y Nivel Principiante en Rust

✓A. Introducción, historia, filosofía y modelo de compilación de Rust

- Comprender el origen histórico de Rust (Graydon Hoare, Mozilla Research, Rust Foundation) y las ediciones 2015, 2018, 2021 y 2024.
- Trazar la tubería de compilación desde el código fuente hasta código máquina nativo pasando por HIR, THIR, MIR y LLVM IR.
- Contrastar las abstracciones de coste cero y la seguridad espacial y temporal de memoria sin recolector de basura frente a C, C++ y Go.

2 conceptos planificados.

✓B−C. Instalación, toolchain (rustup, Cargo) y nivel principiante completo

- Configurar y gestionar toolchains con rustup, perfiles de compilación en Cargo.toml, resolución determinista con Cargo.lock, rustfmt y clippy.
- Dominar el modelo de vinculación inmutable por defecto (let vs let mut), sombreado (shadowing), tipos escalares y compuestos, y control de flujo expresivo.
- Distinguir la representación UTF-8 de String y &str frente a arrays [T; N], tuplas y vectores Vec<T>.

2 conceptos planificados.

Bloque D: Nivel Intermedio – Propiedad, Tipos Algebraicos, Rasgos y Modularidad

✓D.1. Propiedad (Ownership), Préstamo (Borrowing), Rebanadas y Tiempos de Vida (Lifetimes)

- Enunciar y aplicar las tres leyes del sistema de propiedad (ownership) y diferenciar semántica Move, Copy y Clone.
- Aplicar la ley de exclusión mutua de referencias (XOR entre alias y mutabilidad: múltiples &T o una única &mut T).
- Anotar explícitamente tiempos de vida genéricos ('a, 'static), comprender las tres reglas de elisión y el comprobador NLL.

2 conceptos planificados.

✓D.2. Tipos Algebraicos (struct, enum), Rasgos (Traits), Genéricos y Manejo de Errores (Option y Result)

- Modelar invariantes de dominio mediante tipos producto (struct) y tipos suma (enum) haciendo irrepresentables los estados inválidos.
- Diseñar jerarquías de polimorfismo paramétrico y ad-hoc con rasgos (traits), tipos asociados, cotas where y coherencia (regla del huérfano).

- Gestionar errores recuperables e irrecuperables mediante Option<T>, Result<T, E>, el operador de propagación ‘¿ y conversiones From/Into.

2 conceptos planificados.

Bloques E–F: Nivel Avanzado y Experto – Smart Pointers, Concurrencia, Async y Unsafe

✓E.1. Nivel Avanzado: Clausuras (Fn/FnMut/FnOnce), Iteradores, Punteros Inteligentes y Mutabilidad Interior

- Distinguir la representación y captura de entorno en clausuras según la jerarquía de rasgos FnOnce, FnMut y Fn.
- Diseñar tuberías perezosas de iteradores de coste cero y demostrar su equivalencia en ensamblador frente a bucles manuales.
- Seleccionar con rigor entre Box<T>, Rc<T>, Arc<T>, Cell<T>, RefCell<T>, Cow<’a, B> y Weak<T> evitando fugas por ciclos de referencias.

2 conceptos planificados.

✓E.2. Concurrencia sin Miedo (Send/Sync, Arc, Mutex, Canales) y Asincronía (Future, async/await, Pin, Tokio)

- Demostrar cómo los rasgos automáticos Send y Sync garantizan en tiempo de compilación la ausencia de carreras de datos.
- Combinar hilos con ámbito (std::thread::scope), canales MPSC, Arc<Mutex<T», Arc<RwLock<T» y tipos atómicos con órdenes de memoria.
- Comprender la compilación de funciones async a máquinas de estados sin pila (stackless coroutines), el rasgo Future::poll y la garantía estructural de Pin<P>.

2 conceptos planificados.

✓F. Nivel Experto: Rust Inseguro (unsafe), FFI con C, Macros, WebAssembly, Embedded (no_std) y Optimización

- Enumerar exactamente los cinco superpoderes de un bloque unsafe y encapsularlos tras interfaces seguras con contratos Safety documentados.
- Construir puentes FFI bidireccionales con C (#[repr(C)], extern "C", #[no_mangle]) y auditar ausencias de Undefined Behavior con Miri.
- Escribir macros declarativas (macro_rules!) e higiénicas, comprender macros procedimentales y compilar crates para WebAssembly y sistemas empotrados (#![no_std]).

2 conceptos planificados.

Bloques G–H: Ecosistema Industrial, Pruebas, Benchmarking y Proyectos Completos

✓G–H. Ecosistema Profesional, Idioms, Antipatrones, Evaluación y 4 Proyectos en Rust

- Dominar el ecosistema estándar de producción en Rust: serde, clap, tokio, axum, sqlx, tracing, thiserror, anyhow, proptest y criterion.

- Diseñar arquitecturas modulares con Cargo Workspaces, feature flags aditivas, semántica SemVer e idioms (AsRef, Borrow, FromStr, Builder).
- Construir y verificar los 4 proyectos integradores progresivos en Rust, incluyendo el motor Key-Value concurrente con WAL y CRC32.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-rust-nomicon. https://doc.rust-lang.org/reference/src-rust-nomicon.html

[27] Referencia Normativa src-unicode-15. https://doc.rust-lang.org/reference/src-unicode-15.html

[28] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html

[29] Referencia Normativa src-iso-c17. https://doc.rust-lang.org/reference/src-iso-c17.html

[30] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen IV: Monografías de Lenguajes de Sistemas — C, C++, Zig, Ensamblador, Ada y Fortran

Parte V (Bloques A–H): Memoria manual, RAII, metaprogramación con Concepts, ABI de bajo nivel y cómputo científico

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografía Autónoma de C (ISO/IEC 9899:2018 / C23) 1

1. Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante 2
1.1. Bloques A y B: Historia de C, modelo de máquina abstracta, GCC/Clang, CMake y flags de diagnóstico . . . 2
1.2. Bloque C: Tipos fundamentales, <stdint.h>, promociones aritméticas, operadores y control de flujo . . . 3

2. Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing 5
2.1. Bloque D: Punteros, arrays, cadenas terminadas en NUL, structs, unions y memoria dinámica 5
2.2. Bloque E: Punteros a función, calificador restrict, strict aliasing y asignadores de región (Arena) . . . 5

3. Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos 8
3.1. Bloque F: Concurrencia C11 (<stdatomic.h>), pthreads, llamadas POSIX y programación defensiva . . . 8
3.2. Bloques G y H: Ecosistema (CMake, Meson, pkg-config, Sanitizers) y los 4 proyectos en C 9

II Monografía Autónoma de C++ (ISO/IEC 14882:2020 / C++23) 11

4. Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes 12
4.1. Bloques A–C: Evolución C++11/14/17/20/23, toolchain, referencias, sobrecarga, constexpr y RAII . . . 12
4.2. Bloque D: Categorías de valores (lvalue/rvalue), Regla de Cero/Cinco, std::unique_ptr y std::span . . . 12

5. Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos 15
5.1. Bloques E y F: Plantillas, C++20 Concepts, std::ranges, std::jthread y corrutinas . . . 15
5.2. Bloques G y H: Ecosistema CMake/vcpkg/Conan/Clang-Tidy y los 4 proyectos (incluyendo cpp-task-scheduler) . . . 16

III Monografías Especializadas de Sistemas: Zig, Ensamblador, Ada y Fortran 18

6. Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C 19
6.1. Bloques A–D: Filosofía de Zig, toolchain integrada, tipos, rebanadas, uniones de error y defer/errdefer . . . 19
6.2. Bloques E–H: Asignadores explícitos (Arena, FixedBuffer, GPA), comptime, FFI con C y proyectos . . . 20

7. Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A−H): Registros, ABI, Pila y SIMD **22**
7.1. Bloques A−D: Arquitectura de registros x86-64 y ARM64 (AArch64), modos de direccionamiento y saltos . . . 22
7.2. Bloques E−H: Convenciones ABI (System V y AAPCS64), alineamiento de pila, SIMD (NEON/AVX) e inspección con objdump . . . 23

8. Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A−H) **25**
8.1. Monografía de Ada 2022 y SPARK (Bloques A−H): Tipado fuerte de dominio, contratos y sistemas críticos . . . 25
8.2. Monografía de Fortran 2018/2023 (Bloques A−H): Arrays multidimensionales, vectorización y Coarrays . . . 26

A. Notación **28**

B. Hoja de fórmulas **29**

C. Soluciones **30**
C.1. Monografía de C (I − Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante . . . 30
C.2. Monografía de C (II − Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing . . . 30
C.3. Monografía de C (III − Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos . . . 31
C.4. Monografía de C++ (I − Bloques A−D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes . . . 32
C.5. Monografía de C++ (II − Bloques E−H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos . . . 32
C.6. Monografía de Zig (Bloques A−H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C . . . 33
C.7. Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A−H): Registros, ABI, Pila y SIMD . . . 34
C.8. Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A−H) . . . 34

D. Auditoría de fuentes **36**

E. Plan de estudio **38**

Parte I

Monografía Autónoma de C (ISO/IEC 9899:2018 / C23)

Capítulo 1

Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante

Este capítulo inicia la monografía autónoma del lenguaje C (Bloques A, B y C). C es la lingua franca de los sistemas operativos (Linux, Windows NT, XNU/macOS), los motores de bases de datos (PostgreSQL, SQLite, Redis), los intérpretes de lenguajes (CPython, Lua) y los sistemas empotrados. El lector aprenderá a programar en C moderno (C17/C23) desde cero con disciplina de tipos y herramientas de verificación rigurosas.

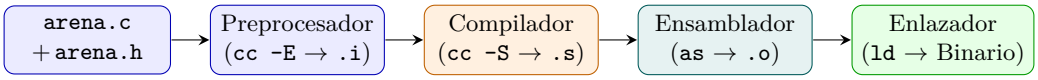


Figura 1. Tubería clásica de compilación separada en C: el preprocesador expande directivas sobre cada unidad de traducción (.i), el compilador emite código ensamblador (.s), el ensamblador genera objetos reubicables (.o) y el enlazador resuelve símbolos externos.

Consejo

Incluye siempre guardas de inclusión (`#ifndef MODULO_H` / `#define MODULO_H` / `#endif`) en todas tus cabeceras `.h` y nunca declares variables globales con definición en una cabecera sin `extern`.

Definición

Comportamiento Indefinido (Undefined Behavior, ISO C17 §3.4.3): uso de una construcción no portable o de datos erróneos para los cuales el estándar internacional no impone requisito alguno, permitiendo que el compilador optimice bajo la premisa axiomática de que dicha situación es inalcanzable.

Autoficha

¿Por qué comparar un índice `int i = -1` con `size_t n = 10` mediante `if (i < n)` evalúa a `false` en C debido a las conversiones aritméticas usuales?

1.1 Bloques A y B: Historia de C, modelo de máquina abstracta, GCC/Clang, CMake y flags de diagnóstico

Creado por Dennis Ritchie entre 1969 y 1973 en los Laboratorios Bell para reescribir el núcleo de UNIX (originalmente en ensamblador PDP-11), C fue estandarizado por ANSI en 1989 (C89/C90) y por ISO en las revisiones C99 (que introdujo `<stdint.h>`, `<stdbool.h>`, inicializadores designados y el calificador `restrict`), C11 (modelo de memoria multihilo y `_Atomic`), C17 y C23 (`nullptr`, `constexpr`, `typeof`, `<stdckdint.h>`). Como muestra la Figura \ref{fig:fig-c-translation-pipeline}, cada fichero `.c` junto con las cabeceras `.h` incluidas por `#include` constituye una *unidad de traducción* independiente. En ingeniería profesional, ningún programa en C debe compilarse sin activar las banderas estrictas de diagnóstico y los sanitizadores dinámicos: `clang -std=c17 -O2 -Wall -Wextra -Wpedantic -Werror -Wshadow -Wconversion -fsanitize=address,undefined`.

1.2 Bloque C: Tipos fundamentales, <stdint.h>, promociones aritméticas, operadores y control de flujo

Dado que el estándar C solo fija anchuras mínimas para `short`, `int`, `long` y `long long`, el código de sistemas portable utiliza siempre los tipos de anchura exacta de `<stdint.h>`: `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`, `int32_t`, `int64_t`, además de `size_t` (para tamaños y conteos de objetos en memoria) y `ptrdiff_t` (para diferencias entre punteros). Una regla crítica de la máquina abstracta de C es la asimetría entre enteros sin signo y con signo: mientras que los enteros sin signo (`uint32_t`, `size_t`) operan por definición en aritmética modular mód2^w, el desbordamiento de un entero **con signo** (`int32_t`) constituye **comportamiento indefinido** (*Undefined Behavior*, UB), lo que autoriza al optimizador a asumir que jamás ocurre y a eliminar comprobaciones ingenuas a posteriori como `if (a + b < a)`. Por ello, el desbordamiento con signo debe prevenirse **antes** de ejecutar la suma:

```
#include <stdint.h>
#include <stdbool.h>

bool safe_add_i32(int32_t a, int32_t b, int32_t *out) {
    if ((b > 0 && a > INT32_MAX - b) || (b < 0 && a < INT32_MIN - b)) {
        return false;
    }
    *out = a + b;
    return true;
}
```

En C23, la cabecera estándar `<stdckdint.h>` estandariza esta operación mediante la primitiva intrínseca `ckd_add(out, a, b)`.

Problema resuelto

Enunciado En una arquitectura de 32 bits para enteros `uint32_t` ($w = 32$, módulo $2^{32} = 4294967296$), un entero con signo con valor -100 se convierte implícitamente a `uint32_t` durante una comparación. Calcula el valor numérico exacto que toma tras la conversión según la regla $2^{32} - 100$.

Datos

- $w = 32\,\text{bits}$ — Anchura en bits de uint32_t
- $x = -100$ — Valor con signo original

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Conversión a entero sin signo módulo 2^32 (ISO C §6.3.1.3)

$$U = 2^{32} - 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos 2^32 al valor negativo -100

$$U = 2^{32} - 100$$

$$\left((2)^{32} - 100\right)$$

Resultado

$Resultado = 4294967196. El valor resultante en uint32_t es 4294967196, razón por la cual `-Wsign-conversion` es indispens$

Interpretación El valor resultante en uint32$_t$ es 4 294 967 196, razón por la cual `-Wsign-conversion` es indispensable.
Comprobación de los pasos

- Paso 1: 4294967196 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) Calcula el valor en `uint16_t` ($w = 16$) resultante de convertir -1 a sin signo ($2^{16} - 1$).
2 [PRB-0004] (Básico, Numérico) Un registro de control de 8 bits activa los bits 0, 3 y 7 ($\$2^0 + 2^3 + 2^7\$$). ¿Cuál es el valor decimal de la máscara `uint8_t`?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) ¿Cuántos bytes ocupa en total una estructura con 4 campos `uint32_t` y 2 campos `uint64_t` perfectamente alineados ($4 \times 4 + 2 \times 8$)?
4 [PRB-0006] (Intermedio, Diseño) Si compilamos un proyecto en C formado por 18 unidades de traducción `.c` y solo modificamos un fichero `.c`, ¿cuántas unidades de traducción evita recompilar `make`/`ninja`?

Capítulo 2

Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing

*Este capítulo desarrolla los niveles intermedio y avanzado de C (Bloques D y E). Aquí desentrañamos la relación exacta entre arrays y punteros, la representación en memoria de estructuras y uniones, los riesgos de `malloc/free` individuales y por qué el patrón de ingeniería de sistemas por excelencia en C es el asignador por región (*Arena Allocator*).*

Consejo

Para reinterpretar los bits de un `float` como `uint32_t` o leer enteros de un paquete de red en C, utiliza siempre `memcpy(&destino, &origen, sizeof(destino))`: es 100 % seguro frente a strict aliasing y alineamiento, y el compilador lo optimiza a una sola instrucción de registro.

Definición

Regla de Alias Estricto (Strict Aliasing, ISO C17 §6.5): restricción semántica que prohíbe acceder a un valor almacenado mediante un puntero de un tipo distinto no relacionado (salvo tipos de carácter `unsigned char`), permitiendo al compilador mantener valores en registros sin recargarlos desde memoria tras escrituras a otros tipos.

Autoficha

¿Por qué tras ejecutar `free(ptr)` en C es una buena práctica defensiva asignar `ptr = NULL` si el puntero permanece aún en ámbito?

2.1 Bloque D: Punteros, arrays, cadenas terminadas en NUL, structs, unions y memoria dinámica

En C, un puntero `T *p` almacena la dirección virtual de un objeto de tipo `T`. La aritmética de punteros está escalada automáticamente por el tamaño del tipo apuntado: si `p` tiene dirección base A, la expresión `p + k` (y su equivalente por definición `&(p[k])`) evalúa a la dirección $A + k \cdot \text{sizeof}(T)$. El estándar ISO C permite formar y comparar un puntero únicamente dentro del mismo array asignado o exactamente una posición más allá del último elemento (`p + N`, denominado *one-past-the-end*), aunque desreferenciar `p + N` es comportamiento indefinido. Las cadenas en C son secuencias de bytes terminadas por el carácter nulo `'\0'` (`0x00`); calcular `strlen(s)` cuesta $O(n)$ porque debe recorrer los bytes hasta hallar el centinela, y usar funciones no acotadas permite desbordar el búfer si no se pasa explícitamente la capacidad máxima del destino.

2.2 Bloque E: Punteros a función, calificador restrict, strict aliasing y asignadores de región (Arena)

La regla de *alias estricto* (*strict aliasing*, ISO C17 §6.5 p7) estipula que un objeto en memoria solo puede ser accedido a través de un lvalue de tipo compatible con su tipo efectivo o a través de un puntero a carácter (`unsigned char *` o `char *`). Por tanto, convertir un `uint8_t *buf` arbitrario mediante `*(uint32_t *)buf` viola tanto el alias estricto como potencialmente el alineamiento de hardware; la forma conforme al estándar —que GCC y Clang compilan a una única instrucción máquina `mov` sin llamada real a función— consiste en usar `memcpy(&val, buf, sizeof(val))`. Asimismo, el calificador `restrict` en `void saxpy(size_t n, float a, const float *restrict x, float *restrict`

`y)` promete al compilador que las regiones apuntadas por `x` e `y` no se solapan, habilitando la vectorización SIMD automática:

```
#include <stddef.h>
#include <stdint.h>

void vector_scale_add(size_t n, float alpha, const float *restrict x, float *restrict y) {
    for (size_t i = 0; i < n; ++i) {
        y[i] += alpha * x[i];
    }
}
```

En sistemas de alto rendimiento (compiladores, servidores HTTP, videojuegos), en lugar de invocar miles de veces `malloc` y `free`, se reserva un bloque contiguo y se sirve memoria alineada incrementando un desplazamiento (`Arena Allocator`), liberando toda la región en $O(1)$ al finalizar la petición o fase.

Problema resuelto

Enunciado En el asignador por región `projects/c-arena-allocator`, el desplazamiento actual del búfer es `offset = 19` bytes y se solicita reservar una estructura de 40 B con requisito de alineamiento `align = 8` bytes. El asignador avanza primero `offset` al siguiente múltiplo de 8 (sumando 5 B de relleno hasta 24) y luego suma los 40 B del objeto. Calcula el nuevo valor exacto de `offset` tras la asignación.
Datos

- $offset_0 = 19\,\mathrm{B}$ — Desplazamiento actual antes de la reserva
- $pad = 5\,\mathrm{B}$ — Relleno para alcanzar el múltiplo de 8 (24 B)
- $size = 40\,\mathrm{B}$ — Tamaño del bloque solicitado

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Avance de puntero con alineamiento en Arena Allocator

$$\mathrm{offset}_1 = (19 + 5) + 40$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Alineamos $19 + 5 = 24$ y sumamos los 40 bytes del bloque

$$\mathrm{offset}_1 = (19 + 5) + 40$$

$$((19 + 5) + 40)$$

Resultado

$$Resultado = 64. El desplazamiento final de la arena queda en 64 bytes.$$

Interpretación El desplazamiento final de la arena queda en 64 bytes.
Comprobación de los pasos

- Paso 1: 64 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0009**] **(Básico, Numérico)** Si un puntero `uint64_t *p` apunta a la dirección decimal 4096, ¿a qué dirección decimal apunta `p + 5` sabiendo que `sizeof(uint64_t) = 8`?
2 [**PRB-0010**] **(Básico, Numérico)** Una cadena en C contiene 31 caracteres visibles más su byte terminador `'\0'`. ¿Cuántos bytes mínimos debe reservar `malloc`?
3 [**PRB-0011**] **(Intermedio, Desarrollo por pasos)** En una arena con capacidad total de 4096 bytes, se reservan 12 nodos de 64 bytes ya alineados. ¿Cuántos bytes libres quedan en la arena?
4 [**PRB-0012**] **(Intermedio, Diseño)** Si un paquete binario de telemetría tiene 8 bytes de cabecera y transporta 16 muestras `int16_t` (2 bytes cada una), ¿cuántos bytes mide el paquete completo?

Capítulo 3

Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos

Este capítulo cierra la monografía de C abordando el nivel experto (Bloque F), el ecosistema de construcción y auditoría (Bloque G) y los proyectos prácticos (Bloque H), incluyendo la verificación real del proyecto `projects/c-arena-allocator` compilado con `clang -std=c17 -fsanitize=address,undefined`.

Consejo

Nunca utilices `volatile` como sustituto de `<stdatomic.h>` o `pthread_mutex_t`: una carrera de datos sobre una variable `volatile` no atómica sigue siendo comportamiento indefinido según ISO C11/C17.

Definición

Memoria sombra en AddressSanitizer (ASan Shadow Memory): técnica de instrumentación de compilador donde cada bloque alineado de 8 bytes de la memoria de aplicación se proyecta mediante `(addr » 3) + Offset` sobre 1 byte de memoria sombra que codifica si dichos 8 bytes son direccionables o pertenecen a una zona roja (*redzone*) envenenada.

Autoficha

¿Por qué el patrón `goto cleanup;` hacia adelante al final de una función en C mejora la seguridad de liberación de recursos frente a tener múltiples sentencias `return` dispersas que olvidan liberar descriptores previos?

3.1 Bloque F: Concurrencia C11 (<stdatomic.h>), pthreads, llamadas POSIX y programación defensiva

Un error histórico frecuente en C consiste en emplear el calificador `volatile` creyendo que proporciona sincronización entre hilos. En el estándar ISO C, `volatile` únicamente instruye al compilador para no omitir ni reordenar lecturas/escrituras respecto a registros de hardware mapeados en memoria (MMIO) o manejadores de señales (`sig_atomic_t`), pero **no** garantiza atomicidad ni emite barreras de coherencia de caché entre núcleos de CPU. Para concurrencia multihilo real, C11 introdujo `<stdatomic.h>` (`_Atomic uint64_t`, `atomic_fetch_add_explicit`, `memory_order_acquire`, `memory_order_release`). En sistemas UNIX/POSIX (IEEE Std 1003.1), la gestión de recursos que pueden fallar en cascada dentro de una misma función se estructura idiomáticamente mediante el patrón de limpieza centralizada con etiquetas `goto cleanup:`:

```
int process_file_buffer(size_t bytes) {
    int rc = -1;
    uint8_t *buf_a = malloc(bytes);
    if (!buf_a) goto fail_a;
    uint8_t *buf_b = malloc(bytes);
    if (!buf_b) goto fail_b;
    rc = 0; // operación exitosa
    free(buf_b);
fail_b:
    free(buf_a);
fail_a:
    return rc;
}
```

Este patrón asegura un único camino determinista de liberación en orden inverso de adquisición sin duplicar llamadas a `free`.

3.2 Bloques G y H: Ecosistema (CMake, Meson, pkg-config, Sanitizers) y los 4 proyectos en C

El ecosistema moderno de C se apoya en generadores de sistemas de construcción como `CMake` y `Meson` sobre el ejecutor `Ninja`, el sistema de metadatos de bibliotecas `pkg-config`, pruebas unitarias ligeras y, de forma obligatoria en integración continua, la tríada de sanitizadores de compilador: `AddressSanitizer` (detecta desbordamientos en pila/montículo/globales y *use-after-free* con una sobrecarga media inferior a 2×), `UndefinedBehaviorSanitizer` (detecta desbordamientos con signo, desplazamientos ilegales y punteros nulos o desalineados) y `ThreadSanitizer` (detecta carreras de datos). Los cuatro proyectos del bloque en C son: (1) una utilidad CLI compatible con POSIX (`wc/sha256`) usando E/S por bloques; (2) el asignador por arena y analizador de paquetes binarios con checksum `projects/c-arena-allocator`; (3) una tabla hash abierta genérica con sondeo lineal y factor de carga controlado; y (4) un servidor TCP multicliente no bloqueante basado en multiplexación de descriptores.

Problema resuelto

Enunciado Cuando un programa en C se compila con `-fsanitize=address` (AddressSanitizer), cada 8 B de memoria de aplicación requieren 1 B de memoria sombra (*shadow memory*), más zonas rojas (*redzones*) de 32 B alrededor de cada asignación. Si una aplicación reserva $N = 1000$ bloques de 64 B cada uno (con 32 B de zona roja por bloque), calcula cuántos bytes suman los bloques más sus zonas rojas más la memoria sombra correspondiente ($1/8$ del total).
Datos

- $N = 1000$ — Número de bloques asignados
- $S_{bloque} = 64\,\text{B}$ — Tamaño útil de cada bloque
- $S_{redzone} = 32\,\text{B}$ — Zona roja envenenada por bloque

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Consumo de memoria bajo instrumentación AddressSanitizer (ASan)

$$M = 1000 \cdot (64 + 32) \cdot \frac{9}{8}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos bloque + redzone (96000 B) y multiplicamos por 9/8 para incluir el byte sombra por cada 8 B

$$M = 1000 \cdot (64 + 32) \cdot \frac{9}{8}$$

$$\frac{((1000 \cdot (64 + 32)) \cdot 9)}{8}$$

Resultado

$$\mathit{Resultado} = 108000. \mathit{El conjunto instrumentado ocupa} 108000 \mathit{bytes} (96000 \mathit{B de bloques con redzones y} 12000 \mathit{B de memoria so}$$

Interpretación El conjunto instrumentado ocupa 108 000 bytes (96 000 B de bloques con redzones y 12 000 B de memoria sombra).
Comprobación de los pasos

- Paso 1: 108000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En AddressSanitizer, ¿cuántos bytes de memoria sombra corresponden a 64 000 bytes de memoria de aplicación ($64000 / 8$)?
2 [PRB-0016] (Básico, Numérico) En `projects/c-arena-allocator`, la cabecera de telemetría suma los campos `magic` (10), `version` (2), `flags` (3) y `payload_len` (25). ¿Cuál es el valor del checksum esperado?
3 [PRB-0017] (Intermedio, Desarrollo por pasos) Una función en C adquiere secuencialmente 5 recursos y falla al intentar adquirir el cuarto. ¿Cuántos recursos ya adquiridos debe liberar la cascada `goto cleanup` antes de retornar?
4 [PRB-0018] (Intermedio, Diseño) Un anillo de descriptores E/S tiene capacidad para 256 entradas de 16 bytes cada una. ¿Cuántos bytes mide el anillo completo mapeado en memoria?

Parte II

Monografía Autónoma de C++ (ISO/IEC 14882:2020 / C++23)

Capítulo 4

Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes

Este capítulo inicia la monografía autónoma de C++ moderno (ISO C++20 / C++23). Lejos de tratar a C++ como «C con clases y `new/delete` manuales», enseñamos desde el primer día el paradigma moderno: propiedad automática determinista mediante RAII, vistas no propietarias (`std::string_view`, `std::span`), evaluación en tiempo de compilación (`constexpr/consteval`) y semántica de movimiento.

Consejo

En C++20/C++23, no escribas jamás `new` ni `delete` desnudos en código de aplicación: utiliza `std::make_unique<T>(...)` para propiedad única (cuyo tamaño y coste en ensamblador son idénticos a un puntero crudo `T*`) y pasa vistas `std::span<const T>` o `std::string_view` cuando una función no necesite adquirir propiedad.

Definición

Garantía fuerte frente a excepciones (Strong Exception Safety Guarantee): propiedad de una operación en C++ según la cual, si la operación termina lanzando una excepción, el estado observable del programa permanece idéntico al que tenía antes de iniciar la operación (semántica commit-or-rollback).

Autoficha

¿Por qué `std::vector<T>::push_back` se ve obligado a copiar en lugar de mover los elementos existentes al duplicar su capacidad si el constructor de movimiento `T(T&&)` no está declarado como `noexcept`?

4.1 Bloques A–C: Evolución C++11/14/17/20/23, toolchain, referencias, sobrecarga, constexpr y RAII

Diseñado por Bjarne Stroustrup desde 1979 y estandarizado por ISO en 1998, C++ experimentó una profunda modernización con C++11 (semántica de movimiento, lambdas, `auto`, `constexpr`, `std::unique_ptr`, modelo de memoria multihilo), C++14, C++17 (`std::optional`, `std::variant`, `std::string_view`, `if constexpr`) y C++20/C++23 (`concepts`, `std::ranges`, corrutinas, módulos, `std::span`, `std::expected`). El pilar arquitectónico de C++ es **RAII** (*Resource Acquisition Is Initialization*): todo recurso (memoria del montículo, socket, fichero, cerrojo `std::scoped_lock`) se vincula al ciclo de vida de un objeto alojado en la pila cuyo constructor establece la invariante y cuyo destructor `~T()` libera infaliblemente el recurso al salir del ámbito, incluso si se desenrolla la pila (*stack unwinding*) por el lanzamiento de una excepción.

4.2 Bloque D: Categorías de valores (lvalue/rvalue), Regla de Cero/Cinco, std::unique_ptr y std::span

C++11 clasificó toda expresión según su identidad y movibilidad en `lvalue` (tiene identidad persistente), `xvalue` (*expiring value*, tiene identidad pero sus recursos pueden ser robados) y `prvalue` (temporal puro). La **Regla de Cero** establece que las clases de lógica de negocio no deben escribir destructores, constructores de copia ni de movimiento manuales, sino componer miembros RAII (`std::vector`,

`std::string`, `std::unique_ptr`). Si una clase gestiona directamente un manejador crudo del sistema, aplica la **Regla de Cinco**: definir explícitamente destructor, constructor de copia, asignación por copia, constructor de movimiento (`noexcept`) y asignación por movimiento (`noexcept`):

```
#include <memory>
#include <span>
#include <numeric>

// std::span<const double> observa cualquier array, std::vector o std::array sin copiar:
constexpr double sum_window(std::span<const double> samples) noexcept {
    double acc = 0.0;
    for (double v : samples) { acc += v; }
    return acc;
}
```

Es fundamental marcar los constructores de movimiento como `noexcept`: `std::vector` solo mueve sus elementos durante una realojación si el constructor de movimiento promete `noexcept`; de lo contrario, cae de vuelta a copiar cada elemento para preservar la garantía fuerte frente a excepciones (*strong exception safety guarantee*).

Problema resuelto

Enunciado En una implementación estándar de C++ de 64 bits donde `sizeof(std::string) = 24` bytes y la optimización de cadena corta (*Small String Optimization*, SSO) almacena en línea cadenas de hasta 22 B sin alojar en el montículo, un programa crea un `std::vector<std::string>` con $N = 5000$ identificadores cortos de 12 B cada uno. Calcula cuántos bytes totales ocupan los 5000 objetos `std::string` dentro del búfer contiguo del vector (sabiendo que realizan 0 alojamientos adicionales fuera del vector).

Datos

- $N = 5000$ — Número de cadenas cortas de 12 B (<= 22 B SSO)
- $S_{str} = 24\,\mathrm{B}$ — Tamaño de cada objeto std::string en 64 bits

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Huella de memoria con Small String Optimization (SSO) en C++

$$M = 5000 \cdot 24$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 5000 instancias por 24 B (sin alojamientos externos gracias a SSO)

$$M = 5000 \cdot 24$$

$$(5000 \cdot 24)$$

Resultado

$$\mathit{Resultado} = 120000. \mathit{Los} 5000 \mathit{identificadores se almacenan contiguamente en} 120000 \mathit{bytes con cero llamadas individuales a}$$

Interpretación Los 5000 identificadores se almacenan contiguamente en 120 000 bytes con cero llamadas individuales al montículo por cadena.

Comprobación de los pasos

- Paso 1: 120000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) ¿Cuántos bytes ocupa `std::unique_ptr<double>` con deleción por defecto en una arquitectura de 64 bits?
2 [PRB-0022] (Básico, Numérico) Un `std::span<const int>` en C++20 con extensión dinámica almacena un puntero (8 B) y una longitud `size_t` (8 B). ¿Cuántos bytes ocupa en la pila?
3 [PRB-0023] (Intermedio, Desarrollo por pasos) Un `std::vector<int>` duplica su capacidad geométricamente desde 16 hasta 1024 elementos (`sizeof(int) = 4`). ¿Cuántos bytes reserva el búfer final de 1024 enteros?
4 [PRB-0024] (Intermedio, Diseño) Un bloque de control de `std::make_shared<T>` combina en una única reserva de montículo los dos contadores atómicos (16 B) más el objeto `T` de 48 B. ¿Cuántos bytes mide esa única reserva?

Capítulo 5

Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos

Este capítulo completa la monografía de C++ desarrollando los Bloques E, F, G y H: desde la programación genérica con `concepts` y `std::ranges` hasta la concurrencia cooperativa con `std::jthread` y la verificación del planificador de tareas `projects/cpp-task-scheduler`.

Consejo

Sustituye `std::thread` por `std::jthread` en todo código C++20 nuevo: si una función lanza una excepción antes de llamar a `.join()` sobre un `std::thread` convencional, su destructor invoca `std::terminate()` abortando el proceso.

Definición

Concepto en C++20 (`concept`): predicado nombrado evaluado en tiempo de compilación que restringe el conjunto de argumentos de plantilla aceptados por una función o clase genérica y participa en la resolución de sobrecarga por subsunción lógica.

Autoficha

¿Cómo decide el compilador de C++20 qué sobrecarga de plantilla es más específica cuando dos funciones restringidas por `concepts` distintos son válidas para un mismo tipo (regla de subsunción)?

5.1 Bloques E y F: Plantillas, C++20 Concepts, std::ranges, std::jthread y corrutinas

Históricamente, las plantillas (*templates*) en C++ dependían de SFINAE (*Substitution Failure Is Not An Error*), produciendo diagnósticos de compilación crípticos cuando un tipo no cumplía una operación interna. C++20 resolvió este problema introduciendo **Concepts**: predicados en tiempo de compilación que especifican formalmente los requisitos sintácticos y semánticos de un parámetro de plantilla. En nuestro proyecto verificado `projects/cpp-task-scheduler`, el concepto `ExecutableTask` exige que cualquier tipo de tarea proporcione `id()`, `priority()` convertible a `int` y `execute()` convertible a `int64_t`:

```
#include <concepts>
#include <cstdint>

template <typename T>
concept ExecutableTask = requires(const T& t) {
    { t.id() } -> std::convertible_to<uint32_t>;
    { t.priority() } -> std::convertible_to<int>;
    { t.execute() } -> std::convertible_to<int64_t>;
};
```

Junto con los conceptos, la biblioteca `<ranges>` permite encadenar vistas perezosas mediante el operador tubería `|`, como `tasks | std::views::filter(...) | std::views::transform(...)`, y `<thread>` incorpora `std::jthread`, que se une automáticamente (`join()`) en su destructor e integra cancelación cooperativa mediante `std::stop_token`.

5.2 Bloques G y H: Ecosistema CMake/vcpkg/Conan/Clang-Tidy y los 4 proyectos (incluyendo cpp-task-scheduler)

En el Bloque G (Ecosistema), los proyectos profesionales de C++ se estructuran mediante `CMake` moderno basado en objetivos (`target_link_libraries`, `target_compile_features(tgt PUBLIC cxx_std_20)`), gestores de paquetes `vcpkg` o `Conan 2`, pruebas con `GoogleTest` o `Catch2`, benchmarking con `Google Benchmark` y auditoría estática continua con `clang-tidy` (perfiles `cppcoreguidelines-*`, `modernize-*`, `bugprone-*`). En el Bloque H, además de una biblioteca de álgebra lineal estática `constexpr`, un motor de eventos con `std::variant` y un servidor de red asíncrono, el lector cuenta con el proyecto completo `projects/cpp-task-scheduler`, que implementa una cola de prioridad determinista `TaskScheduler<Task>` verificada con `clang++ -std=c++20 -fsanitize=address,undefined`.

Problema resuelto

Enunciado En el proyecto `projects/cpp-task-scheduler`, se encolan cuatro tareas `(id, prioridad, base_cost)`: `(1, 2, 10)`, `(2, 9, 10)`, `(3, 5, 20)` y `(4, 9, 15)`. Si se ejecuta `execute_filtered(5)` (que filtra las tareas con `priority >= 5` y suma `base_cost * priority` de cada una), calcula el resultado exacto devuelto.
Datos

- $T_2 = prioridad 9, coste 10$ — Aporta 9 * 10 = 90
- $T_4 = prioridad 9, coste 15$ — Aporta 9 * 15 = 135
- $T_3 = prioridad 5, coste 20$ — Aporta 5 * 20 = 100

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Filtrado y reducción en cpp-task-scheduler con std::views

$$R = 9 \cdot 10 + 9 \cdot 15 + 5 \cdot 20$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Descartamos la tarea 1 (prioridad $2 < 5$) y sumamos los productos de las tres restantes

$$R = 9 \cdot 10 + 9 \cdot 15 + 5 \cdot 20$$

$$(((9 \cdot 10) + (9 \cdot 15)) + (5 \cdot 20))$$

Resultado

$$Resultado = 325. El planificador devuelve exactamente 325, coincidiendo con el test ejecutable de `projects/cpp-task-sch$$

Interpretación El planificador devuelve exactamente 325, coincidiendo con el test ejecutable de `projects/cpp-task-scheduler`.
Comprobación de los pasos

- Paso 1: 325 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) En `cpp-task-scheduler`, ¿qué valor devuelve `ComputeTask{10, 7, 12}.execute()` (12×7)?

2 [PRB-0028] (Básico, Numérico) Una plantilla variádica `sum_all(10, 20, 30, 40)` utiliza una expresión pliegue (*fold expression* `(args + ...)`). ¿Cuál es el resultado evaluado en `constexpr`?

3 [PRB-0029] (Intermedio, Desarrollo por pasos) Un `std::variant<int, double, std::array<char, 32>>` almacena la alternativa mayor (32 B) más un índice alineado a 8 B. ¿Cuántos bytes ocupa la instancia?

4 [PRB-0030] (Intermedio, Diseño) Si una vista `std::views::iota(1, 1001) | std::views::filter(...) | std::views::take(5)` detiene la evaluación al encontrar los primeros 5 pares ($2, 4, 6, 8, 10$), ¿cuántos enteros del rango $[1, 1000]$ llegaron a inspeccionarse?

Parte III

Monografías Especializadas de Sistemas: Zig, Ensamblador, Ada y Fortran

Capítulo 6

Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C

*Este capítulo presenta la monografía completa de **Zig** siguiendo los bloques $A-H$. Creado por Andrew Kelley, Zig se concibe como un sucesor moderno y transparente de C donde ninguna expresión oculta llamadas a funciones ni reservas de memoria, los tipos son valores de primera clase en tiempo de compilación (`comptime`) y el propio binario `zig` incluye un compilador C/C++ de compilación cruzada basado en Clang/LLVM.*

Consejo

En todas tus pruebas unitarias de Zig (`test "..." { ... }`), pasa siempre `std.testing.allocator` a las estructuras bajo prueba: el arnés de pruebas hará fallar automáticamente el test si queda un solo byte sin liberar o si ocurre doble liberación.

Definición

Limpieza condicional de error (`errdefer` en Zig): sentencia registrada en la pila léxica del bloque actual cuya expresión asociada se ejecuta si y solo si la función abandona dicho ámbito devolviendo un valor de error de su unión de retorno `!T`.

Autoficha

¿Qué ventaja arquitectónica aporta que `std.ArrayList` en Zig reciba un `Allocator` explícito en lugar de invocar un asignador global fijo como `std::vector` por defecto en C++ o `Vec<T>` en Rust estable?

6.1 Bloques A–D: Filosofía de Zig, toolchain integrada, tipos, rebanadas, uniones de error y defer/errdefer

Zig se rige por tres mandamientos de ingeniería de sistemas: (1) sin flujo de control oculto (no hay sobrecarga de operadores, ni constructores/destructores implícitos, ni excepciones); (2) sin asignaciones de memoria ocultas (toda función de la biblioteca estándar que necesita alojar en el montículo recibe explícitamente un parámetro `allocator: std.mem.Allocator`); y (3) sin preprocesador de macros textuales. Para gestionar recursos y errores con precisión quirúrgica, Zig combina tipos de unión de error (`AllocError![]u8`) con las palabras clave `try`, `catch`, `defer` (que se ejecuta al salir del bloque actual) y `errdefer` (que se ejecuta **únicamente** si el bloque actual retorna con un error):

```
const std = @import("std");

pub fn allocateInitializedRing(allocator: std.mem.Allocator, n: usize) ![]u64 {
    const buf = try allocator.alloc(u64, n);
    errdefer allocator.free(buf);
    for (buf, 0..) |*slot, idx| {
        slot.* = @as(u64, @intCast(idx)) * 10;
    }
    return buf;
}
```

Si cualquier paso posterior dentro de la función fallara y retornara un error mediante `try`, `errdefer` libera automáticamente `buf` sin necesidad de etiquetas `goto`.

6.2 Bloques E–H: Asignadores explícitos (Arena, FixedBuffer, GPA), comptime, FFI con C y proyectos

En lugar de plantillas o macros, el polimorfismo paramétrico en Zig consiste en funciones ordinarias que aceptan argumentos `comptime T: type` y devuelven un `type` (`fn BoundedQueue(comptime T: type, comptime cap: usize) type`). Gracias a que los asignadores se pasan como valores de interfaz `std.mem.Allocator`, el mismo contenedor `std.ArrayList(T)` puede ejecutarse sobre el asignador detector de fugas `std.testing.allocator` durante `zig test`, sobre un `std.heap.ArenaAllocator` por petición HTTP, o sobre un `std.heap.FixedBufferAllocator` respaldado por un array estático en la pila sin tocar jamás el sistema operativo.

Problema resuelto

Enunciado En un microservicio escrito en Zig, cada petición HTTP utiliza un `std.heap.FixedBufferAllocator` respaldado por un array en pila de $B = 16384$ B (16 KiB). Si el manejador aloja una cabecera parseada de 512 B y $N = 60$ cabeceras clave-valor de 128 B cada una, calcula cuántos bytes quedan disponibles en el `FixedBufferAllocator` antes de que devuelva `error.OutOfMemory`.

Datos

- $B = 16384\,\mathrm{B}$ — Capacidad total del búfer estático en pila
- $S_{hdr} = 512\,\mathrm{B}$ — Reserva inicial para el descriptor de petición
- $N_{kv}^{S} = 60,128B$ — 60 pares clave-valor de 128 B

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Contabilidad lineal en FixedBufferAllocator de Zig

$$B_{\text{libre}} = 16384 - (512 + 60 \cdot 128)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos los 512 B y los 60 * 128 = 7680 B a la capacidad total de 16384 B

$$B_{\text{libre}} = 16384 - (512 + 60 \cdot 128)$$

$$(16384 - (512 + (60 \cdot 128)))$$

Resultado

$$Resultado = 8192. Quedan exactamente 8192 bytes (8 KiB) libres en el búfer de la petición.$$

Interpretación Quedan exactamente 8192 bytes (8 KiB) libres en el búfer de la petición.

Comprobación de los pasos

- Paso 1: 8192 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0033**] **(Básico, Numérico)** Una rebanada `[]const u32` en Zig de 64 bits consta de un puntero `[*]const u32` (8 B) y una longitud `usize` (8 B). ¿Cuántos bytes ocupa?
2 [**PRB-0034**] **(Básico, Numérico)** En Zig, el tipo entero de anchura arbitraria `u12` se aloja en memoria ocupando el siguiente número entero de bytes potencia de dos (`@sizeOf(u12)`). ¿Cuántos bytes ocupa en memoria?
3 [**PRB-0035**] **(Intermedio, Desarrollo por pasos)** Un `packed struct` de Zig contiene campos `u3`, `u5`, `u16` y `u8` empaquetados al bit exacto. ¿Cuántos bytes ocupa `(3 + 5 + 16 + 8) / 8`?
4 [**PRB-0036**] **(Intermedio, Diseño)** Si una función con 4 reservas secuenciales protegidas cada una por `errdefer` falla en la cuarta reserva, ¿cuántos bloques previos son liberados automáticamente por los `errdefer` ya registrados?

Capítulo 7

Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A–H): Registros, ABI, Pila y SIMD

*Todo ingeniero de sistemas debe ser capaz de leer el código ensamblador emitido por el compilador (`clang -S -O2` u `objdump -d`), diagnosticar volcados de pila y escribir rutinas críticas vectorizadas o cambios de contexto de sistema operativo. Este capítulo ofrece el bloque completo A – H para las dos arquitecturas dominantes de 64 bits: **x86-64 (AMD64)** y **ARM64 (AArch64)**.*

System V AMD64 (x86-64)	**AAPCS64 (ARM64 / AArch64)**
Args 1–6: `rdi, rsi, rdx, rcx, r8, r9`	Args 1–8: `x0, x1, x2, x3, x4, x5, x6, x7`
Retorno: `rax` (`rdx`)	Retorno: `x0` (`x1`)
Callee-saved: `rbx, rbp, r12-r15`	Callee-saved: `x19-x28, x29(fp), x30(lr)`

Figura 2. Comparativa de paso de los primeros 6 argumentos enteros/punteros y valor de retorno entre la ABI System V x86-64 y la ABI AAPCS64 (ARM64).

Consejo

En lugar de escribir funciones enteras en ensamblador manual que dificultan las optimizaciones del compilador, prefiere funciones intrínsecas SIMD (`<arm_neon.h>` o `<immintrin.h>`) y verifica el ensamblador generado con `clang -O3 -S`.

Definición

Registros preservados por el llamado (Callee-saved registers): subconjunto de registros de la CPU (`rbx, rbp, r12–r15` en System V x86-64; `x19–x28, x29, x30` en AAPCS64) cuyo valor original debe ser restaurado por cualquier subrutina antes de ejecutar `ret` si dicha subrutina decide modificarlos.

Autoficha

¿Por qué en x86-64 al entrar en el cuerpo de una función tras ejecutar la instrucción `call` (que empuja 8 bytes de dirección de retorno en la pila), el registro `rsp` tiene un valor congruente con 8 módulo 16 y requiere restar $8 + 16k$ bytes antes de hacer otro `call`?

7.1 Bloques A–D: Arquitectura de registros x86-64 y ARM64 (AArch64), modos de direccionamiento y saltos

En **x86-64**, la CPU dispone de 16 registros de propósito general de 64 bits (`rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp`, `rsp`, `r8–r15`) e instrucciones de longitud variable (de 1 a 15 bytes) que permiten combinar acceso a memoria y operación aritmética en una sola instrucción. En **ARM64 (AArch64)**, la arquitectura es estrictamente *load/store* con instrucciones de anchura fija de 32 bits (4 bytes) y 31 registros de propósito general de 64 bits (`x0–x30`, con sus mitades inferiores de 32 bits `w0–w30`), más el registro cero `xzr` y el puntero de pila `sp`. En ARM64, las operaciones aritméticas operan exclusivamente entre registros; la memoria solo se toca mediante instrucciones de carga y almacenamiento como `ldr`, `str`, `ldp` (cargar par de registros) y `stp` (almacenar par de registros).

7.2 Bloques E–H: Convenciones ABI (System V y AAPCS64), alineamiento de pila, SIMD (NEON/AVX) e inspección con objdump

Para que módulos compilados por distintos lenguajes (C, C++, Rust, Zig) puedan invocarse entre sí, deben respetar la **ABI** (*Application Binary Interface*) de la plataforma. Como resume la Figura \ref{fig:fig-abi-calling-conventions}, en la ABI System V x86-64 (Linux, macOS, BSD) los primeros 6 argumentos enteros o punteros se pasan en `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, mientras que en AAPCS64 (ARM64) los primeros 8 se pasan en `x0`–`x7`. En ambas arquitecturas, el puntero de pila (`rsp` o `sp`) debe mantenerse alineado a un múltiplo de **16 bytes** en el punto de llamada a cualquier función (`call` o `bl`). Compare la misma función `uint64_t fma_u64(uint64_t a, uint64_t b, uint64_t c)` que calcula $a \cdot b + c$ en ambas arquitecturas:

```
; System V x86-64 (a=rdi, b=rsi, c=rdx -> retorno en rax):
fma_u64_x86:
    mov     rax, rdi
    imul    rax, rsi
    add     rax, rdx
    ret

; AAPCS64 ARM64 (a=x0, b=x1, c=x2 -> retorno en x0, una sola instrucción madd):
fma_u64_arm64:
    madd    x0, x0, x1, x2   ; x0 = (x0 * x1) + x2
    ret
```

En ARM64, el prólogo estándar de una función no hoja guarda el puntero de marco `x29` (`fp`) y el registro de enlace `x30` (`lr`) reservando un múltiplo de 16 bytes en una sola instrucción con pre-indexación: `stp x29, x30, [sp, #-16]!`.

Problema resuelto

Enunciado Una función en ensamblador ARM64 guarda en la pila el par `(x29, x30)` (16 B), dos registros preservados `(x19, x20)` (16 B) y necesita alojar un búfer local de 36 B. Como el hardware AArch64 exige que el puntero de pila `sp` permanezca siempre alineado a múltiplo de 16 B (redondeando los 36 B locales hacia arriba a 48 B), calcula cuántos bytes totales resta al registro `sp` el prólogo de la función.

Datos

- $S_{fplr} = 16\,\mathrm{B}$ — Par de registros (x29, x30)
- $S_{callee} = 16\,\mathrm{B}$ — Par de registros preservados (x19, x20)
- $S_{\updownarrow laligned} = 48\,\mathrm{B}$ — 36 B redondeados al siguiente múltiplo de 16 B

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Cálculo del tamaño de marco de pila alineado a 16 bytes en AAPCS64

$$S_{\text{frame}} = 16 + 16 + 48$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los tres bloques alineados a 16 bytes

$$S_{\text{frame}} = 16 + 16 + 48$$

$$((16 + 16) + 48)$$

Resultado

$$Resultado = 80. El prólogo reserva exactamente 80 bytes (múltiplo exacto de 16 : 5 * 16 = 80 B) en la pila.$$

Interpretación El prólogo reserva exactamente 80 bytes (múltiplo exacto de 16: 5 * 16 = 80 B) en la pila.

Comprobación de los pasos

- Paso 1: 80 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0039] (Básico, Numérico) En ARM64, cada instrucción máquina tiene una anchura fija de 32 bits (4 bytes). ¿Cuántos bytes ocupan en la sección `.text` una secuencia de 250 instrucciones?
2 [PRB-0040] (Básico, Numérico) Un registro vectorial NEON de 128 bits en ARM64 (`v0.4s`) procesa enteros o flotantes de 32 bits en paralelo. ¿Cuántos elementos de 32 bits opera por instrucción?
3 [PRB-0041] (Intermedio, Desarrollo por pasos) Una función en System V x86-64 recibe 9 argumentos enteros de 64 bits. Sabiendo que los primeros 6 van en registros y el resto se pasa por la pila (8 B cada uno), ¿cuántos bytes ocupan en la pila los argumentos restantes?
4 [PRB-0042] (Intermedio, Diseño) Un bucle procesa un array de 4096 flotantes `f32` utilizando instrucciones AVX-512 (16 flotantes `f32` por registro de 512 bits). ¿Cuántas iteraciones vectoriales ejecuta el bucle?

Capítulo 8

Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A–H)

*Este capítulo cierra el Volumen IV con las monografías completas de dos lenguajes insustituibles en sus dominios de alta exigencia: **Ada 2022 / SPARK** en aviónica (DO-178C Nivel A), ferrocarriles y espacio, y **Fortran 2018/2023** en supercomputación (HPC), dinámica de fluidos computacional (CFD) y predicción meteorológica.*

Consejo

En Fortran moderno, comienza siempre todo módulo y programa con `implicit none` e importa `real64` (`dp`) e `int64` desde el módulo estándar `iso_fortran_env`; al anidar bucles sobre `A(i, j)`, sitúa siempre el índice `i` en el bucle más interno y `j` en el externo para recorrer la memoria secuencialmente.

Definición

Orden mayor por columnas (Column-Major Order en Fortran): disposición lineal en memoria de un array bidimensional $A(1..M, 1..N)$ según la cual el elemento $A(i, j)$ (con índices basados en 1) se ubica en el desplazamiento lineal $(j-1)\cdot M+(i-1)$, haciendo contiguos los elementos de una misma columna.

Autoficha

¿Por qué intercambiar el orden de los bucles `do i = 1, M; do j = 1, N` sobre `A(i, j)` en Fortran degrada severamente la tasa de aciertos de caché L1 respecto a `do j = 1, N; do i = 1, M`?

8.1 Monografía de Ada 2022 y SPARK (Bloques A–H): Tipado fuerte de dominio, contratos y sistemas críticos

Encargado por el Departamento de Defensa de EE. UU. (Jean Ichbiah, 1980; revisiones ISO Ada 95, 2005, 2012 y 2022), **Ada** fue diseñado para sistemas embebidos de misión crítica donde un defecto de software puede costar vidas humanas. En Ada, el ingeniero no declara variables como meros `int`, sino que define tipos nominalmente distintos con su rango físico exacto: `type Altitude_Meters is range -500 .. 50_000;` y `type Velocity_Knots is range 0 .. 2_500;`. Sumar una altitud con una velocidad es un error de compilación inmediato, y asignar un valor fuera de rango lanza `Constraint_Error`. En el subconjunto formalmente verificable **SPARK**, los aspectos `Pre`, `Post` y `Global` se traducen en obligaciones de prueba matemática descargadas por solucionadores SMT (`Z3`, `Alt-Ergo`) mediante `gnatprove`, demostrando la ausencia total de desbordamientos o divisiones por cero antes de compilar:

```
package Flight_Envelope with SPARK_Mode is
   type Altitude_M is range 0 .. 20_000;
   function Safe_Climb (Current : Altitude_M; Delta_H : Altitude_M) return Altitude_M
     with Pre  => Integer (Current) + Integer (Delta_H) <= 20_000,
          Post => Safe_Climb'Result = Current + Delta_H;
end Flight_Envelope;
```

Para concurrencia de tiempo real duro, el perfil **Ravenscar** restringe las tareas (`task`) y los objetos protegidos (`protected object`) al protocolo de techo de prioridad inmediata (*Immediate Ceiling Priority Protocol*), garantizando ausencia matemática de interbloqueos (*deadlocks*) y acotación estática del tiempo de respuesta.

8.2 Monografía de Fortran 2018/2023 (Bloques A–H): Arrays multidimensionales, vectorización y Coarrays

Por su parte, **Fortran** (John Backus, IBM, 1957; modernizado radicalmente en Fortran 90/95, 2003, 2008, 2018 y 2023) sigue siendo el estándar de oro en supercomputación científica por dos razones técnicas: primero, los arrays multidimensionales son ciudadanos de primera clase almacenados en **orden mayor por columnas** (*column-major*, donde `A(i, j)` es contiguo en memoria al variar el primer índice `i`); segundo, el estándar Fortran garantiza por defecto que los argumentos `intent(in)` e `intent(inout)` de una subrutina `pure` **no presentan alias entre sí** (a menos que tengan el atributo `pointer` o `target`), lo que permite a compiladores como `gfortran`, `ifx` o `nvfortran` vectorizar bucles matriciales con una agresividad superior a la de C sin `restrict`:

```
module numerics_mod
  use, intrinsic :: iso_fortran_env, only: dp => real64
  implicit none
contains
  pure subroutine laplacian_1d(u, d2u, inv_dx2)
    real(dp), intent(in)  :: u(:), inv_dx2
    real(dp), intent(out) :: d2u(:)
    integer :: n
    n = size(u)
    d2u(1) = 0.0_dp
    d2u(2:n-1) = (u(1:n-2) - 2.0_dp * u(2:n-1) + u(3:n)) * inv_dx2
    d2u(n) = 0.0_dp
  end subroutine laplacian_1d
end module numerics_mod
```

Además, Fortran 2008/2018 incorpora paralelismo de memoria distribuida directamente en la sintaxis del lenguaje mediante **Coarrays** (`real(dp) :: field(1000)[*]`), donde la notación entre corchetes `[imagen]` realiza transferencias RDMA de un solo lado entre nodos de un clúster sin llamadas verbosas a MPI.

Problema resuelto

Enunciado En un programa Fortran 2018, una matriz `real(real64) :: A(100, 200)` ($M = 100$ filas, $N = 200$ columnas, $S = 8$ B por elemento) está almacenada en orden mayor por columnas a partir del desplazamiento base 0. Calcula el desplazamiento exacto en bytes del elemento `A(25, 4)` usando la fórmula $8 \cdot ((4-1) \cdot 100 + (25-1))$.

Datos

- $M = 100$ — Número de filas por columna
- $i^j = 25, 4$ — Fila 25, columna 4 (base 1)
- $S = 8\,\mathrm{B}$ — Tamaño de real(real64)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Direccionamiento Column-Major en Fortran

$$\text{Offset} = 8 \cdot ((4-1) \cdot 100 + (25-1))$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el índice lineal (3 * 100 + 24 = 324) y multiplicamos por 8 bytes

$$\text{Offset} = 8 \cdot ((4-1) \cdot 100 + (25-1))$$

$$(8 \cdot (((4-1) \cdot 100) + (25-1)))$$

Resultado

$$Resultado = 2592. El elemento A(25, 4) se encuentra exactamente a 2592 bytes del inicio de la matriz.$$

Interpretación El elemento A(25, 4) se encuentra exactamente a 2592 bytes del inicio de la matriz.
Comprobación de los pasos

- Paso 1: 2592 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0045**] **(Básico, Numérico)** En Ada, el tipo `type Percent is range 0 .. 100;` comprende desde 0 hasta 100 ambos inclusive. ¿Cuántos valores distintos pertenecen al tipo (`Percent'Last - Percent'First + 1`)?
2 [**PRB-0046**] **(Básico, Numérico)** En Fortran, una rebanada vectorial `u(2:n-1)` para $n = 1002$ evalúa los puntos interiores de una malla 1D. ¿Cuántos puntos interiores actualiza la sentencia?
3 [**PRB-0047**] **(Intermedio, Desarrollo por pasos)** Un programa Coarray Fortran se ejecuta en 16 imágenes (`num_images() = 16`) y cada imagen aloja una malla local de 50000 flotantes `real64` (8 B). ¿Cuántos bytes suman las mallas de las 16 imágenes?
4 [**PRB-0048**] **(Intermedio, Diseño)** En un sistema de tiempo real Ada Ravenscar con periodo de marco mayor $T = 100$ ms, tres tareas periódicas consumen 15 ms, 22 ms y 18 ms por cada 100 ms. ¿Cuál es el porcentaje entero de utilización total de CPU?

Apéndice A

Notación

Símbolo	Significado	Unidades
alignof(T)	Requisito de alineamiento de dirección en C11/C23 y C++11/C++20	B
sizeof(T)	Tamaño en bytes de un objeto o tipo completo en C/C++	B
$k + p$	Aritmética de punteros escalada por k * sizeof(*p) bytes	dirección

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante

Problema resuelto [PRB-0001]
En una arquitectura de 32 bits para enteros `uint32_t` ($w = 32$, módulo $2^{32} = 4294967296$), un entero con signo con valor -100 se convierte implícitamente a `uint32_t` durante una comparación. Calcula el valor numérico exacto que toma tras la conversión según la regla $2^{32} - 100$.
Respuesta

$$Resultado = 4294967196. El valor resultante en uint32_t es 4294967196, razón por la cual `-Wsign-conversion` es indispensable$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
Calcula el valor en `uint16_t` ($w = 16$) resultante de convertir -1 a sin signo ($2^{16} - 1$).
Respuesta 65535
Ejercicio 2 [PRB-0004]
Un registro de control de 8 bits activa los bits 0, 3 y 7 ($\$2^0 + 2^3 + 2^7\$$). ¿Cuál es el valor decimal de la máscara `uint8_t`?
Respuesta 137
Ejercicio 3 [PRB-0005]
¿Cuántos bytes ocupa en total una estructura con 4 campos `uint32_t` y 2 campos `uint64_t` perfectamente alineados ($4 \times 4 + 2 \times 8$)?
Respuesta 32
Ejercicio 4 [PRB-0006]
Si compilamos un proyecto en C formado por 18 unidades de traducción `.c` y solo modificamos un fichero `.c`, ¿cuántas unidades de traducción evita recompilar `make`/`ninja`?
Respuesta 17

Cálculos reproducidos

$$\left((2)^{32} - 100\right) = 4294967196$$

C.2 Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing

Problema resuelto [PRB-0007]
En el asignador por región `projects/c-arena-allocator`, el desplazamiento actual del búfer es `offset = 19` bytes y se solicita reservar una estructura de 40 B con requisito de alineamiento `align = 8` bytes. El asignador avanza primero `offset` al siguiente múltiplo de 8 (sumando 5 B de relleno hasta 24) y luego suma los 40 B del objeto. Calcula el nuevo valor exacto de `offset` tras la asignación.
Respuesta

$$Resultado = 64. El desplazamiento final del la arena queda en 64 bytes.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]

Si un puntero `uint64_t *p` apunta a la dirección decimal 4096, ¿a qué dirección decimal apunta `p + 5` sabiendo que `sizeof(uint64_t) = 8`?
Respuesta 4136
Ejercicio 2 [PRB-0010]
Una cadena en C contiene 31 caracteres visibles más su byte terminador `'\0'`. ¿Cuántos bytes mínimos debe reservar `malloc`?
Respuesta 32
Ejercicio 3 [PRB-0011]
En una arena con capacidad total de 4096 bytes, se reservan 12 nodos de 64 bytes ya alineados. ¿Cuántos bytes libres quedan en la arena?
Respuesta 3328
Ejercicio 4 [PRB-0012]
Si un paquete binario de telemetría tiene 8 bytes de cabecera y transporta 16 muestras `int16_t` (2 bytes cada una), ¿cuántos bytes mide el paquete completo?
Respuesta 40

Cálculos reproducidos

$((19 + 5) + 40) = 64$

C.3 Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos

Problema resuelto [PRB-0013]
Cuando un programa en C se compila con `-fsanitize=address` (AddressSanitizer), cada 8 B de memoria de aplicación requieren 1 B de memoria sombra (*shadow memory*), más zonas rojas (*redzones*) de 32 B alrededor de cada asignación. Si una aplicación reserva $N = 1000$ bloques de 64 B cada uno (con 32 B de zona roja por bloque), calcula cuántos bytes suman los bloques más sus zonas rojas más la memoria sombra correspondiente ($1/8$ del total).
Respuesta

$$Resultado = 108000. El conjunto instrumentado ocupa 108000 bytes (96000 B de bloques con redzones y 12000 B de memoria som$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0015]
En AddressSanitizer, ¿cuántos bytes de memoria sombra corresponden a 64 000 bytes de memoria de aplicación ($64000 / 8$)?
Respuesta 8000
Ejercicio 2 [PRB-0016]
En `projects/c-arena-allocator`, la cabecera de telemetría suma los campos `magic` (10), `version` (2), `flags` (3) y `payload_len` (25). ¿Cuál es el valor del checksum esperado?
Respuesta 40
Ejercicio 3 [PRB-0017]
Una función en C adquiere secuencialmente 5 recursos y falla al intentar adquirir el cuarto. ¿Cuántos recursos ya adquiridos debe liberar la cascada `goto cleanup` antes de retornar?
Respuesta 3
Ejercicio 4 [PRB-0018]
Un anillo de descriptores E/S tiene capacidad para 256 entradas de 16 bytes cada una. ¿Cuántos bytes mide el anillo completo mapeado en memoria?
Respuesta 4096

Cálculos reproducidos

$\frac{((1000 \cdot (64+32)) \cdot 9)}{8} = 108000$

C.4 Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes

Problema resuelto [PRB-0019]
En una implementación estándar de C++ de 64 bits donde `sizeof(std::string) = 24` bytes y la optimización de cadena corta (*Small String Optimization*, SSO) almacena en línea cadenas de hasta 22 B sin alojar en el montículo, un programa crea un `std::vector<std::string>` con $N = 5000$ identificadores cortos de 12 B cada uno. Calcula cuántos bytes totales ocupan los 5000 objetos `std::string` dentro del búfer contiguo del vector (sabiendo que realizan 0 alojamientos adicionales fuera del vector).
Respuesta

$$Resultado = 120000. Los 5000 identificadores se almacenan contiguamente en 120000 bytes con cero llamadas individuales al$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
¿Cuántos bytes ocupa `std::unique_ptr<double>` con deleción por defecto en una arquitectura de 64 bits?
Respuesta 8
Ejercicio 2 [PRB-0022]
Un `std::span<const int>` en C++20 con extensión dinámica almacena un puntero (8 B) y una longitud `size_t` (8 B). ¿Cuántos bytes ocupa en la pila?
Respuesta 16
Ejercicio 3 [PRB-0023]
Un `std::vector<int>` duplica su capacidad geométricamente desde 16 hasta 1024 elementos (`sizeof(int) = 4`). ¿Cuántos bytes reserva el búfer final de 1024 enteros?
Respuesta 4096
Ejercicio 4 [PRB-0024]
Un bloque de control de `std::make_shared<T>` combina en una única reserva de montículo los dos contadores atómicos (16 B) más el objeto `T` de 48 B. ¿Cuántos bytes mide esa única reserva?
Respuesta 64

Cálculos reproducidos

$$(5000 \cdot 24) = 120000$$

C.5 Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos

Problema resuelto [PRB-0025]
En el proyecto `projects/cpp-task-scheduler`, se encolan cuatro tareas `(id, prioridad, base_cost)`: `(1, 2, 10)`, `(2, 9, 10)`, `(3, 5, 20)` y `(4, 9, 15)`. Si se ejecuta `execute_filtered(5)` (que filtra las tareas con `priority >= 5` y suma `base_cost * priority` de cada una), calcula el resultado exacto devuelto.
Respuesta

$$Resultado = 325. El planificador devuelve exactamente 325, coincidiendo con el test ejecutable de `projects/cpp-task-sched$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
En `cpp-task-scheduler`, ¿qué valor devuelve `ComputeTask{10, 7, 12}.execute()` (12×7)?

Respuesta 84

Ejercicio 2 [PRB-0028]
Una plantilla variádica `sum_all(10, 20, 30, 40)` utiliza una expresión pliegue (*fold expression* `(args + ...)`). ¿Cuál es el resultado evaluado en `constexpr`?

Respuesta 100

Ejercicio 3 [PRB-0029]
Un `std::variant<int, double, std::array<char, 32>>` almacena la alternativa mayor (32 B) más un índice alineado a 8 B. ¿Cuántos bytes ocupa la instancia?

Respuesta 40

Ejercicio 4 [PRB-0030]
Si una vista `std::views::iota(1, 1001) | std::views::filter(...) | std::views::take(5)` detiene la evaluación al encontrar los primeros 5 pares ($2, 4, 6, 8, 10$), ¿cuántos enteros del rango $[1, 1000]$ llegaron a inspeccionarse?

Respuesta 10

Cálculos reproducidos

$$(((9 \cdot 10) + (9 \cdot 15)) + (5 \cdot 20)) = 325$$

C.6 Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C

Problema resuelto [PRB-0031]
En un microservicio escrito en Zig, cada petición HTTP utiliza un `std.heap.FixedBufferAllocator` respaldado por un array en pila de $B = 16384$ B (16 KiB). Si el manejador aloja una cabecera parseada de 512 B y $N = 60$ cabeceras clave-valor de 128 B cada una, calcula cuántos bytes quedan disponibles en el `FixedBufferAllocator` antes de que devuelva `error.OutOfMemory`.

Respuesta

$$Resultado = 8192. Quedan exactamente 8192 bytes (8 KiB) libres en el búfer de la petición.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0033]
Una rebanada `[]const u32` en Zig de 64 bits consta de un puntero `[*]const u32` (8 B) y una longitud `usize` (8 B). ¿Cuántos bytes ocupa?

Respuesta 16

Ejercicio 2 [PRB-0034]
En Zig, el tipo entero de anchura arbitraria `u12` se aloja en memoria ocupando el siguiente número entero de bytes potencia de dos (`@sizeOf(u12)`). ¿Cuántos bytes ocupa en memoria?

Respuesta 2

Ejercicio 3 [PRB-0035]
Un `packed struct` de Zig contiene campos `u3`, `u5`, `u16` y `u8` empaquetados al bit exacto. ¿Cuántos bytes ocupa `(3 + 5 + 16 + 8) / 8`?

Respuesta 4

Ejercicio 4 [PRB-0036]
Si una función con 4 reservas secuenciales protegidas cada una por `errdefer` falla en la cuarta reserva, ¿cuántos bloques previos son liberados automáticamente por los `errdefer` ya registrados?

Respuesta 3

Cálculos reproducidos

$$(16384 - (512 + (60 \cdot 128))) = 8192$$

C.7 Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A−H): Registros, ABI, Pila y SIMD

Problema resuelto [PRB-0037]
Una función en ensamblador ARM64 guarda en la pila el par `(x29, x30)` (16 B), dos registros preservados `(x19, x20)` (16 B) y necesita alojar un búfer local de 36 B. Como el hardware AArch64 exige que el puntero de pila `sp` permanezca siempre alineado a múltiplo de 16 B (redondeando los 36 B locales hacia arriba a 48 B), calcula cuántos bytes totales resta al registro `sp` el prólogo de la función.
Respuesta

$$Resultado = 80. El prólogo reserva exactamente 80 bytes (múltiplo exacto de 16 : 5 * 16 = 80 B) en la pila.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0039]
En ARM64, cada instrucción máquina tiene una anchura fija de 32 bits (4 bytes). ¿Cuántos bytes ocupan en la sección `.text` una secuencia de 250 instrucciones?
Respuesta 1000
Ejercicio 2 [PRB-0040]
Un registro vectorial NEON de 128 bits en ARM64 (`v0.4s`) procesa enteros o flotantes de 32 bits en paralelo. ¿Cuántos elementos de 32 bits opera por instrucción?
Respuesta 4
Ejercicio 3 [PRB-0041]
Una función en System V x86-64 recibe 9 argumentos enteros de 64 bits. Sabiendo que los primeros 6 van en registros y el resto se pasa por la pila (8 B cada uno), ¿cuántos bytes ocupan en la pila los argumentos restantes?
Respuesta 24
Ejercicio 4 [PRB-0042]
Un bucle procesa un array de 4096 flotantes `f32` utilizando instrucciones AVX-512 (16 flotantes `f32` por registro de 512 bits). ¿Cuántas iteraciones vectoriales ejecuta el bucle?
Respuesta 256

Cálculos reproducidos

$((16 + 16) + 48) = 80$

C.8 Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A−H)

Problema resuelto [PRB-0043]
En un programa Fortran 2018, una matriz `real(real64) :: A(100, 200)` ($M = 100$ filas, $N = 200$ columnas, $S = 8$ B por elemento) está almacenada en orden mayor por columnas a partir del desplazamiento base 0. Calcula el desplazamiento exacto en bytes del elemento `A(25, 4)` usando la fórmula $8 \cdot ((4 - 1) \cdot 100 + (25 - 1))$.
Respuesta

$$Resultado = 2592. El elemento A(25, 4) se encuentra exactamente a 2592 bytes del inicio de la matriz.$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
En Ada, el tipo `type Percent is range 0 .. 100;` comprende desde 0 hasta 100 ambos inclusive. ¿Cuántos valores distintos pertenecen al tipo (`Percent'Last - Percent'First + 1`)?
Respuesta 101
Ejercicio 2 [PRB-0046]

En Fortran, una rebanada vectorial `u(2:n-1)` para $n = 1002$ evalúa los puntos interiores de una malla 1D. ¿Cuántos puntos interiores actualiza la sentencia?
Respuesta 1000
Ejercicio 3 [PRB-0047]
Un programa Coarray Fortran se ejecuta en 16 imágenes (`num_images() = 16`) y cada imagen aloja una malla local de 50000 flotantes `real64` (8 B). ¿Cuántos bytes suman las mallas de las 16 imágenes?
Respuesta 6400000
Ejercicio 4 [PRB-0048]
En un sistema de tiempo real Ada Ravenscar con periodo de marco mayor $T = 100$ ms, tres tareas periódicas consumen 15 ms, 22 ms y 18 ms por cada 100 ms. ¿Cuál es el porcentaje entero de utilización total de CPU?
Respuesta 55

Cálculos reproducidos

$(8 \cdot (((4 - 1) \cdot 100) + (25 - 1))) = 2592$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 32 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

Referencia Normativa src-iso-c17 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-bryant-ohallaron (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-posix-2017 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-iso-cpp20 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-cormen-clrs (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-patterson-hennessy (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-knuth-taocp (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-c-fundamentos-modelo-compilacion-tipos-principiante scientific
REV-0002: cap02-c-punteros-memoria-dinamica-estructuras-aliasing scientific
REV-0003: cap03-c-experto-atomicos-posix-sanitizers-ecosistema-proyectos scientific
REV-0004: cap04-cpp-fundamentos-raii-move-semantics-smart-pointers scientific
REV-0005: cap05-cpp-templates-concepts-ranges-concurrencia-proyectos scientific
REV-0006: cap06-monografia-zig-asignadores-comptime-interoperabilidad-c scientific
REV-0007: cap07-monografia-ensamblador-arm64-x86-64-abi-simd scientific
REV-0008: cap08-monografias-ada-spark-tiempo-real-y-fortran-cientifico scientific

Apéndice E

Plan de estudio

Monografía Autónoma de C (ISO/IEC 9899:2018 / C23)

✓Monografía de C (I – Bloques A, B y C): Historia, modelo de traducción ISO C17/C23, toolchain y nivel principiante

- Comprender la evolución histórica de C desde UNIX en Bell Labs (Dennis Ritchie, 1972) hasta los estándares ANSI C89, C99, C11, C17 y C23.
- Dominar las 8 fases de traducción del estándar ISO C, el preprocesador, la compilación separada por unidades de traducción y el enlazado ELF/Mach-O.
- Utilizar con precisión los tipos enteros de anchura exacta (<stdint.h>), promociones enteras, operadores bit a bit y estructuras de control.

2 conceptos planificados.

✓Monografía de C (II – Bloques D y E): Punteros, aritmética de direcciones, memoria dinámica, structs y strict aliasing

- Dominar la semántica de punteros, el decaimiento de arrays a punteros, punteros a funciones y la aritmética escalada de direcciones.
- Gestionar el ciclo de vida en el montículo con malloc, calloc, realloc y free, y construir asignadores lineales de región (Arena Allocators).
- Aplicar la regla de alias estricto (strict aliasing), el calificador restrict y la lectura binaria segura mediante memcpy.

2 conceptos planificados.

✓Monografía de C (III – Bloques F, G y H): Concurrencia C11/POSIX, Sanitizers, Ecosistema y Proyectos

- Programar concurrencia segura en C mediante <stdatomic.h>, pthreads, descriptores de fichero POSIX y mmap.
- Integrar AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), ThreadSanitizer (TSan) y analizadores estáticos en CMake/Meson.
- Auditar y extender el proyecto completo ‘projects/c-arena-allocator‘ junto con los 4 proyectos integradores en C.

2 conceptos planificados.

Monografía Autónoma de C++ (ISO/IEC 14882:2020 / C++23)

✓Monografía de C++ (I – Bloques A–D): De C++98 a C++20/23, RAII, Semántica de Movimiento y Punteros Inteligentes

- Comprender la filosofía de C++ moderno (Bjarne Stroustrup): tipos definidos por el usuario con coste cero y determinismo RAII.
- Aplicar la Regla de Cero, la Regla de Cinco, categorías de valores (lvalue, prvalue, xvalue) y semántica de movimiento (std::move).

- Gestionar recursos dinámicos sin fugas mediante std::unique_ptr, std::shared_ptr, std::weak_ptr, std::span y std::string_view.

2 conceptos planificados.

✓Monografía de C++ (II – Bloques E–H): Plantillas, Concepts C++20, std::ranges, Concurrencia, Corrutinas y Proyectos

- Escribir código genérico restringido semánticamente mediante C++20 Concepts (‘requires‘), plantillas variádicas y ‘if constexpr‘.
- Componer tuberías perezosas de vistas con ‘std::ranges‘ y ‘std::views‘ evitando copias intermedias.
- Dominar la concurrencia en C++20 (‘std::jthread‘, ‘std::stop_token‘, ‘std::atomic‘, ‘std::latch‘), el ecosistema CMake/Conan/vcpkg y el proyecto ‘projects/cpp-task-scheduler‘.

2 conceptos planificados.

Monografías Especializadas de Sistemas: Zig, Ensamblador, Ada y Fortran

✓Monografía de Zig (Bloques A–H): Asignadores explícitos, metaprogramación comptime, uniones de error y toolchain C

- Comprender los principios de diseño de Zig: ausencia de flujo de control oculto, ausencia de asignaciones implícitas de montículo y ausencia de preprocesador.
- Programar con uniones de error (‘!T‘), limpieza diferida determinista (‘defer‘ y ‘errdefer‘), rebanadas con longitud y asignadores explícitos (‘std.mem.Allocator‘).
- Utilizar ‘comptime‘ para tipos genéricos y evaluación en tiempo de compilación, e integrar cabeceras C con ‘@cImport‘.

2 conceptos planificados.

✓Monografía de Ensamblador ARM64 (AArch64) y x86-64 (Bloques A–H): Registros, ABI, Pila y SIMD

- Distinguir la arquitectura RISC de carga/almacenamiento AArch64 (ARM64) frente a la arquitectura CISC de longitud variable x86-64.
- Dominar las convenciones de llamada ABI System V AMD64 (‘rdi, rsi, rdx, rcx, r8, r9‘ y retorno en ‘rax‘) y AAPCS64 (‘x0–x7‘ y retorno en ‘x0‘).
- Analizar el prólogo y epílogo de marcos de pila, el alineamiento obligatorio de 16 bytes del puntero de pila (‘sp‘/‘rsp‘) e instrucciones vectoriales SIMD.

2 conceptos planificados.

✓Monografías de Ada/SPARK (Sistemas Críticos) y Fortran Moderno (Cómputo Científico HPC) (Bloques A–H)

- Diseñar tipos escalares con rangos restringidos, contratos ejecutables (‘Pre‘, ‘Post‘, ‘Type_Invariant‘) y tareas críticas bajo el perfil Ravenscar en Ada 2022 y SPARK.
- Comprender cómo el demostrador formal GNATprove en SPARK demuestra estáticamente la ausencia de errores en tiempo de ejecución (AoRTE).
- Programar cálculo numérico de alto rendimiento en Fortran 2018/2023 mediante operaciones matriciales intrínsecas, ‘intent(in/out)‘, módulos y paralelismo Coarray.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-iso-c17. https://doc.rust-lang.org/reference/src-iso-c17.html

[27] Referencia Normativa src-bryant-ohallaron. https://doc.rust-lang.org/reference/src-bryant-ohallaron.html

[28] Referencia Normativa src-posix-2017. https://doc.rust-lang.org/reference/src-posix-2017.html

[29] Referencia Normativa src-iso-cpp20. https://doc.rust-lang.org/reference/src-iso-cpp20.html

[30] Referencia Normativa src-cormen-clrs. https://doc.rust-lang.org/reference/src-cormen-clrs.html

[31] Referencia Normativa src-patterson-hennessy. https://doc.rust-lang.org/reference/src-patterson-hennessy.html

[32] Referencia Normativa src-knuth-taocp. https://doc.rust-lang.org/reference/src-knuth-taocp.html

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen V: Monografías de Python, JavaScript y TypeScript

Parte V (Bloques A–H): De cero a máquinas virtuales de bytecode, asincronía, Event Loop V8 y sistemas de tipos estructurales

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografía Autónoma de Python (Python 3.12 – 3.14) 1

1. Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante 2
1.1. Bloques A y B: Historia, máquina virtual CPython, GIL / Free-Threading y toolchain moderna (uv, ruff, pyright) . . . 2
1.2. Bloque C: Modelo de nombres y objetos, tipos nativos, colecciones, comprensiones y control de flujo (match/case) . . . 3

2. Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers 5
2.1. Bloque D: POO en Python, Data Model (___dunder___), @dataclass, ___slots___ y sistema de tipos (PEP 484 / PEP 695) . . . 5
2.2. Bloque E: Clausuras, decoradores, iteradores, generadores (yield) y gestores de contexto (with) . . . 6

3. Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos 8
3.1. Bloque F: Asincronía con asyncio.TaskGroup, multiprocesamiento, metaclases y compilación AST . . . 8
3.2. Bloques G y H: Ecosistema industrial (FastAPI, Pydantic, SQLAlchemy, Pytest) y los 4 proyectos (python-bytecode-vm) . . . 9

II Monografías Autónomas de JavaScript (ES2024+) y TypeScript (5.x) 11

4. Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM 12
4.1. Bloques A–C: Historia de ECMAScript, motores JIT (V8), tipos, coerción vs ===, let/const y funciones . . . 12
4.2. Bloque D: Clausuras, delegación por prototipos, clases, desestructuración y módulos ES (ESM vs CommonJS) . . . 12

5. Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos 15
5.1. Bloques E y F: Event Loop, cola de microtareas vs macrotareas, Promises, async/await, Streams y Workers . . . 15
5.2. Bloques G y H: Ecosistema (Node.js, Deno, Bun, npm/pnpm, Vitest, ESLint) y los 4 proyectos en JavaScript . . . 16

6. Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas 18
6.1. Bloques A–C: Filosofía de TypeScript, borrado de tipos, tsconfig.json estricto y primitivos (unknown, never) . . . 18
6.2. Bloque D: Tipado estructural, interfaces vs type aliases, estrechamiento (narrowing) y uniones discriminadas . . . 18

7. Monografía de TypeScript (II − Bloques E−H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos **21**
7.1. Bloques E y F: Generics, keyof, Mapped Types, Conditional Types, infer, Template Literals, satisfies y Branded Types . . . 21
7.2. Bloques G y H: Ecosistema (Zod, tRPC, Vite, tsx, Vitest) y los 4 proyectos (incluyendo ts-reactive-http) . . . 22

8. Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript **24**
8.1. Comparativa profunda de ejecución, memoria, concurrencia y sistemas de tipos (Python vs JS vs TS) . . . 24
8.2. Arquitectura políglota: contratos OpenAPI/JSON Schema entre Python y TypeScript y evaluación global . . . 24

A. Notación **27**

B. Hoja de fórmulas **28**

C. Soluciones **29**
C.1. Monografía de Python (I − Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante . . . 29
C.2. Monografía de Python (II − Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers . . . 29
C.3. Monografía de Python (III − Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos . . . 30
C.4. Monografía de JavaScript (I − Bloques A−D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM . . . 31
C.5. Monografía de JavaScript (II − Bloques E−H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos . . . 31
C.6. Monografía de TypeScript (I − Bloques A−D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas . . . 32
C.7. Monografía de TypeScript (II − Bloques E−H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos . . . 33
C.8. Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript . . . 33

D. Auditoría de fuentes **35**

E. Plan de estudio **37**

Parte I

Monografía Autónoma de Python (Python 3.12 – 3.14)

Capítulo 1

Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante

*Este capítulo inaugura la monografía autónoma de **Python** (Bloques A, B y C). Python es simultáneamente el lenguaje hegemónico en ciencia de datos, inteligencia artificial y automatización, y un lenguaje de ingeniería de software de primer orden para backends web y herramientas de infraestructura. El lector aprenderá su modelo exacto de ejecución bajo el capó de CPython.*

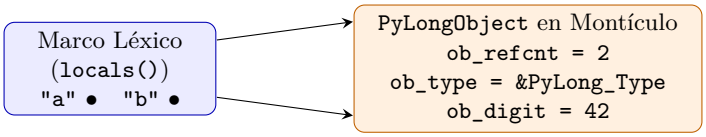


Figura 1. Diferencia semántica fundamental en Python: las variables no son cajas con tipo fijo, sino etiquetas en un diccionario de espacio de nombres que apuntan a estructuras PyObject en el montículo con cabecera $\mathrm{ob}_{r}efcnt$ y $\mathrm{ob}_{t}ype$.

Consejo

Utiliza siempre `is None` e `is not None` para comprobar identidad frente al singleton `None`, y reserva `==` para comparar igualdad de valor (`__eq__`).

Definición

Vinculación de nombres en Python (Name Binding): asociación en una tabla de símbolos (`locals()`, `globals()`) entre un identificador y la dirección de un `PyObject` en el montículo; el tipo pertenece exclusivamente al objeto referenciado y nunca a la variable léxica.

Autoficha

¿Por qué una tupla `(1, [2, 3])` lanza `TypeError: unhashable type: 'list'` al intentar usarla como clave de un `dict` a pesar de que el contenedor exterior es un `tuple` inmutable?

1.1 Bloques A y B: Historia, máquina virtual CPython, GIL / Free-Threading y toolchain moderna (uv, ruff, pyright)

Creado por Guido van Rossum en el CWI de Ámsterdam y publicado en 1991, Python ejecuta el código fuente compilándolo primero a un árbol de sintaxis abstracta (AST) y seguidamente a código de bytes (*bytecode*, inspeccionable con el módulo estándar `dis`) interpretado por una máquina virtual de pila en CPython. Cada valor en Python −incluidos los enteros y las propias clases− es una estructura `PyObject` en el montículo que almacena como mínimo su contador de referencias (`ob_refcnt`, 8 bytes en 64 bits) y un puntero a su descriptor de tipo (`ob_type`, 8 bytes), como ilustra la Figura \ref{fig:fig-cpython-object-layout}. La gestión de memoria combina liberación inmediata cuando `ob_refcnt` cae a cero con un recolector generacional detector de ciclos. Históricamente, CPython protegió el estado del intérprete con el cerrojo global **GIL** (*Global Interpreter Lock*), pero a partir de Python 3.13/3.14 (PEP 703) incorpora soporte para compilaciones *free-threaded* sin GIL basadas en conteo de referencias sesgado (*biased reference counting*). Todo proyecto moderno de Python se configura mediante un único fichero declarativo `pyproject.toml` (PEP 621), gestionado con `uv` o `pip` sobre `venv`, formateado y auditado con `ruff`, verificado estáticamente con `pyright` o `mypy` y probado con `pytest`.

1.2 Bloque C: Modelo de nombres y objetos, tipos nativos, colecciones, comprensiones y control de flujo (match/case)

En Python, la sentencia `a = [1, 2, 3]; b = a` **no** copia la lista: vincula ambos nombres al mismo objeto `PyListObject` incrementando su `ob_refcnt` a 2. La distinción entre tipos inmutables (`int` de precisión arbitraria, `float` IEEE 754, `str` Unicode, `bytes`, `tuple`, `frozenset`) y mutables (`list`, `dict`, `set`, `bytearray`) determina qué objetos son *hashables* y pueden actuar como claves de un diccionario. Desde Python 3.6/3.7, `dict` se implementa como una tabla hash compacta que preserva por especificación el orden de inserción separando un array denso de índices hash de la tabla contigua de entradas `(hash, key_ptr, val_ptr)`. Asimismo, Python 3.10+ incorpora coincidencia estructural de patrones (`match/case`, PEP 634):

```
def evaluate_ast_node(node: tuple) -> int:
    match node:
        case ("lit", int(v)):
            return v
        case ("add", left, right):
            return evaluate_ast_node(left) + evaluate_ast_node(right)
        case ("mul", left, right):
            return evaluate_ast_node(left) * evaluate_ast_node(right)
        case _:
            raise ValueError(f"Nodo AST desconocido: {node!r}")
```

Un escollo clásico que todo programador debe evitar desde el primer día es emplear argumentos por defecto mutables en definiciones de función (`def append_item(x, target=[]):`), ya que la expresión por defecto se evalúa una única vez al ejecutar la sentencia `def`, compartiendo la misma lista entre todas las llamadas; el patrón correcto es `def append_item(x, target: list | None = None):`.

Problema resuelto

Enunciado En CPython de 64 bits, cada entrada de un diccionario almacena tres palabras de 8 B (`me_hash`, `me_key`, `me_value` = 24 B). Para una tabla con $K = 128$ ranuras en el índice (1 B por ranura en `dk_indices`) y capacidad para $N = 85$ entradas en `dk_entries`, calcula cuántos bytes ahorra el diseño de diccionario compacto ($128 \cdot 1 + 85 \cdot 24$) frente al diseño disperso antiguo ($128 \cdot 24$).

Datos

- $K = 128$ ranuras — Tamaño del array de índices hash (1 B por índice)
- $N = 85$ entradas — Capacidad densa de entradas (2/3 de K)
- $S_{entry} = 24$ B — Tamaño de cada entrada (hash, key, value)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria del diccionario compacto de CPython (Raymond Hettinger)

$$\Delta M = 128 \cdot 24 - (128 \cdot 1 + 85 \cdot 24)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el tamaño compacto (2168 B) al tamaño antiguo (3072 B)

$$\Delta M = 128 \cdot 24 - (128 \cdot 1 + 85 \cdot 24)$$

$$((128 \cdot 24) - ((128 \cdot 1) + (85 \cdot 24)))$$

Resultado

$$Resultado = 904$$

Interpretación El diccionario compacto ahorra 904 bytes (casi un 30 % menos de memoria) y acelera la iteración al recorrer un array contiguo.
Comprobación de los pasos

- Paso 1: 904 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) En CPython de 64 bits, la cabecera base `PyObject` contiene `ob_refcnt` (8 B) y `ob_type` (8 B). ¿Cuántos bytes mide esta cabecera mínima?
2 [PRB-0004] (Básico, Numérico) Una lista `PyListObject` mantiene un array de punteros `PyObject*` (8 B cada uno). Si su capacidad reservada es de 64 elementos, ¿cuántos bytes ocupa ese array de punteros?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) ¿Cuál es la suma de los cuadrados impares evaluada por `sum(x * x for x in range(1, 10, 2))` ($1^2 + 3^2 + 5^2 + 7^2 + 9^2$)?
4 [PRB-0006] (Intermedio, Diseño) En el proyecto `projects/python-bytecode-vm`, la función compila y evalúa `(x + 4) * (y - 2) + (10 if x < y else 20)` para $x = 6, y = 9$. ¿Cuál es el resultado entero devuelto?

Capítulo 2

Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers

*Este capítulo cubre los Bloques D y E de Python. La verdadera fluidez en Python consiste en comprender que la sintaxis del lenguaje (`len(x)`, `x[i]`, `for v in x`, `with ctx:`, `a + b`) se traduce directamente en invocaciones a métodos especiales del **Modelo de Datos** (`__len__`, `__getitem__`, `__iter__`, `__enter__`/`__exit__`, `__add__`).*

Consejo

Usa siempre `@dataclass(slots=True)` en clases de datos de alto volumen: reduce el consumo de memoria entre un 40 % y un 60 % y acelera el acceso a atributos al sustituir búsquedas hash en `__dict__` por desplazamientos directos en C.

Definición

Protocolo Descriptor en Python: mecanismo fundamental del modelo de objetos según el cual cualquier atributo de clase que defina `__get__`, `__set__` o `__delete__` intercepta el acceso a atributos en las instancias, sirviendo de base para métodos ligados, `@property`, `@classmethod`, `@staticmethod` y `__slots__`.

Autoficha

¿Qué diferencia existe en el protocolo descriptor entre un *data descriptor* (define `__get__` y `__set__`) y un *non-data descriptor* (solo define `__get__`) frente a una entrada con el mismo nombre en `instance.__dict__`?

2.1 Bloque D: POO en Python, Data Model (___dunder___), @dataclass, ___slots___ y sistema de tipos (PEP 484 / PEP 695)

Por defecto, cada instancia de una clase definida por el usuario almacena sus atributos en un diccionario dinámico `__dict__`, lo que aporta flexibilidad pero añade más de 100 B de sobrecoste por objeto. Cuando se instancian millones de objetos de dominio o nodos del AST (como en nuestro proyecto `projects/python-bytecode-vm`), declarar `@dataclass(frozen=True, slots=True)` genera automáticamente `__init__`, `__repr__`, `__eq__`, `__hash__` y reemplaza `__dict__` por un descriptor compacto de ranuras fijas (`__slots__`). Asimismo, Python 3.12 (PEP 695) simplifica la sintaxis de clases y funciones genéricas y de `typing.Protocol` (subtipado estructural o *static duck typing*):

```
from dataclasses import dataclass
from typing import Protocol

class Measurable(Protocol):
    def byte_size(self) -> int: ...

@dataclass(frozen=True, slots=True)
class TelemetryFrame:
    sensor_id: int
```

```
    payload: bytes
    def byte_size(self) -> int:
        return 8 + len(self.payload)

def total_footprint[T: Measurable](items: list[T]) -> int:
    return sum(item.byte_size() for item in items)
```

Observa que `TelemetryFrame` satisface `Measurable` estructuralmente para `pyright`/`mypy` sin necesidad de heredar nominalmente de `Measurable`.

2.2 Bloque E: Clausuras, decoradores, iteradores, generadores (yield) y gestores de contexto (with)

Un **generador** es cualquier función cuyo cuerpo contiene la palabra clave `yield`. Al llamarla, no ejecuta su cuerpo inmediatamente, sino que devuelve un objeto iterador que suspende y reanuda su marco de ejecución (`PyGenObject`) en cada llamada a `next()`, manteniendo un consumo de memoria $O(1)$ independientemente de si la secuencia subyacente tiene miles de millones de registros. Por su parte, los **gestores de contexto** implementan el protocolo `__enter__` / `__exit__(exc_type, exc_val, exc_tb)` (o se escriben de forma concisa con `@contextlib.contextmanager` y un único `yield` dentro de un bloque `try ... finally:`), garantizando la liberación determinista de ficheros, conexiones de red, cerrojos `threading.Lock` y transacciones de base de datos.

Problema resuelto

Enunciado En una aplicación de procesamiento de grafos en CPython de 64 bits, una clase `Edge` con $k = 3$ atributos ocupa $16 + 3 \times 8 = 40$ B cuando se declara con `__slots__`, mientras que sin `__slots__` cada instancia aloja además un `__dict__` que añade 112 B por objeto (152 B en total). Calcula cuántos megabytes exactos (dividiendo entre 10^6) se ahorran al instanciar $N = 1000000$ de aristas con `__slots__`.

Datos

- $N = 1000000$ — Un millón de instancias de Edge
- $S_{condict} = 152\,\text{B}$ — Tamaño por instancia con ___dict___
- $S_{conslots} = 40\,\text{B}$ — Tamaño por instancia con ___slots___

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de RAM con ___slots___ en millones de instancias en CPython

$$\Delta M_{\text{MB}} = \frac{10^6 \cdot (152 - 40)}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos el millón de instancias por los 112 B ahorrados y dividimos entre 10^6

$$\Delta M_{\text{MB}} = \frac{10^6 \cdot (152 - 40)}{10^6}$$

$$\frac{(1000000 \cdot (152 - 40))}{1000000}$$

Resultado

$$\textit{Resultado} = 112$$

Interpretación El uso de ___slots___ ahorra exactamente 112 MB de memoria RAM.

Comprobación de los pasos

- Paso 1: 112 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) En CPython de 64 bits, ¿cuántos bytes ocupa una instancia con `__slots__` de 4 atributos (16 B de cabecera más 4×8 B)?
2 [PRB-0010] (Básico, Numérico) Un generador perezoso produce bloques de 4096 bytes uno a uno para procesar un fichero de 10 GiB. ¿Cuántos bloques mantiene simultáneamente en RAM?
3 [PRB-0011] (Intermedio, Desarrollo por pasos) En una jerarquía con herencia múltiple `class D(B, C)` donde `B(A)` y `C(A)` heredan de `A(object)`, ¿cuántas clases aparecen en la tupla de orden de resolución de métodos `D.__mro__` (`D, B, C, A, object`)?
4 [PRB-0012] (Intermedio, Diseño) Un decorador `@lru_cache(maxsize=256)` recibe 1000 consultas sobre un conjunto de solo 200 claves distintas. ¿Cuántas veces se ejecuta realmente la función subyacente?

Capítulo 3

Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos

Este capítulo completa la monografía de Python abordando el Nivel Experto (Bloque F), el Ecosistema (Bloque G) y los Proyectos (Bloque H), analizando en detalle el compilador AST y la máquina virtual de código de bytes `projects/python-bytecode-vm`.

Consejo

Nunca llames a funciones bloqueantes síncronas (`time.sleep`, `requests.get`, lectura síncrona de disco) dentro de una función `async def`: bloquearás todo el bucle de eventos de `asyncio` congelando el resto de tareas concurrentes; utiliza `await asyncio.sleep(...)`, `httpx.AsyncClient` o `await asyncio.to_thread(...)`.

Definición

Concurrencia estructurada (`asyncio.TaskGroup`): abstracción de control de flujo concurrente que garantiza que el tiempo de vida de todas las tareas asíncronas lanzadas dentro de un bloque léxico `async with` está acotado por dicho bloque, sin dejar tareas huérfanas ejecutándose en segundo plano tras salir del ámbito.

Autoficha

¿Por qué en `projects/python-bytecode-vm` un salto condicional `JMP_IF_FALSE` necesita parchar (*backpatch*) su índice de destino después de haber emitido las instrucciones de la rama verdadera?

3.1 Bloque F: Asincronía con asyncio.TaskGroup, multiprocesamiento, metaclases y compilación AST

En Python, la elección del modelo de concurrencia depende de la naturaleza del cuello de botella: para operaciones limitadas por E/S de red o disco (*I/O-bound*) con miles de conexiones, `asyncio` ejecuta corrutinas cooperativas sobre un único hilo respaldado por `epoll`/`kqueue`/`iocp`. Python 3.11+ introdujo **concurrencia estructurada** mediante `async with asyncio.TaskGroup() as tg:`: si cualquiera de las tareas hijas lanza una excepción no capturada, el `TaskGroup` cancela automáticamente todas las tareas hermanas y propaga un `ExceptionGroup` manejable con `except*`. Para cargas intensivas de CPU (*CPU-bound*), se emplea `concurrent.futures.ProcessPoolExecutor`, compilación sin GIL (PEP 703) o núcleos vectorizados en Rust/C mediante `PyO3`, `NumPy` o `PyTorch`. En metaprogramación, el módulo estándar `ast` permite inspeccionar, transformar y compilar árboles sintácticos de forma segura sin usar `eval()` arbitrario, técnica demostrada en nuestro proyecto verificado `projects/python-bytecode-vm`:

```
import ast

def count_binary_ops(expr_source: str) -> int:
    tree = ast.parse(expr_source, mode="eval")
    return sum(1 for node in ast.walk(tree) if isinstance(node, ast.BinOp))
```

Para registrar subclases o validar esquemas de clase en el momento de su creación, Python 3.6+ ofrece el gancho ligero `__init_subclass__(cls, **kwargs)`, evitando en el 95 % de los casos la complejidad de definir una metaclase personalizada que herede de `type`.

3.2 Bloques G y H: Ecosistema industrial (FastAPI, Pydantic, SQLAlchemy, Pytest) y los 4 proyectos (python-bytecode-vm)

El ecosistema profesional de Python (Bloque G) destaca por cuatro pilares: (1) desarrollo web y APIs con `FastAPI` (ASGI asíncrono), `Django` y validación de esquemas en Rust con `Pydantic v2`; (2) persistencia y migraciones con `SQLAlchemy 2.0` y `Alembic`; (3) ingeniería de datos y cálculo científico con `NumPy`, `SciPy`, `Pandas` y `Polars`; y (4) calidad y empaquetado con `pytest` (fixtures, parametrización), `Hypothesis` (property-based testing), `ruff`, `pyright` y construcción de paquetes `.whl` con `hatchling` o `maturin`. En el Bloque H, el lector desarrolla cuatro proyectos completos: un gestor CLI de tareas con persistencia SQLite, un cliente/servidor HTTP asíncrono con reintentos exponenciales, un pipeline de validación de datos y la máquina virtual de pila `projects/python-bytecode-vm`, verificada al 100 % con `pytest`, `ruff` y `pyright`.

Problema resuelto

Enunciado Un agregador de microservicios en Python debe consultar $N = 20$ endpoints HTTP independientes, cada uno con una latencia de red de $L = 150$ ms y 1 ms de procesamiento de CPU local en el bucle de eventos. Calcula cuántos milisegundos se ahorran al ejecutarlos concurrentemente con `asyncio.TaskGroup` ($150 + 20 \times 1 = 170$ ms) frente a ejecutarlos secuencialmente de forma síncrona ($20 \times (150 + 1) = 3020$ ms).
Datos

- $N = 20$ — Número de peticiones HTTP
- $L = 150\,\text{ms}$ — Latencia de red por petición
- $t_{cpu} = 1\,\text{ms}$ — Tiempo de CPU por respuesta

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Solapamiento de latencia de E/S en el bucle de eventos de asyncio

$$\Delta T = 20 \cdot 151 - (150 + 20)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el tiempo concurrente (170 ms) al tiempo secuencial (3020 ms)

$$\Delta T = 20 \cdot 151 - (150 + 20)$$

$$((20 \cdot (150 + 1)) - (150 + (20 \cdot 1)))$$

Resultado

$$\textit{Resultado} = 2850$$

Interpretación La ejecución concurrente con asyncio ahorra 2850 ms, reduciendo el tiempo de respuesta total de 3.02 s a 0.17 s.
Comprobación de los pasos

- Paso 1: 2850 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 **[PRB-0015]** **(Básico, Numérico)** En `projects/python-bytecode-vm`, la expresión `2 * 3 + 4 * 5` emite 4 instrucciones `LOAD_CONST` y 3 instrucciones binarias (`MUL`, `MUL`, `ADD`). ¿Cuántas instrucciones totales contiene el bloque compilado?

2 **[PRB-0016]** **(Básico, Numérico)** Al evaluar `2 * 3 + 4 * 5` en una máquina de pila, ¿cuál es la altura máxima (número máximo de operandos apilados simultáneamente) que alcanza la pila?

3 **[PRB-0017]** **(Intermedio, Desarrollo por pasos)** Un pool de 8 procesos (`ProcessPoolExecutor(max_workers=8)`) ejecuta 40 tareas de CPU de 2 segundos cada una. ¿Cuántos segundos tarda en completar las 40 tareas ideales ($40/8 \times 2$)?

4 **[PRB-0018]** **(Intermedio, Diseño)** Una política de reintentos con retroceso exponencial (`backoff`) espera $100 \cdot 2^k$ ms para $k = 0, 1, 2, 3$. ¿Cuántos milisegundos totales habrá esperado acumulados tras esos 4 reintentos?

Parte II

Monografías Autónomas de JavaScript (ES2024+) y TypeScript (5.x)

Capítulo 4

Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM

*Este capítulo inicia la monografía autónoma de **JavaScript (ECMAScript 2024+)**, diferenciándolo con rigor de TypeScript. JavaScript es el único lenguaje nativo universal de los navegadores web y, a través de motores como V8 (Node.js, Deno) y JavaScriptCore (Bun), un entorno de servidor de altísima velocidad cuando se comprende cómo optimiza el compilador JIT sus estructuras dinámicas.*

Consejo

Para preservar el polimorfismo monomórfico en las *Inline Caches* de V8, inicializa siempre todas las propiedades de un objeto en su constructor en el mismo orden y evita eliminar propiedades dinámicamente con `delete obj.prop`.

Definición

Zona Muerta Temporal (Temporal Dead Zone, TDZ en ECMAScript): intervalo de ejecución desde la entrada al bloque léxico hasta la evaluación de la declaración `let` o `const` de una variable, durante el cual cualquier intento de leer o escribir dicha variable lanza determinísticamente un `ReferenceError`.

Autoficha

¿Por qué en JavaScript `0.1 + 0.2 === 0.3` evalúa a `false` y cómo deben representarse importes monetarios exactos usando `BigInt` en céntimos?

4.1 Bloques A–C: Historia de ECMAScript, motores JIT (V8), tipos, coerción vs ===, let/const y funciones

Creado por Brendan Eich en Netscape en 1995 y estandarizado por Ecma International bajo la especificación **ECMA-262**, JavaScript experimentó su transformación decisiva en ES6 (ES2015) con la introducción de `let/const`, funciones flecha, clases, promesas, generadores, `Map/Set`, `Symbol` y módulos nativos ESM. En los motores modernos como V8, el código fuente no se interpreta línea a línea: se compila a bytecode (*Ignition*) y las funciones calientes se optimizan a código máquina nativo (*TurboFan*) asociando a cada objeto una **clase oculta** (*Hidden Class* o *Shape/Map*). Si todas las instancias de un punto se inicializan con las mismas propiedades en el mismo orden (`{ x, y }`), comparten la misma Shape y los accesos `pt.x` se resuelven en una sola instrucción mediante *Inline Caching* (monomórfico). En cuanto a los tipos primitivos, `number` es un flotante IEEE 754 de doble precisión cuyos enteros exactos llegan hasta `Number.MAX_SAFE_INTEGER` ($2^{53} - 1 = 9007199254740991$), mientras que `BigInt` (`42n`) representa enteros de precisión arbitraria. En código profesional se utiliza siempre la igualdad estricta `===` y `!==` (que no realiza coerción implícita de tipos) o `Object.is(a, b)` (que distingue `+0` de `-0` y evalúa `Object.is(NaN, NaN)` a `true`).

4.2 Bloque D: Clausuras, delegación por prototipos, clases, desestructuración y módulos ES (ESM vs CommonJS)

A diferencia de los lenguajes basados en clases estáticas, el modelo de objetos de JavaScript se fundamenta en la **delegación prototípica**: cada objeto posee un enlace interno `[[Prototype]]` (accesible mediante `Object.getPrototypeOf(obj)`). Cuando se lee una propiedad `obj.metodo`, si no existe como

propiedad propia (`Object.hasOwn(obj, 'metodo')`), el motor asciende por la cadena de prototipos hasta encontrarla o llegar a `null`. La sintaxis `class` de ES2015/ES2022 (incluyendo campos privados reales `#saldo` verificados por la gramática) se construye limpiamente sobre este mecanismo:

```
export class TokenBucket {
  #capacity;
  #tokens;
  constructor(capacity) {
    this.#capacity = capacity;
    this.#tokens = capacity;
  }
  tryConsume(amount = 1) {
    if (amount <= 0 || this.#tokens < amount) return false;
    this.#tokens -= amount;
    return true;
  }
  get available() { return this.#tokens; }
}
```

Asimismo, mientras que las funciones tradicionales determinan el valor de `this` dinámicamente según el sitio de llamada (`obj.fn()`, `fn.call(ctx)`), las funciones flecha (`() => ...`) carecen de `this` propio y lo capturan léxicamente del ámbito circundante.

Problema resuelto

Enunciado En el motor V8 con compresión de punteros (*Pointer Compression*), cada referencia a propiedad en un objeto de forma fija (*Fast Properties / In-object properties*) ocupa 4 B en lugar de 8 B, más una cabecera de objeto de 12 B (`map`, `properties`, `elements` comprimidos). Calcula cuántos bytes ahorran $N = 10000$ objetos con $k = 5$ propiedades en línea bajo compresión de punteros ($12 + 5 \times 4 = 32$ B por objeto) frente a punteros de 64 bits sin comprimir ($24 + 5 \times 8 = 64$ B por objeto).

Datos

- $N = 10000$ — Número de objetos instanciados
- $S_{64} = 64\,\text{B}$ — Tamaño por objeto con punteros de 8 B (24 + 40)
- $S_{comp} = 32\,\text{B}$ — Tamaño por objeto con punteros comprimidos de 4 B (12 + 20)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de montículo mediante Pointer Compression en el motor V8

$$\Delta M = 10000 \cdot (64 - 32)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 10000 objetos por los 32 B ahorrados en cada uno

$$\Delta M = 10000 \cdot (64 - 32)$$

$$(10000 \cdot ((24 + (5 \cdot 8)) - (12 + (5 \cdot 4))))$$

Resultado

$$\textit{Resultado} = 320000$$

Interpretación La compresión de punteros de V8 reduce el consumo a la mitad exacta, ahorrando 320 000 bytes.

Comprobación de los pasos

- Paso 1: 320000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) En V8 con punteros comprimidos de 4 bytes, ¿cuántos bytes ocupa la cabecera de 3 campos (`map`, `properties`, `elements`) de un objeto JS (3×4)?
2 [PRB-0022] (Básico, Numérico) Un `ArrayBuffer(1024)` se observa mediante un `Float64Array` (8 bytes por elemento). ¿Cuántos elementos `length` tiene la vista tipada?
3 [PRB-0023] (Intermedio, Desarrollo por pasos) Si una cadena de prototipos tiene 4 niveles (`instancia -> SubClase.prototype -> Base.prototype -> Object.prototype -> null`) y se busca una propiedad inexistente, ¿cuántos eslabones de objeto se inspeccionan antes de llegar a `null`?
4 [PRB-0024] (Intermedio, Diseño) En `TokenBucket` con capacidad inicial de 100 tokens, se procesan 7 peticiones que consumen 12 tokens cada una. ¿Cuántos tokens quedan disponibles?

Capítulo 5

Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos

*Este capítulo completa la monografía de JavaScript cubriendo los Bloques E, F, G y H: el funcionamiento exacto del **Event Loop**, la diferencia crítica entre microtareas y macrotareas, el manejo de contrapresión (*backpressure*) en flujos de datos y el paralelismo real mediante `Workers`.*

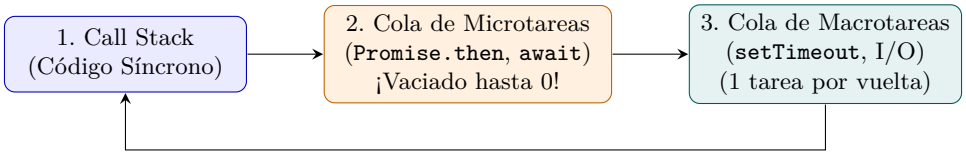


Figura 2. Ciclo determinista del Event Loop: tras vaciar la pila de llamadas síncrona (Call Stack), el motor drena exhaustivamente TODas las microtareas encoladas (incluidas las añadidas por otras microtareas) antes de tomar una única macrotarea.

Consejo

Cuando lances múltiples operaciones asíncronas independientes entre sí dentro de una función `async`, no escribas `const a = await fetchA(); const b = await fetchB();` en serie: lánzalas en paralelo con `const [a, b] = await Promise.all([fetchA(), fetchB()]);`.

Definición

Contrapresión (Backpressure en Streams): mecanismo de control de flujo mediante el cual un consumidor lento señala al productor asíncrono que su búfer interno ha alcanzado la marca de agua alta (`highWaterMark`), pausando la lectura de la fuente subyacente hasta que el búfer se drena.

Autoficha

¿Qué diferencia existe entre `Promise.all`, `Promise.allSettled`, `Promise.race` y `Promise.any` cuando una de las promesas de entrada se rechaza con un error?

5.1 Bloques E y F: Event Loop, cola de microtareas vs macrotareas, Promises, async/await, Streams y Workers

El modelo de concurrencia de JavaScript se basa en un hilo principal con ejecución hasta completitud (*run-to-completion*) gobernado por el **Event Loop**. Como muestra la Figura \ref{fig:fig-js-event-loop-queues}, existen dos colas con semánticas de vaciado radicalmente distintas: la **cola de microtareas** (*Jobs* en ECMA-262: continuaciones `.then/.catch/.finally` de `Promise`, código posterior a un `await` y `queueMicrotask`) y la **cola de tareas o macrotareas** (`setTimeout`, `setInterval`, eventos del DOM, callbacks de red/disco en `libuv`). Regla fundamental: cada vez que el *Call Stack* queda vacío, el motor ejecuta **todas** las microtareas pendientes hasta dejar la cola de microtareas completamente vacía antes de procesar la siguiente macrotarea o repintar la interfaz:

```
// Orden determinista de salida: [1, 4, 3, 2]
export function traceEventLoopOrder() {
  const log = [];
  log.push(1);
  setTimeout(() => log.push(2), 0);               // Macrotarea
```

```
  Promise.resolve().then(() => log.push(3));      // Microtarea
  log.push(4);
  return log;
}
```

Por consiguiente, un bucle recursivo infinito de `queueMicrotask` o `Promise.resolve().then(...)` hambrea por completo las macrotareas de E/S y congela el proceso aunque nunca bloquee mediante un `while(true)` síncrono.

5.2 Bloques G y H: Ecosistema (Node.js, Deno, Bun, npm/pnpm, Vitest, ESLint) y los 4 proyectos en JavaScript

En el Nivel Experto (Bloque F), cuando se procesan ficheros o respuestas HTTP de varios gigabytes en Node.js o la Web API, cargar todo el cuerpo en un `Buffer` agota el montículo de V8; en su lugar se utilizan `ReadableStream` / `WritableStream` con `pipeline()`, que regulan automáticamente la velocidad del productor según la capacidad del consumidor (*backpressure*). Para cómputo intensivo de CPU (criptografía, compresión, procesamiento de imágenes), se emplean `Worker Threads` (Node.js) o `Web Workers` (navegador) comunicados por paso de mensajes (`Structured Clone`), transferencia de propiedad de coste cero (`ArrayBuffer` transferibles) o memoria compartida sincronizada con `SharedArrayBuffer` y `Atomics`. El Bloque G cubre la gestión de paquetes con `pnpm`/`npm`, `package.json` (`exports` condicionales ESM), `ESLint`, `Prettier` y `Vitest` / `node -test`, y el Bloque H presenta cuatro proyectos completos en JavaScript puro (motor de plantillas reactivo sin dependencias, planificador de colas asíncrono con límite de concurrencia `p-limit`, procesador de logs por streams y servidor HTTP REST).

Problema resuelto

Enunciado Un descargador asíncrono en Node.js debe procesar $N = 120$ ficheros remotos con un limitador de concurrencia que mantiene exactamente $k = 15$ promesas en vuelo simultáneamente para no saturar el servidor de origen. Si cada descarga tarda $D = 200$ ms, calcula el tiempo total en milisegundos necesario para completar las 120 descargas $((120/15) \times 200)$.

Datos

- $N = 120$ — Total de tareas asíncronas
- $k = 15$ — Concurrencia máxima permitida
- $D = 200\,\text{ms}$ — Duración de cada descarga

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Tiempo de finalización con concurrencia acotada (Semaphore / Pool de Promesas)

$$T = \frac{120}{15} \cdot 200$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos las 120 tareas en 8 oleadas de 15 y multiplicamos por 200 ms

$$T = \frac{120}{15} \cdot 200$$

$$\left(\frac{120}{15} \cdot 200\right)$$

Resultado

$$\textit{Resultado} = 1600$$

Interpretación Las 120 descargas finalizan en 1600 ms (1.6 segundos) manteniendo el uso de sockets acotado en 15.
Comprobación de los pasos

- Paso 1: 1600 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) En un turno del Event Loop se encolan 3 macrotareas (`setTimeout`) y 5 microtareas (`Promise.then`). ¿Cuántas microtareas se ejecutan antes de que comience la primera macrotarea?
2 [PRB-0028] (Básico, Numérico) Un `ReadableStream` con `highWaterMark = 65536` bytes (64 KiB) contiene actualmente 49 152 bytes sin consumir. ¿Cuántos bytes más puede aceptar antes de activar la señal de contrapresión?
3 [PRB-0029] (Intermedio, Desarrollo por pasos) Si 3 promesas independientes tardan 120 ms, 350 ms y 210 ms respectivamente, ¿en cuántos milisegundos se resuelve `Promise.all` de las tres?
4 [PRB-0030] (Intermedio, Diseño) En el problema anterior, si en lugar de `Promise.all` se ejecutaran secuencialmente con tres `await` uno tras otro, ¿cuántos milisegundos adicionales tardaría la operación ($120 + 350 + 210 - 350$)?

Capítulo 6

Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas

*Este capítulo inicia la monografía diferenciada de **TypeScript (5.x)** (Bloques A, B, C y D). Explicamos con rigor matemático cómo su sistema de tipos estructural modela toda la flexibilidad dinámica de JavaScript mediante tipos algebraicos y análisis de flujo de control, y por qué el **borrado de tipos** (*type erasure*) exige combinar tipos estáticos con validadores en las fronteras de red.*

Consejo

Activa siempre en `tsconfig.json` tanto `"strict": true` como `"noUncheckedIndexedAccess": true`: sin esta última opción, indexar un array `arr[i]` o un diccionario `dict[k]` se tipa como `T` en lugar de `T | undefined`, ocultando posibles accesos fuera de límites.

Definición

Subtipado Estructural por Anchura y Profundidad: relación de subtipado $S <: T$ en la que un registro $S = \{l_i : S_i\}_{i \in I}$ es subtipo de $T = \{l_j : T_j\}_{j \in J}$ si el conjunto de etiquetas de T está contenido en el de S ($J \subseteq I$, anchura) y para cada $j \in J$ inmutable se cumple $S_j <: T_j$ (profundidad).

Autoficha

¿Por qué TypeScript aplica una comprobación adicional de *propiedades en exceso* (*excess property checking*) cuando se pasa un literal de objeto directamente a una función, pero la omite si ese mismo objeto se guarda primero en una variable intermedia?

6.1 Bloques A–C: Filosofía de TypeScript, borrado de tipos, tsconfig.json estricto y primitivos (unknown, never)

Creado por Anders Hejlsberg en Microsoft y publicado en 2012, TypeScript es un superconjunto sintáctico estáticamente tipado de ECMAScript. Su principio arquitectónico fundamental es el **borrado de tipos** (*type erasure*): una vez que el comprobador de tipos (`tsc`) verifica la ausencia de errores estáticos, todas las anotaciones de tipos, interfaces, alias `type` y parámetros genéricos desaparecen por completo al emitir JavaScript (o al ejecutar directamente el fichero `.ts` mediante `node -experimental-strip-types` en Node.js 22+/24+). En consecuencia, las anotaciones de TypeScript tienen **coste cero** en tiempo de ejecución, pero tampoco realizan validaciones automáticas sobre un JSON recibido de la red (`JSON.parse` devuelve datos cuya forma real debe verificarse en tiempo de ejecución antes de estrechar su tipo). En teoría de tipos, `unknown` es el tipo superior seguro (*top type*: acepta cualquier valor pero prohíbe operar sobre él sin estrecharlo primero), `never` es el tipo inferior vacío (*bottom type*, conjunto $\emptyset$: habita las funciones que nunca retornan y las ramas inalcanzables), mientras que `any` desactiva el sistema de tipos y debe prohibirse mediante `@typescript-eslint/no-explicit-any`.

6.2 Bloque D: Tipado estructural, interfaces vs type aliases, estrechamiento (narrowing) y uniones discriminadas

A diferencia de Java o C#, TypeScript utiliza **subtipado estructural** (*structural subtyping*): un tipo S es subtipo de T ($S <: T$) si y solo si S posee al menos todas las propiedades requeridas por T con tipos compatibles, sin importar cómo se declararon sus nombres. La herramienta de modelado de

dominio más potente de TypeScript es la **unión discriminada** (*discriminated union*): una unión de tipos objeto que comparten una propiedad literal discriminante común (como `kind` o `ok`). Como se implementa en nuestro proyecto verificado `projects/ts-reactive-http`, el compilador estrecha (*narrows*) automáticamente el tipo en cada rama condicional y verifica la exhaustividad asignando el caso residual al tipo vacío `never`:

```
export type DomainEvent =
  | { readonly kind: "created"; readonly id: string; readonly timestamp: number }
  | { readonly kind: "paid"; readonly id: string; readonly amountCents: number }
  | { readonly kind: "cancelled"; readonly id: string; readonly reason: string };

export function formatEventSummary(ev: DomainEvent): string {
  switch (ev.kind) {
    case "created":   return `Created ${ev.id} at ${ev.timestamp}`;
    case "paid":      return `Paid ${ev.id}: ${ev.amountCents} cents`;
    case "cancelled": return `Cancelled ${ev.id}: ${ev.reason}`;
    default: {
      const _exhaustive: never = ev;
      return _exhaustive;
    }
  }
}
```

Si en el futuro un ingeniero añade una cuarta variante a `DomainEvent`, el bloque `default` fallará en compilación porque la nueva variante no es asignable a `never`.

Problema resuelto

Enunciado En un módulo de pagos en TypeScript, un estado mal modelado con banderas opcionales independientes `{ status: 'idle' | 'loading' | 'success' | 'error'; isRetry: boolean; hasCache: boolean; isOffline: boolean }` tiene $4 \times 2 \times 2 \times 2 = 32$ combinaciones posibles. Al refactorizarlo en una unión discriminada exacta donde solo existen 9 estados válidos de dominio, ¿cuántos estados inválidos se vuelven estáticamente irrepresentables en compilación?
Datos

- $C_{prod} = 32$ — Estados del tipo producto cartesiano $(4 * 2^3)$
- $C_{suma} = 9$ — Estados legítimos de la unión discriminada

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Reducción de cardinalidad de estados mediante uniones discriminadas en TypeScript

$$\Delta C = 4 \cdot 2^3 - 9$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos los 9 estados legítimos a las 32 combinaciones del producto cartesiano

$$\Delta C = 4 \cdot 2^3 - 9$$

$$((((4 \cdot 2) \cdot 2) \cdot 2) - 9)$$

Resultado

$$Resultado = 23$$

Interpretación Se eliminan de raíz 23 estados imposibles o incoherentes sin escribir una sola prueba adicional.
Comprobación de los pasos

- Paso 1: 23 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0033**] **(Básico, Numérico)** Un tipo literal unión `type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH'` se combina en una tupla `[HttpMethod, boolean]`. ¿Cuántos valores distintos habitan esa tupla (5×2)?
2 [**PRB-0034**] **(Básico, Numérico)** ¿Cuántos bytes adicionales ocupa en el fichero `.js` emitido por `tsc` una interfaz `interface UserProfile` de 45 líneas tras el borrado de tipos?
3 [**PRB-0035**] **(Intermedio, Desarrollo por pasos)** Una unión discriminada tiene 6 variantes y un `switch` cubre actualmente 4 de ellas. ¿Cuántas variantes quedan en el tipo estrechado de la variable al llegar a la rama `default`?
4 [**PRB-0036**] **(Intermedio, Diseño)** En `projects/ts-reactive-http`, el limitador de tasa se inicializa con capacidad 3 y se envían 4 peticiones válidas seguidas. ¿Cuántas respuestas retornan con código HTTP `201` antes de que salte el código `429`?

Capítulo 7

Monografía de TypeScript (II – Bloques E–H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos

Este capítulo lleva al lector a los niveles Avanzado y Experto de TypeScript (Bloques E, F, G y H). El sistema de tipos de TypeScript constituye por sí mismo un lenguaje funcional puro que se evalúa en tiempo de compilación sobre conjuntos de tipos, permitiendo calcular tipos de retorno, rutas tipadas y esquemas recursivos.

Consejo

Utiliza el operador `satisfies` para diccionarios de configuración y tablas de rutas: `const routes = { ... } satisfies Record<string, RouteConfig>` comprueba cada entrada contra `RouteConfig` pero conserva las claves literales exactas en `keyof typeof routes`.

Definición

Distributividad de Tipos Condicionales en TypeScript: propiedad según la cual, para un tipo condicional `type F<T> = T extends U ? X : Y` donde `T` es un parámetro genérico desnudo, instanciar `F<A | B>` se distribuye automáticamente como `F<A> | F<B>` (pudiendo desactivarse escribiendo `[T] extends [U] ? X : Y`).

Autoficha

¿Por qué `Exclude<'a' | 'b' | 'c', 'a'>` devuelve `'b' | 'c'` gracias a la distributividad de `T extends U ? never : T` y por qué `never` actúa como el elemento neutro de la unión (`T | never = T`)?

7.1 Bloques E y F: Generics, keyof, Mapped Types, Conditional Types, infer, Template Literals, satisfies y Branded Types

La programación a nivel de tipos en TypeScript se apoya en cuatro primitivas composicionales: (1) el operador de consulta de claves `keyof T` y el acceso indexado `T[K]`; (2) los **tipos mapeados** (`{ readonly [K in keyof T as ...]: ... }`), sobre los que se construyen utilidades estándar como `Partial<T>`, `Required<T>`, `Readonly<T>`, `Pick<T, K>` y `Record<K, V>`; (3) los **tipos condicionales** `T extends U ? X : Y`, que se distribuyen automáticamente sobre uniones cuando `T` es un parámetro de tipo desnudo y permiten extraer tipos internos mediante `infer` (como `type AwaitedValue<T> = T extends Promise<infer U> ? AwaitedValue<U> : T`); y (4) los **Template Literal Types**, que calculan productos cartesianos de cadenas en compilación. Además, TypeScript 4.9+ introdujo el operador `satisfies`, que valida que una expresión cumpla un contrato `T` **sin ensanchar** (*widening*) su tipo literal inferido, y para los casos donde el tipado estructural resulta demasiado permisivo (por ejemplo, evitar pasar un `UserId` donde se espera un `OrderId`, siendo ambos `string`), se emplean **Branded Types**:

```
declare const __brand: unique symbol;
export type Brand<T, B extends string> = T & { readonly [__brand]: B };

export type UserId  = Brand<string, "UserId">;
export type OrderId = Brand<string, "OrderId">;
export type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
```

```
};
```

Con `Brand<string, "UserId">`, la marca fantasma `unique symbol` existe únicamente en el comprobador de tipos y se borra por completo en ejecución, proporcionando seguridad nominal con coste cero.

7.2 Bloques G y H: Ecosistema (Zod, tRPC, Vite, tsx, Vitest) y los 4 proyectos (incluyendo ts-reactive-http)

En el Bloque G (Ecosistema), la práctica estándar de la industria combina TypeScript con validadores de esquema en las fronteras de entrada/salida (como `Zod` o `Valibot`, donde `type Payload = z.infer<typeof PayloadSchema>` deriva el tipo estático directamente del validador de ejecución para que jamás diverjan), comunicación extremo a extremo con seguridad de tipos (`tRPC`, OpenAPI/GraphQL codegen), empaquetadores rápidos (`Vite`, `esbuild`, `tsup`) y ejecución directa en Node.js (`node -experimental-strip-types` o `tsx`). En el Bloque H, junto con un cliente HTTP tipado por rutas, un motor de máquinas de estados finitos (`StateMachine<S, E>`) y un bus de eventos tipado, estudiamos el proyecto completo `projects/ts-reactive-http`, verificado con `node -experimental-strip-types -test`.

Problema resuelto

Enunciado En un sistema de diseño tipado en TypeScript, se define el tipo de clases utilitarias mediante Template Literal Types: `type SpacingUtility =` ${Direction}-${Axis}-${Scale}`, donde` Direction `tiene $2$ miembros ('m' | 'p'),` Axis `tiene $7$ miembros ('t' | 'b' | 'l' | 'r' | 'x' | 'y' | 'a') y` Scale `tiene $12$ miembros ('0' a '11'). Calcula cuántos tipos literales de cadena exactos genera el compilador en la unión` SpacingUtility`.`
Datos

- $Direction = 2$ — Margen o padding ('m' | 'p')
- $Axis = 7$ — Ejes direccionales
- $Scale = 12$ — Niveles de la escala espacial

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Producto cartesiano en Template Literal Types de TypeScript

$$|U| = 2 \cdot 7 \cdot 12$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las cardinalidades de las tres uniones literales

$$|U| = 2 \cdot 7 \cdot 12$$

$$((2 \cdot 7) \cdot 12)$$

Resultado

$$Resultado = 168$$

Interpretación El compilador genera y autocompleta exactamente 168 literales de cadena válidos en tiempo de compilación.
Comprobación de los pasos

- Paso 1: 168 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0039**] **(Básico, Numérico)** Una interfaz `Config` tiene 8 propiedades obligatorias. Si se aplica `Pick<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades tiene el tipo resultante?
2 [**PRB-0040**] **(Básico, Numérico)** En el problema anterior, si en su lugar se aplica `Omit<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades conserva el tipo resultante?
3 [**PRB-0041**] **(Intermedio, Desarrollo por pasos)** Un Template Literal Type `${'http' | 'https'}://${'api' | 'auth' | 'cdn'}.ejemplo.es/${'v1' | 'v2'}` expande todas las rutas válidas. ¿Cuántos literales de URL compone la unión?
4 [**PRB-0042**] **(Intermedio, Diseño)** En `projects/ts-reactive-http`, cada lectura de telemetría se redondea con `Math.round(v * 100) / 100`. Para el vector `[12.5, 7.25, 10.25]`, ¿cuál es el valor exacto de la suma?

Capítulo 8

Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript

Este capítulo de síntesis compara con rigor científico los tres lenguajes del Volumen V y muestra cómo combinarlos en arquitecturas reales de producción (por ejemplo, un frontend y pasarela API en TypeScript comunicándose con un motor de cálculo científico o de aprendizaje automático en Python).

Consejo

En arquitecturas políglotas Python + TypeScript, utiliza el esquema OpenAPI exportado por FastAPI/Pydantic o definiciones Protobuf como única fuente de verdad en el pipeline de CI para detectar rupturas de contrato antes de desplegar.

Definición

Ejecución ansiosa frente a perezosa de tareas asíncronas (Eager Promises vs Lazy Coroutines): en ECMAScript el ejecutor de `new Promise(fn)` se invoca síncronamente al instanciarse, mientras que en Python (y Rust) invocar una función `async` devuelve un objeto corrutina inerte hasta que el bucle de eventos lo impulsa.

Autoficha

¿Por qué llamar a una función `async def f()` en Python sin anteponer `await` ni `asyncio.create_task(f())` no ejecuta ni una sola línea del cuerpo de `f` y emite un `RuntimeWarning: coroutine 'f' was never awaited`?

8.1 Comparativa profunda de ejecución, memoria, concurrencia y sistemas de tipos (Python vs JS vs TS)

Aunque Python y JavaScript comparten tipado dinámico en tiempo de ejecución y gestión automática de memoria, sus decisiones de diseño divergen en aspectos esenciales. Primero, en **semántica numérica y de tipos**: Python posee enteros de precisión arbitraria (`int`) y distingue estrictamente entre tipos sin coerción implícita entre cadenas y números (`"5" + 1` lanza `TypeError`), mientras que JavaScript unifica números en `Number` (IEEE 754) + `BigInt` y realiza coerciones implícitas si no se usa `===` o TypeScript. Segundo, en **modelo de objetos**: Python se basa en clases reales donde incluso las metaclases son objetos (`type`), mientras que JavaScript se basa en delegación de prototipos (`[[Prototype]]`). Tercero, en **asincronía**: en JavaScript una `Promise` es *ansiosa* (*eager*, comienza a ejecutarse en el instante en que se construye y el Event Loop está siempre activo por defecto), mientras que una corrutina `async def` en Python es estrictamente *perezosa* hasta que es agendada o esperada dentro de `asyncio.run(...)`.

8.2 Arquitectura políglota: contratos OpenAPI/JSON Schema entre Python y TypeScript y evaluación global

Tanto el sistema de anotaciones de Python (`typing`, PEP 484/695) como TypeScript aplican borrado de tipos en tiempo de ejecución (*type erasure*). Sin embargo, dado que en Python las anotaciones pueden conservarse opcionalmente en el diccionario `__annotations__` de la clase o función, bibliotecas como `Pydantic` y `FastAPI` inspeccionan dichas anotaciones al arrancar el proceso para compilar validadores en Rust y emitir automáticamente un documento **OpenAPI 3.1 / JSON Schema**. A partir de ese

único contrato normativo, el equipo de TypeScript genera sus tipos estáticos y clientes HTTP sin duplicar definiciones manualmente, garantizando consistencia matemática entre ambos extremos del sistema distribuido.

Problema resuelto

Enunciado Una pasarela en TypeScript envía lotes de $N = 5000$ eventos de telemetría a un servicio de análisis en Python. En formato JSON textual cada objeto `{ "sensorId": ..., "timestamp": ..., "value": ... }` ocupa en promedio $S_{\text{json}} = 76$ B, mientras que codificado en binario compacto ocupa $S_{\text{bin}} = 20$ B. Calcula cuántos kilobytes (dividiendo entre 1000) se ahorran por cada lote transmitido.
Datos

- $N = 5000$ — Eventos por lote
- $S_{json} = 76\,\text{B}$ — Tamaño medio en JSON UTF-8
- $S_{bin} = 20\,\text{B}$ — Tamaño en binario compacto

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de tráfico de red entre representación JSON y binaria

$$\Delta B_{\text{kB}} = \frac{5000 \cdot (76 - 20)}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 5000 eventos por los 56 B ahorrados y dividimos entre 1000 B/kB

$$\Delta B_{\text{kB}} = \frac{5000 \cdot (76 - 20)}{1000}$$

$$\frac{(5000 \cdot (76 - 20))}{1000}$$

Resultado

$$Resultado = 280$$

Interpretación Se ahorran 280 kB por lote (pasando de 380 kB a 100 kB).
Comprobación de los pasos

- Paso 1: 280 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0045] (Básico, Numérico) Un contrato OpenAPI define 18 endpoints REST y cada endpoint especifica 3 esquemas (petición, respuesta 200 y error 400). ¿Cuántos esquemas genera automáticamente el generador de tipos de TypeScript (18×3)?
2 [PRB-0046] (Básico, Numérico) Si una respuesta JSON de 120 000 bytes se comprime con Brotli a un 25 % de su tamaño original ($120000 / 4$), ¿cuántos bytes se transmiten por la red?
3 [PRB-0047] (Intermedio, Desarrollo por pasos) Un servicio en Python delega un cálculo matricial a una extensión compilada en Rust (`PyO3`) reduciendo el tiempo por petición de 45 ms a 3 ms. Para 200 peticiones secuenciales, ¿cuántos milisegundos totales se ahorran?

4 [PRB-0048] (Intermedio, Diseño) Una aplicación web carga un módulo WebAssembly de 180 KiB y un bundle TypeScript/JS de 95 KiB. ¿Cuántos KiB totales descarga el cliente en el arranque inicial?

Apéndice A

Notación

Símbolo	Significado	Unidades
T extends $U ? X : Y$	Tipo condicional distributivo en el sistema de tipos de TypeScript	tipo
$\tau_{\text{micro}}, \tau_{\text{macro}}$	Colas de microtareas (Promises) y macrotareas (timers/I/O) del Event Loop	tareas
refcnt(o)	Contador de referencias de un PyObject en CPython	referencias

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante

Problema resuelto [PRB-0001]
En CPython de 64 bits, cada entrada de un diccionario almacena tres palabras de 8 B (`me_hash`, `me_key`, `me_value` = 24 B). Para una tabla con $K = 128$ ranuras en el índice (1 B por ranura en `dk_indices`) y capacidad para $N = 85$ entradas en `dk_entries`, calcula cuántos bytes ahorra el diseño de diccionario compacto ($128 \cdot 1 + 85 \cdot 24$) frente al diseño disperso antiguo ($128 \cdot 24$).
Respuesta

$$\textit{Resultado} = 904$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
En CPython de 64 bits, la cabecera base `PyObject` contiene `ob_refcnt` (8 B) y `ob_type` (8 B). ¿Cuántos bytes mide esta cabecera mínima?
Respuesta 16
Ejercicio 2 [PRB-0004]
Una lista `PyListObject` mantiene un array de punteros `PyObject*` (8 B cada uno). Si su capacidad reservada es de 64 elementos, ¿cuántos bytes ocupa ese array de punteros?
Respuesta 512
Ejercicio 3 [PRB-0005]
¿Cuál es la suma de los cuadrados impares evaluada por `sum(x * x for x in range(1, 10, 2))` ($1^2 + 3^2 + 5^2 + 7^2 + 9^2$)?
Respuesta 165
Ejercicio 4 [PRB-0006]
En el proyecto `projects/python-bytecode-vm`, la función compila y evalúa `(x + 4) * (y - 2) + (10 if x < y else 20)` para $x = 6, y = 9$. ¿Cuál es el resultado entero devuelto?
Respuesta 80

Cálculos reproducidos

$((128 \cdot 24) - ((128 \cdot 1) + (85 \cdot 24))) = 904$

C.2 Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers

Problema resuelto [PRB-0007]
En una aplicación de procesamiento de grafos en CPython de 64 bits, una clase `Edge` con $k = 3$ atributos ocupa $16 + 3 \times 8 = 40$ B cuando se declara con `__slots__`, mientras que sin `__slots__` cada instancia aloja además un `__dict__` que añade 112 B por objeto (152 B en total). Calcula cuántos megabytes exactos (dividiendo entre 10^6) se ahorran al instanciar $N = 1000000$ de aristas con `__slots__`.
Respuesta

$$\textit{Resultado} = 112$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0009]
En CPython de 64 bits, ¿cuántos bytes ocupa una instancia con `__slots__` de 4 atributos (16 B de cabecera más 4×8 B)?
Respuesta 48
Ejercicio 2 [PRB-0010]
Un generador perezoso produce bloques de 4096 bytes uno a uno para procesar un fichero de 10 GiB. ¿Cuántos bloques mantiene simultáneamente en RAM?
Respuesta 1
Ejercicio 3 [PRB-0011]
En una jerarquía con herencia múltiple `class D(B, C)` donde `B(A)` y `C(A)` heredan de `A(object)`, ¿cuántas clases aparecen en la tupla de orden de resolución de métodos `D.__mro__` (`D, B, C, A, object`)?
Respuesta 5
Ejercicio 4 [PRB-0012]
Un decorador `@lru_cache(maxsize=256)` recibe 1000 consultas sobre un conjunto de solo 200 claves distintas. ¿Cuántas veces se ejecuta realmente la función subyacente?
Respuesta 200

Cálculos reproducidos

$\frac{(1000000 \cdot (152-40))}{1000000} = 112$

C.3 Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos

Problema resuelto [PRB-0013]
Un agregador de microservicios en Python debe consultar $N = 20$ endpoints HTTP independientes, cada uno con una latencia de red de $L = 150$ ms y 1 ms de procesamiento de CPU local en el bucle de eventos. Calcula cuántos milisegundos se ahorran al ejecutarlos concurrentemente con `asyncio.TaskGroup` ($150 + 20 \times 1 = 170$ ms) frente a ejecutarlos secuencialmente de forma síncrona ($20 \times (150 + 1) = 3020$ ms).
Respuesta

$$Resultado = 2850$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0015]
En `projects/python-bytecode-vm`, la expresión `2 * 3 + 4 * 5` emite 4 instrucciones `LOAD_CONST` y 3 instrucciones binarias (`MUL`, `MUL`, `ADD`). ¿Cuántas instrucciones totales contiene el bloque compilado?
Respuesta 7
Ejercicio 2 [PRB-0016]
Al evaluar `2 * 3 + 4 * 5` en una máquina de pila, ¿cuál es la altura máxima (número máximo de operandos apilados simultáneamente) que alcanza la pila?
Respuesta 3
Ejercicio 3 [PRB-0017]
Un pool de 8 procesos (`ProcessPoolExecutor(max_workers=8)`) ejecuta 40 tareas de CPU de 2 segundos cada una. ¿Cuántos segundos tarda en completar las 40 tareas ideales ($40/8 \times 2$)?
Respuesta 10
Ejercicio 4 [PRB-0018]
Una política de reintentos con retroceso exponencial (`backoff`) espera $100 \cdot 2^k$ ms para $k = 0, 1, 2, 3$. ¿Cuántos milisegundos totales habrá esperado acumulados tras esos 4 reintentos?
Respuesta 1500

Cálculos reproducidos

$((20 \cdot (150 + 1)) - (150 + (20 \cdot 1))) = 2850$

C.4 Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM

Problema resuelto [PRB-0019]
En el motor V8 con compresión de punteros (*Pointer Compression*), cada referencia a propiedad en un objeto de forma fija (*Fast Properties / In-object properties*) ocupa 4 B en lugar de 8 B, más una cabecera de objeto de 12 B (`map`, `properties`, `elements` comprimidos). Calcula cuántos bytes ahorran $N = 10000$ objetos con $k = 5$ propiedades en línea bajo compresión de punteros ($12 + 5 \times 4 = 32$ B por objeto) frente a punteros de 64 bits sin comprimir ($24 + 5 \times 8 = 64$ B por objeto).
Respuesta

$$Resultado = 320000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
En V8 con punteros comprimidos de 4 bytes, ¿cuántos bytes ocupa la cabecera de 3 campos (`map`, `properties`, `elements`) de un objeto JS (3×4)?
Respuesta 12
Ejercicio 2 [PRB-0022]
Un `ArrayBuffer(1024)` se observa mediante un `Float64Array` (8 bytes por elemento). ¿Cuántos elementos `length` tiene la vista tipada?
Respuesta 128
Ejercicio 3 [PRB-0023]
Si una cadena de prototipos tiene 4 niveles (`instancia -> SubClase.prototype -> Base.prototype -> Object.prototype -> null`) y se busca una propiedad inexistente, ¿cuántos eslabones de objeto se inspeccionan antes de llegar a `null`?
Respuesta 4
Ejercicio 4 [PRB-0024]
En `TokenBucket` con capacidad inicial de 100 tokens, se procesan 7 peticiones que consumen 12 tokens cada una. ¿Cuántos tokens quedan disponibles?
Respuesta 16

Cálculos reproducidos

$(10000 \cdot ((24 + (5 \cdot 8)) - (12 + (5 \cdot 4)))) = 320000$

C.5 Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos

Problema resuelto [PRB-0025]
Un descargador asíncrono en Node.js debe procesar $N = 120$ ficheros remotos con un limitador de concurrencia que mantiene exactamente $k = 15$ promesas en vuelo simultáneamente para no saturar el servidor de origen. Si cada descarga tarda $D = 200$ ms, calcula el tiempo total en milisegundos necesario para completar las 120 descargas ($(120/15) \times 200$).
Respuesta

$$Resultado = 1600$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
En un turno del Event Loop se encolan 3 macrotareas (`setTimeout`) y 5 microtareas (`Promise.then`). ¿Cuántas microtareas se ejecutan antes de que comience la primera macrotarea?
Respuesta 5
Ejercicio 2 [PRB-0028]

Un `ReadableStream` con `highWaterMark = 65536` bytes (64 KiB) contiene actualmente 49 152 bytes sin consumir. ¿Cuántos bytes más puede aceptar antes de activar la señal de contrapresión?

Respuesta 16384

Ejercicio 3 [PRB-0029]

Si 3 promesas independientes tardan 120 ms, 350 ms y 210 ms respectivamente, ¿en cuántos milisegundos se resuelve `Promise.all` de las tres?

Respuesta 350

Ejercicio 4 [PRB-0030]

En el problema anterior, si en lugar de `Promise.all` se ejecutaran secuencialmente con tres `await` uno tras otro, ¿cuántos milisegundos adicionales tardaría la operación ($120 + 350 + 210 - 350$)?

Respuesta 330

Cálculos reproducidos

$\left(\frac{120}{15} \cdot 200\right) = 1600$

C.6 Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas

Problema resuelto [PRB-0031]

En un módulo de pagos en TypeScript, un estado mal modelado con banderas opcionales independientes `{ status: 'idle' | 'loading' | 'success' | 'error'; isRetry: boolean; hasCache: boolean; isOffline: boolean }` tiene $4 \times 2 \times 2 \times 2 = 32$ combinaciones posibles. Al refactorizarlo en una unión discriminada exacta donde solo existen 9 estados válidos de dominio, ¿cuántos estados inválidos se vuelven estáticamente irrepresentables en compilación?

Respuesta

$$Resultado = 23$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0033]

Un tipo literal unión `type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH'` se combina en una tupla `[HttpMethod, boolean]`. ¿Cuántos valores distintos habitan esa tupla (5×2)?

Respuesta 10

Ejercicio 2 [PRB-0034]

¿Cuántos bytes adicionales ocupa en el fichero `.js` emitido por `tsc` una interfaz `interface UserProfile` de 45 líneas tras el borrado de tipos?

Respuesta 0

Ejercicio 3 [PRB-0035]

Una unión discriminada tiene 6 variantes y un `switch` cubre actualmente 4 de ellas. ¿Cuántas variantes quedan en el tipo estrechado de la variable al llegar a la rama `default`?

Respuesta 2

Ejercicio 4 [PRB-0036]

En `projects/ts-reactive-http`, el limitador de tasa se inicializa con capacidad 3 y se envían 4 peticiones válidas seguidas. ¿Cuántas respuestas retornan con código HTTP `201` antes de que salte el código `429`?

Respuesta 3

Cálculos reproducidos

$((((4 \cdot 2) \cdot 2) \cdot 2) - 9) = 23$

C.7 Monografía de TypeScript (II – Bloques E–H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos

Problema resuelto [PRB-0037]
En un sistema de diseño tipado en TypeScript, se define el tipo de clases utilitarias mediante Template Literal Types: `type SpacingUtility = ${Direction}-${Axis}-${Scale}`, donde Direction `tiene $2$ miembros ('m' | 'p')`, Axis `tiene $7$ miembros ('t' | 'b' | 'l' | 'r' | 'x' | 'y' | 'a')` y Scale `tiene $12$ miembros ('0' a '11'). Calcula cuántos tipos literales de cadena exactos genera el compilador en la unión` SpacingUtility`.
Respuesta

$$\textit{Resultado} = 168$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0039]
Una interfaz `Config` tiene 8 propiedades obligatorias. Si se aplica `Pick<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades tiene el tipo resultante?
Respuesta 3
Ejercicio 2 [PRB-0040]
En el problema anterior, si en su lugar se aplica `Omit<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades conserva el tipo resultante?
Respuesta 5
Ejercicio 3 [PRB-0041]
Un Template Literal Type `${'http' | 'https'}://${'api' | 'auth' | 'cdn'}.ejemplo.es/${'v1' | 'v2'}` expande todas las rutas válidas. ¿Cuántos literales de URL compone la unión?
Respuesta 12
Ejercicio 4 [PRB-0042]
En `projects/ts-reactive-http`, cada lectura de telemetría se redondea con `Math.round(v * 100) / 100`. Para el vector `[12.5, 7.25, 10.25]`, ¿cuál es el valor exacto de la suma?
Respuesta 30

Cálculos reproducidos

$((2 \cdot 7) \cdot 12) = 168$

C.8 Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript

Problema resuelto [PRB-0043]
Una pasarela en TypeScript envía lotes de $N = 5000$ eventos de telemetría a un servicio de análisis en Python. En formato JSON textual cada objeto `{ "sensorId": ..., "timestamp": ..., "value": ... }` ocupa en promedio $S_{\text{json}} = 76$ B, mientras que codificado en binario compacto ocupa $S_{\text{bin}} = 20$ B. Calcula cuántos kilobytes (dividiendo entre 1000) se ahorran por cada lote transmitido.
Respuesta

$$\textit{Resultado} = 280$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
Un contrato OpenAPI define 18 endpoints REST y cada endpoint especifica 3 esquemas (petición, respuesta 200 y error 400). ¿Cuántos esquemas genera automáticamente el generador de tipos de TypeScript (18×3)?
Respuesta 54

Ejercicio 2 [PRB-0046]
Si una respuesta JSON de 120 000 bytes se comprime con Brotli a un 25 % de su tamaño original ($120000 / 4$), ¿cuántos bytes se transmiten por la red?
Respuesta 30000
Ejercicio 3 [PRB-0047]
Un servicio en Python delega un cálculo matricial a una extensión compilada en Rust (`PyO3`) reduciendo el tiempo por petición de 45 ms a 3 ms. Para 200 peticiones secuenciales, ¿cuántos milisegundos totales se ahorran?
Respuesta 8400
Ejercicio 4 [PRB-0048]
Una aplicación web carga un módulo WebAssembly de 180 KiB y un bundle TypeScript/JS de 95 KiB. ¿Cuántos KiB totales descarga el cliente en el arranque inicial?
Respuesta 275

Cálculos reproducidos

$\frac{(5000 \cdot (76 - 20))}{1000} = 280$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 32 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.

¶2: dos testimonios.

¶3: dos testimonios.

¶4: dos testimonios.

¶5: dos testimonios.

¶6: dos testimonios.

¶7: dos testimonios.

¶8: dos testimonios.

¶9: dos testimonios.

¶10: dos testimonios.

¶11: dos testimonios.

¶12: dos testimonios.

¶13: dos testimonios.

¶14: dos testimonios.

¶15: dos testimonios.

¶16: dos testimonios.

¶17: dos testimonios.

¶18: dos testimonios.

¶19: dos testimonios.

¶20: dos testimonios. Ecuaciones: 1 reproducida numéricamente.

¶21: dos testimonios.

¶22: dos testimonios.

¶23: dos testimonios.

¶24: dos testimonios.

¶25: dos testimonios.

¶26: dos testimonios.

¶27: dos testimonios.

¶28: dos testimonios.

¶29: dos testimonios.

¶30: dos testimonios.

¶31: dos testimonios.

¶32: dos testimonios.

¶33: dos testimonios.

¶34: dos testimonios. Ecuaciones: 2 valor o condición.

¶35: dos testimonios.

¶36: dos testimonios.

¶37: dos testimonios.

¶38: dos testimonios.

¶39: dos testimonios.

¶40: dos testimonios.

¶41: dos testimonios.

¶42: dos testimonios.

¶43: dos testimonios.

¶44: dos testimonios.

¶45: dos testimonios.

¶46: dos testimonios.

¶47: dos testimonios.

¶48: dos testimonios.

Referencia Normativa src-python-ref (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-unicode-15 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
Referencia Normativa src-gamma-gof (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-ecmascript-2024 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-mdn-html-css (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
The TypeScript Handbook — Structural Typing, Narrowing, and Type Manipulation (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
REV-0001: cap01-python-fundamentos-modelo-objetos-nivel-principiante scientific
REV-0002: cap02-python-intermedio-avanzado-modelo-datos-tipado-iteradores scientific
REV-0003: cap03-python-experto-asyncio-concurrencia-metaclases-packaging-proyectos scientific
REV-0004: cap04-javascript-fundamentos-closures-prototipos-nivel-intermedio scientific
REV-0005: cap05-javascript-avanzado-event-loop-promesas-web-node-proyectos scientific
REV-0006: cap06-typescript-fundamentos-sistema-tipos-estructural-uniones scientific
REV-0007: cap07-typescript-avanzado-experto-mapped-conditional-infer-proyectos scientific
REV-0008: cap08-interoperabilidad-comparativa-arquitectura-python-js-ts scientific

Apéndice E

Plan de estudio

Monografía Autónoma de Python (Python 3.12 – 3.14)

✓Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante

- Comprender la historia de Python (Guido van Rossum, 1991), la transición Python 2 a 3 y el modelo de ejecución de bytecode de CPython (incluyendo el compilador especializador PEP 659 y el modo free-threaded PEP 703).
- Configurar entornos reproducibles conforme a PEP 517/518/621 (‘pyproject.toml‘, ‘venv‘, ‘uv‘, ‘ruff‘, ‘pyright‘, ‘pytest‘).
- Dominar la semántica de nombres vinculados a objetos (‘PyObject‘), mutabilidad vs inmutabilidad, tipos básicos y estructuras de datos nativas (‘list‘, ‘tuple‘, ‘dict‘, ‘set‘).

2 conceptos planificados.

✓Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers

- Dominar el Data Model de Python (‘___repr___‘, ‘___eq___‘, ‘___hash___‘, ‘___len___‘, ‘___getitem___‘, ‘___enter___‘/‘___exit___‘, ‘___slots___‘).
- Diseñar jerarquías limpias con ‘@dataclass(frozen=True, slots=True)‘, protocolos estructurales (‘typing.Protocol‘) y genéricos de Python 3.12 (PEP 695).
- Construir clausuras, decoradores con ‘functools.wraps‘, generadores perezosos (‘yield‘, ‘yield from‘) y gestores de contexto (‘contextlib‘).

2 conceptos planificados.

✓Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos

- Dominar la concurrencia asíncrona con ‘asyncio‘ (‘TaskGroup‘, ‘gather‘, corrutinas nativas ‘async def‘/‘await‘) y distinguir cuándo usar ‘multiprocessing‘ o ‘concurrent.futures‘.
- Aplicar metaprogramación limpia con ‘___init_subclass___‘, metaclases, el módulo ‘ast‘ y extensiones nativas C/Rust (‘PyO3‘/‘maturin‘, ‘ctypes‘).
- Empaquetar y publicar distribuciones Wheel/sdist y estudiar el proyecto completo verificado ‘projects/python-bytecode-vm‘.

2 conceptos planificados.

Monografías Autónomas de JavaScript (ES2024+) y TypeScript (5.x)

✓Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM

- Comprender la evolución de ECMAScript (ES3, ES5, ES6/ES2015 hasta ES2024+) y la arquitectura JIT multipaso (Ignition + Sparkplug + Maglev + TurboFan en V8).

- Dominar los tipos primitivos (‘number‘ IEEE 754, ‘bigint‘, ‘string‘, ‘symbol‘), la igualdad estricta (‘===‘), ámbitos léxicos (‘let‘/‘const‘ y Temporal Dead Zone) y clausuras.
- Explicar con rigor la delegación prototípica (‘[[Prototype]]‘, ‘Object.create‘, sintaxis ‘class‘), la vinculación dinámica de ‘this‘ y el sistema de módulos estándar ESM (‘import‘/‘export‘).

2 conceptos planificados.

✓Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos

- Trazar con exactitud el orden de ejecución entre la pila síncrona, la cola de microtareas (Promises, queueMicrotask) y la cola de macrotareas (setTimeout, I/O).
- Dominar los combinadores concurrentes de Promesas (‘Promise.all‘, ‘Promise.allSettled‘, ‘Promise.race‘, ‘Promise.any‘), ‘AbortController‘, generadores asíncronos (‘for await...of‘) y Web/Node Streams.
- Gestionar fugas de memoria en el recolector generacional (Orinoco), usar ‘WeakMap‘/‘WeakRef‘, ‘Worker Threads‘ y desarrollar los 4 proyectos en JavaScript.

2 conceptos planificados.

✓Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas

- Comprender con exactitud la relación entre TypeScript y JavaScript: superconjunto sintáctico tipado con borrado completo de tipos (type erasure) en tiempo de ejecución.
- Configurar ‘tsconfig.json‘ con máxima rigurosidad (‘strict: true‘, ‘noUncheckedIndexedAccess: true‘, ‘exactOptionalPropertyTypes: true‘, ‘verbatimModuleSyntax: true‘).
- Dominar el subtipado estructural, tipos unión e intersección (‘|‘, ‘&‘), estrechamiento de flujo de control (*control-flow narrowing*), ‘unknown‘ vs ‘any‘ vs ‘never‘ y uniones discriminadas.

2 conceptos planificados.

✓Monografía de TypeScript (II – Bloques E–H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos

- Dominar los operadores avanzados del sistema de tipos de TypeScript: ‘keyof‘, ‘typeof‘, tipos indexados ‘T[K]‘, predicados de tipo (‘x is T‘), funciones de aserción (‘asserts x is T‘) y el operador ‘satisfies‘.
- Construir transformaciones a nivel de tipos mediante Mapped Types (‘{ [K in keyof T]: ... }‘), Conditional Types (‘T extends U ? X : Y‘), inferencia estructural (‘infer R‘) y Template Literal Types.
- Integrar validación de fronteras en tiempo de ejecución (Zod / TypeBox) y auditar el proyecto completo ‘projects/ts-reactive-http‘.

2 conceptos planificados.

✓Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript

- Comparar rigurosamente los modelos de ejecución, sistemas de tipos, concurrencia y ecosistemas de Python, JavaScript y TypeScript.
- Diseñar contratos de interoperabilidad entre backends en Python (FastAPI/Pydantic) y clientes/servidores en TypeScript mediante esquemas OpenAPI 3.1 y JSON Schema.
- Superar la evaluación integradora del Volumen V resolviendo problemas de rendimiento, tipado y concurrencia en los tres lenguajes.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-python-ref. https://doc.rust-lang.org/reference/src-python-ref.html

[27] Referencia Normativa src-unicode-15. https://doc.rust-lang.org/reference/src-unicode-15.html

[28] Referencia Normativa src-gamma-gof. https://doc.rust-lang.org/reference/src-gamma-gof.html

[29] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

[30] Referencia Normativa src-ecmascript-2024. https://doc.rust-lang.org/reference/src-ecmascript-2024.html

[31] Referencia Normativa src-mdn-html-css. https://doc.rust-lang.org/reference/src-mdn-html-css.html

[32] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen VI: Monografías de C#, Java, Go, Kotlin y Swift

Parte V (Bloques A–H): Máquinas virtuales gestionadas (CLR y JVM), concurrencia G-M-P en Go, Coroutines y ARC/Actores

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografías de Plataformas Gestionadas: C# (.NET 8/9) y Java (JVM 21 LTS) 1

1. Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ 2

1.1. Bloques A–C: Historia de C# y .NET, CLI (‘dotnet‘), CIL, tipos valor vs referencia y nulabilidad . . . 2

1.2. Bloque D: Propiedades, records, interfaces, genéricos reificados, delegados y LINQ 2

2. Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos 5

2.1. Bloques E y F: Span<T>, ref struct, ArrayPool<T>, máquinas de estados async/await, ValueTask<T> y Channels . . . 5

2.2. Bloques G y H: Ecosistema (ASP.NET Core, DI, EF Core, xUnit, BenchmarkDotNet) y los 4 proyectos en C# . . . 6

3. Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams 8

3.1. Bloques A–C: Evolución hasta Java 21 LTS, arquitectura JVM, JIT C1/C2, G1/ZGC y sintaxis fundamental . . . 8

3.2. Bloque D: Records, Sealed Classes, Pattern Matching, Genéricos (PECS), Colecciones y Streams API . . . 8

4. Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos 11

4.1. Bloques E y F: Java Memory Model, Virtual Threads (Project Loom), StructuredTaskScope y API Panama (FFM) . . . 11

4.2. Bloques G y H: Ecosistema (Maven, Gradle, JUnit 5, Spring Boot, GraalVM) y los 4 proyectos en Java . . . 12

II Monografías de Sistemas Cloud y Móviles: Go, Kotlin y Swift 14

5. Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores 15

5.1. Bloques A–C: Filosofía de Go, toolchain integrada (‘go‘), tipos, valores cero, arrays vs slices y maps . . . 15

5.2. Bloque D: Métodos, composición de structs, interfaces implícitas, errores como valores y genéricos . . . 15

6. Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos 18

6.1. Bloques E y F: Planificador G-M-P, goroutines, canales, select, context.Context, sync/errgroup y análisis de escape . . . 18

6.2. Bloques G y H: Ecosistema (net/http, grpc-go, sqlc, pprof, -race) y los 4 proyectos en Go 19

7. Monografía de Kotlin (Bloques A−H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform **21**
7.1. Bloques A−D: Filosofía de Kotlin, interoperabilidad JVM/KMP, null-safety, data/sealed classes y extensiones . . . 21
7.2. Bloques E−H: Funciones inline/reified, DSLs, Coroutines (suspend, CPS, Flow), ecosistema (Ktor, Compose) y proyectos . . . 21

8. Monografía de Swift 6 (Bloques A−H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores **24**
8.1. Bloques A−D: Filosofía de Swift, SIL/LLVM, SwiftPM, semántica de valor (struct/enum), Copy-on-Write y protocolos . . . 24
8.2. Bloques E−H: Gestión de memoria ARC (weak/unowned), concurrencia estricta en Swift 6 (actor, Sendable) y proyectos . . . 25

A. Notación **27**

B. Hoja de fórmulas **28**

C. Soluciones **29**
C.1. Monografía de C# (I − Bloques A−D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ . . . 29
C.2. Monografía de C# (II − Bloques E−H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos . . . 29
C.3. Monografía de Java (I − Bloques A−D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams . . . 30
C.4. Monografía de Java (II − Bloques E−H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos . . . 31
C.5. Monografía de Go (I − Bloques A−D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores . . . 32
C.6. Monografía de Go (II − Bloques E−H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos . . . 32
C.7. Monografía de Kotlin (Bloques A−H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform . . . 33
C.8. Monografía de Swift 6 (Bloques A−H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores . . . 34

D. Auditoría de fuentes **35**

E. Plan de estudio **37**

Parte I

Monografías de Plataformas Gestionadas: C# (.NET 8/9) y Java (JVM 21 LTS)

Capítulo 1

Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ

*Este capítulo inicia la monografía autónoma de **C# (.NET 8/9)** (Bloques A, B, C y D). Diseñado por Anders Hejlsberg, C# combina la ergonomía expresiva de los lenguajes funcionales modernos con un control de bajo nivel sobre la disposición de memoria (`struct`, `Span<T>`, genéricos reificados en la máquina virtual) único entre los entornos gestionados.*

Consejo

Cuando declares un `struct` en C#, márcalo como `readonly struct` (o `readonly record struct`) siempre que sea posible: si pasas un `struct` no `readonly` mediante `in` o lo almacenas en un campo `readonly`, el compilador se ve obligado a emitir copias defensivas ocultas en cada llamada a método para proteger el original.

Definición

Reificación de Genéricos en el CLR (.NET): preservación de los argumentos de tipo genérico en los metadatos CIL y en tiempo de ejecución, permitiendo que el JIT emita representaciones especializadas en memoria para cada tipo por valor (`struct`) sin encajonamiento (*boxing*).

Autoficha

¿Qué problema grave de rendimiento ocurre en una base de datos de millones de filas si un desarrollador convierte un `IQueryable<Order>` de EF Core a `IEnumerable<Order>` mediante `.AsEnumerable()` o `.ToList()` **antes** de aplicar `.Where(...)`?

1.1 Bloques A–C: Historia de C# y .NET, CLI ('dotnet'), CIL, tipos valor vs referencia y nulabilidad

En la plataforma **.NET**, el compilador Roslyn (`csc`) traduce el código fuente C# a *Common Intermediate Language* (CIL, estándar ECMA-335) almacenado en ensamblados `.dll`. Durante la ejecución, el compilador JIT **RyuJIT** traduce los métodos CIL a código máquina nativo aplicando compilación por niveles (*Tiered Compilation*) y perfilado dinámico (Dynamic PGO), o bien compila directamente a binario nativo autónomo mediante **Native AOT**. Una decisión crucial de ingeniería en C# es la distinción entre **tipos por referencia** (`class`, alojados en el montículo gestionado con una cabecera de objeto de 16 bytes en 64 bits) y **tipos por valor** (`struct`, alojados en línea en el marco de pila o directamente dentro del array/objeto contenedor sin indirección de puntero). Además, a diferencia de la JVM, el CLR **reifica los tipos genéricos**: `List<int>` o `Dictionary<long, double>` generan código máquina especializado con arrays planos de valores primitivos sin *boxing*.

1.2 Bloque D: Propiedades, records, interfaces, genéricos reificados, delegados y LINQ

C# moderno incorpora tipos de referencia anulables verificados por flujo (`<Nullable>enable</Nullable>`), tipos de datos inmutables por valor o referencia (`readonly record struct` y `record class`) con

igualdad estructural sintetizada y **LINQ** (*Language Integrated Query*). En LINQ es indispensable distinguir entre `IEnumerable<T>`, cuyos operadores reciben delegados `Func<T, bool>` evaluados en memoria en el proceso actual, e `IQueryable<T>`, cuyos operadores reciben árboles de expresión (`Expression<Func<T, bool»`) que un proveedor como Entity Framework Core inspecciona para traducirlos a consultas SQL ejecutadas en el motor de base de datos:

```
public readonly record struct SensorSample(int SensorId, double Value, long TimestampMs);

public static double ComputeHighValueAverage(ReadOnlySpan<SensorSample> samples, double
    threshold)
{
    double sum = 0.0;
    int count = 0;
    foreach (ref readonly var s in samples)
    {
        if (s.Value >= threshold) { sum += s.Value; count++; }
    }
    return count > 0 ? sum / count : 0.0;
}
```

Observa cómo `foreach (ref readonly var s in samples)` sobre un `ReadOnlySpan<SensorSample>` recorre las estructuras contiguas por referencia constante sin realizar ni una sola copia ni alojar un iterador en el montículo.

Problema resuelto

Enunciado En una aplicación financiera en C# sobre .NET de 64 bits, se mantiene un búfer de $N = 100000$ cotizaciones donde cada cotización tiene $S_{\text{val}} = 16$ B de datos (`long Timestamp`, `double Price`). Si se modela como `readonly record struct`, el array `Quote[]` ocupa 100000×16 B. Si se modelara como `class`, cada elemento añadiría 8 B de puntero en el array más 16 B de cabecera de objeto en el montículo (24 B extra por instancia). Calcula cuántos kilobytes (dividiendo entre 1000) ahorra el diseño con `struct`.

Datos

- $N = 100000$ — Número de cotizaciones en el array
- $S_{extra} = 24\,\text{B}$ — Sobrecoste por instancia de class (8 B ref + 16 B cabecera)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria entre array de struct y array de class en el CLR (.NET)

$$\Delta M_{\text{kB}} = \frac{100000 \cdot 24}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 100000 instancias por 24 B y dividimos entre 1000 B/kB

$$\Delta M_{\text{kB}} = \frac{100000 \cdot 24}{1000}$$

$$\frac{(100000 \cdot 24)}{1000}$$

Resultado

$$\textit{Resultado} = 2400$$

Interpretación El diseño con readonly record struct ahorra 2400 kB (2.4 MB) y elimina 100 000 nodos del grafo del recolector de basura.

Comprobación de los pasos

- Paso 1: 2400 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) En .NET de 64 bits, la cabecera de todo objeto por referencia (`class`) consta de `ObjHeader` (8 B) y `MethodTable*` (8 B). ¿Cuántos bytes suma esta cabecera?
2 [PRB-0004] (Básico, Numérico) Un `readonly record struct Point3D(double X, double Y, double Z)` contiene tres `double` de 8 bytes. ¿Cuántos bytes ocupa cada instancia?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) Una consulta LINQ encadena `.Where(...)` y `.Take(25)` sobre una secuencia perezosa de 10 000 elementos donde se cumple el predicado en 1 de cada 4 elementos. ¿Cuántos elementos evalúa antes de detenerse (25×4)?
4 [PRB-0006] (Intermedio, Diseño) El recolector generacional del CLR divide el montículo de objetos pequeños en 3 generaciones (`Gen 0`, `Gen 1`, `Gen 2`). Si `Gen 0` tiene un presupuesto de 4 MiB y `Gen 1` de 2 MiB, ¿cuántos KiB suman las dos generaciones efímeras ($(4 + 2) \times 1024$)?

Capítulo 2

Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos

Este capítulo completa la monografía de C# cubriendo los niveles Avanzado y Experto (Bloques E y F), el Ecosistema empresarial (Bloque G) y los Proyectos (Bloque H). Explicamos cómo las primitivas `Span<T>` y `ValueTask<T>` han situado a ASP.NET Core (Kestrel) entre los servidores web más rápidos del mundo.

Consejo

En todos los métodos asíncronos de entrada/salida en C#, acepta siempre un `CancellationToken ct = default` como último parámetro y pásalo hacia abajo a todas las llamadas de red y base de datos para abortar el trabajo inmediatamente cuando el cliente HTTP cierra la conexión.

Definición

Estructura restringida a pila (`ref struct` en C#): tipo por valor cuya ubicación está garantizada estáticamente por el compilador y el verificador del CLR exclusivamente en la pila de ejecución, permitiendo contener punteros interiores gestionados (`ref T` en `Span<T>`) seguros frente a compactaciones del recolector de basura.

Autoficha

¿Por qué un objeto `Span<byte>` no puede ser un campo de una clase ni persistir a través de una expresión `await` en un método `async`?

2.1 Bloques E y F: Span<T>, ref struct, ArrayPool<T>, máquinas de estados async/await, ValueTask<T> y Channels

Para permitir el acceso uniforme y sin copias a memoria gestionada (`T[]`), memoria en pila (`stackalloc`) y memoria nativa no gestionada, C# 7.2+ introdujo `Span<T>` y `ReadOnlySpan<T>`. Un `Span<T>` se declara como `ref struct`, lo que significa que el compilador y el verificador del CLR prohíben estáticamente que escape al montículo (no puede ser un campo de una `class`, ni ser encajonado como `object`, ni capturarse en una lambda, ni cruzar un punto `await`). En programación asíncrona, cada método `async` es transformado por el compilador en una estructura que implementa `IAsyncStateMachine`. Si un método `async Task<T>` completa su trabajo de forma asíncrona, aloja un objeto `Task<T>` en el montículo; pero en rutas calientes (como lecturas desde caché o sockets con datos ya presentes en el búfer del kernel) donde el 90 % de las llamadas se resuelven síncronamente, devolver `ValueTask<T>` (un `readonly struct`) evita por completo cualquier asignación en el montículo:

```
using System.Buffers;
using System.Threading;
using System.Threading.Tasks;

public sealed class CachedMetricReader
{
    private double? _cached;
    public ValueTask<double> ReadMetricAsync(CancellationToken ct)
    {
        if (_cached is double ready)
            return new ValueTask<double>(ready); // 0 bytes alojados en el montículo
        return new ValueTask<double>(FetchSlowAsync(ct));
```

```
    }
    private async Task<double> FetchSlowAsync(CancellationToken ct)
    {
        await Task.Delay(10, ct).ConfigureAwait(false);
        _cached = 42.5;
        return 42.5;
    }
}
```

Recuerda la restricción contractual de `ValueTask<T>`: nunca debe ser esperado (`await`) más de una vez ni consumido concurrentemente antes de que complete.

2.2 Bloques G y H: Ecosistema (ASP.NET Core, DI, EF Core, xUnit, BenchmarkDotNet) y los 4 proyectos en C#

En el Bloque G (Ecosistema), las aplicaciones .NET modernas se construyen sobre el contenedor de **Inyección de Dependencias (DI)** integrado en `Microsoft.Extensions.DependencyInjection`, que distingue tres ciclos de vida: `Transient` (una instancia nueva en cada resolución), `Scoped` (una instancia única por ámbito o petición HTTP, obligatoria para `DbContext` de Entity Framework Core) y `Singleton` (una única instancia segura para hilos durante toda la vida del proceso). Un antipatrón crítico (detectado automáticamente por `ValidateScopes = true` en entorno de desarrollo) es la **dependencia cautiva** (*Captive Dependency*): inyectar un servicio `Scoped` dentro de un `Singleton`, lo que retiene el servicio `Scoped` indefinidamente y provoca accesos concurrentes ilegales a `DbContext`.

Problema resuelto

Enunciado Un servicio de alta frecuencia en ASP.NET Core procesa $R = 50000$ llamadas por segundo donde el 90 % (45000 llamadas/s) se resuelven síncronamente desde memoria local y solo el 10 % (5000 llamadas/s) requieren E/S asíncrona real. Si cada objeto `Task<Result>` alojado en el montículo mide $S_{\text{task}} = 72$ B, calcula cuántos kilobytes por segundo (dividiendo entre 1000) deja de alojar en `Gen 0` el servicio al cambiar el tipo de retorno de `Task<Result>` a `ValueTask<Result>`.

Datos

- $R_{sync} = 45000\,\text{llamadas}\,\text{s}^{-1}$ — Llamadas resueltas síncronamente por segundo
- $S_{task} = 72\,\text{B}$ — Tamaño del objeto Task<Result> evitado

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de asignaciones en Gen 0 con ValueTask<T> en rutas calientes

$$\Delta A_{\text{kB/s}} = \frac{45000 \cdot 72}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 45000 llamadas síncronas por 72 B y dividimos entre 1000 B/kB

$$\Delta A_{\text{kB/s}} = \frac{45000 \cdot 72}{1000}$$

$$\frac{(45000 \cdot 72)}{1000}$$

Resultado

$$\mathit{Resultado} = 3240$$

Interpretación Se evita alojar y recolectar 3240 kB/s (3.24 MB/s) de objetos efímeros en Gen 0.

Comprobación de los pasos

- Paso 1: 3240 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) En .NET de 64 bits, un `Span<byte>` contiene una referencia interior `ref byte` (8 B) y una longitud `int` alineada a 8 B. ¿Cuántos bytes ocupa en la pila?

2 [PRB-0010] (Básico, Numérico) Si se alquila un búfer con `ArrayPool<byte>.Shared.Rent(4000)`, el pool devuelve un array cuya longitud es la siguiente potencia de dos (4096). ¿Cuántos bytes adicionales respecto a los 4000 solicitados tiene el array devuelto?

3 [PRB-0011] (Intermedio, Desarrollo por pasos) Un canal acotado `Channel.CreateBounded<Job>(new BoundedChannelOptions(128))` contiene actualmente 95 trabajos. ¿Cuántos trabajos más pueden escribirse sin suspender al productor?

4 [PRB-0012] (Intermedio, Diseño) Un contenedor de inyección de dependencias resuelve por cada petición HTTP 1 servicio `Singleton` (ya creado), 2 servicios `Scoped` y 4 servicios `Transient` (invocados 2 veces cada uno en la petición). ¿Cuántas instancias nuevas se crean durante esa petición $(2 + 4 \times 2)$?

Capítulo 3

Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams

*Este capítulo inicia la monografía autónoma de **Java (Java 21 LTS)** (Bloques A, B, C y D). Java impulsa la infraestructura crítica financiera, bancaria y de comercio electrónico a escala planetaria, y con la llegada de Java 17 y Java 21 LTS ha incorporado tipos algebraicos de datos (`record` + `sealed interface`) e hilos virtuales que transforman por completo la escritura de código en la JVM.*

Consejo

Cuando operes con flujos numéricos en la API de Streams de Java, utiliza siempre las especializaciones primitivas (`IntStream`, `LongStream`, `DoubleStream` mediante `.mapToLong(...)` o `.mapToDouble(...)`) en lugar de `Stream<Integer>` o `Stream<Double>` para evitar el encajonamiento (*boxing*) de millones de objetos.

Definición

Principio PECS (Producer Extends, Consumer Super en Java): regla de varianza en el sitio de uso (*use-site variance*) según la cual una estructura parametrizada que actúa como productora de valores `T` debe tiparse de forma covariante (`? extends T`), y una que actúa como consumidora de valores `T` de forma contravariante (`? super T`).

Autoficha

¿Por qué en Java `Integer.valueOf(100) == Integer.valueOf(100)` evalúa a `true` (por la caché obligatoria de `-128` a `127` de la JLS §5.1.7), pero `Integer.valueOf(1000) == Integer.valueOf(1000)` evalúa a `false` y debe compararse siempre con `.equals()`?

3.1 Bloques A–C: Evolución hasta Java 21 LTS, arquitectura JVM, JIT C1/C2, G1/ZGC y sintaxis fundamental

Creado por James Gosling en Sun Microsystems en 1995, Java compila los ficheros `.java` mediante `javac` a código de bytes de pila (`.class`) independiente de la arquitectura física. En tiempo de ejecución, la máquina virtual **HotSpot** aplica compilación escalonada (*Tiered Compilation*): comienza interpretando el bytecode (Nivel 0), compila métodos frecuentes con el compilador rápido C1 recolectando perfiles de ejecución (Niveles $1-3$) y finalmente optimiza el código caliente con el compilador servidor **C2** (Nivel 4), capaz de desvirtualizar llamadas polimórficas monomórficas/bimórficas y aplicar análisis de escape (*Escape Analysis*) para alojar objetos efímeros en registros o escalarizarlos en la pila. Para la gestión del montículo, Java 21 ofrece **G1GC** (recolector regional por defecto) y **Generational ZGC** (`-XX:+UseZGC -XX:+ZGenerational`), un recolector concurrente con barreras de punteros coloreados (*colored pointers*) cuyas pausas *Stop-The-World* son inferiores a 1 ms incluso en montículos de varios terabytes.

3.2 Bloque D: Records, Sealed Classes, Pattern Matching, Genéricos (PECS), Colecciones y Streams API

Java 17 y Java 21 LTS introdujeron el paradigma de **Programación Orientada a Datos** (*Data-Oriented Programming*): la combinación de clases `record` (portadores transparentes e inmutables de

datos), interfaces selladas (`sealed interface ... permits ...`) y coincidencia de patrones exhaustiva en expresiones `switch` con desestructuración:

```
public sealed interface PaymentResult permits PaymentResult.Approved, PaymentResult.Declined {
    record Approved(String txId, long cents) implements PaymentResult {}
    record Declined(String reason, int retryAfterSec) implements PaymentResult {}

    static String describe(PaymentResult res) {
        return switch (res) {
            case Approved(var id, var cents) when cents >= 100_00 -> "High-value approved: " +
     id;
            case Approved(var id, var cents)                      -> "Approved: " + id + " ("
    + cents + ")";
            case Declined(var reason, var wait)                   -> "Declined (" + reason + "
    ), wait " + wait + "s";
        };
    }
}
```

En el sistema de tipos genéricos de Java (implementado con borrado de tipos a `Object` o a su cota superior para mantener compatibilidad binaria), la varianza en sitios de uso se rige por la regla **PECS** (*Producer Extends, Consumer Super*, formulada por Joshua Bloch): usa `List<? extends T>` cuando solo leas elementos `T` de la estructura productora, y `Comparator<? super T>` o `Collection<? super T>` cuando insertes o consumas elementos `T`.

Problema resuelto

Enunciado En la JVM HotSpot de 64 bits con punteros comprimidos activos (`-XX:+UseCompressedOops`), un objeto `Double` encajonado (*boxed*) ocupa 24 B (12 B de cabecera + 8 B del `double` + 4 B de relleno a múltiplo de 8), más 4 B de la referencia en un `ArrayList<Double>` (28 B por número), mientras que un array primitivo `double[]` o un `DoubleStream` ocupa únicamente 8 B por número. Calcula cuántos megabytes (dividiendo entre 10^6) se ahorran al almacenar $N = 2000000$ de valores en `double[]` frente a `ArrayList<Double>`.

Datos

- $N = 2000000$ — Dos millones de números en coma flotante
- $S_{boxed} = 28\,\text{B}$ — 24 B del objeto Double + 4 B de referencia comprimida
- $S_{prim} = 8\,\text{B}$ — 8 B por elemento en double[]

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Sobrecoste de memoria por boxing en la JVM frente a tipos primitivos

$$\Delta M_{\text{MB}} = \frac{2 \cdot 10^6 \cdot (28 - 8)}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 2 millones de elementos por los 20 B ahorrados y dividimos entre 10^6

$$\Delta M_{\text{MB}} = \frac{2 \cdot 10^6 \cdot (28 - 8)}{10^6}$$

$$\frac{(2000000 \cdot (28 - 8))}{1000000}$$

Resultado

$$\mathit{Resultado} = 40$$

Interpretación Usar arrays o flujos primitivos ahorra exactamente 40 MB de montículo y elimina 2 millones de objetos del recolector de basura.
Comprobación de los pasos

- Paso 1: 40 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En la JVM HotSpot con Compressed OOPs, la cabecera de un objeto ordinario mide 8 B (`Mark Word`) más 4 B (`Klass Pointer`). ¿Cuántos bytes mide la cabecera antes de alinear campos?
2 [PRB-0016] (Básico, Desarrollo por pasos) En `HashMap` de Java 8+, cuando un cubo (*bucket*) alcanza el umbral `TREEIFY_THRESHOLD = 8` colisiones (con capacidad de tabla ≥ 64), la lista enlazada se transforma en un árbol rojo-negro. Si hay 256 elementos colisionando en ese mismo cubo, ¿cuál es la altura máxima $2\log_2(256)$ a comparar frente a los 256 saltos lineales?
3 [PRB-0017] (Intermedio, Desarrollo por pasos) Un `ConcurrentHashMap` redimensiona su tabla de 1024 a 2048 cubos manteniendo el factor de carga en 0,75. ¿Cuántas entradas admite la nueva tabla antes del siguiente redimensionado ($2048 \times 3/4$)?
4 [PRB-0018] (Intermedio, Diseño) Si el tamaño de un montículo de la JVM supera los 32 GiB y desactiva Compressed OOPs, cada referencia pasa de 4 B a 8 B. En un grafo en memoria con 50 millones de referencias, ¿cuántos megabytes (10^6 B) adicionales consumen las referencias?

Capítulo 4

Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos

*Este capítulo completa la monografía de Java (Bloques E, F, G y H). Analizamos con rigor la innovación más importante de la JVM en la última década: los **Hilos Virtuales** (*Virtual Threads*, JEP 444 en Java 21 LTS), que reconcilian el modelo sencillo de un hilo por petición (*thread-per-request*) con la escalabilidad masiva de la E/S no bloqueante.*

Consejo

No reutilices en un pool (`ThreadPool`) los hilos virtuales: los hilos virtuales son baratos y efímeros; crea uno nuevo por cada tarea con `Executors.newVirtualThreadPerTaskExecutor()` y usa un `Semaphore` cuando necesites limitar el número de llamadas simultáneas a una base de datos externa.

Definición

Hilo Virtual en la JVM (Virtual Thread, JEP 444): instancia de `java.lang.Thread` planificada en espacio de usuario por la máquina virtual sobre un conjunto reducido de hilos portadores del sistema operativo (*carrier threads*), cuya pila de marcos crece y se encoge dinámicamente en el montículo durante bloqueos de E/S.

Autoficha

¿Qué es el *anclaje de hilo portador* (*carrier thread pinning*) cuando un hilo virtual se bloquea en E/S dentro de un bloque `synchronized` tradicional en Java 21 y por qué sustituirlo por `ReentrantLock` evita el anclaje?

4.1 Bloques E y F: Java Memory Model, Virtual Threads (Project Loom), StructuredTaskScope y API Panama (FFM)

El **Java Memory Model (JMM)** define formalmente cuándo las escrituras de un hilo son visibles para otro hilo a través de la relación transitiva *happens-before*: la liberación de un monitor (`synchronized` o `Lock.unlock()`) *happens-before* toda adquisición posterior del mismo monitor, y una escritura en un campo `volatile` *happens-before* toda lectura posterior de ese campo. En Java 21 LTS, **Project Loom** introdujo los **Virtual Threads** (`Thread.ofVirtual().start(...)` o `Executors.newVirtualThreadPerTaskExecutor()`): hilos gestionados por la JVM que se montan sobre un pequeño grupo de hilos del sistema operativo (*carrier threads* en un `ForkJoinPool`). Cuando el código ejecutado dentro de un hilo virtual realiza una operación bloqueante de red o base de datos (como `Socket.read()` o `Thread.sleep()`), el runtime de la JVM **desmonta** automáticamente la pila de continuación del hilo virtual hacia el montículo y deja libre al hilo portador del sistema operativo para ejecutar otro hilo virtual:

```
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public final class VirtualThreadDispatcher {
    public static long runConcurrentTasks(int taskCount) throws Exception {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
```

```
            var futures = IntStream.range(0, taskCount)
                .mapToObj(i -> executor.submit(() -> (long) i * 2L))
                .toList();
            long total = 0L;
            for (var f : futures) { total += f.get(); }
            return total;
        }
    }
}
```

Por la Ley de Little ($L = \lambda \cdot W$), un servidor ya no está limitado por el número de hilos de plataforma ($\approx$ 200–1000), sino que puede sostener cientos de miles de peticiones bloqueadas en E/S escribiendo código imperativo directo, fácil de depurar y con trazas de pila legibles.

4.2 Bloques G y H: Ecosistema (Maven, Gradle, JUnit 5, Spring Boot, GraalVM) y los 4 proyectos en Java

Asimismo, Java 21/22 reemplaza a JNI y `sun.misc.Unsafe` con la **Foreign Function & Memory API (Project Panama)**: `Arena.ofConfined()` y `MemorySegment` permiten gestionar memoria nativa fuera del montículo (*off-heap*) con límites espaciales y temporales comprobados determinísticamente. En el Bloque G (Ecosistema), estudiamos la gestión declarativa de proyectos con `Maven` (`pom.xml`) y `Gradle` (`build.gradle.kts`), pruebas unitarias y de integración con `JUnit 5` y `Testcontainers`, microservicios con `Spring Boot 3` (`spring.threads.virtual.enabled=true`) y compilación nativa con `GraalVM Native Image`.

Problema resuelto

Enunciado Según la Ley de Little ($\lambda_{\text{máx}} = L_{\text{máx}}/W$), un microservicio donde cada petición pasa $W = 0{,}25$ s esperando respuestas de APIs externas está limitado con hilos tradicionales de plataforma a $L_{\text{plat}} = 400$ hilos (caudal máximo $400/0{,}25 = 1600$ req/s). Al migrar a **Virtual Threads** en Java 21 soportando $L_{\text{virt}} = 20000$ hilos virtuales simultáneos con la misma latencia $W = 0{,}25$ s, calcula cuántas peticiones por segundo adicionales puede sostener el servidor ($20000 / 0.25 - 400 / 0.25$).
Datos

- $L_{plat} = 400\,\text{hilos}$ — Pool clásico de hilos del sistema operativo
- $L_{virt} = 20000\,\text{hilos}$ — Hilos virtuales simultáneos en Java 21
- $W = 0{,}25\,\text{s}$ — Tiempo medio de espera de E/S por petición

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Incremento de caudal máximo por Ley de Little con Virtual Threads (JEP 444)

$$\Delta\lambda = \frac{20000}{0{,}25} - \frac{400}{0{,}25}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el caudal antiguo (1600 req/s) al nuevo caudal (80000 req/s)

$$\Delta\lambda = \frac{20000}{0{,}25} - \frac{400}{0{,}25}$$

$$\left(\frac{20000}{0{,}25} - \frac{400}{0{,}25}\right)$$

Resultado

$$Resultado = 78400$$

Interpretación El servidor incrementa su capacidad en 78 400 peticiones/s (pasando de 1600 req/s a 80 000 req/s).

Comprobación de los pasos

- Paso 1: 78400 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) Por la Ley de Little ($L = \lambda \cdot W$), si un sistema recibe $\lambda = 2500$ peticiones/s y cada una tarda $W = 0{,}2$ segundos, ¿cuántas peticiones hay en vuelo simultáneamente en promedio?

2 [PRB-0022] (Básico, Numérico) En una CPU de 16 núcleos lógicos, el `ForkJoinPool` portador de hilos virtuales crea por defecto un hilo portador (*carrier thread*) por núcleo. ¿Cuántos hilos del sistema operativo mantienen activos los hilos virtuales?

3 [PRB-0023] (Intermedio, Desarrollo por pasos) En el código `VirtualThreadDispatcher.runConcurrentTasks(100)`, se suma `i * 2` para $i = 0, 1, \ldots, 99$. ¿Cuál es el valor exacto de la suma (99×100)?

4 [PRB-0024] (Intermedio, Diseño) Una `Arena.ofConfined()` de Project Panama reserva 64 segmentos nativos de 4096 bytes cada uno. ¿Cuántos bytes de memoria *off-heap* se liberan determinísticamente al cerrarse el bloque `try-with-resources`?

Parte II

Monografías de Sistemas Cloud y Móviles: Go, Kotlin y Swift

Capítulo 5

Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores

*Este capítulo inicia la monografía autónoma de **Go (Golang 1.22+)** (Bloques A, B, C y D). Go es el lenguaje fundacional de la infraestructura nativa en la nube (Docker, Kubernetes, etcd, Prometheus, Terraform, Caddy). Estudiaremos por qué su deliberada economía de conceptos produce sistemas distribuidos mantenibles por grandes equipos.*

Consejo

Sigue el axioma idiomático de Go: *«Acepta interfaces (pequeñas, como `io.Reader`), devuelve estructuras concretas»*, y evalúa siempre los argumentos de `defer` teniendo en cuenta que se capturan en el momento en que se ejecuta la sentencia `defer`, no cuando retorna la función.

Definición

Expresión de rebanada completa (`s[i:j:k]` en Go): operación de sub-slicing que produce una nueva rebanada con longitud `j - i` y capacidad restringida a `k - i`, forzando que cualquier `append` posterior sobre la sub-rebanada aloje un nuevo array en lugar de sobrescribir los elementos a partir del índice `k`.

Autoficha

¿Por qué una función en Go que declara `var p *MyCustomError = nil; return p` como valor de retorno de tipo `error` hace que el llamador observe `err != nil`?

5.1 Bloques A–C: Filosofía de Go, toolchain integrada (‘go‘), tipos, valores cero, arrays vs slices y maps

Diseñado en Google en 2007 por Robert Griesemer, Rob Pike y Ken Thompson, Go nació como respuesta a los tiempos de compilación masivos y la complejidad accidental de C++ y Java en servidores multinúcleo. La herramienta unificada `go` integra compilación, gestión de módulos (`go.mod` / `go.sum` con selección de versión mínima, *Minimal Version Selection*), formateo canónico (`gofmt`), análisis estático (`go vet`), pruebas y *fuzzing* nativo (`go test -fuzz`). En Go, toda variable no inicializada explícitamente recibe su **valor cero** determinista (`0`, `false`, `""`, `nil`), y los principios de diseño recomiendan que el valor cero de una estructura sea inmediatamente utilizable sin constructor (como `var mu sync.Mutex` o `var buf bytes.Buffer`). Un `slice []T` es un descriptor de 24 bytes (en 64 bits) formado por un puntero al array subyacente, la longitud `len` y la capacidad `cap`. Si dos rebanadas comparten el mismo array subyacente y una ejecuta `append` sin superar `cap`, modifica el array compartido; para independizar la capacidad al hacer sub-slicing se utiliza la expresión de tres índices `s[low:high:max]`.

5.2 Bloque D: Métodos, composición de structs, interfaces implícitas, errores como valores y genéricos

Go prescinde por completo de la herencia de clases y la sustituye por **composición de estructuras** y **interfaces satisfechas implícitamente**: un tipo satisface una interfaz como `io.Reader` (`Read(p []byte) (n int, err error)`) simplemente implementando sus métodos, sin escribir nunca

`implements`. Esto permite definir interfaces pequeñas de uno o dos métodos en el paquete consumidor en lugar de acoplarse al paquete proveedor. Asimismo, Go trata los errores como valores ordinarios devueltos en la última posición de retorno `(val, error)`, encadenados con `fmt.Errorf("contexto: %w", err)` e inspeccionados mediante `errors.Is` y `errors.As`:

```
package storage

import (
    "errors"
    "fmt"
    "io"
)

var ErrEmptyPayload = errors.New("empty payload")

func ReadExactHeader(r io.Reader, size int) ([]byte, error) {
    if size <= 0 {
        return nil, ErrEmptyPayload
    }
    buf := make([]byte, size)
    if _, err := io.ReadFull(r, buf); err != nil {
        return nil, fmt.Errorf("read exact header (%d bytes): %w", size, err)
    }
    return buf, nil
}
```

Un matiz técnico esencial de las interfaces en Go es su representación interna de dos palabras (`tab` / tipo dinámico y `data` / valor dinámico): una interfaz solo es `== nil` si **ambos** punteros internos son nulos; asignar un puntero tipado `(*MyError)(nil)` a una variable `var err error` produce una interfaz **no nula** (`err != nil`), por lo que las funciones deben devolver siempre el literal `nil` directamente cuando no hay error.

Problema resuelto

Enunciado En un programa en Go, se crea un slice `base := make([]int64, 100, 200)` (cada `int64` ocupa 8 B) y se obtiene la sub-rebanada de tres índices `sub := base[20:60:80]`. Calcula la suma de la longitud `len(sub)` ($60 - 20$) más la capacidad `cap(sub)` ($80 - 20$) multiplicada por los 8 B de cada elemento.

Datos

- $i = 20$ — Índice inicial
- $j = 60$ — Índice final de longitud
- $k = 80$ — Índice límite de capacidad

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Cálculo de len y cap en expresiones de rebanada de tres índices s[i:j:k] en Go

$$R = ((60 - 20) + (80 - 20)) \cdot 8$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos len (40) + cap (60) = 100 y multiplicamos por 8 bytes

$$R = ((60 - 20) + (80 - 20)) \cdot 8$$

$$(((60 - 20) + (80 - 20)) \cdot 8)$$

Resultado

$$Resultado = 800$$

Interpretación La longitud de sub es 40, su capacidad es 60 y la métrica ponderada por 8 bytes es 800.

Comprobación de los pasos

- Paso 1: 800 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) En Go de 64 bits, el encabezado de un slice (`SliceHeader`) consta de `Data` (8 B), `Len` (8 B) y `Cap` (8 B). ¿Cuántos bytes ocupa en la pila?

2 [PRB-0028] (Básico, Numérico) En Go, cada cubo (*bucket* `bmap`) de un `map[K]V` almacena exactamente 8 pares clave-valor. Si un mapa tiene $2^6 = 64$ cubos, ¿cuántos pares clave-valor caben en esos 64 cubos sin cubos de desbordamiento (64×8)?

3 [PRB-0029] (Intermedio, Desarrollo por pasos) Para la rebanada `s[15:45:50]`, ¿cuántos elementos pueden añadirse con `append` antes de superar la capacidad y forzar la reserva de un nuevo array ($(50 - 15) - (45 - 15)$)?

4 [PRB-0030] (Intermedio, Diseño) Una interfaz `io.Reader` en Go se representa como un par de palabras `(itab*, data*)` de 8 bytes cada una. ¿Cuántos bytes ocupan 500 descriptores de interfaz en un slice `[]io.Reader`?

Capítulo 6

Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos

*Este capítulo completa la monografía de Go (Bloques E, F, G y H). Analizamos el motor de ejecución de Go: su planificador de **goroutines G-M-P**, los patrones de canales y `context.Context` para evitar fugas de goroutines, el recolector de basura tricolor concurrente de baja latencia y las herramientas integradas de diagnóstico de producción (`pprof`, `trace`, `-race`).*

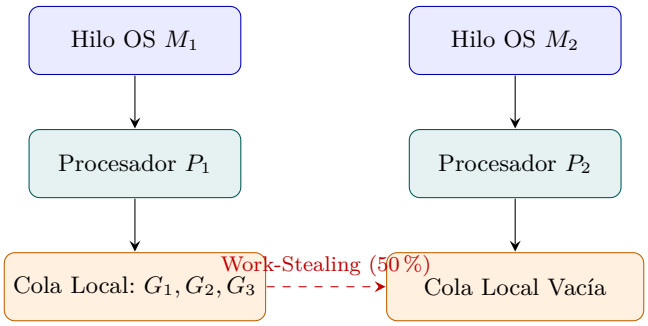


Figura 1. Arquitectura del planificador M:N de Go: cada contexto lógico P (GOMAXPROCS) posee una cola local sin cerrojos de Goroutines (G) y se vincula a un hilo del sistema operativo (M) para ejecutar código Go o robar la mitad de la cola de otro P.

Consejo

Antes de lanzar cualquier `go func() { ... }()`, responde siempre a dos preguntas: (1) ¿bajo qué condición exacta terminará esta goroutine (por ejemplo, cancelación de `ctx.Done()` o cierre del canal de entrada)? y (2) ¿quién esperará a que termine (`sync.WaitGroup` o `errgroup.Group`) antes de cerrar el proceso?

Definición

Fuga de Goroutines (Goroutine Leak): defecto de recursos en Go en el que una goroutine queda suspendida permanentemente en una operación de envío o recepción de canal que ningún otro participante completará jamás, impidiendo que el recolector de basura libere su pila y las variables referenciadas por ella.

Autoficha

¿Qué ocurre exactamente en tiempo de ejecución en Go cuando una goroutine intenta enviar un valor a un canal cerrado (`close(ch)`) frente a cuando intenta recibir de un canal cerrado?

6.1 Bloques E y F: Planificador G-M-P, goroutines, canales, select, context.Context, sync/errgroup y análisis de escape

Una **goroutine** (`go fn()`) es un hilo de ejecución ligero gestionado por el runtime de Go cuya pila inicial mide tan solo **2 KiB** (2048 B) y crece o decrece dinámicamente según la profundidad de llamadas. Como muestra la Figura \ref{fig:fig-go-gmp-scheduler}, el planificador M:N multiplexa N goroutines (`G`) sobre M hilos del sistema operativo (`M`) utilizando hasta `GOMAXPROCS` procesadores lógicos (`P`). Cada `P` mantiene una cola local de hasta 256 goroutines ejecutables; cuando la cola de un `P` se vacía, intenta

robar la mitad de las goroutines de la cola de otro `P` elegido al azar (*work-stealing*) o toma trabajo de la cola global. La comunicación entre goroutines sigue el cálculo **CSP** (*Communicating Sequential Processes*): *«No compartas memoria para comunicarte; comunícate para compartir memoria»*. Para evitar que una goroutine quede bloqueada eternamente intentando enviar en un canal cuyo receptor ya ha abandonado la operación (*goroutine leak*), todo ciclo concurrente debe seleccionar también sobre `<-ctx.Done()` de un `context.Context`:

```
package worker

import "context"

func FanOutGenerator(ctx context.Context, items []int, out chan<- int) error {
    for _, v := range items {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case out <- v * 2:
        }
    }
    return nil
}
```

A partir de Go 1.22, las variables de bucle `for i, v := range items` tienen ámbito por iteración en lugar de por bucle completo, eliminando el error histórico de capturar la misma variable `v` en múltiples clausuras concurrentes.

6.2 Bloques G y H: Ecosistema (net/http, grpc-go, sqlc, pprof, -race) y los 4 proyectos en Go

En el Nivel Experto (Bloque F), el compilador de Go decide automáticamente si una variable local se aloja en la pila de 2 KiB de la goroutine o en el montículo gestionado mediante **análisis de escape** (*escape analysis*), inspeccionable con `go build -gcflags="-m"`. En el Bloque G y Bloque H, la biblioteca estándar de Go es tan completa que permite construir servidores HTTP/2 de producción con enrutamiento por patrones (`net/http` en Go 1.22+), observabilidad (`log/slog`, `runtime/pprof`), clientes SQL con pools de conexiones (`database/sql`) y pruebas de concurrencia instrumentadas con ThreadSanitizer (`go test -race`). Los cuatro proyectos del bloque incluyen un proxy inverso con balanceo *round-robin* y *health checks*, un grupo de trabajadores (*worker pool*) con apagado ordenado (`os/signal` + `context.WithTimeout`), un limitador de tasa distribuido y un servidor gRPC.

Problema resuelto

Enunciado Cada goroutine recién creada en Go 1.22 reserva una pila inicial de $S_G = 2$ KiB (2048 B), mientras que un hilo nativo de sistema operativo reserva por defecto una pila de $S_M = 2048$ KiB (2 MiB). Si un servidor de telemetría en Go mantiene $N = 100000$ goroutines activas sobre $M = 16$ hilos del sistema operativo, calcula cuántos mebibytes totales (dividiendo los KiB entre 1024) ocupan las pilas de las 100 000 goroutines más los 16 hilos OS.

Datos

- $N = 100000$ — Número de goroutines concurrentes
- $S_G = 2\,\text{KiB}$ — Pila inicial por goroutine
- $M = 16$ — Hilos del sistema operativo de 2048 KiB

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Consumo de memoria de pilas en el modelo M:N de Go

$$M_{\text{MiB}} = \frac{100000 \cdot 2 + 16 \cdot 2048}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los KiB de las goroutines (200000 KiB) y de los 16 hilos OS (32768 KiB) y dividimos entre 1024

$$M_{\text{MiB}} = \frac{100000 \cdot 2 + 16 \cdot 2048}{1024}$$

$$\frac{((100000 \cdot 2) + (16 \cdot 2048))}{1024}$$

Resultado

$$Resultado = 227{,}3125$$

Interpretación Las 100 000 goroutines más los 16 hilos del sistema operativo consumen tan solo 227.3125 MiB de memoria de pila.
Comprobación de los pasos

- Paso 1: 227,312 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) ¿Cuántos bytes mide la pila inicial de una goroutine en Go (2×1024)?
2 [PRB-0034] (Básico, Numérico) En el planificador de Go, un procesador P_1 tiene 180 goroutines en su cola local y un procesador inactivo P_2 le roba la mitad. ¿Cuántas goroutines roba P_2?
3 [PRB-0035] (Intermedio, Desarrollo por pasos) Un canal con búfer `ch := make(chan uint64, 512)` almacena elementos de 8 bytes. Si hay 300 elementos encolados en el búfer, ¿cuántos elementos adicionales caben antes de bloquear al emisor?
4 [PRB-0036] (Intermedio, Diseño) En el problema anterior, ¿cuántos bytes ocupa el búfer circular completo de 512 elementos `uint64` (512×8)?

Capítulo 7

Monografía de Kotlin (Bloques A–H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform

*Este capítulo ofrece la monografía completa (Bloques A – H) de **Kotlin (2.0+)**. Desarrollado por JetBrains, Kotlin es el lenguaje oficial prioritario para el desarrollo en Android, una potencia en microservicios de servidor sobre la JVM (Spring Boot, Ktor) y un lenguaje multiplataforma nativo (**Kotlin Multiplatform**, KMP).*

Consejo

Cuando captures excepciones genéricas (`catch (e: Throwable)` o `catch (e: Exception)`) dentro de una corrutina de Kotlin, **relanza siempre `CancellationException`** (`if (e is CancellationException) throw e`): tragarse `CancellationException` rompe el mecanismo de cancelación cooperativa de la concurrencia estructurada.

Definición

Transformación CPS de funciones `suspend` en Kotlin: reescritura realizada por el compilador en la que toda función `suspend fun f(a: A): R` se compila en bytecode a `Object f(A a, Continuation<? super R> completion)`, donde `completion` almacena el índice `label` del estado actual y las variables locales vivas entre puntos de suspensión.

Autoficha

¿Qué diferencia de comportamiento ante el fallo de una corrutina hija existe entre `coroutineScope { ... }` y `supervisorScope { ... }` en Kotlin Coroutines?

7.1 Bloques A–D: Filosofía de Kotlin, interoperabilidad JVM/KMP, null-safety, data/sealed classes y extensiones

Kotlin resuelve las limitaciones históricas de Java manteniendo una interoperabilidad del 100 % a nivel de bytecode. Su sistema de tipos incorpora nulabilidad en la raíz de la jerarquía: `String` no puede almacenar `null`, mientras que `String?` exige comprobación explícita mediante el operador de llamada segura `?.`, el operador Elvis `?:` o *smart casts* tras verificar `if (x != null)`. Para modelar dominios sin sobrecoste de ejecución, `@JvmInline value class UserId(val raw: Long)` envuelve un tipo primitivo en compilación pero se emite como un `long` desnudo en el bytecode de la JVM. Asimismo, dado que en Kotlin las funciones de orden superior se usan intensivamente, la palabra clave `inline` inserta el cuerpo de la función y el de las lambdas recibidas directamente en el sitio de llamada, eliminando la creación de objetos clausura y permitiendo preservar el tipo genérico en ejecución con `reified T`.

7.2 Bloques E–H: Funciones inline/reified, DSLs, Coroutines (suspend, CPS, Flow), ecosistema (Ktor, Compose) y proyectos

La concurrencia en Kotlin se fundamenta en **Coroutines** estructuradas a partir de una única palabra clave en el lenguaje: `suspend`. El compilador K2 transforma cada `suspend fun fetch(id: Long):`

`Profile` mediante **Continuation-Passing Style (CPS)** en una función que recibe un parámetro adicional oculto `Continuation<Profile>` implementado como una máquina de estados sin alojar un hilo por espera:

```
import kotlinx.coroutines.*

data class NodeHealth(val nodeId: Int, val latencyMs: Long)

suspend fun aggregateClusterLatency(nodeIds: List<Int>): Long = coroutineScope {
    val deferreds: List<Deferred<Long>> = nodeIds.map { id ->
        async(Dispatchers.Default) {
            // Simulación de consulta paralela por nodo:
            id.toLong() * 5L
        }
    }
    deferreds.awaitAll().sum()
}
```

Dentro de `coroutineScope { ... }`, la concurrencia es estrictamente jerárquica: la función no retorna hasta que todas las corrutinas hijas lanzadas con `launch` o `async` han terminado, y si una sola falla con una excepción, todas las demás hermanas reciben cancelación cooperativa inmediata. Para secuencias asíncronas de múltiples valores, Kotlin distingue entre `Flow<T>` (flujo frío perezoso con contrapresión natural por suspensión) y `StateFlow<T>` / `SharedFlow<T>` (flujos calientes observables en arquitecturas reactivas como Jetpack Compose).

Problema resuelto

Enunciado En una aplicación en Kotlin sobre la JVM, una función de orden superior que captura una variable local del entorno se invoca en un bucle caliente $N = 500000$ veces por segundo. Si la función NO está marcada como `inline`, cada invocación aloja una instancia de clase anónima de $S_{\text{lambda}} = 24$ B. Al marcar la función de orden superior con `inline`, el alojamiento cae a 0 B. Calcula cuántos megabytes por segundo (dividiendo entre 10^6) deja de alojar el bucle gracias a `inline`.
Datos

- $N = 500000$ — Invocaciones por segundo
- $S_\lambda = 24\,\text{B}$ — Tamaño del objeto clausura capturado

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de montículo mediante expansión inline de lambdas con captura en Kotlin

$$\Delta M_{\text{MB/s}} = \frac{500000 \cdot 24}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 500000 invocaciones por 24 B y dividimos entre 10^6

$$\Delta M_{\text{MB/s}} = \frac{500000 \cdot 24}{10^6}$$

$$\frac{(500000 \cdot 24)}{1000000}$$

Resultado

$$\textit{Resultado} = 12$$

Interpretación La palabra clave inline ahorra exactamente 12 MB/s de asignaciones efímeras en el montículo.
Comprobación de los pasos

- Paso 1: 12 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0039**] **(Básico, Numérico)** En la función `aggregateClusterLatency` con `nodeIds = [1, 2, 3, 4, 5]` donde cada nodo devuelve `id * 5`, ¿cuál es la suma total devuelta $((1+2+3+4+5)\times 5)$?
2 [**PRB-0040**] **(Básico, Numérico)** Un `@JvmInline value class Meter(val v: Double)` se almacena en un array primitivo de 1000 elementos. ¿Cuántos bytes ocupan los 1000 elementos `double` (1000×8)?
3 [**PRB-0041**] **(Intermedio, Desarrollo por pasos)** Un `Flow<Int>` emite 60 elementos a razón de 1 elemento cada 10 ms y el colector aplica `.buffer(10)` tardando 25 ms por elemento. ¿Cuántos milisegundos tarda el colector en procesar los 60 elementos $(10 + 60 \times 25)$?
4 [**PRB-0042**] **(Intermedio, Diseño)** Un proyecto Kotlin Multiplatform comparte 14 000 líneas de lógica de dominio en `commonMain` entre Android e iOS, requiriendo solo 1500 líneas específicas en `androidMain` y 1500 en `iosMain`. ¿Cuántas líneas de código duplicado se ahorran respecto a escribir los 14 000 líneas dos veces?

Capítulo 8

Monografía de Swift 6 (Bloques A–H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores

*Este capítulo cierra el Volumen VI con la monografía completa (Bloques A – H) de **Swift (6.0+)**. Creado por Chris Lattner y el equipo de Apple sobre la infraestructura LLVM, Swift es un lenguaje compilado nativamente sin recolector de basura de trazado que combina semántica de valor profunda, conteo automático de referencias (**ARC**) y verificación estática de aislamiento de actores en tiempo de compilación.*

Consejo

En firmas de funciones en Swift, prefiere siempre `func render(item: some Drawable)` sobre `func render(item: any Drawable)` salvo que necesites almacenar elementos heterogéneos en un mismo array `[any Drawable]`: `some` permite al compilador especializar la función y evitar el contenedor existencial.

Definición

Aislamiento de Actor (`actor` en Swift): tipo por referencia que protege su estado mutable encapsulado mediante un ejecutor serie exclusivo, exigiendo estáticamente en tiempo de compilación que cualquier acceso desde fuera del actor sea asíncrono (`await`) y que los valores transferidos a través de su frontera cumplan `Sendable`.

Autoficha

¿Cómo ocurre la *reentrancia de actores* (*actor reentrancy*) en Swift cuando un método de un `actor` ejecuta un punto de suspensión `await` intermedio, y por qué deben revalidarse las invariantes del actor después del `await`?

8.1 Bloques A–D: Filosofía de Swift, SIL/LLVM, SwiftPM, semántica de valor (struct/enum), Copy-on-Write y protocolos

A diferencia de Java o Objective-C, en Swift casi todos los tipos fundamentales de la biblioteca estándar –incluyendo enteros, flotantes, `String`, `Array<T>`, `Dictionary<K, V>` y `Set<T>`– son **tipos por valor** (`struct`), no clases (`class`). Para que pasar un `Array<Double>` de un millón de elementos por valor a una función no implique copiar el búfer inmediatamente, las colecciones de Swift implementan **Copy-on-Write (CoW)**: al asignar `var b = a`, ambas estructuras comparten el mismo búfer interno en el montículo; solo cuando se ejecuta una mutación como `b.append(1.0)`, `Array` comprueba en $O(1)$ mediante `isKnownUniquelyReferenced` si el contador de referencias del búfer es mayor que 1 y, únicamente en ese caso, duplica el búfer antes de modificarlo. En el diseño con protocolos (`protocol`), Swift 5.7+/6.0 distingue explícitamente entre `some Protocol` (tipo opaco conocido estáticamente en compilación, despachado sin coste) y `any Protocol` (contenedor existencial dinámico de 5 palabras máquina: 3 palabras de búfer de valor en línea más metadatos de tipo y *witness table*).

8.2 Bloques E–H: Gestión de memoria ARC (weak/unowned), concurrencia estricta en Swift 6 (actor, Sendable) y proyectos

Para las clases (`class`) y clausuras, Swift gestiona la memoria de forma determinista en tiempo de compilación mediante **ARC** (*Automatic Reference Counting*): el compilador inserta llamadas `swift_retain` y `swift_release` en la representación intermedia SIL. Como el conteo de referencias no recolecta ciclos aislados, todo ciclo entre instancias de clases o clausuras que capturan `self` debe romperse declarando una de las aristas como `weak` (opcional `T?` que se pone automáticamente a `nil` al destruirse el objeto) o `unowned` (no opcional, cuando ambas vidas útiles están ligadas). Finalmente, **Swift 6** garantiza en tiempo de compilación la ausencia de carreras de datos mediante el protocolo marcador `Sendable` y el tipo de referencia **`actor`**, que serializa el acceso a su estado mutable interno exigiendo `await` desde cualquier contexto externo:

```
public actor TelemetryVault {
    private var readings: [String: Double] = [:]
    public init() {}
    public func record(sensor: String, value: Double) {
        readings[sensor] = value
    }
    public func current(sensor: String) -> Double? {
        return readings[sensor]
    }
}
```

El capítulo concluye con el ecosistema Swift Package Manager (`Package.swift`), `SwiftTesting` (`@Test`, `#expect`), `SwiftUI` / `Vapor` en servidor Linux y cuatro proyectos completos en Swift 6.

Problema resuelto

Enunciado En Swift sobre una arquitectura de 64 bits ($W = 8$ B), un contenedor existencial `any P` para un protocolo ocupa $5 \times 8 = 40$ B (24 B de búfer de valor en línea, 8 B de *Value Witness Table* y 8 B de *Protocol Witness Table*), mientras que un valor concreto `Int64` pasado mediante un genérico especializado `some P` ocupa únicamente 8 B. Calcula cuántos bytes ahorra un array de $N = 10000$ elementos al usar el tipo especializado `[Int64]` frente a un array existencial `[any P]`.

Datos

- $N = 10000$ — Número de elementos en la colección
- $S_{any} = 40\,\text{B}$ — Tamaño del contenedor existencial de 5 palabras
- $S_{some} = 8\,\text{B}$ — Tamaño del tipo concreto especializado

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Sobrecoste espacial de contenedores existenciales (any P) frente a tipos concretos (some P) en Swift

$$\Delta M = 10000 \cdot (40 - 8)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 10000 elementos por los 32 B ahorrados por elemento

$$\Delta M = 10000 \cdot (40 - 8)$$

$$(10000 \cdot (40 - 8))$$

Resultado

$$Resultado = 320000$$

Interpretación La especialización estática ahorra 320 000 bytes (un 80 % de reducción) y elimina el despacho indirecto por tabla de testigos.
Comprobación de los pasos

- Paso 1: 320000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0045] (Básico, Numérico) En Swift de 64 bits ($W = 8$ B), el `Value Buffer` interno de un contenedor existencial tiene 3 palabras. ¿Hasta cuántos bytes puede medir un `struct` para caber en línea dentro del contenedor existencial sin alojar en el montículo (3×8)?
2 [PRB-0046] (Básico, Numérico) Si se copia un `Array<Int>` de 50 000 enteros (`var b = a`) y solo se realizan lecturas sobre `a` y `b` sin mutar ninguno de los dos, ¿cuántos elementos se duplican en el montículo gracias a Copy-on-Write?
3 [PRB-0047] (Intermedio, Desarrollo por pasos) Un contenedor existencial para una composición de $K = 3$ protocolos (`any P1 & P2 & P3`) ocupa $(3 + 1 + 3) \times 8$ bytes en 64 bits. ¿Cuántos bytes mide en total?
4 [PRB-0048] (Intermedio, Diseño) En la cabecera de un objeto `class` en Swift de 64 bits, se almacenan el puntero `isa`/metadatos (8 B) y la palabra de contadores ARC (8 B). ¿Cuántos bytes suman las cabeceras de 2000 instancias?

Apéndice A

Notación

Símbolo	Significado	Unidades
$(G,\ M,\ P)$	Goroutine, Machine (hilo OS) y Processor (contexto lógico) en el runtime de Go	entidades
$\mathrm{ARC}(o)$	Contador fuerte/débil/unowned gestionado en compilación en Swift	referencias
$\mathrm{Span}\langle T\rangle$	Vista contigua de memoria alojada en pila (ref struct) sin asignación en el montículo en .NET	vista

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ

Problema resuelto [PRB-0001]
En una aplicación financiera en C# sobre .NET de 64 bits, se mantiene un búfer de $N = 100000$ cotizaciones donde cada cotización tiene $S_{\text{val}} = 16$ B de datos (`long Timestamp`, `double Price`). Si se modela como `readonly record struct`, el array `Quote[]` ocupa 100000×16 B. Si se modelara como `class`, cada elemento añadiría 8 B de puntero en el array más 16 B de cabecera de objeto en el montículo (24 B extra por instancia). Calcula cuántos kilobytes (dividiendo entre 1000) ahorra el diseño con `struct`.
Respuesta

$$Resultado = 2400$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
En .NET de 64 bits, la cabecera de todo objeto por referencia (`class`) consta de `ObjHeader` (8 B) y `MethodTable*` (8 B). ¿Cuántos bytes suma esta cabecera?
Respuesta 16
Ejercicio 2 [PRB-0004]
Un `readonly record struct Point3D(double X, double Y, double Z)` contiene tres `double` de 8 bytes. ¿Cuántos bytes ocupa cada instancia?
Respuesta 24
Ejercicio 3 [PRB-0005]
Una consulta LINQ encadena `.Where(...)` y `.Take(25)` sobre una secuencia perezosa de 10 000 elementos donde se cumple el predicado en 1 de cada 4 elementos. ¿Cuántos elementos evalúa antes de detenerse (25×4)?
Respuesta 100
Ejercicio 4 [PRB-0006]
El recolector generacional del CLR divide el montículo de objetos pequeños en 3 generaciones (`Gen 0`, `Gen 1`, `Gen 2`). Si `Gen 0` tiene un presupuesto de 4 MiB y `Gen 1` de 2 MiB, ¿cuántos KiB suman las dos generaciones efímeras ($(4 + 2) \times 1024$)?
Respuesta 6144

Cálculos reproducidos

$\frac{(100000 \cdot 24)}{1000} = 2400$

C.2 Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos

Problema resuelto [PRB-0007]
Un servicio de alta frecuencia en ASP.NET Core procesa $R = 50000$ llamadas por segundo donde el 90 % (45000 llamadas/s) se resuelven síncronamente desde memoria local y solo el 10 % (5000 llamadas/s) requieren E/S asíncrona real. Si cada objeto `Task<Result>` alojado en el montículo mide $S_{\text{task}} = 72$ B, calcula cuántos kilobytes por segundo (dividiendo entre 1000) deja de alojar en `Gen 0` el servicio al cambiar el tipo de retorno de `Task<Result>` a `ValueTask<Result>`.

Respuesta

$$Resultado = 3240$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0009]
En .NET de 64 bits, un `Span<byte>` contiene una referencia interior `ref byte` (8 B) y una longitud `int` alineada a 8 B. ¿Cuántos bytes ocupa en la pila?
Respuesta 16

Ejercicio 2 [PRB-0010]
Si se alquila un búfer con `ArrayPool<byte>.Shared.Rent(4000)`, el pool devuelve un array cuya longitud es la siguiente potencia de dos (4096). ¿Cuántos bytes adicionales respecto a los 4000 solicitados tiene el array devuelto?
Respuesta 96

Ejercicio 3 [PRB-0011]
Un canal acotado `Channel.CreateBounded<Job>(new BoundedChannelOptions(128))` contiene actualmente 95 trabajos. ¿Cuántos trabajos más pueden escribirse sin suspender al productor?
Respuesta 33

Ejercicio 4 [PRB-0012]
Un contenedor de inyección de dependencias resuelve por cada petición HTTP 1 servicio `Singleton` (ya creado), 2 servicios `Scoped` y 4 servicios `Transient` (invocados 2 veces cada uno en la petición). ¿Cuántas instancias nuevas se crean durante esa petición $(2 + 4 \times 2)$?
Respuesta 10

Cálculos reproducidos

$\frac{(45000 \cdot 72)}{1000} = 3240$

C.3 Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams

Problema resuelto [PRB-0013]
En la JVM HotSpot de 64 bits con punteros comprimidos activos (`-XX:+UseCompressedOops`), un objeto `Double` encajonado (*boxed*) ocupa 24 B (12 B de cabecera + 8 B del `double` + 4 B de relleno a múltiplo de 8), más 4 B de la referencia en un `ArrayList<Double>` (28 B por número), mientras que un array primitivo `double[]` o un `DoubleStream` ocupa únicamente 8 B por número. Calcula cuántos megabytes (dividiendo entre 10^6) se ahorran al almacenar $N = 2000000$ de valores en `double[]` frente a `ArrayList<Double>`.
Respuesta

$$Resultado = 40$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]
En la JVM HotSpot con Compressed OOPs, la cabecera de un objeto ordinario mide 8 B (`Mark Word`) más 4 B (`Klass Pointer`). ¿Cuántos bytes mide la cabecera antes de alinear campos?
Respuesta 12

Ejercicio 2 [PRB-0016]
En `HashMap` de Java 8+, cuando un cubo (*bucket*) alcanza el umbral `TREEIFY_THRESHOLD = 8` colisiones (con capacidad de tabla ≥ 64), la lista enlazada se transforma en un árbol rojo-negro. Si hay 256 elementos colisionando en ese mismo cubo, ¿cuál es la altura máxima $2\log_2(256)$ a comparar frente a los 256 saltos lineales?
Respuesta 16

Ejercicio 3 [PRB-0017]

Un `ConcurrentHashMap` redimensiona su tabla de 1024 a 2048 cubos manteniendo el factor de carga en 0,75. ¿Cuántas entradas admite la nueva tabla antes del siguiente redimensionado ($2048 \times 3/4$)?

Respuesta 1536

Ejercicio 4 [PRB-0018]

Si el tamaño de un montículo de la JVM supera los 32 GiB y desactiva Compressed OOPs, cada referencia pasa de 4 B a 8 B. En un grafo en memoria con 50 millones de referencias, ¿cuántos megabytes (10^6 B) adicionales consumen las referencias?

Respuesta 200

Cálculos reproducidos

$\frac{(2000000 \cdot (28-8))}{1000000} = 40$

C.4 Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos

Problema resuelto [PRB-0019]

Según la Ley de Little ($\lambda_{\text{máx}} = L_{\text{máx}}/W$), un microservicio donde cada petición pasa $W = 0{,}25$ s esperando respuestas de APIs externas está limitado con hilos tradicionales de plataforma a $L_{\text{plat}} = 400$ hilos (caudal máximo $400/0{,}25 = 1600$ req/s). Al migrar a **Virtual Threads** en Java 21 soportando $L_{\text{virt}} = 20000$ hilos virtuales simultáneos con la misma latencia $W = 0{,}25$ s, calcula cuántas peticiones por segundo adicionales puede sostener el servidor ($20000 / 0.25 - 400 / 0.25$).

Respuesta

$$Resultado = 78400$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0021]

Por la Ley de Little ($L = \lambda \cdot W$), si un sistema recibe $\lambda = 2500$ peticiones/s y cada una tarda $W = 0{,}2$ segundos, ¿cuántas peticiones hay en vuelo simultáneamente en promedio?

Respuesta 500

Ejercicio 2 [PRB-0022]

En una CPU de 16 núcleos lógicos, el `ForkJoinPool` portador de hilos virtuales crea por defecto un hilo portador (*carrier thread*) por núcleo. ¿Cuántos hilos del sistema operativo mantienen activos los hilos virtuales?

Respuesta 16

Ejercicio 3 [PRB-0023]

En el código `VirtualThreadDispatcher.runConcurrentTasks(100)`, se suma `i * 2` para $i = 0, 1, \ldots, 99$. ¿Cuál es el valor exacto de la suma (99×100)?

Respuesta 9900

Ejercicio 4 [PRB-0024]

Una `Arena.ofConfined()` de Project Panama reserva 64 segmentos nativos de 4096 bytes cada uno. ¿Cuántos bytes de memoria *off-heap* se liberan determinísticamente al cerrarse el bloque `try-with-resources`?

Respuesta 262144

Cálculos reproducidos

$\left(\frac{20000}{0{,}25} - \frac{400}{0{,}25}\right) = 78400$

C.5 Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores

Problema resuelto [PRB-0025]
En un programa en Go, se crea un slice `base := make([]int64, 100, 200)` (cada `int64` ocupa 8 B) y se obtiene la sub-rebanada de tres índices `sub := base[20:60:80]`. Calcula la suma de la longitud `len(sub)` ($60 - 20$) más la capacidad `cap(sub)` ($80 - 20$) multiplicada por los 8 B de cada elemento.
Respuesta

$$Resultado = 800$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
En Go de 64 bits, el encabezado de un slice (`SliceHeader`) consta de `Data` (8 B), `Len` (8 B) y `Cap` (8 B). ¿Cuántos bytes ocupa en la pila?
Respuesta 24
Ejercicio 2 [PRB-0028]
En Go, cada cubo (*bucket* `bmap`) de un `map[K]V` almacena exactamente 8 pares clave-valor. Si un mapa tiene $2^6 = 64$ cubos, ¿cuántos pares clave-valor caben en esos 64 cubos sin cubos de desbordamiento (64×8)?
Respuesta 512
Ejercicio 3 [PRB-0029]
Para la rebanada `s[15:45:50]`, ¿cuántos elementos pueden añadirse con `append` antes de superar la capacidad y forzar la reserva de un nuevo array $((50 - 15) - (45 - 15))$?
Respuesta 5
Ejercicio 4 [PRB-0030]
Una interfaz `io.Reader` en Go se representa como un par de palabras `(itab*, data*)` de 8 bytes cada una. ¿Cuántos bytes ocupan 500 descriptores de interfaz en un slice `[]io.Reader`?
Respuesta 8000

Cálculos reproducidos

$(((60 - 20) + (80 - 20)) \cdot 8) = 800$

C.6 Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos

Problema resuelto [PRB-0031]
Cada goroutine recién creada en Go 1.22 reserva una pila inicial de $S_G = 2$ KiB (2048 B), mientras que un hilo nativo de sistema operativo reserva por defecto una pila de $S_M = 2048$ KiB (2 MiB). Si un servidor de telemetría en Go mantiene $N = 100000$ goroutines activas sobre $M = 16$ hilos del sistema operativo, calcula cuántos mebibytes totales (dividiendo los KiB entre 1024) ocupan las pilas de las 100 000 goroutines más los 16 hilos OS.
Respuesta

$$Resultado = 227{,}3125$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
¿Cuántos bytes mide la pila inicial de una goroutine en Go (2×1024)?
Respuesta 2048
Ejercicio 2 [PRB-0034]
En el planificador de Go, un procesador P_1 tiene 180 goroutines en su cola local y un procesador inactivo P_2 le roba la mitad. ¿Cuántas goroutines roba P_2?

Respuesta 90

Ejercicio 3 [PRB-0035]

Un canal con búfer `ch := make(chan uint64, 512)` almacena elementos de 8 bytes. Si hay 300 elementos encolados en el búfer, ¿cuántos elementos adicionales caben antes de bloquear al emisor?

Respuesta 212

Ejercicio 4 [PRB-0036]

En el problema anterior, ¿cuántos bytes ocupa el búfer circular completo de 512 elementos `uint64` (512×8)?

Respuesta 4096

Cálculos reproducidos

$\frac{((100000 \cdot 2)+(16 \cdot 2048))}{1024} = 227{,}3125$

C.7 Monografía de Kotlin (Bloques A−H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform

Problema resuelto [PRB-0037]

En una aplicación en Kotlin sobre la JVM, una función de orden superior que captura una variable local del entorno se invoca en un bucle caliente $N = 500000$ veces por segundo. Si la función NO está marcada como `inline`, cada invocación aloja una instancia de clase anónima de $S_{\text{lambda}} = 24$ B. Al marcar la función de orden superior con `inline`, el alojamiento cae a 0 B. Calcula cuántos megabytes por segundo (dividiendo entre $\$10^6\$$) deja de alojar el bucle gracias a `inline`.

Respuesta

$$Resultado = 12$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0039]

En la función `aggregateClusterLatency` con `nodeIds = [1, 2, 3, 4, 5]` donde cada nodo devuelve `id * 5`, ¿cuál es la suma total devuelta ($(1+2+3+4+5) \times 5$)?

Respuesta 75

Ejercicio 2 [PRB-0040]

Un `@JvmInline value class Meter(val v: Double)` se almacena en un array primitivo de 1000 elementos. ¿Cuántos bytes ocupan los 1000 elementos `double` (1000×8)?

Respuesta 8000

Ejercicio 3 [PRB-0041]

Un `Flow<Int>` emite 60 elementos a razón de 1 elemento cada 10 ms y el colector aplica `.buffer(10)` tardando 25 ms por elemento. ¿Cuántos milisegundos tarda el colector en procesar los 60 elementos ($10 + 60 \times 25$)?

Respuesta 1510

Ejercicio 4 [PRB-0042]

Un proyecto Kotlin Multiplatform comparte 14 000 líneas de lógica de dominio en `commonMain` entre Android e iOS, requiriendo solo 1500 líneas específicas en `androidMain` y 1500 en `iosMain`. ¿Cuántas líneas de código duplicado se ahorran respecto a escribir los 14 000 líneas dos veces?

Respuesta 14000

Cálculos reproducidos

$\frac{(500000 \cdot 24)}{1000000} = 12$

C.8 Monografía de Swift 6 (Bloques A–H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores

Problema resuelto [PRB-0043]
En Swift sobre una arquitectura de 64 bits ($W = 8$ B), un contenedor existencial `any P` para un protocolo ocupa $5 \times 8 = 40$ B (24 B de búfer de valor en línea, 8 B de *Value Witness Table* y 8 B de *Protocol Witness Table*), mientras que un valor concreto `Int64` pasado mediante un genérico especializado `some P` ocupa únicamente 8 B. Calcula cuántos bytes ahorra un array de $N = 10000$ elementos al usar el tipo especializado `[Int64]` frente a un array existencial `[any P]`.
Respuesta

$$\mathit{Resultado} = 320000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
En Swift de 64 bits ($W = 8$ B), el `Value Buffer` interno de un contenedor existencial tiene 3 palabras. ¿Hasta cuántos bytes puede medir un `struct` para caber en línea dentro del contenedor existencial sin alojar en el montículo (3×8)?
Respuesta 24
Ejercicio 2 [PRB-0046]
Si se copia un `Array<Int>` de 50 000 enteros (`var b = a`) y solo se realizan lecturas sobre `a` y `b` sin mutar ninguno de los dos, ¿cuántos elementos se duplican en el montículo gracias a Copy-on-Write?
Respuesta 0
Ejercicio 3 [PRB-0047]
Un contenedor existencial para una composición de $K = 3$ protocolos (`any P1 & P2 & P3`) ocupa $(3 + 1 + 3) \times 8$ bytes en 64 bits. ¿Cuántos bytes mide en total?
Respuesta 56
Ejercicio 4 [PRB-0048]
En la cabecera de un objeto `class` en Swift de 64 bits, se almacenan el puntero `isa`/metadatos (8 B) y la palabra de contadores ARC (8 B). ¿Cuántos bytes suman las cabeceras de 2000 instancias?
Respuesta 32000

Cálculos reproducidos

$(10000 \cdot (40 - 8)) = 320000$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 31 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.

¶2: dos testimonios.

¶3: dos testimonios.

¶4: dos testimonios.

¶5: dos testimonios.

¶6: dos testimonios.

¶7: dos testimonios.

¶8: dos testimonios.

¶9: dos testimonios.

¶10: dos testimonios.

¶11: dos testimonios.

¶12: dos testimonios.

¶13: dos testimonios.

¶14: dos testimonios.

¶15: dos testimonios.

¶16: dos testimonios.

¶17: dos testimonios.

¶18: dos testimonios.

¶19: dos testimonios.

¶20: dos testimonios. Ecuaciones: 1 respaldada por una fuente.

¶21: dos testimonios.

¶22: dos testimonios.

¶23: dos testimonios.

¶24: dos testimonios.

¶25: dos testimonios.

¶26: dos testimonios.

¶27: dos testimonios.

¶28: dos testimonios.

¶29: dos testimonios.

¶30: dos testimonios.

¶31: dos testimonios.

¶32: dos testimonios.

¶33: dos testimonios.

¶34: dos testimonios.

¶35: dos testimonios.

¶36: dos testimonios.

¶37: dos testimonios.

¶38: dos testimonios.

¶39: dos testimonios.

¶40: dos testimonios.

¶41: dos testimonios.

¶42: dos testimonios.

¶43: dos testimonios.

¶44: dos testimonios.

¶45: dos testimonios.

¶46: dos testimonios.

¶47: dos testimonios.

¶48: dos testimonios.

Referencia Normativa src-csharp-spec (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-gamma-gof (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-java-jls21 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Go Programming Language Specification & Memory Model (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-kotlin-spec (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Swift Programming Language (Swift 6.0 Official Book) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-csharp-dotnet-clr-tipos-valor-referencia-linq scientific
REV-0002: cap02-csharp-avanzado-experto-span-async-valuetask-aspnet-proyectos scientific
REV-0003: cap03-java-jvm-bytecode-memoria-generics-streams scientific
REV-0004: cap04-java-avanzado-experto-virtual-threads-jmm-spring-proyectos scientific
REV-0005: cap05-go-fundamentos-slices-interfaces-errores-genericos scientific
REV-0006: cap06-go-avanzado-experto-goroutines-gmp-canales-context-proyectos scientific
REV-0007: cap07-monografia-kotlin-null-safety-coroutines-flow-kmp scientific
REV-0008: cap08-monografia-swift-valor-cow-protocolos-arc-actores scientific

Apéndice E

Plan de estudio

Monografías de Plataformas Gestionadas: C# (.NET 8/9) y Java (JVM 21 LTS)

✓Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ

- Comprender la arquitectura del Common Language Runtime (CLR), el lenguaje intermedio CIL, la compilación JIT (RyuJIT) y Native AOT en .NET 8/9.
- Distinguir con precisión semántica y espacial entre tipos por valor (‘struct‘, ‘readonly struct‘, ‘ref struct‘) y tipos por referencia (‘class‘, ‘record‘), evitando el boxing involuntario.
- Dominar la nulabilidad verificada (‘string¿), coincidencia de patrones avanzada y consultas declarativas con LINQ (‘IEnumerable<T>‘ vs ‘IQueryable<T>‘).

2 conceptos planificados.

✓Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos

- Programar procesamiento de alto rendimiento sin asignaciones en el montículo usando ‘Span<T>‘, ‘ReadOnlySpan<T>‘, ‘Memory<T>‘, ‘ArrayPool<T>‘ y ‘System.IO.Pipelines‘.
- Comprender la máquina de estados generada por el compilador para ‘async‘/‘await‘, cuándo utilizar ‘ValueTask<T>‘ frente a ‘Task<T>‘ y la propagación de ‘CancellationToken‘.
- Dominar el ecosistema empresarial de .NET (ASP.NET Core, inyección de dependencias, EF Core, Dapper, xUnit, BenchmarkDotNet) y los 4 proyectos integradores.

2 conceptos planificados.

✓Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams

- Explicar la arquitectura de la Java Virtual Machine (JVM): subsistema ClassLoader, áreas de memoria (Heap, Metaspace, pilas de hilo), intérprete y compiladores JIT C1/C2.
- Dominar las innovaciones de Java 17/21 LTS: ‘record‘, ‘sealed interface‘/‘permits‘, coincidencia de patrones en ‘switch‘ y desestructuración de records.
- Aplicar genéricos con comodines acotados (principio PECS: Producer Extends, Consumer Super), colecciones (‘ArrayList‘, ‘HashMap‘, ‘ConcurrentHashMap‘) y la API ‘Stream<T>‘.

2 conceptos planificados.

✓Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos

- Dominar el Java Memory Model (JMM, JLS §17.4): la relación de orden parcial *happens-before*, ‘volatile‘, ‘VarHandle‘, ‘CompletableFuture‘ y ‘ReentrantLock‘.
- Escalar aplicaciones síncronas legibles a millones de tareas concurrentes mediante **Virtual Threads** (JEP 444 en Java 21 LTS) y Concurrencia Estructurada.

- Integrar el ecosistema empresarial de Java (Maven, Gradle, JUnit 5, Testcontainers, Spring Boot 3, Micrometer/OpenTelemetry) y construir los 4 proyectos del bloque.

2 conceptos planificados.

Monografías de Sistemas Cloud y Móviles: Go, Kotlin y Swift

✓Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores

- Comprender la filosofía de simplicidad ortogonal de Go (Rob Pike, Ken Thompson, Robert Griesemer), su compilador ultrarrápido y el enlazado de binarios estáticos.
- Dominar la anatomía en memoria de ‘slice‘ (puntero, longitud, capacidad) y ‘map‘, el valor cero útil (*zero value*), composición de ‘struct‘ por incrustación (*embedding*) y ‘defer‘.
- Diseñar abstracciones desacopladas mediante interfaces satisfechas implícitamente, manejo explícito de errores como valores (‘errors.Is‘, ‘errors.As‘, ‘ %w‘) y genéricos de Go 1.18+.

2 conceptos planificados.

✓Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos

- Explicar el funcionamiento interno del planificador M:N con robo de trabajo (*work-stealing*) de Go basado en las entidades G (Goroutine), M (Machine) y P (Processor).
- Diseñar patrones concurrentes basados en CSP (Tony Hoare) con canales con y sin búfer, ‘select‘, cancelación jerárquica con ‘context.Context‘ y ‘errgroup.Group‘.
- Auditar el escape al montículo (‘go build -gcflags='-m'‘), detectar carreras de datos (‘go test -race‘), perfilar CPU/memoria con ‘pprof‘ y construir los 4 proyectos en Go.

2 conceptos planificados.

✓Monografía de Kotlin (Bloques A–H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform

- Dominar el sistema de tipos de Kotlin 2.0 (compilador K2): distinción estricta ‘T‘ vs ‘T¿, smart casts, ‘data class‘, ‘sealed interface‘, ‘value class‘ y delegación (‘by‘).
- Escribir código expresivo con funciones de orden superior ‘inline‘, tipos genéricos reificados (‘reified T‘), funciones de extensión y DSLs con receptores (‘T.() -> Unit‘).
- Construir sistemas concurrentes y reactivos con **Kotlin Coroutines**: funciones ‘suspend‘, transformaciones CPS, ‘CoroutineScope‘, ‘Dispatchers‘, ‘Structured Concurrency‘ y ‘Flow‘ / ‘StateFlow‘.

2 conceptos planificados.

✓Monografía de Swift 6 (Bloques A–H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores

- Comprender la arquitectura del compilador ‘swiftc‘ (SIL: Swift Intermediate Language y LLVM), la semántica de valor (‘struct‘, ‘enum‘) y la optimización Copy-on-Write (CoW).
- Diseñar abstracciones con Programación Orientada a Protocolos (‘protocol‘, tipos asociados, tipos opacos ‘some P‘ vs existenciales ‘any P‘).
- Dominar la gestión determinista de memoria mediante **ARC** (‘strong‘, ‘weak‘, ‘unowned‘) y el aislamiento estricto de carreras de datos de **Swift 6** (‘async‘/‘await‘, ‘actor‘, ‘@MainActor‘, ‘Sendable‘).

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-csharp-spec. https://doc.rust-lang.org/reference/src-csharp-spec.html

[27] Referencia Normativa src-gamma-gof. https://doc.rust-lang.org/reference/src-gamma-gof.html

[28] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

[29] Referencia Normativa src-java-jls21. https://doc.rust-lang.org/reference/src-java-jls21.html

[30] Referencia Normativa src-kotlin-spec. https://doc.rust-lang.org/reference/src-kotlin-spec.html

[31] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen VII: Monografías de SQL, Shell, PHP, Ruby, Dart, Funcionales y Científicos

Parte V (Bloques A–H): Consultas declarativas, automatización UNIX, ecosistemas web/móvil, programación funcional pura, BEAM y cálculo científico

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografías de Datos, Automatización de Sistemas y Desarrollo Web/Móvil 1

1. Monografía de SQL (Bloques A−H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización 2
1.1. Bloques A−D: Modelo relacional, DDL con restricciones, orden lógico de SELECT, lógica trivaluada (NULL) y JOINs . . . 2
1.2. Bloques E−H: CTEs recursivas, funciones de ventana (OVER), transacciones, EXPLAIN y projects/sql-transactional-ledger . . . 3

2. Monografía de Bash y POSIX Shell (Bloques A−H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos 5
2.1. Bloques A−D: Filosofía UNIX, POSIX sh vs Bash, descriptores de fichero, redirecciones, entrecomillado y expansiones . . . 5
2.2. Bloques E−H: Modo estricto, arrays, trap de limpieza, herramientas (awk, sed, jq), ShellCheck y 4 proyectos . . . 5

3. Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A−H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma 8
3.1. Monografía de PHP 8.3+ (Bloques A−H): Zend Engine, tipado estricto, enums, Composer/PSR y arquitectura web . . . 8
3.2. Monografías de Ruby 3.3+ y Dart 3.x (Bloques A−H): Bloques/Metaprogramación en Ruby e Isolates/AOT en Dart . . . 8

II Monografías de Lenguajes Funcionales Estáticos y Concurrentes (Haskell, OCaml, F#, Scala, BEAM, Lisp) 11

4. Monografía de Haskell (Bloques A−H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas 12
4.1. Bloques A−D: Filosofía pura de Haskell, GHC/Cabal/Stack, currificación, evaluación perezosa, ADTs y Typeclasses . . . 12
4.2. Bloques E−H: Functor, Applicative, Monad (IO, Reader, State, Either), concurrencia STM, QuickCheck y proyectos . . . 12

5. Monografías de OCaml 5.x y F# (.NET) (Bloques A−H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación 15
5.1. Monografía de OCaml 5.x (Bloques A−H): Inferencia ML, Functors de módulos, representación en memoria y efectos algebraicos . . . 15
5.2. Monografía de F# sobre .NET (Bloques A−H): Tuberías, unidades de medida ([<Measure>]), Computation Expressions y proyectos . . . 16

6. Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A−H) 18
6.1. Monografías de Erlang/OTP y Elixir sobre la BEAM (Bloques A−H): Actores aislados, GenServer y Supervisores . . . 18
6.2. Monografías de Scala 3, Clojure y Common Lisp / Scheme (Bloques A−H): Tipos dependientes, HAMT y macros . . . 19

III Monografías de Lenguajes Científicos, Embebidos y Lógicos (Julia, R, Lua, Perl, Objective-C, Prolog, GDScript) 21

7. Monografías de Julia y R (Bloques A−H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos 22
7.1. Monografía de Julia (Bloques A−H): Despacho múltiple, estabilidad de tipos, broadcasting fusionado y HPC . . . 22
7.2. Monografía de R (Bloques A−H): Vectores, data frames, fórmulas estadísticas, Tidyverse/data.table y Quarto . . . 23

8. Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A−H): Scripting Embebido, Unificación Lógica y Motores 25
8.1. Monografías de Lua 5.4+, Perl 5 y Objective-C (Bloques A−H): Tablas/Metatablas, Regex y objc_msgSend . . . 25
8.2. Monografías de Prolog y GDScript (Bloques A−H): Unificación lógica (SLD) y programación de motores de videojuegos . . . 26

A. Notación 28

B. Hoja de fórmulas 29

C. Soluciones 30
C.1. Monografía de SQL (Bloques A−H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización . . . 30
C.2. Monografía de Bash y POSIX Shell (Bloques A−H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos . . . 30
C.3. Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A−H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma . . . 31
C.4. Monografía de Haskell (Bloques A−H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas . . . 32
C.5. Monografías de OCaml 5.x y F# (.NET) (Bloques A−H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación . . . 32
C.6. Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A−H) . . . 33
C.7. Monografías de Julia y R (Bloques A−H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos . . . 34
C.8. Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A−H): Scripting Embebido, Unificación Lógica y Motores . . . 34

D. Auditoría de fuentes 36

E. Plan de estudio 38

Parte I

Monografías de Datos, Automatización de Sistemas y Desarrollo Web/Móvil

Capítulo 1

Monografía de SQL (Bloques A–H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización

*Este capítulo desarrolla la monografía completa (Bloques A – H) de **SQL (ISO/IEC 9075:2023)**. SQL no es un simple lenguaje de consulta superficial, sino un lenguaje declarativo funcional basado en el álgebra y el cálculo relacional de primer orden (con recursión de punto fijo y funciones analíticas de ventana) que todo ingeniero de software y datos debe dominar en profundidad.*

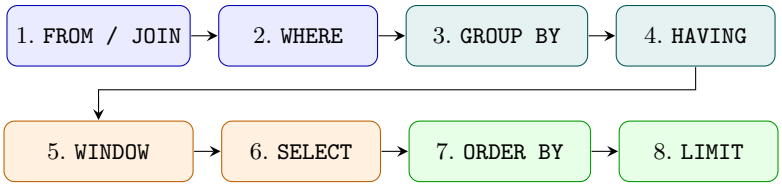


Figura 1. Orden lógico de evaluación de una consulta SQL frente al orden léxico de escritura: un alias definido en SELECT no existe aún cuando se evalúan WHERE, GROUP BY o HAVING.

Consejo

En funciones de ventana con `ORDER BY` que calculan acumulados (`SUM(...) OVER (ORDER BY fecha)`), especifica explícitamente `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`: el marco por defecto del estándar SQL cuando hay `ORDER BY` es `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, que agrupa de golpe todas las filas empatadas en `fecha`.

Definición

Predicado Sargable en SQL (Search Argument Able): condición de filtro de la forma `columna_indexada OP expresión_constante` que permite al motor de almacenamiento posicionarse directamente en las páginas hoja de un árbol B+ (`Index Seek / Index Range Scan`) sin evaluar una transformación fila por fila.

Autoficha

¿Qué diferencia exacta de numeración producen `ROW_NUMBER()`, `RANK()` y `DENSE_RANK()` ante una secuencia de puntuaciones `[100, 90, 90, 80]`?

1.1 Bloques A–D: Modelo relacional, DDL con restricciones, orden lógico de SELECT, lógica trivaluada (NULL) y JOINs

Aunque en SQL escribimos léxicamente `SELECT ... FROM ... WHERE ...`, el motor relacional evalúa la consulta siguiendo el orden lógico estricto mostrado en la Figura \ref{fig:fig-sql-logical-execution-order}: `FROM` y `JOIN` construyen primero el producto cartesiano filtrado por `ON`; `WHERE` descarta las filas cuya condición no evalúa a `TRUE`; `GROUP BY` particiona las filas en grupos; `HAVING` filtra grupos agregados; `WINDOW` calcula funciones de ventana; `SELECT` proyecta las columnas y asigna alias; `DISTINCT` elimina duplicados; `ORDER BY` ordena el conjunto resultante; y `LIMIT/FETCH FIRST` recorta la salida. Un escollo clásico proviene de la **lógica trivaluada (3VL)** de `NULL` (`TRUE`, `FALSE`, `UNKNOWN`): cualquier comparación ordinaria `col = NULL` o `col <> NULL` produce `UNKNOWN`. Si una subconsulta de `WHERE id NOT IN`

`(SELECT parent_id FROM t)` devuelve un solo `NULL`, la expresión `id <> v1 AND id <> NULL` evalúa a `UNKNOWN` para todas las filas y la consulta devuelve un conjunto vacío; por ello debe utilizarse siempre `WHERE NOT EXISTS (...)` o `IS DISTINCT FROM`.

1.2 Bloques E–H: CTEs recursivas, funciones de ventana (OVER), transacciones, EXPLAIN y projects/sql-transactional-ledger

En el Nivel Avanzado y Experto (Bloques E y F), las **expresiones de tabla común recursivas** (`WITH RECURSIVE`) permiten recorrer árboles jerárquicos y grafos (como el plan contable de nuestro proyecto verificado `projects/sql-transactional-ledger`), mientras que las **funciones de ventana** (`OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`) calculan saldos acumulados, medias móviles y clasificaciones (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`) en una sola pasada sin colapsar las filas individuales:

```
SELECT
    account_id,
    entry_id,
    debit_cents - credit_cents AS delta_cents,
    SUM(debit_cents - credit_cents) OVER (
        PARTITION BY account_id
        ORDER BY entry_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_balance_cents
FROM journal_lines;
```

Para que el optimizador de costes pueda emplear un índice B+ Tree, los predicados de `WHERE` deben ser **sargables** (*Search Argument Able*): aplicar una función sobre la columna indexada (`WHERE YEAR(created_at) = 2026`) impide el recorrido por rango del índice y fuerza un escaneo completo de tabla (`Seq Scan`), debiendo reescribirse como `WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01'`.

Problema resuelto

Enunciado En el proyecto `projects/sql-transactional-ledger`, la cuenta de Caja (`account_id = 110`) registra tres apuntes sucesivos `(debit_cents, credit_cents)`: `(500000, 0)`, `(125000, 0)` y `(0, 40000)`. Calcula el saldo acumulado en céntimos (`running_balance_cents`) que devuelve la función de ventana `SUM(debit_cents - credit_cents) OVER (...)` en la tercera fila.

Datos

- $d_1 = 500000$ centimos — Aportación inicial de capital
- $d_2 = 125000$ centimos — Cobro de factura SaaS
- $c_3 = 40000$ centimos — Pago de infraestructura cloud

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Cálculo de saldo acumulado por función de ventana SQL en sql-transactional-ledger

$$B_3 = 500000 + 125000 - 40000$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los tres deltas algebraicos de la partición account$_i d = 110$

$$B_3 = 500000 + 125000 - 40000$$

$$(((500000 - 0) + (125000 - 0)) + (0 - 40000))$$

Resultado

$$Resultado = 585000$$

Interpretación El saldo acumulado en la tercera fila es exactamente 585 000 céntimos (5850.00 EUR), idéntico a la aserción de `verify_ledger.py`.
Comprobación de los pasos

- Paso 1: 585000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) Ante las puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `RANK()` al cuarto elemento (`80`) tras el empate en la posición 2?
2 [PRB-0004] (Básico, Numérico) Ante las mismas puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `DENSE_RANK()` al cuarto elemento (`80`)?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) Un producto cartesiano `CROSS JOIN` entre una tabla de 120 tiendas y una tabla de 365 días del año genera todas las combinaciones `(tienda, día)`. ¿Cuántas filas produce?
4 [PRB-0006] (Intermedio, Diseño) Una consulta con paginación `LIMIT 50 OFFSET 400` devuelve la página 9 de resultados. ¿Cuál es el número de orden (base 1) de la primera fila devuelta en esa página ($400 + 1$)?

Capítulo 2

Monografía de Bash y POSIX Shell (Bloques A–H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos

*Este capítulo presenta la monografía completa (Bloques $A - H$) de **Bash y POSIX Shell**. Los scripts de shell son el pegamento universal de los sistemas Linux/macOS, los contenedores Docker y los pipelines de CI/CD. Enseñamos a escribirlos con rigor de ingeniería de sistemas: inmunes a espacios en nombres de fichero, con limpieza atómica de directorios temporales y auditados con* `shellcheck`.

Consejo

Integra siempre `shellcheck -S warning script.sh` en tu editor y en CI: detecta al instante variables sin entrecomillar (SC2086), pérdida de estado de salida al usar `local x=$(cmd)` en una sola línea (SC2155) y problemas de subshells en tuberías.

Definición

Código de estado de salida por señal en POSIX (`128 + N`): convención del intérprete de comandos según la cual un proceso terminado de forma abrupta por la señal número N reporta en `$?` el código de salida $128 + N$ (por ejemplo, `130` para `SIGINT = 2` o `137` para `SIGKILL = 9`).

Autoficha

¿Por qué la línea `local salida="$(comando_que_falla)"` en Bash oculta el código de error de `comando_que_falla` (porque `local` es un comando builtin cuyo propio código de retorno es `0`) y debe escribirse en dos líneas separadas (`local salida; salida="$(comando_que_falla)"`)?

2.1 Bloques A–D: Filosofía UNIX, POSIX sh vs Bash, descriptores de fichero, redirecciones, entrecomillado y expansiones

En el modelo de procesos POSIX, cada comando hereda tres descriptores de fichero abiertos: `0` (entrada estándar), `1` (salida estándar) y `2` (error estándar), y al terminar devuelve un código de estado entero entre `0` y `255` donde **`0` significa éxito** (verdadero en `if cmd; then` y en los operadores de cortocircuito `cmd1 && cmd2` y `cmd1 || cmd2`) y cualquier valor distinto de cero indica fallo. El orden de las redirecciones se evalúa de izquierda a derecha: `cmd > salida.log 2>&1` apunta primero el descriptor `1` a `salida.log` y luego duplica el descriptor `2` hacia donde apunta `1` (ambos van al fichero). La causa número uno de fallos y vulnerabilidades de inyección en scripts de shell es dejar expansiones sin comillas dobles (`$archivo` en lugar de `"$archivo"`), lo que dispara la división por palabras (*word splitting* según `IFS`) y la expansión de comodines (*pathname expansion* `*`, `?`).

2.2 Bloques E–H: Modo estricto, arrays, trap de limpieza, herramientas (awk, sed, jq), ShellCheck y 4 proyectos

Todo script de automatización en Bash debe comenzar configurando el modo estricto (`set -euo pipefail`, que aborta ante comandos fallidos, variables no definidas y errores intermedios en una tubería) y registrando un manejador `trap` sobre la pseudo-señal `EXIT` para eliminar de forma garantizada los recursos temporales:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'

readonly WORK_DIR="$(mktemp -d)"
cleanup() {
    rm -rf -- "$WORK_DIR"
}
trap cleanup EXIT INT TERM

count_non_empty_lines() {
    local target_file="${1:?Ruta de fichero obligatoria}"
    grep -cve '^[[:space:]]*$' -- "$target_file"
}
```

Observa el uso de `-` antes de `"$target_file"` y `"$WORK_DIR"`: indica el fin de las opciones de bandera, impidiendo que un fichero cuyo nombre empiece por guion (`-rf`) sea interpretado como una opción por `grep` o `rm`. Para procesar listas de ficheros con nombres arbitrarios (incluyendo saltos de línea), se combina siempre el delimitador nulo: `find . -type f -name '*.log' -print0 | xargs -0 sha256sum`.

Problema resuelto

Enunciado Un contenedor en un pipeline de despliegue termina abruptamente con código de salida `$? = 137` tras intervenir el *OOM Killer* del kernel Linux (`SIGKILL = 9`), y otro proceso aborta por violación de segmento (`SIGSEGV = 11`, código $128+11=139$). Calcula la suma de los números de las dos señales POSIX subyacentes obtenidas restando 128 a cada código de salida $((137-128)+(139-128))$.

Datos

- $E_1 = 137$ — Código de salida por SIGKILL (128 + 9)
- $E_2 = 139$ — Código de salida por SIGSEGV (128 + 11)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Decodificación de señales POSIX a partir del código de estado 128 + N

$$S = (137-128)+(139-128)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos 128 a cada código para recuperar los números de señal 9 y 11 y los sumamos

$$S = (137-128)+(139-128)$$

$$((137-128)+(139-128))$$

Resultado

$$\mathit{Resultado} = 20$$

Interpretación Las señales subyacentes son la 9 (SIGKILL) y la 11 (SIGSEGV), cuya suma es 20.

Comprobación de los pasos

- Paso 1: 20 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) Un script en ejecución es interrumpido por el usuario pulsando `Ctrl+C` (`SIGINT = 2`). ¿Qué código de salida `$?` reporta el shell ($128 + 2$)?
2 [PRB-0010] (Básico, Numérico) Un proceso recibe la señal de terminación ordenada `SIGTERM = 15` sin capturarla. ¿Cuál es su código de salida ($128 + 15$)?
3 [PRB-0011] (Intermedio, Desarrollo por pasos) Un comando `xargs -0 -n 50 -P 4` procesa una lista de 600 ficheros pasándolos en lotes de 50 ficheros por invocación. ¿Cuántos procesos hijos lanza en total `xargs` ($600 / 50$)?
4 [PRB-0012] (Intermedio, Diseño) Una máscara `umask 027` se resta de los permisos base `666` (octal) para la creación de ficheros regulares ($666 - 27$ en dígito a dígito: $6-0=6$, $6-2=4$, $6-7 \to 0$). ¿Cuál es el permiso octal resultante expresado como entero de tres dígitos?

Capítulo 3

Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A–H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma

*Este capítulo agrupa las monografías completas (Bloques A – H) de tres lenguajes esenciales en el desarrollo web y móvil de producción: **PHP 8.3+**, **Ruby 3.3+** y **Dart 3.x**.*

Consejo

En PHP 8+, incluye siempre `declare(strict_types=1);` en la primera línea de todos tus ficheros `.php` y utiliza la expresión `match ($valor) { ... }` en lugar de `switch ($valor)`: `match` compara con identidad estricta (`===`), no requiere `break` y lanza `UnhandledMatchError` si olvidas un caso.

Definición

Nulabilidad Sólida (Sound Null-Safety en Dart 3): propiedad del sistema de tipos según la cual, una vez que el programa compila, es matemáticamente imposible en tiempo de ejecución que una expresión de tipo no anulable `T` evalúe a `null`, permitiendo al compilador AOT eliminar comprobaciones de puntero nulo.

Autoficha

¿Qué diferencia exacta existe en Ruby entre una instancia de `Proc.new { return 1 }` y un `lambda` (`-> { return 1 }`) en cuanto al efecto de `return` y la comprobación del número de argumentos?

3.1 Monografía de PHP 8.3+ (Bloques A–H): Zend Engine, tipado estricto, enums, Composer/PSR y arquitectura web

PHP 8.x ha experimentado una transformación profunda respecto a sus versiones históricas: al encabezar cada fichero con `declare(strict_types=1);`, el motor Zend desactiva las coerciones escalares silenciosas y verifica contratos de tipos de unión (`int|float`), intersección (`Countable&Iterator`), clases inmutables `readonly class`, enumeraciones respaldadas (`enum Status: string`), atributos de metadatos (`#[Route(...)]`) y expresiones `match` con comparación estricta (`===`). En producción, **OPcache** mantiene el bytecode precompilado en memoria compartida entre trabajadores PHP-FPM (o en servidores persistentes como FrankenPHP / RoadRunner), y el gestor de paquetes `Composer` junto con las normas **PSR** (*PHP Standards Recommendations*: PSR-4 autoloading, PSR-7 mensajes HTTP, PSR-11 contenedores DI, PSR-15 middlewares) articula los ecosistemas de `Symfony` y `Laravel`, verificados estáticamente con `PHPStan` (nivel máximo) y probados con `PHPUnit`.

3.2 Monografías de Ruby 3.3+ y Dart 3.x (Bloques A–H): Bloques/Metaprogramación en Ruby e Isolates/AOT en Dart

En **Ruby 3.3+** (creado por Yukihiro «Matz» Matsumoto), absolutamente todo valor es un objeto y toda operación es un envío de mensaje; las clases son abiertas, la herencia múltiple se resuelve linealmente mediante *mixins* (`include` e `prepend` de módulos) y cada método puede recibir un bloque anónimo invocado con `yield` o convertido en clausura `Proc`. Con el compilador JIT en proceso **YJIT** (escrito en Rust dentro de CRuby), las aplicaciones `Ruby on Rails` logran mejoras notables de latencia, mientras

que los **Ractors** proporcionan paralelismo de actores sin compartir estado mutable. Por su parte, **Dart 3.x** ofrece **nulabilidad sólida** (*sound null-safety*: el compilador AOT garantiza que una variable no anulable jamás contiene `null`, optimizando el código máquina emitido), clases selladas (`sealed class`) con *pattern matching* exhaustivo y un modelo de concurrencia basado en **Isolates** (hebras con montículos de memoria independientes que se comunican por puertos de mensajes o transferencia de `TransferableTypedData`), compilando en modo JIT con *stateful hot reload* durante el desarrollo y en binario nativo **AOT** para producción en iOS, Android, escritorio y web:

```
sealed class ApiResponse<T> {}
final class ApiSuccess<T> extends ApiResponse<T> {
  final T data;
  const ApiSuccess(this.data);
}
final class ApiFailure<T> extends ApiResponse<T> {
  final int statusCode;
  const ApiFailure(this.statusCode);
}

String describeResponse<T>(ApiResponse<T> res) => switch (res) {
  ApiSuccess(data: final d)       => 'OK: $d',
  ApiFailure(statusCode: final c) => 'HTTP Error $c',
};
```

Problema resuelto

Enunciado Un servidor de producción con 8192 MiB de RAM reserva 2048 MiB para el sistema operativo y el búfer de OPcache/Redis, dejando $M_{\text{avail}} = 6144$ MiB para el pool de trabajadores PHP-FPM. Si cada trabajador consume en carga punta $m_{\text{worker}} = 48$ MiB de memoria privada, calcula el valor óptimo de `pm.max_children` ($6144 / 48$).

Datos

- $M_{avail} = 6144\,\text{MiB}$ — RAM neta disponible para trabajadores PHP-FPM
- $m_{worker} = 48\,\text{MiB}$ — Consumo privado RSS por proceso trabajador

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Dimensionamiento de pm.max_children en PHP-FPM sin agotamiento de RAM

$$N_{\text{máx}} = \frac{6144}{48}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos la memoria disponible entre el consumo por trabajador

$$N_{\text{máx}} = \frac{6144}{48}$$

$$\frac{6144}{48}$$

Resultado

$$Resultado = 128$$

Interpretación El servidor puede ejecutar de forma segura hasta 128 trabajadores PHP-FPM concurrentes.

Comprobación de los pasos

- Paso 1: 128 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En Flutter/Dart, para sostener una tasa de refresco de pantalla de 120 Hz, el presupuesto total por fotograma (construcción + diseño + pintado) es 1000/120 ms. ¿Cuántos milisegundos caben en 12 fotogramas consecutivos?
2 [PRB-0016] (Básico, Numérico) En Ruby, la expresión `(1..10).select(&:even?).map { |x| x * 3 }.sum` suma el triple de los pares del 2 al 10 ($(2+4+6+8+10)\times 3$). ¿Cuál es el resultado?
3 [PRB-0017] (Intermedio, Desarrollo por pasos) Si OPcache evita recompilar 85 ficheros PHP por petición HTTP (ahorrando 0,4 ms por fichero), ¿cuántos milisegundos de CPU ahorra en cada petición ($85 \times 0{,}4$)?
4 [PRB-0018] (Intermedio, Diseño) Un Isolate de fondo en Dart procesa un lote de 400 imágenes en grupos de 25 imágenes devolviendo el progreso al Isolate principal tras cada grupo. ¿Cuántos mensajes de progreso envía?

Parte II

Monografías de Lenguajes Funcionales Estáticos y Concurrentes (Haskell, OCaml, F#, Scala, BEAM, Lisp)

Capítulo 4

Monografía de Haskell (Bloques A–H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas

*Este capítulo desarrolla la monografía completa (Bloques A – H) de **Haskell (GHC 9.x)**. Estudiar Haskell enseña en su forma más pura los conceptos que posteriormente han adoptado Rust (`Trait`, `Option`, `Result`), Scala, Swift y TypeScript: tipos algebraicos, clases de tipos, composición monádica y pruebas basadas en propiedades.*

Consejo

En Haskell de producción, utiliza siempre `newtype` en lugar de `data` cuando envuelvas un único campo con un solo constructor (`newtype CustomerId = CustomerId Int`): `newtype` garantiza distinción de tipos en compilación pero se borra por completo en ejecución con cero sobrecoste de puntero.

Definición

Forma Normal Débil a la Cabeza (Weak Head Normal Form, WHNF en Haskell): estado de reducción de una expresión en el que su capa más externa ya no es un redex (es un constructor de datos como `Just thunk`, un literal primitivo o una abstracción lambda `\x -> ...`), aunque sus subexpresiones internas permanezcan sin evaluar.

Autoficha

¿Por qué el sistema de tipos de Haskell impide ejecutar `putStrLn " enviando misil "` dentro de una transacción `atomically :: STM a -> IO a` de memoria transaccional por software?

4.1 Bloques A–D: Filosofía pura de Haskell, GHC/Cabal/Stack, currificación, evaluación perezosa, ADTs y Typeclasses

En **Haskell**, toda función es matemática y **referencialmente transparente**: para una misma entrada, produce siempre la misma salida sin mutar variables globales ni realizar entrada/salida oculta. Todas las funciones de varios argumentos están currificadas por defecto (`add :: Int -> Int -> Int` es `Int -> (Int -> Int)`), permitiendo la aplicación parcial directa (`map (+10) [1, 2, 3]`). La estrategia de evaluación por defecto es no estricta (*lazy evaluation* / *call-by-need*): las expresiones no se evalúan al pasarse como argumento, sino que se empaquetan en una suspensión en el montículo denominada **thunk** que se reduce a *Weak Head Normal Form* (WHNF) solo cuando la coincidencia de patrones exige conocer su constructor exterior, compartiendo el resultado para no recalcularlo. Cuando se acumulan sumas en un pliegue izquierdo, usar `foldl` perezoso construye una cadena de N *thunks* `(((0 + x1) + x2) ...)` que desborda la memoria (*space leak*); la solución profesional consiste en usar siempre la variante estricta `Data.List.foldl'`.

4.2 Bloques E–H: Functor, Applicative, Monad (IO, Reader, State, Either), concurrencia STM, QuickCheck y proyectos

Haskell parametriza constructores de tipos de *kind* `* -> *` mediante la jerarquía algebraica `Functor` (`fmap :: (a -> b) -> f a -> f b`), `Applicative` (`pure :: a -> f a`, `(<*>) :: f (a -> b) -> f a -> f b`) y **`Monad`** (`(>>=) :: m a -> (a -> m b) -> m b`). Toda instancia de `Monad` debe satisfacer las tres leyes monádicas: identidad por la izquierda (`return a >>= k == k a`), identidad por la derecha

(`m >>= return == m`) y asociatividad (`(m >>= k) >>= h == m >>= (\x -> k x >>= h)`). La notación `do` es azúcar sintáctico directo sobre `(>>=)`:

```
module SafeEval (Expr(..), eval) where

data Expr = Lit Int | Add Expr Expr | Div Expr Expr
  deriving (Show, Eq)

eval :: Expr -> Either String Int
eval (Lit n)     = Right n
eval (Add e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  Right (v1 + v2)
eval (Div e1 e2) = do
  v1 <- eval e1
  v2 <- eval e2
  if v2 == 0 then Left "division by zero" else Right (div v1 v2)
```

Gracias a que las lecturas y escrituras de estado están controladas por el sistema de tipos, Haskell ofrece **Software Transactional Memory (STM)**: transacciones en memoria (`TVar`, `atomically`, `retry`, `orElse`) que se componen sin posibilidad de interbloqueos (*deadlocks*), ya que el tipo `STM a` prohíbe estáticamente ejecutar efectos `IO` irreversibles dentro de un bloque que podría reintentarse.

Problema resuelto

Enunciado En GHC de 64 bits, cada celda `(:)` de una lista enlazada `[Int]` en el montículo ocupa 3 palabras de 8 B (24 B por nodo). Una tubería `sum (map (*2) (map (+1) [1..N]))` para $N = 100000$ elementos, si se evaluara de forma estricta sin fusión, alojaría 3 listas de 100000 celdas ($3 \times 100000 \times 24$ B). Gracias a la fusión `foldr/build` de GHC con `-O2`, las tres listas desaparecen y el bucle se ejecuta en registros alojando 0 B de celdas. Calcula cuántos kilobytes (dividiendo entre 1000) ahorra la desforestación.

Datos

- $k_{listas} = 3$ — Listas intermedias eliminadas
- $N = 100000$ — Elementos de la secuencia
- $S_{cons} = 24\,\mathrm{B}$ — Tamaño de una celda cons en GHC 64 bits

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de montículo mediante desforestación foldr/build en GHC

$$\Delta M_{\mathrm{kB}} = \frac{3 \cdot 100000 \cdot 24}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos 3 * 100000 * 24 B y dividimos entre 1000 B/kB

$$\Delta M_{\mathrm{kB}} = \frac{3 \cdot 100000 \cdot 24}{1000}$$

$$\frac{((3 \cdot 100000) \cdot 24)}{1000}$$

Resultado

$$\mathit{Resultado} = 7200$$

Interpretación La fusión de flujos de GHC evita alojar 7200 kB (7.2 MB) de celdas intermedias.

Comprobación de los pasos

- Paso 1: 7200 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0021**] **(Básico, Numérico)** En el evaluador monádico `eval (Div (Add (Lit 20) (Lit 22)) (Lit 6))`, ¿qué valor entero `Right v` devuelve la expresión $((20 + 22)/6)$?
2 [**PRB-0022**] **(Básico, Numérico)** `QuickCheck` ejecuta por defecto 100 casos generados aleatoriamente por cada propiedad `prop_*`. Si un módulo define 18 propiedades, ¿cuántos casos de prueba verifica `quickCheck` en total?
3 [**PRB-0023**] **(Intermedio, Desarrollo por pasos)** En la expresión aplicativa `[(+10), (*2), (+100)] <*> [1, 2, 3, 4]`, la instancia `Applicative` de listas aplica cada una de las 3 funciones a cada uno de los 4 valores. ¿Cuántos elementos contiene la lista resultante?
4 [**PRB-0024**] **(Intermedio, Diseño)** Un `newtype` en Haskell envuelve un `Int64` (8 B). ¿Cuántos bytes de sobrecoste adicional añade el constructor del `newtype` en tiempo de ejecución tras el borrado de GHC?

Capítulo 5

Monografías de OCaml 5.x y F# (.NET) (Bloques A–H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación

*Este capítulo presenta las monografías completas (Bloques $A - H$) de los dos principales exponentes industriales de la familia **ML** (*Meta Language*): **OCaml 5.x** (el lenguaje de elección para escribir compiladores, demostradores de teoremas como Coq/Rocq y sistemas de negociación algorítmica de baja latencia como en Jane Street) y **F#** (el lenguaje funcional primero del ecosistema .NET).*

Consejo

En OCaml, define siempre un fichero de interfaz explícito `.mli` para cada módulo `.ml`: mantener abstractos los tipos internos en el `.mli` garantiza el encapsulamiento de invariantes y acelera la compilación incremental con `dune`.

Definición

Entero etiquetado de 63 bits en OCaml (Tagged Integer): representación en máquina de 64 bits en la que un entero matemático n se codifica en registro como la palabra $(2n + 1)$ (desplazado 1 bit a la izquierda con el bit menos significativo fijado a `1`), permitiendo al recolector de basura distinguirlo instantáneamente de cualquier puntero alineado (cuyo bit 0 es `0`) sin alojar en el montículo.

Autoficha

¿Por qué la codificación de un entero n como $2n+1$ en OCaml permite que la suma de dos enteros etiquetados se implemente en ensamblador como `(a + b) - 1` en un solo ciclo de reloj sin desencajonar punteros?

5.1 Monografía de OCaml 5.x (Bloques A–H): Inferencia ML, Functors de módulos, representación en memoria y efectos algebraicos

OCaml (desarrollado en INRIA por Xavier Leroy y su equipo) combina evaluación estricta predecible, un compilador nativo rapidísimo (`ocamlopt`) con un recolector generacional incremental de latencias sub-milisegundo y el sistema de módulos más potente de la industria: en OCaml, los módulos (`struct ... end`) tienen tipos llamados firmas (`sig ... end`), y un **Functor** es una función a nivel de módulos que recibe un módulo como argumento y devuelve un nuevo módulo especializado (por ejemplo, `Map.Make(String)` construye un árbol AVL tipado para claves `string`). En cuanto a su representación de bajo nivel en 64 bits, OCaml reserva el bit menos significativo de cada palabra como etiqueta del recolector de basura (`1` para enteros inmediatos sin puntero `int` de **63 bits**, y `0` para punteros alineados al montículo). Además, **OCaml 5** incorpora paralelismo multinúcleo real (`Domain.spawn`) y **manejadores de efectos algebraicos** (*Effect Handlers*), que permiten definir planificadores asíncronos como bibliotecas puras sin colorear las funciones con `async`.

5.2 Monografía de F# sobre .NET (Bloques A–H): Tuberías, unidades de medida ([<Measure>]), Computation Expressions y proyectos

Por su parte, **F#** (diseñado por Don Syme en Microsoft Research) traslada la elegancia de OCaml al CLR de .NET. Entre sus innovaciones más notables destacan las **unidades de medida verificadas en compilación** (`[<Measure>] type m`, `[<Measure>] type s`, `[<Measure>] type N = kg m / s^2`), que previenen errores dimensionales (como el accidente de la sonda Mars Climate Orbiter) y se borran por completo al emitir el bytecode CIL con coste cero en tiempo de ejecución:

```
[<Measure>] type m
[<Measure>] type s
[<Measure>] type kg

let kineticEnergy (mass: float<kg>) (velocity: float<m/s>) : float<kg * m^2 / s^2> =
    0.5 * mass * velocity * velocity
```

Asimismo, F# generaliza las mónadas, los generadores y las consultas mediante una construcción sintáctica unificada: las **expresiones de computación** (*Computation Expressions*: `seq { yield ... }`, `async { let! x = ... }`, `task { ... }`, `result { ... }`).

Problema resuelto

Enunciado En el compilador nativo `ocamlopt` de 64 bits, cualquier entero `int` n se almacena en un registro de CPU mediante la representación etiquetada $R(n) = 2n + 1$ (con el bit 0 puesto a 1 para indicar al recolector de basura que no es un puntero). Calcula el valor decimal exacto de la palabra en registro $R(n)$ para el entero de aplicación $n = 1500$.

Datos

- $n = 1500$ — Valor entero lógico en el programa OCaml

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Codificación de enteros inmediatos de 63 bits (2n + 1) en OCaml

$$R(1500) = 2 \cdot 1500 + 1$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Desplazamos 1 bit a la izquierda (multiplicar por 2) y activamos el bit de etiqueta (+1)

$$R(1500) = 2 \cdot 1500 + 1$$

$$((2 \cdot 1500) + 1)$$

Resultado

$$\mathit{Resultado} = 3001$$

Interpretación El registro de CPU almacena el valor impar 3001 (0xBB9), que el recolector identifica en O(1) como un valor inmediato no rastreable.

Comprobación de los pasos

- Paso 1: 3001 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0027**] **(Básico, Numérico)** En OCaml, si un registro contiene la palabra etiquetada $R(n) = 85$, ¿cuál es el entero lógico $n = (85 - 1)/2$?

2 [**PRB-0028**] **(Básico, Numérico)** En F#, la función `kineticEnergy` calcula $0{,}5 \cdot m \cdot v^2$. Para una masa $m = 4$ kg y velocidad $v = 15$ m/s, ¿cuántos julios ($\mathrm{kg \cdot m^2/s^2}$) devuelve?

3 [**PRB-0029**] **(Intermedio, Desarrollo por pasos)** ¿Cuántos bytes adicionales ocupa en el ensamblado `.dll` de .NET la anotación de unidades `[<Measure>] type kg` tras la compilación de F#?

4 [**PRB-0030**] **(Intermedio, Diseño)** En OCaml de 64 bits, cada bloque en el montículo tiene 1 palabra de cabecera (8 B) que codifica tamaño, color GC y etiqueta de constructor, seguida de k campos de 8 B. ¿Cuántos bytes ocupa un nodo de árbol binario con 3 campos (`valor`, `izq`, `der`)?

Capítulo 6

Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A–H)

*Este capítulo desarrolla las monografías completas (Bloques A–H) de cinco lenguajes fundamentales de la computación concurrente y funcional: **Erlang/OTP** y **Elixir** (sobre la máquina virtual BEAM), **Scala 3**, **Clojure** y **Lisp/Scheme**.*

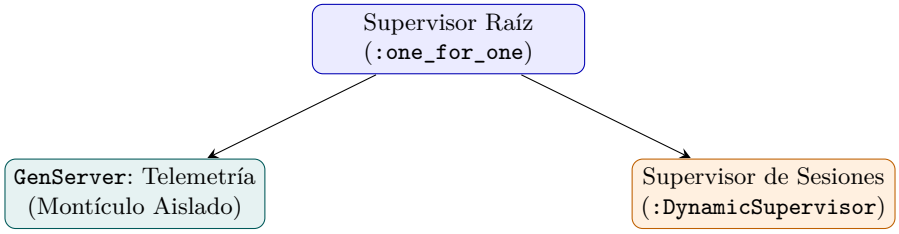


Figura 2. Árbol de supervisión OTP en Erlang/Elixir: los supervisores monitorizan procesos trabajadores (GenServer) aislados en memoria y aplican estrategias deterministas de reinicio (one_for_one, rest_for_one, one_for_all).

Consejo

En Erlang y Elixir, evita acumular millones de mensajes sin procesar en el buzón (*mailbox*) de un único `GenServer` usando `cast` asíncrono sin control: utiliza `GenServer.call` (que aplica contrapresión síncrona al llamador) o procesadores con demanda controlada como `GenStage` y `Broadway`.

Definición

Homoiconicidad (Code-as-Data en la familia Lisp): propiedad de un lenguaje cuya representación sintáctica primaria (S-expressions en Common Lisp/Scheme y EDN en Clojure) coincide directamente con las estructuras de datos primitivas manipulables por el propio lenguaje, haciendo trivial la metaprogramación por macros.

Autoficha

¿Por qué en un vector persistente inmutable de Clojure con factor de ramificación $B = 32$, acceder o actualizar un elemento entre $32^4 = 1048576$ elementos requiere recorrer únicamente 4 niveles de punteros?

6.1 Monografías de Erlang/OTP y Elixir sobre la BEAM (Bloques A–H): Actores aislados, GenServer y Supervisores

Creado por Joe Armstrong, Robert Virding y Mike Williams en Ericsson para conmutadores telefónicos con una disponibilidad del 99,9999999 % (*nueve nueves*), **Erlang** y su compañero moderno **Elixir** (creado por José Valim) se ejecutan sobre la máquina virtual **BEAM**. En la BEAM, cada actor (proceso Erlang/Elixir, que ocupa apenas ≈ 2,6 KiB iniciales) posee su **propio montículo privado inmutable** y su propio recolector de basura independiente: no existe memoria mutable compartida ni pausas globales *Stop-The-World* que detengan a todo el nodo. La tolerancia a fallos se estructura mediante **OTP** (*Open Telecom Platform*) y la filosofía *«Let it crash»*: como ilustra la Figura \ref{fig:fig-otp-supervision-tree}, cuando un proceso trabajador (`GenServer`) encuentra un estado corrup-

to inesperado, muere limpiamente liberando su montículo instantáneamente y su `Supervisor` padre lo reinicia a un estado inicial conocido válido:

```
defmodule TelemetryCounter do
  use GenServer

  def start_link(initial \\ 0), do: GenServer.start_link(__MODULE__, initial, name: __MODULE__
    )
  def increment(delta),          do: GenServer.call(__MODULE__, {:inc, delta})

  @impl true
  def init(initial), do: {:ok, initial}

  @impl true
  def handle_call({:inc, delta}, _from, state) do
    new_state = state + delta
    {:reply, new_state, new_state}
  end
end
```

6.2 Monografías de Scala 3, Clojure y Common Lisp / Scheme (Bloques A–H): Tipos dependientes, HAMT y macros

Scala 3 (Martin Odersky, EPFL) unifica la programación orientada a objetos y funcional sobre el cálculo **DOT** (*Dependent Object Types*), reemplazando los antiguos `implicit` sobrecargados de Scala 2 por las abstracciones contextuales limpias `given` y `using`, tipos unión (`A | B`) e intersección (`A & B`) y enumeraciones algebraicas (`enum`). Por su parte, **Lisp** (John McCarthy, 1958; estandarizado en **Common Lisp** y **Scheme** R7RS) y **Clojure** (Rich Hickey, sobre la JVM) se distinguen por la **homoiconicidad** (*el código es datos*: el programa se escribe directamente como listas de símbolos o vectores que son el propio AST del lenguaje), lo que permite escribir macros sintácticas que transforman el lenguaje desde dentro. Clojure aporta además vectores y mapas persistentes inmutables implementados como árboles trie de factor de ramificación 32 (**HAMT**, *Hash Array Mapped Trie* con compartición estructural $O(\log_{32} N)$), donde producir una versión modificada de un vector de un millón de elementos copia a lo sumo $\lceil \log_{32}(10^6) \rceil = 4$ nodos internos.

Problema resuelto

Enunciado En el vector persistente inmutable de Clojure y Scala (`Vector`), cada nodo interno contiene 32 punteros. Para un vector con $N = 1048576$ elementos (32^4, profundidad $d = 4$ niveles), crear un nuevo vector con un elemento actualizado mediante **compartición estructural** (*path copying*) clona únicamente los $d = 4$ nodos de 32 referencias a lo largo del camino modificado, reutilizando todos los demás nodos. Calcula cuántas referencias de puntero se copian en esa actualización (4×32).

Datos

- $d = 4$ niveles — Profundidad del trie para $32^4 = 1048576$ elementos
- $B = 32$ punteros — Factor de ramificación por nodo

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Coste de actualización por Path Copying en vectores persistentes de ramificación 32

$$P_{\text{copiados}} = 4 \cdot 32$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 4 niveles modificados por las 32 ranuras de cada nodo

$$P_{\text{copiados}} = 4 \cdot 32$$

$$(4 \cdot 32)$$

Resultado

$$Resultado = 128$$

Interpretación Actualizar inmutablemente un vector de más de un millón de elementos copia únicamente 128 punteros, compartiendo el 99.99 % de la estructura.
Comprobación de los pasos

- Paso 1: 128 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) Un nodo de la máquina virtual BEAM ejecuta 25 000 procesos Elixir aislados, cada uno con un montículo inicial de 3 KiB. ¿Cuántos KiB suman en total esos 25 000 procesos?
2 [PRB-0034] (Básico, Numérico) En el planificador preventivo de la BEAM, cada proceso dispone de un presupuesto de `2000` reducciones (llamadas a función) antes de ceder la CPU a otro proceso. Si un proceso ejecuta 14 000 llamadas a función, ¿cuántas rodajas de planificación (*context switches*) consume?
3 [PRB-0035] (Intermedio, Desarrollo por pasos) En un árbol persistente de Clojure con factor de ramificación 32, ¿cuántos elementos caben en $d = 3$ niveles (32^3)?
4 [PRB-0036] (Intermedio, Diseño) Un supervisor OTP configurado con estrategia `:one_for_all` supervisa 6 procesos trabajadores y uno de ellos falla. ¿Cuántos procesos trabajadores reinicia el supervisor?

Parte III

Monografías de Lenguajes Científicos, Embebidos y Lógicos (Julia, R, Lua, Perl, Objective-C, Prolog, GDScript)

Capítulo 7

Monografías de Julia y R (Bloques A–H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos

*Este capítulo desarrolla las monografías completas (Bloques $A - H$) de los dos lenguajes especializados en cálculo numérico, simulación física y estadística avanzada: **Julia** y **R**.*

Consejo

En estructuras `struct` de Julia que van a participar en bucles numéricos intensivos, nunca declares campos con tipos abstractos (`x::Real` o `v::Vector{Number}`): parametriza siempre la estructura (`struct Particle{T<:Real} x::T end`) para que el compilador conozca el tamaño exacto del campo y lo aloje en línea sin punteros.

Definición

Estabilidad de Tipos (Type Stability en Julia): propiedad de un método según la cual el compilador puede inferir un único tipo concreto (no abstracto ni unión abierta `Any`) para su valor de retorno y todas sus variables internas a partir únicamente de los tipos de los argumentos de entrada.

Autoficha

¿Por qué en Julia la función `f(x) = x > 0 ? x : 0.0` es inestable en tipos cuando se llama con un `Int64` (porque una rama devuelve `Int64` y la otra `Float64`), y cómo la corrige `zero(x)`?

7.1 Monografía de Julia (Bloques A–H): Despacho múltiple, estabilidad de tipos, broadcasting fusionado y HPC

Creado en el MIT por Jeff Bezanson, Stefan Karpinski, Viral B. Shah y Alan Edelman, **Julia** fue diseñado para eliminar el «problema de los dos lenguajes» (prototipar en un lenguaje dinámico lento y reescribir en C/Fortran para producción). El núcleo semántico de Julia es el **despacho múltiple** (*multiple dispatch*): las funciones genéricas seleccionan el método más específico considerando simultáneamente el tipo concreto de **todos** sus argumentos, y cuando se invoca un método con una tupla de tipos concretos, el compilador JIT genera código máquina LLVM especializado para esa firma exacta. La regla de oro del rendimiento en Julia es la **estabilidad de tipos** (*type stability*, verificable con `@code_warntype`): el tipo del valor de retorno de una función debe depender exclusivamente de los tipos de sus argumentos y nunca de sus valores en tiempo de ejecución. Además, la sintaxis de difusión con punto `y .= a .* x .+ b` fusiona a nivel sintáctico múltiples operaciones elementales en un único bucle SIMD en el sitio sin alojar ningún array temporal:

```
abstract type DissipationLaw end
struct LinearDrag <: DissipationLaw
    coeff::Float64
end

# Despacho múltiple especializado y bucle fusionado sin alojamientos (0 bytes):
function apply_damping!(v::AbstractVector{Float64}, law::LinearDrag, dt::Float64)
    @. v = v * (1.0 - law.coeff * dt)
    return v
end
```

7.2 Monografía de R (Bloques A−H): Vectores, data frames, fórmulas estadísticas, Tidyverse/data.table y Quarto

Por su parte, **R** (Ross Ihaka y Robert Gentleman, heredero del lenguaje S de John Chambers en Bell Labs) es el estándar en bioestadística, econometría y ciencia de datos reproducible. En R no existen escalares aislados: el número `42` es un vector numérico de longitud 1, todas las operaciones aritméticas están vectorizadas en C/Fortran bajo el capó, los valores faltantes tipados `NA` (*Not Available*) son ciudadanos de primera clase y los objetos siguen semántica funcional **Copy-on-Modify**. Asimismo, R posee evaluación perezosa de argumentos mediante promesas (*Non-Standard Evaluation*, NSE) y un operador de fórmulas simbólicas (`y ~ x1 + x2 + x1:x2`) que construye matrices de diseño para regresión lineal (`lm`), modelos lineales generalizados (`glm`) y efectos mixtos.

Problema resuelto

Enunciado Una expresión vectorial `w = a .* x .+ b .* y .- c` contiene $k = 4$ operaciones binarias sobre vectores de $N = 1000000$ de elementos `Float64` (8 B cada uno). En un lenguaje sin fusión de expresiones, evaluar los 4 operadores materializaría 4 vectores temporales en memoria ($4 \times 10^6 \times 8$ B). En Julia, al escribir `@. w = a * x + b * y - c` sobre un vector preasignado `w`, la fusión sintáctica aloja 0 B. Calcula cuántos megabytes exactos (dividiendo entre 10^6) ahorra la fusión de Julia.
Datos

- $k = 4$ operaciones — Vectores temporales evitados
- $N = 1000000$ — Longitud de los vectores Float64
- $S_{f64} = 8\,\mathrm{B}$ — Tamaño de cada Float64

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de asignaciones mediante Loop Fusion de broadcasting en Julia

$$\Delta M_{\mathrm{MB}} = \frac{4 \cdot 10^6 \cdot 8}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos $4 * 10^6 * 8B$ y dividimos entre $10^6 B/MB$

$$\Delta M_{\mathrm{MB}} = \frac{4 \cdot 10^6 \cdot 8}{10^6}$$

$$\frac{((4 \cdot 1000000) \cdot 8)}{1000000}$$

Resultado

$$Resultado = 32$$

Interpretación La fusión sintáctica de Julia ahorra 32 MB de asignaciones temporales por evaluación y maximiza la localidad en caché L1/L2.
Comprobación de los pasos

- Paso 1: 32 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0039**] **(Básico, Numérico)** En Julia, si una función genérica tiene 4 métodos especializados para el primer argumento y 4 para el segundo (todas las combinaciones de pares), ¿cuántos métodos especializados forman la tabla de despacho (4×4)?

2 [**PRB-0040**] **(Básico, Numérico)** En R, un `data.frame` tiene 12 columnas numéricas (`double` de 8 B) y 50 000 filas. ¿Cuántos bytes suman los datos de las 12 columnas ($12 \times 50000 \times 8$)?

3 [**PRB-0041**] **(Intermedio, Desarrollo por pasos)** En un modelo lineal en R con intercepto y $p = 5$ predictores continuos más todas las interacciones de pares entre los 5 predictores ($5 \times 4/2 = 10$), ¿cuántos coeficientes β estima en total `lm()` ($1 + 5 + 10$)?

4 [**PRB-0042**] **(Intermedio, Diseño)** Una validación cruzada de $K = 10$ pliegues (*10-fold CV*) repetida 5 veces evalúa una rejilla de 8 hiperparámetros. ¿Cuántos ajustes del modelo ejecuta en total ($10 \times 5 \times 8$)?

Capítulo 8

Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A–H): Scripting Embebido, Unificación Lógica y Motores

*Este capítulo cierra las monografías de lenguajes de la Parte V completando los Bloques $A - H$ para **Lua**, **Perl**, **Objective-C**, **Prolog** y **GDScript**.*

Consejo

En GDScript (Godot 4), añade anotaciones de tipos estáticos explícitas (`var vel: Vector2`, `func calc(x: float) -> float:`) en todos los scripts de física e inteligencia artificial: permite a la máquina virtual de Godot emitir opcodes tipados directos evitando la resolución dinámica de variantes en cada fotograma.

Definición

Unificador General Máximo (Most General Unifier, MGU en Prolog): sustitución de variables θ tal que, dados dos términos lógicos s y t, los hace sintácticamente idénticos ($s\theta = t\theta$) y cualquier otro unificador σ de s y t es una instancia de θ ($\sigma = \theta\gamma$).

Autoficha

¿Qué diferencia fundamental existe entre la asignación destructiva `x = x + 1` de un lenguaje imperativo y la unificación `X = Y + 1` frente a la evaluación aritmética `X is Y + 1` en Prolog?

8.1 Monografías de Lua 5.4+, Perl 5 y Objective-C (Bloques A–H): Tablas/Metatablas, Regex y objc_msgSend

Diseñado en la PUC-Rio (Roberto Ierusalimschy, Waldemar Celes y Luiz Henrique de Figueiredo), **Lua** es el lenguaje de scripting embebido por excelencia en motores de videojuegos, Redis, Nginx/OpenResty y Neovim: todo su intérprete ANSI C ocupa menos de 300 KiB y se ejecuta sobre una máquina virtual basada en registros. La única estructura de datos compuesta de Lua es la **tabla**, implementada internamente como una estructura híbrida que combina una parte de array denso (indexada desde `1`) y una parte de tabla hash; asociando a una tabla una **metatabla** con metamétodos como `__index` y `__call`, se implementan sistemas de objetos, proxies y sobrecarga de operadores:

```
local Vector2D = {}
Vector2D.__index = Vector2D

function Vector2D.new(x, y)
    return setmetatable({ x = x or 0, y = y or 0 }, Vector2D)
end

function Vector2D:magnitude_sq()
    return self.x * self.x + self.y * self.y
end
```

En **Perl 5** (Larry Wall), todo módulo moderno comienza con `use v5.36; use strict; use warnings;` y distingue por sigilo e invocación entre contexto escalar (`$x`), lista (`@arr`) y tabla asociativa

(`%hash`), habiendo definido el estándar moderno de expresiones regulares (PCRE). En **Objective-C** (Brad Cox), las llamadas `[receiver message:arg]` no se resuelven por desplazamiento de tabla virtual fija como en C++, sino mediante el despachador dinámico en tiempo de ejecución `objc_msgSend(receiver, @selector(message:), arg)`, lo que permite enviar mensajes a `nil` de forma segura (no-op que devuelve cero) e introspeccionar selectores dinámicamente.

8.2 Monografías de Prolog y GDScript (Bloques A–H): Unificación lógica (SLD) y programación de motores de videojuegos

En el paradigma lógico declarativo, un programa en **Prolog** (Alain Colmerauer y Philippe Roussel, 1972; ISO Prolog / SWI-Prolog) consta de una base de conocimientos de **cláusulas de Horn**: hechos (`padre(juan, maria).`) y reglas (`ancestro(X, Y) :- padre(X, Z), ancestro(Z, Y).`). Ante una consulta, el motor ejecuta **resolución SLD** combinando el algoritmo de **unificación sintáctica de Robinson** (que calcula el unificador general máximo, MGU, vinculando variables lógicas en ambos sentidos) con búsqueda en profundidad y retroceso (*backtracking*) automático, además de aritmética con restricciones sobre dominios finitos (`use_module(library(clpfd))`). Finalmente, **GDScript** (el lenguaje específico de dominio del motor libre **Godot Engine 4.x**) ofrece tipado gradual estático (`var speed: float = 250.0`), referencias directas a nodos del árbol de escenas y comunicación desacoplada mediante el patrón Observador con señales tipadas (`signal health_changed(new_hp: int)`), separando estrictamente el bucle visual (`_process(delta)`) del paso fijo del motor de física (`_physics_process(delta)`).

Problema resuelto

Enunciado En la implementación C de las tablas híbridas de Lua 5.4, cuando una tabla almacena claves enteras consecutivas `1..N`, se alojan en la parte array (`TValue` de 16 B por elemento: 8 B de valor más 1 B de etiqueta alineado), mientras que en la parte hash cada nodo (`Node`) ocupa 32 B (clave + valor + enlace `next`). Calcula cuántos bytes ahorra una tabla Lua de $N = 4000$ elementos al indexarse secuencialmente desde `1` hasta `4000` (parte array de 16 B) frente a usar claves dispersas en la parte hash (32 B por nodo).

Datos

- $N = 4000$ — Elementos en la tabla Lua
- $S_{hash} = 32\,\text{B}$ — Tamaño por nodo en la parte hash
- $S_{array} = 16\,\text{B}$ — Tamaño por TValue en la parte array

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria de la parte Array frente a la parte Hash en tablas híbridas de Lua 5.4

$$\Delta M = 4000 \cdot (32 - 16)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 4000 elementos por los 16 B ahorrados al residir en la parte array

$$\Delta M = 4000 \cdot (32 - 16)$$

$$(4000 \cdot (32 - 16))$$

Resultado

$$\textit{Resultado} = 64000$$

Interpretación Usar índices enteros consecutivos desde 1 ahorra 64 000 bytes (la mitad exacta de memoria) y evita el cálculo de hashes.

Comprobación de los pasos

- Paso 1: 64000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0045**] **(Básico, Numérico)** En el ejemplo de Lua `Vector2D.new(6, 8):magnitude_sq()`, ¿qué valor devuelve $6^2 + 8^2$?

2 [**PRB-0046**] **(Básico, Numérico)** En Godot Engine (GDScript), el bucle `_physics_process(delta)` se ejecuta por defecto a una frecuencia fija de 60 tics por segundo. ¿Cuántos pasos de física ejecuta exactamente en 15 segundos (60×15)?

3 [**PRB-0047**] **(Intermedio, Desarrollo por pasos)** Un predicado no determinista en Prolog tiene 3 cláusulas que se ramifican en profundidad $d = 4$ niveles (3^4 hojas en el árbol de búsqueda SLD). Si el operador de corte `!` poda 2 de las 3 ramas en la raíz, ¿cuántas hojas se ahorra explorar ($3^4 - 3^3$)?

4 [**PRB-0048**] **(Intermedio, Diseño)** En la API de pila en C de Lua (`lua_State`), una función nativa empuja 5 argumentos a la pila, invoca `lua_pcall(L, 4, 2, 0)` (que consume la función más los 4 argumentos y empuja 2 resultados). Si antes de empezar había 3 elementos en la pila, ¿cuántos elementos hay en la pila al terminar ($3 + 2$)?

Apéndice A

Notación

Símbolo	Significado	Unidades
$\sigma_p(R), \pi_L(R), R \bowtie S$	Operadores de selección, proyección y reunión natural del álgebra relacional	relación
$m \gg= f : M\langle B \rangle$	Operador bind (»=) de una mónada en programación funcional	tipo
w OVER (P, O)	Marco de evaluación de una función de ventana en SQL estándar	ventana

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de SQL (Bloques A–H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización

Problema resuelto [PRB-0001]
En el proyecto `projects/sql-transactional-ledger`, la cuenta de Caja (`account_id = 110`) registra tres apuntes sucesivos `(debit_cents, credit_cents)`: `(500000, 0)`, `(125000, 0)` y `(0, 40000)`. Calcula el saldo acumulado en céntimos (`running_balance_cents`) que devuelve la función de ventana `SUM(debit_cents - credit_cents) OVER (...)` en la tercera fila.
Respuesta

$$\textit{Resultado} = 585000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
Ante las puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `RANK()` al cuarto elemento (`80`) tras el empate en la posición 2?
Respuesta 4
Ejercicio 2 [PRB-0004]
Ante las mismas puntuaciones `[100, 90, 90, 80]`, ¿qué valor entero asigna `DENSE_RANK()` al cuarto elemento (`80`)?
Respuesta 3
Ejercicio 3 [PRB-0005]
Un producto cartesiano `CROSS JOIN` entre una tabla de 120 tiendas y una tabla de 365 días del año genera todas las combinaciones `(tienda, día)`. ¿Cuántas filas produce?
Respuesta 43800
Ejercicio 4 [PRB-0006]
Una consulta con paginación `LIMIT 50 OFFSET 400` devuelve la página 9 de resultados. ¿Cuál es el número de orden (base 1) de la primera fila devuelta en esa página ($400 + 1$)?
Respuesta 401

Cálculos reproducidos

$(((500000 - 0) + (125000 - 0)) + (0 - 40000)) = 585000$

C.2 Monografía de Bash y POSIX Shell (Bloques A–H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos

Problema resuelto [PRB-0007]
Un contenedor en un pipeline de despliegue termina abruptamente con código de salida `$? = 137` tras intervenir el *OOM Killer* del kernel Linux (`SIGKILL = 9`), y otro proceso aborta por violación de segmento (`SIGSEGV = 11`, código $128 + 11 = 139$). Calcula la suma de los números de las dos señales POSIX subyacentes obtenidas restando 128 a cada código de salida $((137 - 128) + (139 - 128))$.
Respuesta

$$\textit{Resultado} = 20$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0009]
Un script en ejecución es interrumpido por el usuario pulsando `Ctrl+C` (`SIGINT = 2`). ¿Qué código de salida `$?` reporta el shell ($128 + 2$)?
Respuesta 130

Ejercicio 2 [PRB-0010]
Un proceso recibe la señal de terminación ordenada `SIGTERM = 15` sin capturarla. ¿Cuál es su código de salida ($128 + 15$)?
Respuesta 143

Ejercicio 3 [PRB-0011]
Un comando `xargs -0 -n 50 -P 4` procesa una lista de 600 ficheros pasándolos en lotes de 50 ficheros por invocación. ¿Cuántos procesos hijos lanza en total `xargs` ($600 / 50$)?
Respuesta 12

Ejercicio 4 [PRB-0012]
Una máscara `umask 027` se resta de los permisos base `666` (octal) para la creación de ficheros regulares ($666 - 27$ en dígito a dígito: $6-0=6$, $6-2=4$, $6-7\to 0$). ¿Cuál es el permiso octal resultante expresado como entero de tres dígitos?
Respuesta 640

Cálculos reproducidos

$$((137-128)+(139-128))=20$$

C.3 Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A–H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma

Problema resuelto [PRB-0013]
Un servidor de producción con 8192 MiB de RAM reserva 2048 MiB para el sistema operativo y el búfer de OPcache/Redis, dejando $M_{\text{avail}} = 6144$ MiB para el pool de trabajadores PHP-FPM. Si cada trabajador consume en carga punta $m_{\text{worker}} = 48$ MiB de memoria privada, calcula el valor óptimo de `pm.max_children` ($6144 / 48$).
Respuesta

$$\mathit{Resultado} = 128$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]
En Flutter/Dart, para sostener una tasa de refresco de pantalla de 120 Hz, el presupuesto total por fotograma (construcción + diseño + pintado) es 1000/120 ms. ¿Cuántos milisegundos caben en 12 fotogramas consecutivos?
Respuesta 100

Ejercicio 2 [PRB-0016]
En Ruby, la expresión `(1..10).select(&:even?).map { |x| x * 3 }.sum` suma el triple de los pares del 2 al 10 ($(2+4+6+8+10)\times 3$). ¿Cuál es el resultado?
Respuesta 90

Ejercicio 3 [PRB-0017]
Si OPcache evita recompilar 85 ficheros PHP por petición HTTP (ahorrando 0,4 ms por fichero), ¿cuántos milisegundos de CPU ahorra en cada petición ($85 \times 0{,}4$)?
Respuesta 34

Ejercicio 4 [PRB-0018]
Un Isolate de fondo en Dart procesa un lote de 400 imágenes en grupos de 25 imágenes devolviendo el progreso al Isolate principal tras cada grupo. ¿Cuántos mensajes de progreso envía?
Respuesta 16

Cálculos reproducidos

$\frac{6144}{48} = 128$

C.4 Monografía de Haskell (Bloques A–H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas

Problema resuelto [PRB-0019]
En GHC de 64 bits, cada celda `(:)` de una lista enlazada `[Int]` en el montículo ocupa 3 palabras de 8 B (24 B por nodo). Una tubería `sum (map (*2) (map (+1) [1..N]))` para $N = 100000$ elementos, si se evaluara de forma estricta sin fusión, alojaría 3 listas de 100000 celdas ($3 \times 100000 \times 24$ B). Gracias a la fusión `foldr/build` de GHC con `-O2`, las tres listas desaparecen y el bucle se ejecuta en registros alojando 0 B de celdas. Calcula cuántos kilobytes (dividiendo entre 1000) ahorra la desforestación.
Respuesta

$$Resultado = 7200$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
En el evaluador monádico `eval (Div (Add (Lit 20) (Lit 22)) (Lit 6))`, ¿qué valor entero `Right v` devuelve la expresión $((20 + 22)/6)$?
Respuesta 7
Ejercicio 2 [PRB-0022]
`QuickCheck` ejecuta por defecto 100 casos generados aleatoriamente por cada propiedad `prop_*`. Si un módulo define 18 propiedades, ¿cuántos casos de prueba verifica `quickCheck` en total?
Respuesta 1800
Ejercicio 3 [PRB-0023]
En la expresión aplicativa `[(+10), (*2), (+100)] <*> [1, 2, 3, 4]`, la instancia `Applicative` de listas aplica cada una de las 3 funciones a cada uno de los 4 valores. ¿Cuántos elementos contiene la lista resultante?
Respuesta 12
Ejercicio 4 [PRB-0024]
Un `newtype` en Haskell envuelve un `Int64` (8 B). ¿Cuántos bytes de sobrecoste adicional añade el constructor del `newtype` en tiempo de ejecución tras el borrado de GHC?
Respuesta 0

Cálculos reproducidos

$\frac{((3 \cdot 100000) \cdot 24)}{1000} = 7200$

C.5 Monografías de OCaml 5.x y F# (.NET) (Bloques A–H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación

Problema resuelto [PRB-0025]
En el compilador nativo `ocamlopt` de 64 bits, cualquier entero `int` n se almacena en un registro de CPU mediante la representación etiquetada $R(n) = 2n + 1$ (con el bit 0 puesto a 1 para indicar al recolector de basura que no es un puntero). Calcula el valor decimal exacto de la palabra en registro $R(n)$ para el entero de aplicación $n = 1500$.
Respuesta

$$Resultado = 3001$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0027]
En OCaml, si un registro contiene la palabra etiquetada $R(n) = 85$, ¿cuál es el entero lógico $n = (85-1)/2$?
Respuesta 42

Ejercicio 2 [PRB-0028]
En F#, la función `kineticEnergy` calcula $0{,}5 \cdot m \cdot v^2$. Para una masa $m = 4$ kg y velocidad $v = 15$ m/s, ¿cuántos julios ($\mathrm{kg \cdot m^2/s^2}$) devuelve?
Respuesta 450

Ejercicio 3 [PRB-0029]
¿Cuántos bytes adicionales ocupa en el ensamblado `.dll` de .NET la anotación de unidades `[<Measure>] type kg` tras la compilación de F#?
Respuesta 0

Ejercicio 4 [PRB-0030]
En OCaml de 64 bits, cada bloque en el montículo tiene 1 palabra de cabecera (8 B) que codifica tamaño, color GC y etiqueta de constructor, seguida de k campos de 8 B. ¿Cuántos bytes ocupa un nodo de árbol binario con 3 campos (`valor`, `izq`, `der`)?
Respuesta 32

Cálculos reproducidos

$((2 \cdot 1500) + 1) = 3001$

C.6 Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A–H)

Problema resuelto [PRB-0031]
En el vector persistente inmutable de Clojure y Scala (`Vector`), cada nodo interno contiene 32 punteros. Para un vector con $N = 1048576$ elementos (32^4, profundidad $d = 4$ niveles), crear un nuevo vector con un elemento actualizado mediante **compartición estructural** (*path copying*) clona únicamente los $d = 4$ nodos de 32 referencias a lo largo del camino modificado, reutilizando todos los demás nodos. Calcula cuántas referencias de puntero se copian en esa actualización (4×32).
Respuesta

$$\mathit{Resultado} = 128$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0033]
Un nodo de la máquina virtual BEAM ejecuta 25 000 procesos Elixir aislados, cada uno con un montículo inicial de 3 KiB. ¿Cuántos KiB suman en total esos 25 000 procesos?
Respuesta 75000

Ejercicio 2 [PRB-0034]
En el planificador preventivo de la BEAM, cada proceso dispone de un presupuesto de `2000` reducciones (llamadas a función) antes de ceder la CPU a otro proceso. Si un proceso ejecuta 14 000 llamadas a función, ¿cuántas rodajas de planificación (*context switches*) consume?
Respuesta 7

Ejercicio 3 [PRB-0035]
En un árbol persistente de Clojure con factor de ramificación 32, ¿cuántos elementos caben en $d = 3$ niveles (32^3)?
Respuesta 32768

Ejercicio 4 [PRB-0036]
Un supervisor OTP configurado con estrategia `:one_for_all` supervisa 6 procesos trabajadores y uno de ellos falla. ¿Cuántos procesos trabajadores reinicia el supervisor?
Respuesta 6

Cálculos reproducidos

$(4 \cdot 32) = 128$

C.7 Monografías de Julia y R (Bloques A–H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos

Problema resuelto [PRB-0037]
Una expresión vectorial `w = a .* x .+ b .* y .- c` contiene $k = 4$ operaciones binarias sobre vectores de $N = 1000000$ de elementos `Float64` (8 B cada uno). En un lenguaje sin fusión de expresiones, evaluar los 4 operadores materializaría 4 vectores temporales en memoria ($4 \times 10^6 \times 8$ B). En Julia, al escribir `@. w = a * x + b * y - c` sobre un vector preasignado `w`, la fusión sintáctica aloja 0 B. Calcula cuántos megabytes exactos (dividiendo entre 10^6) ahorra la fusión de Julia.
Respuesta

$$Resultado = 32$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0039]
En Julia, si una función genérica tiene 4 métodos especializados para el primer argumento y 4 para el segundo (todas las combinaciones de pares), ¿cuántos métodos especializados forman la tabla de despacho (4×4)?
Respuesta 16
Ejercicio 2 [PRB-0040]
En R, un `data.frame` tiene 12 columnas numéricas (`double` de 8 B) y 50 000 filas. ¿Cuántos bytes suman los datos de las 12 columnas $(12 \times 50000 \times 8)$?
Respuesta 4800000
Ejercicio 3 [PRB-0041]
En un modelo lineal en R con intercepto y $p = 5$ predictores continuos más todas las interacciones de pares entre los 5 predictores $(5 \times 4/2 = 10)$, ¿cuántos coeficientes β estima en total `lm()` ($1 + 5 + 10$)?
Respuesta 16
Ejercicio 4 [PRB-0042]
Una validación cruzada de $K = 10$ pliegues (*10-fold CV*) repetida 5 veces evalúa una rejilla de 8 hiperparámetros. ¿Cuántos ajustes del modelo ejecuta en total $(10 \times 5 \times 8)$?
Respuesta 400

Cálculos reproducidos

$\frac{((4 \cdot 1000000) \cdot 8)}{1000000} = 32$

C.8 Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A–H): Scripting Embebido, Unificación Lógica y Motores

Problema resuelto [PRB-0043]
En la implementación C de las tablas híbridas de Lua 5.4, cuando una tabla almacena claves enteras consecutivas `1..N`, se alojan en la parte array (`TValue` de 16 B por elemento: 8 B de valor más 1 B de etiqueta alineado), mientras que en la parte hash cada nodo (`Node`) ocupa 32 B (clave + valor + enlace `next`). Calcula cuántos bytes ahorra una tabla Lua de $N = 4000$ elementos al indexarse secuencialmente desde `1` hasta `4000` (parte array de 16 B) frente a usar claves dispersas en la parte hash (32 B por nodo).
Respuesta

$$Resultado = 64000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0045]
En el ejemplo de Lua `Vector2D.new(6, 8):magnitude_sq()`, ¿qué valor devuelve $6^2 + 8^2$?
Respuesta 100

Ejercicio 2 [PRB-0046]
En Godot Engine (GDScript), el bucle `_physics_process(delta)` se ejecuta por defecto a una frecuencia fija de 60 tics por segundo. ¿Cuántos pasos de física ejecuta exactamente en 15 segundos (60×15)?
Respuesta 900

Ejercicio 3 [PRB-0047]
Un predicado no determinista en Prolog tiene 3 cláusulas que se ramifican en profundidad $d = 4$ niveles (3^4 hojas en el árbol de búsqueda SLD). Si el operador de corte `!` poda 2 de las 3 ramas en la raíz, ¿cuántas hojas se ahorra explorar ($3^4 - 3^3$)?
Respuesta 54

Ejercicio 4 [PRB-0048]
En la API de pila en C de Lua (`lua_State`), una función nativa empuja 5 argumentos a la pila, invoca `lua_pcall(L, 4, 2, 0)` (que consume la función más los 4 argumentos y empuja 2 resultados). Si antes de empezar había 3 elementos en la pila, ¿cuántos elementos hay en la pila al terminar ($3 + 2$)?
Respuesta 5

Cálculos reproducidos

$(4000 \cdot (32 - 16)) = 64000$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 36 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios. Ecuaciones: 1 respaldada por una fuente.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios. Ecuaciones: 1 reproducida numéricamente, 1 valor o condición.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶47: dos testimonios.
¶48: dos testimonios.

Referencia Normativa src-sql-2023 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
Referencia Normativa src-codd-1970 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
Referencia Normativa src-posix-2017 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-bryant-ohallaron (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-ecmascript-2024 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-unicode-15 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-knuth-taocp (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-csharp-spec (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
Referencia Normativa src-iso-c17 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-monografia-sql-ddl-dml-cte-ventanas-optimizacion scientific
REV-0002: cap02-monografia-bash-posix-shell-automatizacion-sistemas scientific
REV-0003: cap03-monografias-php-ruby-dart-web-movil scientific
REV-0004: cap04-monografia-haskell-funcional-puro-tipos-monadas scientific
REV-0005: cap05-monografias-ocaml-fsharp-ml-sistemas-tipos scientific
REV-0006: cap06-monografias-beam-elixir-erlang-scala3-clojure-lisp scientific
REV-0007: cap07-monografias-julia-r-computacion-cientifica-estadistica scientific
REV-0008: cap08-monografias-lua-perl-objc-prolog-gdscript scientific

Apéndice E

Plan de estudio

Monografías de Datos, Automatización de Sistemas y Desarrollo Web/Móvil

✓Monografía de SQL (Bloques A−H): Álgebra Relacional, DDL/DML, Lógica Trivaluada, CTEs Recursivas, Funciones de Ventana y Optimización

- Comprender el fundamento relacional de SQL (E. F. Codd, 1970; estándar ISO/IEC 9075:2023) y el orden lógico exacto de evaluación de una consulta (‘FROM -> WHERE -> GROUP BY -> HAVING -> WINDOW -> SELECT -> DISTINCT -> ORDER BY -> LIMIT‘).
- Dominar la lógica trivaluada de ‘NULL‘ (‘TRUE‘, ‘FALSE‘, ‘UNKNOWN‘), todos los tipos de ‘JOIN‘ (incluso ‘LATERAL‘), subconsultas correlacionadas, CTEs recursivas (‘WITH RECURSIVE‘) y funciones de ventana (‘ROW_NUMBER‘, ‘RANK‘, ‘LAG‘, ‘SUM() OVER (...)‘).
- Interpretar planes de ejecución (‘EXPLAIN ANALYZE‘), diseñar índices compuestos sargables y verificar el libro mayor financiero ‘projects/sql-transactional-ledger‘.

2 conceptos planificados.

✓Monografía de Bash y POSIX Shell (Bloques A−H): Tuberías, Redirecciones, Expansiones Seguras, Trap, Herramientas UNIX y Proyectos

- Distinguir con rigor entre el estándar portable POSIX ‘sh‘ (IEEE Std 1003.1) y las extensiones de GNU Bash (‘[[...]]‘, arrays indexados/asociativos, sustitución de procesos ‘<(...)‘).
- Dominar los descriptores de fichero (‘0‘ stdin, ‘1‘ stdout, ‘2‘ stderr), tuberías (‘|‘), códigos de salida (‘$¿), entrecomillado seguro y expansiones de parámetros (‘${var:-def}‘, ‘${var:?err}‘).
- Escribir scripts de producción robustos con modo estricto (‘set -euo pipefail‘), limpieza atómica mediante ‘trap ... EXIT‘, integración con ‘find‘/‘xargs -0‘/‘jq‘/‘awk‘ y verificación con ‘shellcheck‘.

2 conceptos planificados.

✓Monografías de PHP 8.3+, Ruby 3.3+ y Dart 3.x (Bloques A−H): Motores Zend, YJIT y Compilación AOT/JIT Multiplataforma

- Dominar PHP moderno (8.3+): tipado estricto (‘declare(strict_types=1)‘), tipos de unión/intersección, ‘readonly class‘, ‘enum‘ respaldados, Fibers, OPcache/JIT, Composer, estándares PSR y Laravel/Symfony.
- Dominar Ruby (3.3+): modelo de objetos puro, bloques/‘Proc‘/lambdas, mixins con ‘Module‘, metaprogramación, compilador YJIT, concurrencia con Fibers/Ractors y ecosistema Bundler/RSpec/Rails.
- Dominar Dart (3.x): *sound null-safety*, modificadores de clase (‘sealed‘, ‘final‘, ‘interface‘), coincidencia de patrones, concurrencia por memoria aislada (*Isolates*) y compilación JIT/AOT para Flutter.

2 conceptos planificados.

Monografías de Lenguajes Funcionales Estáticos y Concurrentes (Haskell, OCaml, F#, Scala, BEAM, Lisp)

✓Monografía de Haskell (Bloques A−H): Pureza, Evaluación Perezosa, Inferencia Hindley-Milner, Typeclasses y Mónadas

- Comprender la transparencia referencial, la evaluación no estricta (perezosa por necesidad, *call-by-need* con *thunks*), la currificación universal y el sistema de tipos Hindley-Milner de GHC.
- Diseñar tipos de datos algebraicos (‘data‘, ‘newtype‘), sinónimos de tipos, polimorfismo de clases de tipos (‘Eq‘, ‘Ord‘, ‘Monoid‘) y tipos de orden superior (*Higher-Kinded Types*, ‘* -> *‘).
- Dominar la jerarquía ‘Functor‘ (‘fmap‘), ‘Applicative‘ (‘<*>‘) y ‘Monad‘ (‘»=‘, notación ‘do‘), el aislamiento puro de efectos con ‘IO‘, ‘STM‘ (*Software Transactional Memory*) y pruebas de propiedades con ‘QuickCheck‘.

2 conceptos planificados.

✓Monografías de OCaml 5.x y F# (.NET) (Bloques A−H): Módulos de Orden Superior (Functors), Efectos Algebraicos y Expresiones de Computación

- Dominar la familia ML (Robin Milner): evaluación estricta por valor con inferencia Hindley-Milner completa, tipos algebraicos, coincidencia de patrones y mutabilidad explícita (‘ref‘, ‘mutable‘).
- En OCaml 5.x: programar módulos parametrizados de orden superior (*Functors*), variantes polimórficas, paralelismo nativo con *Domains* y concurrencia mediante manejadores de efectos algebraicos (*Effect Handlers*).
- En F#: dominar el operador tubería (‘|>‘), unidades de medida estáticas (‘[<Measure>]‘), proveedores de tipos y expresiones de computación (‘async { ... }‘, ‘seq { ... }‘).

2 conceptos planificados.

✓Monografías de Elixir, Erlang/OTP (BEAM), Scala 3, Clojure y Lisp/Scheme (Bloques A−H)

- Comprender la máquina virtual BEAM (Erlang/Elixir): aislamiento total de memoria por proceso, recolección de basura por proceso, paso de mensajes asíncrono y árboles de supervisión OTP (*let it crash*).
- Dominar Elixir (tuberías ‘|>‘, protocolos, macros higiénicas, ‘GenServer‘, ‘Supervisor‘, ‘Phoenix LiveView‘) y Erlang/OTP (coincidencia de patrones binarios, actualización de código en caliente).
- Dominar Scala 3 (cálculo DOT, ‘enum‘, ‘given‘/‘using‘, tipos unión/intersección, ZIO/Cats Effect), Clojure (estructuras persistentes HAMT, átomos/STM, REPL) y Lisp/Scheme (homoiconicidad, S-expressions y macros ‘defmacro‘/‘syntax-rules‘).

2 conceptos planificados.

Monografías de Lenguajes Científicos, Embebidos y Lógicos (Julia, R, Lua, Perl, Objective-C, Prolog, GDScript)

✓Monografías de Julia y R (Bloques A−H): Despacho Múltiple, Especialización JIT LLVM, Vectorización y Ciencia de Datos

- Comprender cómo Julia resuelve el «problema de los dos lenguajes» mediante despacho múltiple (*multiple dispatch*), inferencia de estabilidad de tipos y compilación JIT sobre LLVM.
- Dominar en Julia los tipos paramétricos abstractos y concretos, la difusión vectorizada fusionada con punto (‘f.(x) .+ g.(y)‘), macros metaprogramáticas y álgebra lineal/ecuaciones diferenciales (‘DifferentialEquations.jl‘).

- Dominar R (4.x): vectores atómicos con semántica Copy-on-Modify, factores, matrices, ‘data.frame‘ / ‘tibble‘ / ‘data.table‘, programación funcional (‘purrr‘), modelado estadístico (‘y ~ x1 + x2‘) y visualización con la Gramática de Gráficos (‘ggplot2‘).

2 conceptos planificados.

✓Monografías de Lua 5.4+, Perl 5, Objective-C, Prolog y GDScript (Bloques A−H): Scripting Embebido, Unificación Lógica y Motores

- Dominar Lua (5.4+ / LuaJIT): máquina virtual basada en registros, tablas híbridas (array + hash), metatablas (‘__index‘, ‘__newindex‘, ‘__add‘), corrutinas asimétricas y la API de pila en C (‘lua_State*‘).
- Comprender Perl 5 (contexto escalar vs lista, expresiones regulares PCRE, CPAN, ‘use strict; use warnings;‘) y Objective-C (paso de mensajes dinámico Smalltalk ‘objc_msgSend‘, categorías y puente con Swift/C).
- Dominar Prolog (cláusulas de Horn, resolución SLD, unificación sintáctica de Robinson, retroceso y corte ‘¡) y GDScript (árbol de escenas de Godot, señales desacopladas y procesado por fotograma ‘_process‘/‘_physics_process‘).

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-sql-2023. https://doc.rust-lang.org/reference/src-sql-2023.html

[27] Referencia Normativa src-codd-1970. https://doc.rust-lang.org/reference/src-codd-1970.html

[28] Referencia Normativa src-posix-2017. https://doc.rust-lang.org/reference/src-posix-2017.html

[29] Referencia Normativa src-bryant-ohallaron. https://doc.rust-lang.org/reference/src-bryant-ohallaron.html

[30] Referencia Normativa src-ecmascript-2024. https://doc.rust-lang.org/reference/src-ecmascript-2024.html

[31] Referencia Normativa src-unicode-15. https://doc.rust-lang.org/reference/src-unicode-15.html

[32] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html

[33] Referencia Normativa src-knuth-taocp. https://doc.rust-lang.org/reference/src-knuth-taocp.html

[34] Referencia Normativa src-csharp-spec. https://doc.rust-lang.org/reference/src-csharp-spec.html

[35] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

[36] Referencia Normativa src-iso-c17. https://doc.rust-lang.org/reference/src-iso-c17.html

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen VIII: Desarrollo Web, Formatos de Datos, Bases de Datos y Frameworks

Enciclopedia de Programación e Ingeniería del Software — Partes VI, VII y VIII

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Partes VI−VIII: Web, Almacenamiento y Ecosistemas de Producción 1

1. HTML Living Standard y Accesibilidad Web (WCAG / ARIA) 2
1.1. Naturaleza declarativa de HTML y el estándar vivo WHATWG y Semántica estructural, árbol de accesibilidad (AOM) y WAI-ARIA . . . 2
1.2. Formularios nativos, validación de restricciones y seguridad en enlaces y Carga eficiente de recursos, módulos ES y Web Components . . . 3

2. CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries 5
2.1. Algoritmo de la Cascada, Capas (@layer) y Especificidad Lexicográfica y Modelo de Caja, Box-Sizing y Propiedades Lógicas Internacionalizadas . . . 5
2.2. Contextos de Formato: Flexbox Unidimensional y CSS Grid Bidimensional y Container Queries, Rendimiento de Animaciones y Compositor GPU . . . 6

3. Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf 8
3.1. JSON (RFC 8259) y Validación Contractual con JSON Schema y YAML 1.2 vs TOML v1.0: Configuración Humana y Trampas Semánticas . . . 8
3.2. XML, Namespaces, XSD y Prevención de Ataques XXE y CSV (RFC 4180), Markdown (CommonMark) y Serialización Binaria con Protocol Buffers . . . 9

4. Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF) 11
4.1. El Modelo Relacional de Codd y las Claves Relacionales y Dependencias Funcionales, Axiomas de Armstrong y Cierre de Atributos . . . 11
4.2. Anomalías de Diseño y Jerarquía de Formas Normales (1NF, 2NF, 3NF y BCNF) y Descomposición sin Pérdida (Lossless-Join) y Preservación de Dependencias . . . 11

5. Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC 14
5.1. Árboles B+ (B+-Trees) frente a LSM-Trees (Log-Structured Merge-Trees) y Diseño de Índices Compuestos, Índices Cubrientes (Covering) y Optimizadores de Costes . . . 14
5.2. Transacciones ACID, Write-Ahead Logging (WAL) y Protocolo ARIES y Niveles de Aislamiento, Anomalías de Concurrencia y MVCC (Snapshot Isolation) . . . 15

6. Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis 17
6.1. Teorema CAP y Clasificación PACELC en Almacenamiento Distribuido y Replicación Líder-Seguidor, Quórums de Lectura/Escritura y Sharding . . . 17
6.2. PostgreSQL y SQLite en Producción: De Sistemas Embebidos a Escala Empresarial y Motores NoSQL Especializados: Documentos (MongoDB) y Memoria Clave-Valor (Redis) 17

7. Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js 20
7.1. Del DOM Imperativo a la UI Declarativa: Función Estado -> Vista y React y Next.js: Reconciliación Fiber, Hooks, Server Components e Hidratación . . . 20
7.2. Angular, Vue y Svelte: Inyección de Dependencias, Proxies Reactivos y Compilación y Gestión de Estado: Estado Local, Estado Global y Caché de Estado Remoto . . . 20

8. Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter **22**
8.1. Arquitectura de Frameworks Backend: Middleware, Inyección de Dependencias y Ciclo HTTP y Comparativa de Ecosistemas Backend: Productividad, Tipado y Rendimiento Extremo . . . 22
8.2. Aplicaciones de Escritorio: Qt (C++), .NET MAUI, Electron y Tauri (Rust + WebView) y Desarrollo Móvil: SwiftUI (iOS), Jetpack Compose (Android), Flutter y React Native . 22

A. Notación **25**

B. Hoja de fórmulas **26**

C. Soluciones **27**
C.1. HTML Living Standard y Accesibilidad Web (WCAG / ARIA) . . . 27
C.2. CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries . . . 27
C.3. Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf . . . 28
C.4. Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF) . . . 28
C.5. Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC . . . 29
C.6. Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis . . . 30
C.7. Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js . . . 30
C.8. Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter . . . 31

D. Auditoría de fuentes **32**

E. Plan de estudio **34**

Parte I

Partes VI–VIII: Web, Almacenamiento y Ecosistemas de Producción

Capítulo 1

HTML Living Standard y Accesibilidad Web (WCAG / ARIA)

Estudio riguroso de HTML como lenguaje de marcado declarativo (no de programación de propósito general), su gramática, ciclo de parseo en el navegador y validación accesible.

Consejo

La API de validación de restricciones de HTML5 verifica tipos, patrones regulares y rangos antes del envío HTTP, pero jamás sustituye la validación en el servidor.

Definición

Naturaleza declarativa de HTML y el estándar vivo WHATWG: HTML es un lenguaje formal de marcado estructural e hipertextual, no un lenguaje de programación Turing-completo.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Los atributos async, defer y type='module' desacoplan la descarga de scripts del bloqueo del parser HTML.

1.1 Naturaleza declarativa de HTML y el estándar vivo WHATWG y Semántica estructural, árbol de accesibilidad (AOM) y WAI-ARIA

A diferencia de un lenguaje de programación imperativo o funcional, HTML (HyperText Markup Language, mantenido como Living Standard por WHATWG) describe de manera declarativa la estructura semántica de un documento mediante elementos delimitados por etiquetas, atributos clave-valor y referencias de entidades. Carece de variables mutables, bucles condicionales o abstracción funcional general. El analizador sintáctico de HTML5 implementa una máquina de estados finitos con reglas deterministas de recuperación de errores: incluso ante etiquetas mal anidadas, construye el mismo árbol DOM en todos los motores conformes (Blink, WebKit, Gecko).

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Panel de Telemetría Industrial</title>
</head>
<body>
  <header role="banner">
    <h1>Estado del Clúster Principal</h1>
  </header>
  <main id="contenido-principal">
    <article aria-labelledby="sec-latencia">
      <h2 id="sec-latencia">Métricas de Latencia p99</h2>
      <p>Latencia actual: <data value="4.2">4,2 ms</data></p>
    </article>
  </main>
</body>
</html>
```

El navegador construye en paralelo al DOM el Árbol de Accesibilidad (Accessibility Object Model), consultado por lectores de pantalla mediante APIs nativas (IAccessible2, UIAutomation, NSAccessibility). La primera regla de WAI-ARIA establece que siempre debe preferirse un elemento HTML nativo (`<button>`, `<nav>`, `<main>`, `<dialog>`) sobre un `<div>` genérico con `role="button"`, ya que el elemento nativo proporciona automáticamente navegación por teclado (`Tab`, `Enter`, `Space`), gestión de foco y estados implícitos. Como se ilustra en la figura 1, el DOM alimenta tanto el Render Tree visual como la estructura semántica.

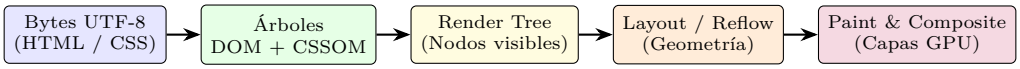


Figura 1. Tubería de renderizado del navegador (Critical Rendering Path): desde el flujo de bytes UTF-8 hasta el árbol DOM, CSSOM, Render Tree, Layout y Composite.

1.2 Formularios nativos, validación de restricciones y seguridad en enlaces y Carga eficiente de recursos, módulos ES y Web Components

Los controles de formulario (`<input>`, `<select>`, `<textarea>`) asociados explícitamente mediante `<label for="id">` exponen validación declarativa (`required`, `type="email"`, `min`, `max`, `pattern`). No obstante, dado que cualquier cliente puede falsificar peticiones HTTP ignorando el navegador, toda validación en el cliente es estrictamente una mejora de experiencia de usuario (UX) y el servidor debe revalidar íntegramente la carga útil. Asimismo, los enlaces externos con `target="_blank"` deben incluir `rel="noopener noreferrer"` para aislar el contexto de navegación y evitar ataques de tabnabbing inverso. Cuando el parser HTML encuentra una etiqueta `<script>` clásica sin atributos, detiene la construcción del DOM hasta descargar y ejecutar el script. El atributo `defer` descarga el script en paralelo y pospone su ejecución en orden hasta que el DOM está completo (`DOMContentLoaded`), mientras que `type="module"` activa el sistema de módulos ECMAScript en modo estricto con diferimiento implícito. Adicionalmente, la especificación HTML incluye `<template>` y `<slot>` como base de Web Components y Shadow DOM para encapsular estructura y estilo.

Problema resuelto

Enunciado Una página HTML descarga un documento base en 40 ms, una hoja CSS bloqueante de renderizado en 65 ms en paralelo con un script clásico bloqueante de 110 ms, y ejecuta el layout final en 15 ms tras ambos recursos. Calcule el tiempo total hasta el First Contentful Paint (FCP) en milisegundos.

Datos

- $t_{html} = 40\,\text{ms}$ — Tiempo de descarga del HTML base
- $t_{css} = 65\,\text{ms}$ — Tiempo de descarga de la hoja CSS
- $t_{js} = 110\,\text{ms}$ — Tiempo del script bloqueante mayor
- $t_{layout} = 15\,\text{ms}$ — Tiempo de Layout y Paint

Incógnita Tiempo total hasta First Contentful Paint T_fcp en ms

Modelo físico

Cálculo del tiempo de bloqueo del Critical Rendering Path

$$T_{\text{fcp}} = 40 + 110 + 15$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos el tiempo base HTML, el máximo de las descargas en paralelo (max(65, 110) = 110 ms) y el tiempo de layout

$$T_{\text{fcp}} = 40 + 110 + 15$$

$$((40 + 110) + 15)$$

Resultado

$$\textit{Resultado} = 165$$

Interpretación El First Contentful Paint ocurre a los 165 ms; añadir defer al script reduciría la espera crítica de recursos de 110 ms a 65 ms.
Comprobación de los pasos

- Paso 1: 165 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) [CH-01.1] Calcule (Reducción de latencia FCP al diferir un script de 110 ms dejando solo el CSS de 65 ms (40 + 65 + 15)) expresado en ms.
2 [PRB-0004] (Intermedio, Desarrollo por pasos) [CH-01.2] Calcule (Tamaño total en KiB de un documento HTML de 18 KiB más 3 iconos SVG inline de 4 KiB cada uno) expresado en KiB.
3 [PRB-0005] (Intermedio, Desarrollo por pasos) [CH-01.3] Calcule (Número total de nodos DOM generados por una tabla de 25 filas con 4 celdas por fila más el nodo tr por fila) expresado en nodos.
4 [PRB-0006] (Avanzado, Diseño) [CH-01.4] Calcule (Ahorro porcentual de transferencia al comprimir con Brotli un HTML de 80 KiB a 16 KiB) expresado en %.

Capítulo 2

CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries

Monografía técnica de CSS como lenguaje declarativo de presentación visual, geometría de cajas, contextos de apilamiento, diseño adaptable e ingeniería de sistemas de diseño.

Consejo

Flexbox distribuye espacio a lo largo de un eje principal dinámico, mientras que CSS Grid impone una malla bidimensional explícita de filas y columnas.

Definición

Algoritmo de la Cascada, Capas (@layer) y Especificidad Lexicográfica: El motor CSS resuelve conflictos entre declaraciones mediante una jerarquía estricta donde el origen y las capas (@layer) preceden a la especificidad del selector.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Animar exclusivamente transform y opacity evita recalcular el Layout (Reflow) y el Paint, delegando la interpolación al hilo compositor de la GPU.

2.1 Algoritmo de la Cascada, Capas (@layer) y Especificidad Lexicográfica y Modelo de Caja, Box-Sizing y Propiedades Lógicas Internacionalizadas

Para cada propiedad de cada elemento DOM, el motor CSS filtra las declaraciones aplicables y elige un único valor ganador evaluando en orden: 1) Origen e importancia (`!important` de usuario/autor), 2) Contexto de encapsulación (Shadow DOM), 3) Capas de cascada explícitas (`@layer reset, base, components, utilities`), 4) Especificidad del selector expresada como el vector lexicográfico $S = (a, b, c)$ donde a cuenta selectores de ID (`#nav`), b cuenta clases, atributos y pseudoclases (`.activo`, `[type="text"]`, `:hover`, mientras `:where()` aporta 0 y `:is()` toma el máximo de su lista), y c cuenta selectores de tipo y pseudoelementos (`article`, `::before`), y finalmente 5) Orden de aparición en el código fuente.

```
@layer base, components;

@layer base {
  :root {
    --espacio-base: 1rem;
    --color-superficie: oklch(98% 0.01 250);
  }
  *, *::before, *::after {
    box-sizing: border-box;
  }
}

@layer components {
  .panel-metricas {
    container-type: inline-size;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(min(100%, 18rem), 1fr));
```

```
    gap: var(--espacio-base);
    background: var(--color-superficie);
  }
}
```

En el modelo tradicional `content-box`, el ancho visual ocupado por una caja es $W_{\text{total}} = W_{\text{content}} + 2P + 2B$. Al declarar `box-sizing: border-box`, el ancho declarado fija directamente W_{total}, descontando `padding` y `border` del área interior disponible. Además, el CSS moderno reemplaza propiedades físicas (`margin-left`, `width`) por propiedades lógicas relativas al flujo de escritura (`margin-inline-start`, `inline-size`, `block-size`), soportando automáticamente idiomas de derecha a izquierda (RTL) como árabe o hebreo sin duplicar hojas de estilo.

2.2 Contextos de Formato: Flexbox Unidimensional y CSS Grid Bidimensional y Container Queries, Rendimiento de Animaciones y Compositor GPU

Un contenedor `display: flex` calcula el tamaño base (`flex-basis`) de sus hijos directos y reparte el espacio libre positivo según los factores `flex-grow` o absorbe el déficit según `flex-shrink` ponderado por el tamaño base. Por su parte, `display: grid` permite definir pistas elásticas con la unidad fraccional `fr`, funciones `minmax()` y alineación de subrejillas mediante `subgrid`, permitiendo que componentes anidados alineen sus cabeceras y pies exactamente con la retícula del contenedor abuelo. Modificar propiedades geométricas (`width`, `top`, `margin`) en cada fotograma invalida el árbol de layout y obliga a la CPU a recalcular posiciones y repintar píxeles, provocando caídas por debajo de 60 FPS (presupuesto de 16,67 ms por fotograma). En cambio, `transform: translate3d(...)` y `opacity` operan sobre texturas ya rasterizadas en la GPU.

Problema resuelto

Enunciado Un contenedor Flex tiene un ancho interior de 960 px y contiene dos paneles con flex-basis de 200 px y 160 px respectivamente. Sus factores flex-grow son 2 y 1. Calcule el ancho final en píxeles del primer panel tras repartir el espacio libre.

Datos

- $W_{cont} = 960\,\text{px}$ — Ancho interior del contenedor Flex
- $B_1 = 200\,\text{px}$ — Tamaño base flex-basis del panel 1
- $B_2 = 160\,\text{px}$ — Tamaño base flex-basis del panel 2
- $g_1 = 2$ — Factor flex-grow del panel 1
- $g_2 = 1$ — Factor flex-grow del panel 2

Incógnita Ancho final del primer panel W_1 en px

Modelo físico

Distribución proporcional de espacio libre en Flexbox

$$W_1 = 200 + \frac{2}{2+1} \cdot (960 - 200 - 160)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el espacio libre (960 - 200 - 160 = 600 px) y asignamos dos tercios al primer panel

$$W_1 = 200 + \frac{2}{2+1} \cdot (960 - 200 - 160)$$

$$\left(200 + \frac{(2 \cdot ((960 - 200) - 160))}{3}\right)$$

Resultado

$$Resultado = 600$$

Interpretación El primer panel alcanza un ancho final exacto de 600 px y el segundo panel ocupa los 360 px restantes.
Comprobación de los pasos

- Paso 1: 600 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) [CH-02.1] Calcule (Ancho final del segundo panel en el contenedor Flex de 960 px (160 + 600 / 3)) expresado en px.
2 [PRB-0010] (Intermedio, Desarrollo por pasos) [CH-02.2] Calcule (Valor entero codificado en base 100 de la especificidad (1, 2, 3) para comparación rápida) expresado en puntos.
3 [PRB-0011] (Intermedio, Desarrollo por pasos) [CH-02.3] Calcule (Ancho de una columna 1fr en un CSS Grid de 1200 px con 3 columnas iguales y 2 huecos (gap) de 24 px) expresado en px.
4 [PRB-0012] (Avanzado, Diseño) [CH-02.4] Calcule (Ancho de área de contenido en border-box para una caja de 320 px con padding de 16 px y borde de 2 px por lado) expresado en px.

Capítulo 3

Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf

Estudio comparativo y riguroso de los lenguajes de serialización, configuración y marcado ligero empleados en sistemas distribuidos e infraestructura.

Consejo

Todo analizador XML en producción debe deshabilitar la resolución de entidades externas (XXE) y la expansión recursiva de DTD.

Definición

JSON (RFC 8259) y Validación Contractual con JSON Schema: JSON proporciona una gramática libre de contexto mínima y determinista, ideal para APIs web cuando se acompaña de validación estructural mediante JSON Schema.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Protocol Buffers reduce entre 3 y 10 veces el tamaño de carga útil y el tiempo de CPU frente a JSON al codificar campos con etiquetas numéricas y enteros Varint.

3.1 JSON (RFC 8259) y Validación Contractual con JSON Schema y YAML 1.2 vs TOML v1.0: Configuración Humana y Trampas Semánticas

Especificado en el RFC 8259, JSON admite únicamente seis tipos de valores: objetos (pares clave-valor con claves de tipo cadena), arrays ordenados, cadenas UTF-8, números en punto flotante IEEE 754, booleanos (`true`, `false`) y `null`. No admite comentarios, comas finales ni referencias cíclicas. Un riesgo clásico de interoperabilidad ocurre cuando enteros de 64 bits (`int64` mayores que $2^{53}-1$) se deserializan en motores JavaScript, perdiendo precisión en el mantisa de 53 bits; por ello, los identificadores de 64 bits deben serializarse como cadenas textuales.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["id_transaccion", "importe_centimos", "moneda"],
  "properties": {
    "id_transaccion": { "type": "string", "pattern": "^[0-9a-fA-F-]{36}$" },
    "importe_centimos": { "type": "integer", "minimum": 1 },
    "moneda": { "enum": ["EUR", "USD", "GBP"] }
  },
  "additionalProperties": false
}
```

YAML 1.2 se utiliza masivamente en Kubernetes, Ansible y pipelines CI/CD gracias a su sintaxis concisa, soporte de comentarios y reutilización mediante anclas (`&base`) y alias (`*base`). Sin embargo, presenta riesgos conocidos: en YAML 1.1 literales sin comillas como `NO` (código ISO de Noruega) u `on` se convertían silenciosamente en booleanos (`false`, `true`), y la carga insegura de etiquetas arbitrarias (`!!python/object/apply`) permite ejecución remota de código si no se invoca exclusivamen-

te `yaml.safe_load`. Por contraste, TOML (`Cargo.toml`, `pyproject.toml`) utiliza secciones explícitas `[tabla]` y tipos literales sin ambigüedad de indentación.

3.2 XML, Namespaces, XSD y Prevención de Ataques XXE y CSV (RFC 4180), Markdown (CommonMark) y Serialización Binaria con Protocol Buffers

XML sigue siendo el estándar normativo en facturación electrónica, banca (ISO 20022) y federación de identidad (SAML 2.0) gracias a sus espacios de nombres (`xmlns`) y validación formal con XML Schema (XSD). No obstante, su soporte histórico de Document Type Definitions (DTD) habilita vulnerabilidades críticas: la inyección de Entidades Externas XML (XXE) permite leer archivos locales del servidor (`file:///etc/passwd`), y la bomba de entidades recursivas (Billion Laughs) agota gigabytes de memoria RAM a partir de menos de 1 KiB de entrada. Mientras CSV (RFC 4180) requiere escapar comillas dobles y neutralizar fórmulas iniciales (`=`, `+`, `-`, `@`) para evitar CSV Injection en hojas de cálculo, y CommonMark estandariza la gramática de Markdown para documentación técnica, las comunicaciones de alta frecuencia entre microservicios emplean formatos binarios con esquema como Protocol Buffers (Protobuf v3). En Protobuf, cada campo se identifica en el flujo binario mediante un único byte de cabecera `(field_number « 3) | wire_type`, permitiendo evolucionar esquemas añadiendo nuevos campos sin romper clientes antiguos.

Problema resuelto

Enunciado Un mensaje JSON de telemetría ocupa 140 bytes en texto UTF-8. Al codificar los mismos 4 campos numéricos en Protocol Buffers v3, cada campo requiere 1 byte de etiqueta (tag) y 3 bytes de valor Varint promedio. Calcule el ahorro porcentual de ancho de banda logrado por Protobuf.
Datos

- $B_{json} = 140\,\text{bytes}$ — Tamaño del mensaje en JSON
- $n_{campos} = 4\,\text{campos}$ — Número de campos del mensaje
- $B_{campo} = 4\,\text{bytes}$ — Bytes por campo en Protobuf (1 tag + 3 valor)

Incógnita Ahorro porcentual de bytes A_pct en %
Modelo físico
Compresión binaria de un mensaje de telemetría con Protocol Buffers frente a JSON

$$A_{\text{pct}} = \frac{140 - 4 \cdot 4}{140} \cdot 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el tamaño en Protobuf (4 * 4 = 16 bytes) y el porcentaje de reducción respecto a 140 bytes

$$A_{\text{pct}} = \frac{140 - 4 \cdot 4}{140} \cdot 100$$

$$\frac{((140 - (4 \cdot 4)) \cdot 100)}{140}$$

Resultado

$$\textit{Resultado} = 88{,}5714285714$$

Interpretación Protocol Buffers reduce el tamaño del mensaje de 140 bytes a 16 bytes, ahorrando aproximadamente un 88,57 % del ancho de banda.
Comprobación de los pasos

- Paso 1: 88,5714 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0015**] **(Básico, Numérico)** [CH-03.1] Calcule (Tamaño en bytes del mensaje Protobuf con 4 campos de 4 bytes cada uno) expresado en bytes.
2 [**PRB-0016**] **(Intermedio, Desarrollo por pasos)** [CH-03.2] Calcule (Valor del byte de cabecera Protobuf para el campo número 3 con $\text{wire}_t ype2$(length-delimited: 3*8 + 2)) expresado en entero.
3 [**PRB-0017**] **(Intermedio, Desarrollo por pasos)** [CH-03.3] Calcule (Codificación ZigZag de sint32 para el entero negativo -15 (-2 * -15 - 1)) expresado en entero.
4 [**PRB-0018**] **(Avanzado, Diseño)** [CH-03.4] Calcule (Tráfico diario ahorrado en MB al enviar 10 millones de mensajes de 16 B en lugar de 116 B (dividiendo entre 10^6)) expresado en MB.

Capítulo 4

Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF)

Fundamentos matemáticos del modelo relacional de E. F. Codd, equivalencia entre álgebra y cálculo relacional, cierre de atributos y teoría formal de la normalización.

Consejo

Un esquema relacional en BCNF elimina toda redundancia derivada de dependencias funcionales al exigir que todo determinante no trivial sea superclave.

Definición

El Modelo Relacional de Codd y las Claves Relacionales: Una relación matemática es un subconjunto finito del producto cartesiano de dominios atómicos donde el orden de tuplas y atributos carece de significado semántico.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La descomposición de R en R1 y R2 es una reunión sin pérdida si y solo si (R1 intersect R2) es superclave de R1 o de R2.

4.1 El Modelo Relacional de Codd y las Claves Relacionales y Dependencias Funcionales, Axiomas de Armstrong y Cierre de Atributos

Formulado por Edgar F. Codd en 1970, el modelo relacional independiza la representación lógica de los datos respecto a su disposición física en almacenamiento secundario. Dado un esquema $R(A_1, A_2, \ldots, A_n)$, un subconjunto de atributos $K \subseteq R$ es una superclave si no pueden existir dos tuplas distintas $t_1 \neq t_2$ en ninguna instancia válida $r(R)$ tales que $t_1[K] = t_2[K]$. Una superclave mínima (ningún subconjunto propio suyo es superclave) constituye una clave candidata, de las cuales se designa una clave primaria (PK) sujeta a integridad de entidad (ningún atributo primo puede ser `NULL`). Una dependencia funcional $X \to Y$ afirma que los valores del conjunto de atributos X determinan unívocamente los valores de Y. Los tres axiomas de Armstrong (reflexividad, aumentatividad y transitividad) son correctos y completos para inferir todas las dependencias implicadas F^+. En la práctica ingenieril, en lugar de enumerar F^+ (cuyo tamaño puede ser exponencial), calculamos el cierre de atributos X^+ partiendo de resultado $= X$ y añadiendo iterativamente W cada vez que $V \to W \in F$ cumple $V \subseteq$ resultado.

4.2 Anomalías de Diseño y Jerarquía de Formas Normales (1NF, 2NF, 3NF y BCNF) y Descomposición sin Pérdida (Lossless-Join) y Preservación de Dependencias

Mezclar múltiples entidades independientes en una sola tabla provoca anomalías de inserción (no poder registrar un departamento sin empleados), actualización (modificar un dato repetido en miles de filas con riesgo de inconsistencia) y borrado. La Primera Forma Normal (1NF) exige dominios atómicos; la 2NF prohíbe dependencias parciales de atributos no primos respecto a una clave compuesta; la 3NF prohíbe dependencias transitivas entre atributos no primos; y la Forma Normal de Boyce-Codd (BCNF) exige que para toda $X \to Y$ no trivial, X sea superclave. Dividir una tabla sin verificar el teorema de Heath

produce descomposiciones con pérdida (lossy joins) donde la reunión natural $R_1 \bowtie R_2$ genera tuplas espurias inexistentes en la relación original. Mientras que toda relación admite una descomposición en BCNF sin pérdida de información, en ocasiones aislar todas las dependencias en tablas individuales sólo es posible en 3NF mediante el algoritmo de síntesis de recubrimiento canónico.

Problema resuelto

Enunciado Una tabla desnormalizada de líneas de pedido con 50 000 filas repite en cada fila los 120 bytes de datos del cliente (habiendo solo 2 000 clientes distintos). Al descomponer a BCNF en Lineas(50 000 filas) y Clientes(2 000 filas x 120 bytes), calcule el espacio total en KiB (1 KiB = 1024 B) ahorrado por no repetir los 120 bytes en las 48 000 filas excedentes.
Datos

- $N_{exc} = 48000\,\text{filas}$ — Filas donde se repetían los datos del cliente
- $B_{cli} = 120\,\text{bytes}$ — Bytes redundantes por fila
- $K = 1024\,\text{bytes}\,\text{KiB}^{-1}$ — Bytes por kibibyte

Incógnita Espacio ahorrado S_kib en KiB
Modelo físico
Eliminación de redundancia mediante descomposición a BCNF

$$S_{\text{kib}} = \frac{48000 \cdot 120}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 48 000 filas redundantes por 120 bytes y dividimos entre 1024

$$S_{\text{kib}} = \frac{48000 \cdot 120}{1024}$$

$$\frac{(48000 \cdot 120)}{1024}$$

Resultado

$$Resultado = 5625$$

Interpretación La normalización a BCNF ahorra exactamente 5 625 KiB de almacenamiento redundante y elimina las anomalías de actualización.
Comprobación de los pasos

- Paso 1: 5625 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) [CH-04.1] Calcule (Espacio en KiB ocupado por la nueva tabla Clientes con 2048 filas de 128 bytes (2048 * 128 / 1024)) expresado en KiB.
2 [PRB-0022] (Intermedio, Desarrollo por pasos) [CH-04.2] Calcule (Número de superclaves en una relación de 5 atributos cuya única clave candidata tiene 2 atributos (2^(5-2))) expresado en superclaves.
3 [PRB-0023] (Intermedio, Desarrollo por pasos) [CH-04.3] Calcule (Tuplas espurias generadas al hacer join de dos proyecciones mal descompuestas que producen 140 filas frente a 100 reales)

expresado en tuplas.
4 [PRB-0024] (Avanzado, Diseño) [CH-04.4] Calcule (Número de atributos en la intersección R1 n R2 si |R1|=4, |R2|=5 y |R1 U R2|=7) expresado en atributos.

Capítulo 5

Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC

Ingeniería interna de sistemas gestores de bases de datos (SGBD): Buffer Pool, Write-Ahead Logging (ARIES), optimizadores basados en costes y control de concurrencia multiversión.

Consejo

La regla WAL garantiza atomicidad y durabilidad exigiendo que todo registro de log (UNDO/REDO) llegue a almacenamiento no volátil antes de confirmar la transacción o volcar la página sucia.

Definición

Árboles B+ (B+-Trees) frente a LSM-Trees (Log-Structured Merge-Trees): Un árbol B+ con factor de ramificación B=250 localiza cualquier clave entre 1 000 millones de filas en tan solo 4 lecturas de página.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: El aislamiento por instantáneas (Snapshot Isolation / Repeatable Read en PostgreSQL) evita lecturas sucias, no repetibles y fantasmas, pero permite Write Skew si no se emplea Serializable Snapshot Isolation (SSI).

5.1 Árboles B+ (B+-Trees) frente a LSM-Trees (Log-Structured Merge-Trees) y Diseño de Índices Compuestos, Índices Cubrientes (Covering) y Optimizadores de Costes

Los motores relacionales clásicos (PostgreSQL, InnoDB, SQLite) organizan tablas e índices en páginas de disco de 8 KiB o 16 KiB estructuradas como árboles B+, como muestra la figura 2. Todos los valores residen en el nivel de las hojas, las cuales están doblemente enlazadas para acelerar escaneos por rango (`BETWEEN`). En contrapartida, motores de alta tasa de ingesta (RocksDB, Cassandra, ScyllaDB) emplean LSM-Trees: toda escritura se registra secuencialmente en un log en disco y en una estructura ordenada en RAM (`MemTable`); al llenarse, se vuelca como un archivo inmutable (`SSTable`) compactado en segundo plano y acelerado mediante filtros de Bloom. El optimizador de consultas (Query Planner) utiliza histogramas de distribución, cardinalidad (NDV) y factores de correlación física para estimar el coste en E/S y CPU de distintos planes de ejecución (`Index Scan`, `Bitmap Heap Scan`, `Seq Scan`, `Hash Join`, `Merge Join`, `Nested Loop`). Cuando un índice incluye todas las columnas requeridas por una consulta (por ejemplo, `CREATE INDEX idx_pedidos ON pedidos (cliente_id, fecha) INCLUDE (total)`), el motor ejecuta un `Index-Only Scan` sin necesidad de acceder a las páginas de la tabla principal (Heap).

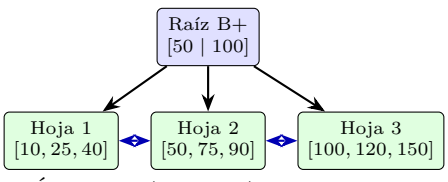


Figura 2. Estructura de un índice Árbol B+ (B+-Tree): los nodos internos dirigen la búsqueda por claves separadoras y las hojas enlazadas almacenan los punteros o tuplas en orden secuencial.

5.2 Transacciones ACID, Write-Ahead Logging (WAL) y Protocolo ARIES y Niveles de Aislamiento, Anomalías de Concurrencia y MVCC (Snapshot Isolation)

Las garantías ACID comprenden Atomicidad (todo o nada mediante registros UNDO), Consistencia (preservación de invariantes e integridad referencial), Aislamiento (ejecución concurrente equivalente a una planificación controlada) y Durabilidad (persistencia mediante registros REDO en el Write-Ahead Log). Tras un fallo eléctrico, el algoritmo de recuperación ARIES ejecuta tres fases a partir del último punto de control (Checkpoint): Análisis, REDO (repitiendo la historia hasta el estado exacto previo al fallo) y UNDO (revirtiendo transacciones inconclusas). Bajo MVCC, cada transacción recibe un identificador monotónico (`txid`) y cada actualización inserta una nueva versión de la tupla marcada con el intervalo de validez `[xmin, xmax)`. Así, las lecturas leen una instantánea consistente sin bloquear a los escritores. Sin embargo, dos transacciones concurrentes que leen un mismo invariante y escriben en filas disjuntas pueden cometer la anomalía de asimetría de escritura (`Write Skew`); prevenirla requiere bloqueos explícitos (`SELECT ... FOR UPDATE`) o el algoritmo SSI (`SERIALIZABLE`).

Problema resuelto

Enunciado En una base de datos con páginas de 8 KiB, cada nodo interno de un índice B+ tiene un factor de ramificación efectivo de 200 hijos y cada página hoja almacena 80 entradas de índice. Calcule el número total de filas (en millones) que puede indexar un árbol B+ de altura h = 3 (1 raíz, 1 nivel intermedio de nodos internos y 1 nivel de hojas).

Datos

- $B_{int} = 200\,\text{hijos}\,\text{nodo}^{-1}$ — Factor de ramificación de nodos internos
- $C_{hoja} = 80\,\text{filas}\,\text{hoja}^{-1}$ — Entradas almacenadas por página hoja

Incógnita Capacidad total en millones de filas $N_millones$

Modelo físico

Capacidad de direccionamiento de un Árbol B+ de 3 niveles

$$N_{\text{millones}} = \frac{200 \cdot 200 \cdot 80}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 200 hijos de la raíz por los 200 hijos del nivel intermedio por las 80 filas por hoja y dividimos entre 10^6

$$N_{\text{millones}} = \frac{200 \cdot 200 \cdot 80}{10^6}$$

$$\frac{((200 \cdot 200) \cdot 80)}{1000000}$$

Resultado

$$Resultado = 3{,}2$$

Interpretación Con solo 3 niveles (y los 2 primeros en RAM), el árbol B+ direcciona hasta 3,2 millones de filas requiriendo como máximo una única lectura física de disco para la hoja.

Comprobación de los pasos

- Paso 1: 3,2 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0027**] **(Básico, Numérico)** [CH-05.1] Calcule (Número de páginas hoja en el árbol B+ de 3 niveles con factor de ramificación 200 (200 * 200)) expresado en hojas.
2 [**PRB-0028**] **(Intermedio, Desarrollo por pasos)** [CH-05.2] Calcule (Memoria RAM en KiB requerida para mantener en Buffer Pool la raíz y los 200 nodos internos de 8 KiB ((1 + 200) * 8)) expresado en KiB.
3 [**PRB-0029**] **(Intermedio, Desarrollo por pasos)** [CH-05.3] Calcule (Coste estimado en unidades de página de un Index Scan que lee 3 niveles de índice más 12 páginas de Heap) expresado en páginas.
4 [**PRB-0030**] **(Avanzado, Diseño)** [CH-05.4] Calcule (Tasa de escritura WAL en MB/s para 5 000 transacciones por segundo que generan 400 bytes de log cada una) expresado en MB/s.

Capítulo 6

Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis

Arquitectura de bases de datos a escala: replicación síncrona/asíncrona, particionamiento por hash consistente y rango, transacciones distribuidas y motores de producción.

Consejo

PostgreSQL combina relacionalidad estricta con tipos JSONB indexados (GIN), mientras que SQLite en modo WAL ofrece lecturas concurrentes embebidas de latencia sub-milisegundo.

Definición

Teorema CAP y Clasificación PACELC en Almacenamiento Distribuido: El modelo PACELC extiende el teorema CAP demostrando que incluso en ausencia de particiones de red existe una disyuntiva fundamental entre latencia (L) y consistencia (C).

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Los almacenes documentales eliminan el coste de joins cuando el agregado se consulta íntegro, y Redis proporciona estructuras atómicas en RAM con latencia de microsegundos.

6.1 Teorema CAP y Clasificación PACELC en Almacenamiento Distribuido y Replicación Líder-Seguidor, Quórums de Lectura/Escritura y Sharding

El teorema CAP (Brewer, demostrado formalmente por Gilbert y Lynch) establece que ante una partición de red (P) entre nodos, un almacén de datos distribuido debe elegir entre cancelar la operación para preservar la Consistencia linealizable (CP) o responder con datos potencialmente desactualizados para preservar la Disponibilidad (AP). El teorema PACELC (Abadi) añade que, incluso cuando el sistema opera normalmente (Else), debe decidirse entre replicar síncronamente (penalizando la Latencia L a cambio de Consistencia C) o asíncronamente. Cuando el volumen de escrituras o el conjunto de datos supera la capacidad de un único nodo, los datos se dividen horizontalmente en particiones (Shards). El particionamiento por rango facilita consultas secuenciales pero puede sufrir puntos calientes (hotspots) si todas las escrituras llevan la marca temporal actual; el particionamiento por Hash Consistente (con nodos virtuales en un anillo de 2^{32} posiciones) distribuye uniformemente la carga y minimiza el movimiento de claves al añadir o retirar servidores.

6.2 PostgreSQL y SQLite en Producción: De Sistemas Embebidos a Escala Empresarial y Motores NoSQL Especializados: Documentos (MongoDB) y Memoria Clave-Valor (Redis)

PostgreSQL constituye el estándar de referencia en ingeniería de datos moderna: soporta índices B-Tree, Hash, GiST, SP-GiST, GIN (para documentos `JSONB` y búsqueda de texto completo) y BRIN (para series temporales masivas), además de particionamiento declarativo y replicación lógica. Por su parte, SQLite ejecuta el motor transaccional completo dentro del espacio de direcciones del proceso anfitrión sin latencia

de red; al activar `PRAGMA journal_mode = WAL;` y `PRAGMA synchronous = NORMAL;`, alcanza decenas de miles de operaciones por segundo en un único archivo de disco. MongoDB almacena documentos en formato binario BSON sobre el motor WiredTiger, resultando idóneo para catálogos polimórficos donde la entidad raíz y sus subdocumentos forman un agregado de consulta cohesivo. Redis (y alternativas compatibles como Valkey) mantiene estructuras de datos ricas en memoria (Strings, Hashes, Sorted Sets, Streams, HyperLogLog) sobre un bucle de eventos monohilo libre de contención de cerrojos, ejecutando scripts Lua de forma atómica para cachés distribuidas, colas de trabajo y limitadores de tasa (rate limiters).

Problema resuelto

Enunciado Un clúster de base de datos distribuida mantiene N = 5 réplicas de cada partición. Si las escrituras se configuran con un quórum de confirmación W = 3 nodos para tolerar la caída de 2 réplicas, calcule el número mínimo de nodos R que debe consultar una lectura para garantizar intersección estricta (W + R > N => R = N - W + 1).
Datos

- $N = 5\,\text{nodos}$ — Factor total de replicación
- $W = 3\,\text{nodos}$ — Quórum de escritura confirmada

Incógnita Quórum mínimo de lectura R en nodos
Modelo físico
Cálculo del quórum mínimo de lectura para consistencia fuerte

$$R = 5 - 3 + 1$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Aplicamos la condición de intersección de quórum estricto R = N - W + 1

$$R = 5 - 3 + 1$$

$$((5 - 3) + 1)$$

Resultado

$$\textit{Resultado} = 3$$

Interpretación Basta con leer de R = 3 réplicas cualesquiera para garantizar que al menos una contiene la última escritura confirmada por W = 3 nodos.
Comprobación de los pasos

- Paso 1: 3 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) [CH-06.1] Calcule (Quórum de lectura requerido si N = 7 réplicas y W = 4 escrituras (7 - 4 + 1)) expresado en nodos.
2 [PRB-0034] (Intermedio, Desarrollo por pasos) [CH-06.2] Calcule (Porcentaje de claves redistribuidas por Hash Consistente al pasar de 9 a 10 nodos (100 / 10)) expresado en %.
3 [PRB-0035] (Intermedio, Desarrollo por pasos) [CH-06.3] Calcule (Memoria RAM total en GB para almacenar en Redis 50 millones de claves de 200 bytes promedio (dividiendo entre 10^9))

expresado en GB.
4 [PRB-0036] (Avanzado, Diseño) [CH-06.4] Calcule (Número máximo de fallos de nodo tolerados por un clúster Raft de 5 nodos ((5 - 1) / 2)) expresado en nodos.

Capítulo 7

Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js

Arquitectura interna y patrones de ingeniería de los principales frameworks de interfaz de usuario web.

Consejo

Vue 3 rastrea dependencias en tiempo de ejecución mediante objetos Proxy de ES6, mientras que Svelte compila las asignaciones reactivas en actualizaciones quirúrgicas del DOM en tiempo de build.

Definición

Del DOM Imperativo a la UI Declarativa: Función Estado -> Vista: Los frameworks frontend modernos modelan la interfaz como una proyección determinista V = f(S) del estado actual, automatizando la sincronización incremental del DOM.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Separar la caché de estado del servidor (TanStack Query / SWR) del estado efímero de la interfaz reduce drásticamente la complejidad de almacenes globales como Redux.

7.1 Del DOM Imperativo a la UI Declarativa: Función Estado -> Vista y React y Next.js: Reconciliación Fiber, Hooks, Server Components e Hidratación

Manipular manualmente nodos DOM ante eventos asíncronos conduce a transiciones de estado no controladas y fugas de memoria. Los frameworks modernos adoptan un paradigma declarativo donde el desarrollador describe cómo debe lucir la interfaz para cada estado S, y el motor de ejecución determina el conjunto mínimo de mutaciones DOM necesarias. Existen dos grandes estrategias: reconciliación de Árbol Virtual (Virtual DOM en React y Vue) y reactividad de grano fino basada en grafos de dependencias o compilación anticipada (Signals en Angular/Solid y compilador de Svelte). En React, los componentes funcionales son funciones puras que utilizan Hooks (`useState`, `useReducer`, `useMemo`, `useEffect`) almacenados en una lista enlazada asociada al nodo Fiber. En aplicaciones full-stack con Next.js (App Router), los React Server Components (RSC) se ejecutan exclusivamente en el servidor −accediendo directamente a bases de datos sin enviar su código JavaScript al navegador−, reservando los Client Components (`"use client"`) para islas de interactividad que requieren hidratación.

7.2 Angular, Vue y Svelte: Inyección de Dependencias, Proxies Reactivos y Compilación y Gestión de Estado: Estado Local, Estado Global y Caché de Estado Remoto

Angular ofrece una plataforma empresarial opinada en TypeScript con inyección de dependencias jerárquica, enrutamiento tipado, RxJS y reactividad moderna mediante `signal()` y `computed()`. Vue 3 combina la Composition API (`ref`, `reactive`) implementada sobre `Proxy` con un compilador de plantillas que marca banderas de parcheo (`PatchFlags`) para saltar subárboles estáticos. Svelte prescinde del Virtual DOM compilando cada componente en código JavaScript imperativo mínimo. Uno de los antipatrones más frecuentes en ingeniería frontend consiste en volcar respuestas HTTP crudas en un almacén global único sin políticas de invalidación, reintento ni deduplicación. La arquitectura moderna distingue estrictamente entre estado del servidor (gestionado mediante cachés declarativas con `stale-while-revalidate`

y actualización optimista), estado de la URL (filtros y paginación sincronizados con el router) y estado puramente local de la interfaz.

Problema resuelto

Enunciado Una aplicación SPA tradicional descarga un bundle monolítico de 480 KiB de JavaScript. Al migrar a una arquitectura híbrida SSR + Server Components, el 65 % del código de presentación y formateo permanece en el servidor. Calcule el tamaño en KiB del bundle JavaScript que aún debe descargar e hidratar el navegador.
Datos

- $B_{spa} = 480\,\text{KiB}$ — Tamaño original del bundle SPA
- $p_{srv} = 65\ \%$ — Porcentaje migrado a Server Components

Incógnita Tamaño del bundle cliente restante B_cli en KiB
Modelo físico
Reducción del bundle JavaScript mediante React Server Components y Code Splitting

$$B_{\text{cli}} = 480 \cdot \frac{100 - 65}{100}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el 35 % restante del bundle original de 480 KiB

$$B_{\text{cli}} = 480 \cdot \frac{100 - 65}{100}$$

$$\frac{(480 \cdot (100 - 65))}{100}$$

Resultado

$$\mathit{Resultado} = 168$$

Interpretación El cliente solo descarga e hidrata 168 KiB de JavaScript, reduciendo el Time to Interactive (TTI) en dispositivos móviles.
Comprobación de los pasos

- Paso 1: 168 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0039] (Básico, Numérico) [CH-07.1] Calcule (Ahorro absoluto en KiB al reducir el bundle de 480 KiB a 168 KiB) expresado en KiB.
2 [PRB-0040] (Intermedio, Desarrollo por pasos) [CH-07.2] Calcule (Tiempo de parseo y compilación JS en móvil a razón de 1,5 ms por cada KiB para 168 KiB) expresado en ms.
3 [PRB-0041] (Intermedio, Desarrollo por pasos) [CH-07.3] Calcule (Nodos re-evaluados con Signals en una lista de 500 filas cuando cambia 1 celda frente a 500 sin memoización (ahorro)) expresado en nodos.
4 [PRB-0042] (Avanzado, Diseño) [CH-07.4] Calcule (Número de peticiones de red tras deduplicar 6 componentes que solicitan simultáneamente el mismo endpoint) expresado en peticiones.

Capítulo 8

Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter

Análisis arquitectónico de los ecosistemas de servidor de alto rendimiento y de las plataformas de interfaz gráfica de escritorio y dispositivos móviles.

Consejo

Tauri reduce en un orden de magnitud el consumo de memoria RAM y el tamaño del instalador frente a Electron al reutilizar el WebView nativo del sistema operativo y ejecutar el backend en Rust.

Definición

Arquitectura de Frameworks Backend: Middleware, Inyección de Dependencias y Ciclo HTTP: Todos los frameworks backend modernos estructuran el procesamiento de peticiones como una composición funcional de middlewares alrededor de un controlador tipado.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Flutter dibuja cada píxel en su propio lienzo GPU mediante el motor Impeller sin depender de widgets OEM, mientras que SwiftUI y Jetpack Compose ofrecen integración nativa declarativa.

8.1 Arquitectura de Frameworks Backend: Middleware, Inyección de Dependencias y Ciclo HTTP y Comparativa de Ecosistemas Backend: Productividad, Tipado y Rendimiento Extremo

Tanto en ecosistemas asíncronos orientados a eventos (Fastify/Express en Node.js, FastAPI sobre ASGI/Uvicorn en Python, Axum/Actix sobre Tokio en Rust, Phoenix sobre BEAM en Elixir) como en plataformas empresariales multihilo o con hilos virtuales (Spring Boot 3 en Java 21, ASP.NET Core en .NET 8, Django, Ruby on Rails, Laravel), cada petición HTTP atraviesa una cadena concéntrica de middlewares (trazabilidad, CORS, autenticación JWT/OIDC, rate limiting) antes de deserializar y validar el payload hacia un DTO tipado. En FastAPI (Python), las anotaciones de tipos estándar junto con Pydantic v2 (escrito en Rust) validan cargas JSON y generan automáticamente la especificación OpenAPI 3.1. En Spring Boot y ASP.NET Core, el contenedor de Inyección de Dependencias (IoC) gestiona el ciclo de vida de servicios (`Singleton`, `Scoped` por petición, `Transient`), facilitando las pruebas unitarias mediante inversión de dependencias. En Rust, Axum aprovecha el sistema de tipos para extraer parámetros y estado compartido (`State<Arc<AppState>>`) con coste de abstracción nulo.

8.2 Aplicaciones de Escritorio: Qt (C++), .NET MAUI, Electron y Tauri (Rust + WebView) y Desarrollo Móvil: SwiftUI (iOS), Jetpack Compose (Android), Flutter y React Native

Para aplicaciones de escritorio de misión crítica (CAD, instrumentación médica, edición de vídeo), Qt 6 en C++ proporciona renderizado acelerado por hardware y un sistema de señales y ranuras (`signals & slots`) nativo. Cuando se desea reutilizar tecnologías web en escritorio, Electron empaqueta una instancia

completa de Chromium y Node.js por aplicación (~150 MB en disco y cientos de MB de RAM base), mientras que Tauri delega la interfaz al WebView del sistema (WebKitGTK, WebView2, WKWebView) y comunica el frontend con un binario nativo en Rust mediante IPC tipado. El desarrollo móvil moderno ha convergido hacia interfaces declarativas reactivas: SwiftUI en el ecosistema Apple y Jetpack Compose en Android eliminan los antiguos archivos XML/Storyboards imperativos. En el ámbito multiplataforma, React Native sincroniza componentes JavaScript/TypeScript con vistas nativas OEM mediante la nueva arquitectura JSI (JavaScript Interface) sin puente serializado, mientras que Flutter (Dart AOT) controla directamente el pipeline gráfico con Impeller.

Problema resuelto

Enunciado Una estación de trabajo ejecuta simultáneamente 4 utilidades corporativas de escritorio. Empaquetadas con Electron, cada instancia consume 220 MB de memoria RAM debido a procesos Chromium independientes; migradas a Tauri (Rust + WebView compartido), cada instancia consume 45 MB. Calcule la memoria RAM total ahorrada en megabytes.
Datos

- $N_{apps} = 4\,\text{aplicaciones}$ — Número de aplicaciones ejecutadas
- $M_{elec} = 220\,\text{MB}$ — Memoria por aplicación en Electron
- $M_{tauri} = 45\,\text{MB}$ — Memoria por aplicación en Tauri

Incógnita Memoria RAM total ahorrada M_ahorro en MB
Modelo físico
Ahorro de memoria RAM en escritorio con Tauri frente a Electron

$$M_{\text{ahorro}} = 4 \cdot (220 - 45)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 4 aplicaciones por la diferencia de consumo unitario (220 - 45 = 175 MB)

$$M_{\text{ahorro}} = 4 \cdot (220 - 45)$$

$$(4 \cdot (220 - 45))$$

Resultado

$$Resultado = 700$$

Interpretación La migración de las 4 utilidades a Tauri libera exactamente 700 MB de memoria RAM en la estación de trabajo.
Comprobación de los pasos

- Paso 1: 700 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0045] (Básico, Numérico) [CH-08.1] Calcule (Reducción del tamaño de descarga en MB al pasar de un instalador Electron de 115 MB a uno Tauri de 8 MB) expresado en MB.
2 [PRB-0046] (Intermedio, Desarrollo por pasos) [CH-08.2] Calcule (Presupuesto de tiempo por fotograma en milisegundos (redondeado a centésimas x100) para una app Flutter a 120 Hz (100000

/ 125)) expresado en centésimas de ms.
3 [PRB-0047] (Intermedio, Desarrollo por pasos) [CH-08.3] Calcule (Capacidad total de peticiones por segundo en un clúster de 6 pods Axum que procesan 35 000 req/s cada uno) expresado en req/s.
4 [PRB-0048] (Avanzado, Diseño) [CH-08.4] Calcule (Instancias de un servicio Scoped creadas durante un lote de 250 peticiones HTTP si se inyecta 2 veces por petición) expresado en instancias.

Apéndice A

Notación

Símbolo	Significado	Unidades
$S = (a, b, c)$	Vector lexicográfico de especificidad CSS	adimensional
$X \to Y$	Dependencia funcional relacional en esquema R	relaciónlógica
$h = \lceil \log_B(N) \rceil$	Altura de un árbol B+ con factor de ramificación B	nivelesdepágina

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 HTML Living Standard y Accesibilidad Web (WCAG / ARIA)

Problema resuelto [PRB-0001]
Una página HTML descarga un documento base en 40 ms, una hoja CSS bloqueante de renderizado en 65 ms en paralelo con un script clásico bloqueante de 110 ms, y ejecuta el layout final en 15 ms tras ambos recursos. Calcule el tiempo total hasta el First Contentful Paint (FCP) en milisegundos.
Respuesta

$$Resultado = 165$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[CH-01.1] Calcule (Reducción de latencia FCP al diferir un script de 110 ms dejando solo el CSS de 65 ms (40 + 65 + 15)) expresado en ms.
Respuesta 120
Ejercicio 2 [PRB-0004]
[CH-01.2] Calcule (Tamaño total en KiB de un documento HTML de 18 KiB más 3 iconos SVG inline de 4 KiB cada uno) expresado en KiB.
Respuesta 30
Ejercicio 3 [PRB-0005]
[CH-01.3] Calcule (Número total de nodos DOM generados por una tabla de 25 filas con 4 celdas por fila más el nodo tr por fila) expresado en nodos.
Respuesta 125
Ejercicio 4 [PRB-0006]
[CH-01.4] Calcule (Ahorro porcentual de transferencia al comprimir con Brotli un HTML de 80 KiB a 16 KiB) expresado en %.
Respuesta 80

Cálculos reproducidos

$((40 + 110) + 15) = 165$

C.2 CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries

Problema resuelto [PRB-0007]
Un contenedor Flex tiene un ancho interior de 960 px y contiene dos paneles con flex-basis de 200 px y 160 px respectivamente. Sus factores flex-grow son 2 y 1. Calcule el ancho final en píxeles del primer panel tras repartir el espacio libre.
Respuesta

$$Resultado = 600$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]
[CH-02.1] Calcule (Ancho final del segundo panel en el contenedor Flex de 960 px (160 + 600 / 3)) expresado en px.
Respuesta 360

Ejercicio 2 [PRB-0010]
[CH-02.2] Calcule (Valor entero codificado en base 100 de la especificidad (1, 2, 3) para comparación rápida) expresado en puntos.
Respuesta 10203
Ejercicio 3 [PRB-0011]
[CH-02.3] Calcule (Ancho de una columna 1fr en un CSS Grid de 1200 px con 3 columnas iguales y 2 huecos (gap) de 24 px) expresado en px.
Respuesta 384
Ejercicio 4 [PRB-0012]
[CH-02.4] Calcule (Ancho de área de contenido en border-box para una caja de 320 px con padding de 16 px y borde de 2 px por lado) expresado en px.
Respuesta 284

Cálculos reproducidos

$\left(200 + \frac{(2\cdot((960-200)-160))}{3}\right) = 600$

C.3 Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf

Problema resuelto [PRB-0013]
Un mensaje JSON de telemetría ocupa 140 bytes en texto UTF-8. Al codificar los mismos 4 campos numéricos en Protocol Buffers v3, cada campo requiere 1 byte de etiqueta (tag) y 3 bytes de valor Varint promedio. Calcule el ahorro porcentual de ancho de banda logrado por Protobuf.
Respuesta

$$Resultado = 88{,}5714285714$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0015]
[CH-03.1] Calcule (Tamaño en bytes del mensaje Protobuf con 4 campos de 4 bytes cada uno) expresado en bytes.
Respuesta 16
Ejercicio 2 [PRB-0016]
[CH-03.2] Calcule (Valor del byte de cabecera Protobuf para el campo número 3 con $\text{wire}_t ype2$(length-delimited: 3*8 + 2)) expresado en entero.
Respuesta 26
Ejercicio 3 [PRB-0017]
[CH-03.3] Calcule (Codificación ZigZag de sint32 para el entero negativo -15 (-2 * -15 - 1)) expresado en entero.
Respuesta 29
Ejercicio 4 [PRB-0018]
[CH-03.4] Calcule (Tráfico diario ahorrado en MB al enviar 10 millones de mensajes de 16 B en lugar de 116 B (dividiendo entre 10^6)) expresado en MB.
Respuesta 1000

Cálculos reproducidos

$\frac{((140-(4\cdot4))\cdot100)}{140} = 88{,}5714285714$

C.4 Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF)

Problema resuelto [PRB-0019]

Una tabla desnormalizada de líneas de pedido con 50 000 filas repite en cada fila los 120 bytes de datos del cliente (habiendo solo 2 000 clientes distintos). Al descomponer a BCNF en Lineas(50 000 filas) y Clientes(2 000 filas x 120 bytes), calcule el espacio total en KiB (1 KiB = 1024 B) ahorrado por no repetir los 120 bytes en las 48 000 filas excedentes.
Respuesta

$$Resultado = 5625$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
[CH-04.1] Calcule (Espacio en KiB ocupado por la nueva tabla Clientes con 2048 filas de 128 bytes (2048 * 128 / 1024)) expresado en KiB.
Respuesta 256
Ejercicio 2 [PRB-0022]
[CH-04.2] Calcule (Número de superclaves en una relación de 5 atributos cuya única clave candidata tiene 2 atributos (2^(5-2))) expresado en superclaves.
Respuesta 8
Ejercicio 3 [PRB-0023]
[CH-04.3] Calcule (Tuplas espurias generadas al hacer join de dos proyecciones mal descompuestas que producen 140 filas frente a 100 reales) expresado en tuplas.
Respuesta 40
Ejercicio 4 [PRB-0024]
[CH-04.4] Calcule (Número de atributos en la intersección R1 n R2 si |R1|=4, |R2|=5 y |R1 U R2|=7) expresado en atributos.
Respuesta 2

Cálculos reproducidos

$\frac{(48000 \cdot 120)}{1024} = 5625$

C.5 Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC

Problema resuelto [PRB-0025]
En una base de datos con páginas de 8 KiB, cada nodo interno de un índice B+ tiene un factor de ramificación efectivo de 200 hijos y cada página hoja almacena 80 entradas de índice. Calcule el número total de filas (en millones) que puede indexar un árbol B+ de altura h = 3 (1 raíz, 1 nivel intermedio de nodos internos y 1 nivel de hojas).
Respuesta

$$Resultado = 3{,}2$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
[CH-05.1] Calcule (Número de páginas hoja en el árbol B+ de 3 niveles con factor de ramificación 200 (200 * 200)) expresado en hojas.
Respuesta 40000
Ejercicio 2 [PRB-0028]
[CH-05.2] Calcule (Memoria RAM en KiB requerida para mantener en Buffer Pool la raíz y los 200 nodos internos de 8 KiB ((1 + 200) * 8)) expresado en KiB.
Respuesta 1608
Ejercicio 3 [PRB-0029]
[CH-05.3] Calcule (Coste estimado en unidades de página de un Index Scan que lee 3 niveles de índice más 12 páginas de Heap) expresado en páginas.
Respuesta 15
Ejercicio 4 [PRB-0030]

[CH-05.4] Calcule (Tasa de escritura WAL en MB/s para 5 000 transacciones por segundo que generan 400 bytes de log cada una) expresado en MB/s.
Respuesta 2

Cálculos reproducidos

$\frac{((200 \cdot 200) \cdot 80)}{1000000} = 3{,}2$

C.6 Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis

Problema resuelto [PRB-0031]
Un clúster de base de datos distribuida mantiene N = 5 réplicas de cada partición. Si las escrituras se configuran con un quórum de confirmación W = 3 nodos para tolerar la caída de 2 réplicas, calcule el número mínimo de nodos R que debe consultar una lectura para garantizar intersección estricta (W + R > N => R = N - W + 1).
Respuesta

$$Resultado = 3$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
[CH-06.1] Calcule (Quórum de lectura requerido si N = 7 réplicas y W = 4 escrituras (7 - 4 + 1)) expresado en nodos.
Respuesta 4
Ejercicio 2 [PRB-0034]
[CH-06.2] Calcule (Porcentaje de claves redistribuidas por Hash Consistente al pasar de 9 a 10 nodos (100 / 10)) expresado en %.
Respuesta 10
Ejercicio 3 [PRB-0035]
[CH-06.3] Calcule (Memoria RAM total en GB para almacenar en Redis 50 millones de claves de 200 bytes promedio (dividiendo entre 10^9)) expresado en GB.
Respuesta 10
Ejercicio 4 [PRB-0036]
[CH-06.4] Calcule (Número máximo de fallos de nodo tolerados por un clúster Raft de 5 nodos ((5 - 1) / 2)) expresado en nodos.
Respuesta 2

Cálculos reproducidos

$((5 - 3) + 1) = 3$

C.7 Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js

Problema resuelto [PRB-0037]
Una aplicación SPA tradicional descarga un bundle monolítico de 480 KiB de JavaScript. Al migrar a una arquitectura híbrida SSR + Server Components, el 65 % del código de presentación y formateo permanece en el servidor. Calcule el tamaño en KiB del bundle JavaScript que aún debe descargar e hidratar el navegador.
Respuesta

$$Resultado = 168$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0039]
[CH-07.1] Calcule (Ahorro absoluto en KiB al reducir el bundle de 480 KiB a 168 KiB) expresado en KiB.
Respuesta 312
Ejercicio 2 [PRB-0040]
[CH-07.2] Calcule (Tiempo de parseo y compilación JS en móvil a razón de 1,5 ms por cada KiB para 168 KiB) expresado en ms.
Respuesta 252
Ejercicio 3 [PRB-0041]
[CH-07.3] Calcule (Nodos re-evaluados con Signals en una lista de 500 filas cuando cambia 1 celda frente a 500 sin memoización (ahorro)) expresado en nodos.
Respuesta 499
Ejercicio 4 [PRB-0042]
[CH-07.4] Calcule (Número de peticiones de red tras deduplicar 6 componentes que solicitan simultáneamente el mismo endpoint) expresado en peticiones.
Respuesta 1

Cálculos reproducidos

$\frac{(480\cdot(100-65))}{100} = 168$

C.8 Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter

Problema resuelto [PRB-0043]
Una estación de trabajo ejecuta simultáneamente 4 utilidades corporativas de escritorio. Empaquetadas con Electron, cada instancia consume 220 MB de memoria RAM debido a procesos Chromium independientes; migradas a Tauri (Rust + WebView compartido), cada instancia consume 45 MB. Calcule la memoria RAM total ahorrada en megabytes.
Respuesta

$$Resultado = 700$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
[CH-08.1] Calcule (Reducción del tamaño de descarga en MB al pasar de un instalador Electron de 115 MB a uno Tauri de 8 MB) expresado en MB.
Respuesta 107
Ejercicio 2 [PRB-0046]
[CH-08.2] Calcule (Presupuesto de tiempo por fotograma en milisegundos (redondeado a centésimas x100) para una app Flutter a 120 Hz (100000 / 125)) expresado en centésimas de ms.
Respuesta 800
Ejercicio 3 [PRB-0047]
[CH-08.3] Calcule (Capacidad total de peticiones por segundo en un clúster de 6 pods Axum que procesan 35 000 req/s cada uno) expresado en req/s.
Respuesta 210000
Ejercicio 4 [PRB-0048]
[CH-08.4] Calcule (Instancias de un servicio Scoped creadas durante un lote de 250 peticiones HTTP si se inyecta 2 veces por petición) expresado en instancias.
Respuesta 250

Cálculos reproducidos

$(4 \cdot (220 - 45)) = 700$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 28 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.

¶2: dos testimonios.

¶3: dos testimonios.

¶4: dos testimonios.

¶5: dos testimonios.

¶6: dos testimonios.

¶7: dos testimonios.

¶8: dos testimonios. Ecuaciones: 2 respaldada por una fuente.

¶9: dos testimonios.

¶10: dos testimonios.

¶11: dos testimonios.

¶12: dos testimonios.

¶13: dos testimonios.

¶14: dos testimonios.

¶15: dos testimonios.

¶16: dos testimonios.

¶17: dos testimonios.

¶18: dos testimonios.

¶19: dos testimonios.

¶20: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.

¶21: dos testimonios.

¶22: dos testimonios.

¶23: dos testimonios.

¶24: dos testimonios.

¶25: dos testimonios.

¶26: dos testimonios.

¶27: dos testimonios.

¶28: dos testimonios.

¶29: dos testimonios.

¶30: dos testimonios.

¶31: dos testimonios.

¶32: dos testimonios.

¶33: dos testimonios.

¶34: dos testimonios.

¶35: dos testimonios.

¶36: dos testimonios.

¶37: dos testimonios.

¶38: dos testimonios.

¶39: dos testimonios.

¶40: dos testimonios.

¶41: dos testimonios.

¶42: dos testimonios.

¶43: dos testimonios.

¶44: dos testimonios.

¶45: dos testimonios.

¶46: dos testimonios.

¶47: dos testimonios.

¶48: dos testimonios.

HTML Living Standard & CSS Cascading and Inheritance Level 5 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Database System Concepts (7th Edition) & Designing Data-Intensive Applications (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Protocol Buffers v3, JSON Schema (RFC 8259) & YAML 1.2.2 Specification (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: ch-01 scientific
REV-0002: ch-02 scientific
REV-0003: ch-03 scientific
REV-0004: ch-04 scientific
REV-0005: ch-05 scientific
REV-0006: ch-06 scientific
REV-0007: ch-07 scientific
REV-0008: ch-08 scientific

Apéndice E

Plan de estudio

Partes VI–VIII: Web, Almacenamiento y Ecosistemas de Producción

✓HTML Living Standard y Accesibilidad Web (WCAG / ARIA)

- Dominar la semántica estructural de HTML5, el algoritmo de construcción del DOM, formularios nativos, multimedia y árboles de accesibilidad (AOM/ARIA).

2 conceptos planificados.

✓CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries

- Calcular determinísticamente la resolución de la cascada CSS, diseñar layouts complejos con Flexbox y Grid, y aplicar propiedades lógicas y variables reactivas.

2 conceptos planificados.

✓Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf

- Seleccionar y validar formatos de datos textuales y binarios, diseñar esquemas con JSON Schema y Protocol Buffers, y prevenir vulnerabilidades de deserialización.

2 conceptos planificados.

✓Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF)

- Traducir requisitos de dominio a esquemas relacionales en Forma Normal de Boyce-Codd (BCNF) garantizando reuniones sin pérdida (lossless-join) y preservación de dependencias.

2 conceptos planificados.

✓Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC

- Analizar la estructura física de páginas de disco, comparar árboles B+ frente a LSM-Trees, diseñar índices compuestos óptimos y depurar anomalías de concurrencia bajo niveles de aislamiento ANSI/serializable.

2 conceptos planificados.

✓Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis

- Diseñar topologías de replicación y particionamiento (sharding), evaluar compromisos CAP/PACELC y seleccionar con criterio técnico entre motores relacionales y NoSQL.

2 conceptos planificados.

✓Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js

- Comprender los motores de reactividad (Virtual DOM, Signals, compilación estática), gestionar estado cliente/servidor e implementar Server-Side Rendering (SSR) e hidratación.

2 conceptos planificados.

✓Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter

- Diseñar tuberías de middleware backend, inyección de dependencias y validación de esquemas, y contrastar arquitecturas de escritorio y móvil nativas frente a multiplataforma.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] HTML Living Standard & CSS Cascading and Inheritance Level 5. https://html.spec.whatwg.org/vol-ref-1

[27] Database System Concepts (7th Edition) & Designing Data-Intensive Applications. https://www.db-book.com/vol-ref-2

[28] Protocol Buffers v3, JSON Schema (RFC 8259) & YAML 1.2.2 Specification. https://datatracker.ietf.org/doc/html/rfc8259/vol-ref-3

Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen IX: Ingeniería del Software, Arquitectura, Sistemas Distribuidos, Seguridad y Proyectos

Enciclopedia de Programación e Ingeniería del Software — Partes IX, X, XI, XII y XIII

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Partes IX–XIII: Ingeniería Profesional, Cloud, Seguridad e Itinerarios 1

1. Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización 2

1.1. Cohesión, Acoplamiento y Métrica de Inestabilidad de Paquetes y Los Cinco Principios SOLID bajo una Lente Moderna . . . 2

1.2. Patrones de Diseño GoF en Lenguajes Modernos: De Clases a Funciones y Tipos Algebraicos y Deuda Técnica, Code Smells y Disciplina de Refactorización Segura . . . 2

2. Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC) 5

2.1. Domain-Driven Design (DDD): Diseño Estratégico y Táctico y Arquitectura Hexagonal (Puertos y Adaptadores) y Arquitectura Limpia . . . 5

2.2. Monolito Modular frente a Microservicios, CQRS y Patrón Outbox Transaccional y Diseño de APIs: REST (OpenAPI), GraphQL, gRPC y WebSockets . . . 6

3. Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing 8

3.1. Arquitectura Interna de Git: Almacén Direccionable por Contenido y Grafo DAG y Pirámide de Testing, TDD y Dobles de Prueba (Stubs, Fakes, Mocks) . . . 8

3.2. Property-Based Testing (QuickCheck, Hypothesis, proptest) y Reducción Automática (Shrinking) y Fuzzing Guiado por Cobertura (libFuzzer, AFL++) y Mutation Testing . . . 8

4. Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE 11

4.1. Tiempo Físico vs Tiempo Lógico: Relojes de Lamport y Relojes Vectoriales y Consenso Distribuido con Raft: Elección de Líder y Replicación de Log . . . 11

4.2. Patrones de Resiliencia: Timeouts, Exponential Backoff con Jitter, Circuit Breaker e Idempotencia y Observabilidad con OpenTelemetry (Métricas, Logs, Trazas) y Disciplina SRE (SLI/SLO) . . . 11

5. Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain 14

5.1. Modelado de Amenazas (STRIDE) y Defensa en Profundidad ante OWASP Top 10 y Criptografía Aplicada Moderna: TLS 1.3, AEAD (AES-GCM / ChaCha20-Poly1305) y Argon2id . . . 14

5.2. Autenticación y Autorización Moderna: OAuth 2.1, OpenID Connect (OIDC), PKCE y Passkeys y Seguridad de la Cadena de Suministro (Supply Chain): Lockfiles, SBOM, Sigstore y SLSA . . . 15

6. Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD 17

6.1. Anatomía de los Contenedores Linux: Namespaces, cgroups v2 y OverlayFS y Orquestación con Kubernetes: Bucle de Reconciliación, Pods, Probes y Autoscaling . . . 17

6.2. Infraestructura como Código (IaC) con Terraform y Gestión de Estado y Pipelines CI/CD y Estrategias de Despliegue Progresivo (Blue/Green, Canary, Feature Flags) . . . 18

7. Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados **20**
7.1. Arquitectura de los 19 Itinerarios de Aprendizaje Personalizados y Proyectos Integradores de Sistemas: Motor KV en Rust, Arena Allocator en C17 y Planificador en C++20 20
7.2. Proyectos Integradores de Alto Nivel y Datos: VM en Python 3.12, Router HTTP en TypeScript y Ledger SQL y Rúbricas de Evaluación por Niveles de Competencia (Principiante a Experto) 21

A. Notación **23**

B. Hoja de fórmulas **24**

C. Soluciones **25**
C.1. Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización . 25
C.2. Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC) 25
C.3. Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing 26
C.4. Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE 27
C.5. Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain 27
C.6. Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD 28
C.7. Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados 28

D. Auditoría de fuentes **30**

E. Plan de estudio **32**

Parte I

Partes IX–XIII: Ingeniería Profesional, Cloud, Seguridad e Itinerarios

Capítulo 1

Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización

Fundamentos del diseño modular de software mantenible: cohesión funcional, inversión de dependencias y catálogo crítico de patrones creacionales, estructurales y de comportamiento.

Consejo

Muchos de los 23 patrones clásicos de Gamma, Helm, Johnson y Vlissides (1994) se simplifican radicalmente mediante funciones de orden superior, tipos suma y traits.

Definición

Cohesión, Acoplamiento y Métrica de Inestabilidad de Paquetes: El diseño modular busca maximizar la cohesión interna de cada módulo mientras dirige el acoplamiento hacia abstracciones estables.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La refactorización es una transformación del código fuente que preserva estrictamente el comportamiento externo observable mientras mejora su estructura interna.

1.1 Cohesión, Acoplamiento y Métrica de Inestabilidad de Paquetes y Los Cinco Principios SOLID bajo una Lente Moderna

La complejidad accidental de un sistema crece cuadráticamente con el acoplamiento implícito entre sus componentes. Dado un módulo o paquete, su acoplamiento aferente C_a cuenta cuántos módulos externos dependen de él, y su acoplamiento eferente C_e cuenta de cuántos módulos externos depende él. La métrica de inestabilidad $I = C_e/(C_a + C_e) \in [0, 1]$ caracteriza desde paquetes máximamente estables ($I = 0$, como el modelo de dominio puro) hasta paquetes volátiles ($I = 1$, como adaptadores de interfaz o puntos de entrada `main`). El Principio de Dependencias Estables dicta que un módulo sólo debe depender de módulos con menor o igual inestabilidad. 1) **Responsabilidad Única (SRP)**: un módulo debe tener una única razón de cambio asociada a un actor de negocio. 2) **Abierto/Cerrado (OCP)**: el comportamiento debe poder extenderse mediante polimorfismo o composición de funciones sin modificar el código ya verificado. 3) **Sustitución de Liskov (LSP)**: formalizado por Barbara Liskov y Jeannette Wing, exige que todo subtipo preserve los invariantes e historia observable del supertipo. 4) **Segregación de Interfaces (ISP)**: ningún cliente debe verse forzado a depender de métodos que no utiliza. 5) **Inversión de Dependencias (DIP)**: los módulos de alto nivel definen las interfaces (traits, protocolos) que implementan los módulos de infraestructura de bajo nivel.

1.2 Patrones de Diseño GoF en Lenguajes Modernos: De Clases a Funciones y Tipos Algebraicos y Deuda Técnica, Code Smells y Disciplina de Refactorización Segura

Los patrones creacionales (`Factory Method`, `Abstract Factory`, `Builder`) encapsulan la construcción válida de objetos complejos; en particular, el patrón `Type-State Builder` en Rust o TypeScript verifica en tiempo de compilación que todos los campos obligatorios se hayan configurado antes de invocar

`.build()`. Los patrones estructurales (`Adapter`, `Decorator`, `Facade`, `Proxy`, `Composite`) traducen interfaces o añaden responsabilidades transversales (telemetría, reintentos, caché) sin alterar la implementación subyacente. Los patrones de comportamiento (`Strategy`, `Observer`, `Command`, `State`, `Iterator`) modelan algoritmos intercambiables y máquinas de estados. Acuñada por Ward Cunningham, la metáfora de la deuda técnica describe el interés compuesto que se paga en cada entrega futura cuando se pospone el reflejo de un aprendizaje de dominio en la arquitectura del código. Toda refactorización (extracción de funciones puras, sustitución de condicionales anidados por polimorfismo o pattern matching exhaustivo, introducción de objetos valor) debe ejecutarse en pasos atómicos respaldados por una batería rápida de pruebas automatizadas.

Problema resuelto

Enunciado En una arquitectura modular, el paquete de facturación es importado por 18 módulos externos (acoplamiento aferente Ca = 18) e importa únicamente 2 bibliotecas estándar de tipos (acoplamiento eferente Ce = 2). Calcule el índice de inestabilidad I expresado en porcentaje.
Datos

- $C_a = 18\,\text{módulos}$ — Acoplamiento aferente (dependientes entrantes)
- $C_e = 2\,\text{módulos}$ — Acoplamiento eferente (dependencias salientes)

Incógnita Índice de inestabilidad porcentual I_pct en %
Modelo físico
Cálculo de la métrica de inestabilidad de un paquete de dominio

$$I_{\text{pct}} = \frac{2}{18+2} \cdot 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Aplicamos la fórmula de Martin I = Ce / (Ca + Ce) multiplicada por 100

$$I_{\text{pct}} = \frac{2}{18+2} \cdot 100$$

$$\frac{(2 \cdot 100)}{(18+2)}$$

Resultado

$$\textit{Resultado} = 10$$

Interpretación El paquete presenta una inestabilidad baja del 10 %, acorde con su rol como núcleo de negocio estable del que dependen otros 18 módulos.
Comprobación de los pasos

- Paso 1: 10 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) [CH-01.1] Calcule (Índice de inestabilidad porcentual de un adaptador HTTP con Ca = 1 y Ce = 9) expresado en %.
2 [PRB-0004] (Intermedio, Desarrollo por pasos) [CH-01.2] Calcule (Distancia a la secuencia principal en porcentaje si A = 40 % e I = 50 % (|40 + 50 - 100|)) expresado en %.

3 [PRB-0005] (Intermedio, Desarrollo por pasos) [CH-01.3] Calcule (Reducción de complejidad ciclomática al descomponer un método de complejidad 19 en 4 funciones de complejidad 2) expresado en puntos.
4 [PRB-0006] (Avanzado, Diseño) [CH-01.4] Calcule (Número de clases eliminadas al reemplazar 6 clases Strategy de un solo método por funciones de primera clase) expresado en clases.

Capítulo 2

Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC)

Arquitectura de sistemas de software a escala empresarial: fronteras de dominio, estilos arquitectónicos, CQRS, Event Sourcing y diseño de contratos de comunicación.

Consejo

Un monolito modular bien acotado ofrece fronteras lógicas limpias con transacciones ACID locales y cero sobrecoste de red, debiendo extraerse microservicios solo ante necesidades reales de escalado o despliegue independiente.

Definición

Domain-Driven Design (DDD): Diseño Estratégico y Táctico: El diseño estratégico divide un dominio complejo en Contextos Acotados (Bounded Contexts) con Lenguaje Ubicuo propio, evitando el antipatrón de un modelo canónico único gigante.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: REST sobre HTTP/JSON sobresale en interoperabilidad pública, GraphQL optimiza consultas de clientes con grafos de vista diversos y gRPC sobre HTTP/2 + Protobuf domina la comunicación interna de baja latencia.

2.1 Domain-Driven Design (DDD): Diseño Estratégico y Táctico y Arquitectura Hexagonal (Puertos y Adaptadores) y Arquitectura Limpia

En el nivel estratégico, DDD identifica subdominios principales (Core, donde reside la ventaja competitiva), de soporte y genéricos, estableciendo mapas de contexto (Context Maps) con capas anticorrupción (ACL) para traducir conceptos entre sistemas. En el nivel táctico, distingue **Entidades** (definidas por su identidad persistente a lo largo del tiempo), **Objetos Valor** (definidos exclusivamente por sus atributos e inmutables por construcción, como `ImporteMonetario`), **Agregados** (fronteras de consistencia transaccional gobernadas por su Aggregate Root) y **Eventos de Dominio** (hechos inmutables ocurridos en el pasado). Formulada por Alistair Cockburn y popularizada junto a la Arquitectura Limpia de Robert C. Martin, la Arquitectura Hexagonal representada en la figura 1 organiza el software en anillos concéntricos. El núcleo contiene las entidades y casos de uso de aplicación; alrededor suyo se sitúan los puertos de entrada (invocados por controladores HTTP, consumidores de colas o CLI) y los puertos de salida (interfaces de repositorios, pasarelas de pago o relojes de sistema), cuyas implementaciones concretas residen en la capa externa de infraestructura.

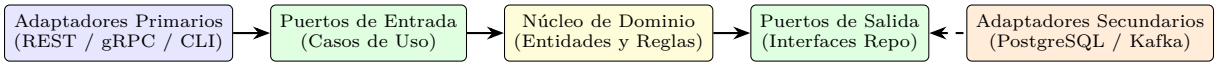


Figura 1. Arquitectura Hexagonal (Puertos y Adaptadores): los adaptadores primarios invocan puertos de entrada (casos de uso) y el dominio invoca puertos de salida implementados por adaptadores secundarios.

2.2 Monolito Modular frente a Microservicios, CQRS y Patrón Outbox Transaccional y Diseño de APIs: REST (OpenAPI), GraphQL, gRPC y WebSockets

Distribuir prematuramente un sistema sin fronteras de dominio maduras engendra un 'monolito distribuido' que sufre la latencia, los fallos parciales y la complejidad operativa de los microservicios sin obtener autonomía de despliegue. Cuando un sistema requiere publicar eventos en un broker (Kafka, RabbitMQ) tras actualizar su base de datos, escribir simultáneamente en ambos sin coordinación dos fases es inseguro; el patrón **Transactional Outbox** resuelve este problema insertando el evento en una tabla `outbox` dentro de la misma transacción ACID local y reenviándolo de forma asíncrona con garantía `at-least-once`. En una API RESTful madura, los recursos se modelan como sustantivos jerárquicos, los métodos HTTP respetan su semántica de seguridad e idempotencia (`GET` y `HEAD` seguros; `PUT` y `DELETE` idempotentes; `POST` con clave `Idempotency-Key` para pagos), y los errores se devuelven en formato estándar RFC 9457 (`Problem Details for HTTP APIs`). En GraphQL es imprescindible implementar el patrón `DataLoader` para agrupar consultas y evitar el problema $N + 1$.

Problema resuelto

Enunciado Una consulta GraphQL solicita una lista de 50 pedidos junto con los datos del autor de cada pedido. Sin agrupación (N+1), el servidor ejecuta 1 consulta para la lista y 50 consultas individuales de 2 ms de latencia cada una. Con DataLoader, ejecuta solo 2 consultas (la lista y un único SELECT ... WHERE id IN (...)) de 2 ms cada una. Calcule el tiempo total de base de datos ahorrado en ms.

Datos

- $N = 50$ pedidos — Número de elementos de la lista
- $t_q = 2$ ms — Latencia por consulta a la base de datos

Incógnita Tiempo de base de datos ahorrado T_ahorro en ms

Modelo físico

Eliminación del problema N+1 en un resolver GraphQL mediante DataLoader

$$T_{\text{ahorro}} = (50 + 1) \cdot 2 - 2 \cdot 2$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el tiempo de las 2 consultas agrupadas al tiempo de las N + 1 = 51 consultas secuenciales

$$T_{\text{ahorro}} = (50 + 1) \cdot 2 - 2 \cdot 2$$

$$(((50 + 1) \cdot 2) - (2 \cdot 2))$$

Resultado

$$Resultado = 98$$

Interpretación DataLoader reduce el tiempo de consultas de 102 ms a 4 ms, ahorrando exactamente 98 ms por petición.

Comprobación de los pasos

- Paso 1: 98 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) [CH-02.1] Calcule (Consultas SQL ahorradas al resolver 120 elementos con DataLoader (2 consultas) frente a N+1 (121 consultas)) expresado en consultas.
2 [PRB-0010] (Intermedio, Desarrollo por pasos) [CH-02.2] Calcule (Latencia acumulada en ms de una cadena síncrona de 6 microservicios con 8 ms por salto) expresado en ms.
3 [PRB-0011] (Intermedio, Desarrollo por pasos) [CH-02.3] Calcule (Disponibilidad porcentual aproximada (en centésimas x100) de 3 servicios en serie al 99 % cada uno (10000 - 3*100)) expresado en centésimas de %.
4 [PRB-0012] (Avanzado, Diseño) [CH-02.4] Calcule (Tamaño de página máximo devuelto si una API REST limita $page_size <= 100$ cuando el cliente solicita 5000) expresado en registros.

Capítulo 3

Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing

Ingeniería de calidad y verificación de software: modelo de objetos de Git, TDD, dobles de prueba, pruebas basadas en propiedades y análisis de mutación.

Consejo

El Property-Based Testing genera cientos de entradas aleatorias estructuradas para refutar invariantes universales y reduce automáticamente el contraejemplo al caso mínimo reproducible.

Definición

Arquitectura Interna de Git: Almacén Direccionable por Contenido y Grafo DAG: Git no almacena diferencias incrementales por commit, sino instantáneas completas modeladas como un grafo acíclico dirigido (DAG) de cuatro objetos inmutables: blob, tree, commit y tag.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La cobertura de líneas del 100 % no garantiza que las aserciones detecten defectos; el Mutation Testing mide la eficacia real de los tests inyectando mutantes sintácticos.

3.1 Arquitectura Interna de Git: Almacén Direccionable por Contenido y Grafo DAG y Pirámide de Testing, TDD y Dobles de Prueba (Stubs, Fakes, Mocks)

Bajo el directorio `.git/objects`, Git opera como un sistema de archivos clave-valor direccionable por contenido (Content-Addressable Storage) donde la clave de cada objeto es el hash criptográfico (SHA-1 o SHA-256) de su cabecera más su contenido comprimido con zlib. Un **blob** almacena los bytes de un archivo sin su nombre; un **tree** asocia nombres de archivo y permisos con hashes de blobs y subtrees; y un **commit** apunta al tree raíz de la instantánea, a sus commits padres y a los metadatos de autoría. Las ramas (`refs/heads/*`) son simples punteros móviles de 41 bytes hacia un commit. El ciclo Test-Driven Development (TDD: Red -> Green -> Refactor) utiliza las pruebas como herramienta de diseño de interfaces antes de escribir la implementación. Siguiendo la taxonomía de Gerard Meszaros, es fundamental distinguir entre **Stubs** (devuelven respuestas preconfiguradas), **Fakes** (implementaciones funcionales ligeras, como un repositorio en memoria) y **Mocks** (verifican interacciones salientes). Abusar de mocks acoplados a detalles internos produce pruebas frágiles que se rompen ante cualquier refactorización legítima.

3.2 Property-Based Testing (QuickCheck, Hypothesis, proptest) y Reducción Automática (Shrinking) y Fuzzing Guiado por Cobertura (libFuzzer, AFL++) y Mutation Testing

Mientras una prueba basada en ejemplos comprueba una tupla concreta (`assert ordenar([3,1,2]) == [1,2,3]`), una prueba basada en propiedades (introducida por Claessen y Hughes en QuickCheck para Haskell y disponible en Hypothesis para Python o `proptest` para Rust) declara propiedades algebraicas que deben cumplirse para toda entrada válida: idempotencia (`f(f(x)) == f(x)`), inversión

(`decode(encode(x)) == x`) o preservación de invariantes estructurales. Al encontrar un fallo, el motor aplica *shrinking* en el árbol de generación para entregar el contraejemplo más simple. En analizadores de protocolos, compiladores y parsers binarios, el Fuzzing guiado por cobertura (Coverage-Guided Fuzzing con `cargo-fuzz` / `libFuzzer`) instrumenta las aristas del grafo de control de flujo y muta el corpus de entradas cada vez que una mutación descubre una nueva ruta de ejecución. Por su parte, herramientas de Mutation Testing (`mutmut`, `cargo-mutants`, `Stryker`) modifican operadores en el código de producción (`<` por `<=`, `+` por `-`) y verifican qué porcentaje de mutantes son detectados ('muertos') por la suite de tests.

Problema resuelto

Enunciado Una herramienta de Mutation Testing genera 250 mutantes sintácticos sobre el módulo de liquidación financiera. De ellos, 10 son mutantes equivalentes (no alteran la semántica) y la batería de pruebas detecta y hace fallar 216 de los mutantes restantes. Calcule el Mutation Score porcentual sobre los mutantes no equivalentes.

Datos

- $M_{tot} = 250\,\text{mutantes}$ — Total de mutantes generados
- $M_{eq} = 10\,\text{mutantes}$ — Mutantes semánticamente equivalentes
- $M_{kill} = 216\,\text{mutantes}$ — Mutantes detectados (killed) por los tests

Incógnita Puntuación de mutación S_mut en %

Modelo físico

Cálculo del Mutation Score en una auditoría de calidad de pruebas

$$S_{\text{mut}} = \frac{216}{250 - 10} \cdot 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos los 216 mutantes detectados entre los 240 mutantes no equivalentes (250 - 10) y multiplicamos por 100

$$S_{\text{mut}} = \frac{216}{250 - 10} \cdot 100$$

$$\frac{(216 \cdot 100)}{(250 - 10)}$$

Resultado

$$Resultado = 90$$

Interpretación La suite de pruebas alcanza un Mutation Score del 90 %, identificando exactamente los 24 mutantes supervivientes que requieren nuevas aserciones.

Comprobación de los pasos

- Paso 1: 90 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) [CH-03.1] Calcule (Mutantes supervivientes que requieren nuevos tests en el módulo auditado (250 - 10 - 216)) expresado en mutantes.

2 [PRB-0016] (Intermedio, Desarrollo por pasos) [CH-03.2] Calcule (Objetos nuevos creados en Git al modificar 1 archivo dentro de un subdirectorio de profundidad 2 (1 blob + 3 trees + 1 commit)) expresado en objetos.
3 [PRB-0017] (Intermedio, Desarrollo por pasos) [CH-03.3] Calcule (Casos generados en 5 segundos por un fuzzer libFuzzer que ejecuta 18 000 iteraciones por segundo) expresado en ejecuciones.
4 [PRB-0018] (Avanzado, Diseño) [CH-03.4] Calcule (Tiempo total de CI en segundos para 1 200 tests unitarios de 2 ms y 30 tests de integración de 200 ms) expresado en s.

Capítulo 4

Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE

Teoría y práctica de los sistemas distribuidos modernos: falacias de la red, relojes de Lamport/vectoriales, consenso Raft, trazabilidad distribuida e ingeniería de fiabilidad.

Consejo

Reintentar llamadas fallidas inmediatamente o con intervalos fijos provoca tormentas de reintentos (Thundering Herd) que tumban al servicio en recuperación; añadir Jitter aleatorio descorrela la carga.

Definición

Tiempo Físico vs Tiempo Lógico: Relojes de Lamport y Relojes Vectoriales: En ausencia de un reloj físico global perfectamente sincronizado, la relación happened-before de Lamport define un orden parcial estricto sobre los eventos de un sistema distribuido.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: El contexto de propagación W3C Trace Context (traceparent) correlaciona logs y métricas a través de decenas de servicios mediante un identificador global de traza y spans jerárquicos.

4.1 Tiempo Físico vs Tiempo Lógico: Relojes de Lamport y Relojes Vectoriales y Consenso Distribuido con Raft: Elección de Líder y Replicación de Log

Los relojes físicos de cuarzo en servidores distintos sufren deriva térmica, y la sincronización NTP presenta incertidumbres de milisegundos que impiden ordenar causalmente eventos de microsegundos. Leslie Lamport formalizó en 1978 la relación causal $a \to b$ (happened-before): si $a \to b$, entonces las marcas lógicas cumplen $L(a) < L(b)$. Los **Relojes Vectoriales** $V \in \mathbb{N}^N$ extienden esta propiedad logrando una equivalencia exacta: $V(a) < V(b) \iff a \to b$, permitiendo detectar exactamente cuándo dos escrituras son concurrentes y requieren resolución de conflictos (CRDTs o reconciliación). En Raft, el tiempo se divide en mandatos (*terms*) numerados monotónicamente. Todo nodo se encuentra en uno de tres estados: *Follower*, *Candidate* o *Leader*. Si un seguidor no recibe latidos (`AppendEntries` vacíos) del líder durante su tiempo de espera aleatorio (p. ej., $150 - 300$ ms), incrementa su mandato, pasa a candidato y solicita votos (`RequestVote`). La restricción de votación garantiza que sólo un candidato cuyo log esté al menos tan actualizado como el de la mayoría de los nodos puede ser elegido líder.

4.2 Patrones de Resiliencia: Timeouts, Exponential Backoff con Jitter, Circuit Breaker e Idempotencia y Observabilidad con OpenTelemetry (Métricas, Logs, Trazas) y Disciplina SRE (SLI/SLO)

Toda llamada de red debe poseer un `timeout` explícito y acotado. Cuando una operación es idempotente (o transporta un token único `Idempotency-Key` registrado atómicamente por el receptor), los fallos transitorios pueden reintentarse aplicando espera exponencial con jitter completo: $t_{\text{wait}} \sim U(0, \text{mín}(T_{\text{máx}}, T_{\text{base}} \cdot$

2^{intento})). Si la tasa de fallos supera un umbral crítico, un **Circuit Breaker** abre el circuito y falla rápidamente en local para permitir que el sistema aguas abajo se recupere. OpenTelemetry estandariza la instrumentación de telemetría neutral respecto al proveedor: **Métricas** (contadores, gauges e histogramas de latencia p50/p95/p99), **Logs estructurados** en JSON y **Trazas distribuidas** (grafos de *spans* con `trace_id` y `span_id`). En Site Reliability Engineering (SRE), la fiabilidad se gobierna mediante Indicadores de Nivel de Servicio (SLI, p. ej., proporción de peticiones HTTP exitosas en menos de 200 ms) y Objetivos (SLO, p. ej., 99,9 % mensual, lo que otorga un presupuesto de error de 43,2 minutos cada 30 días).

Problema resuelto

Enunciado Un servicio crítico de pagos define un SLO de disponibilidad del 99,9 % medido sobre una ventana de 30 días (43 200 minutos totales). Calcule el presupuesto de error mensual en minutos durante los cuales el servicio puede experimentar indisponibilidad sin violar su SLO.
Datos

- $T_{mes} = 43200\,\text{min}$ — Minutos totales en 30 días (30 x 24 x 60)
- $p_{err} = 0{,}001\,\text{fracción}$ — Fracción permitida de error (1 - 0,999)

Incógnita Presupuesto de error E_min en minutos
Modelo físico
Cálculo del Presupuesto de Error (Error Budget) mensual para un SLO del 99,9 %

$$E_{\text{min}} = \frac{43200}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 43 200 minutos del mes por la fracción permitida de indisponibilidad (1 / 1000)

$$E_{\text{min}} = \frac{43200}{1000}$$

$$\frac{43200}{1000}$$

Resultado

$$\textit{Resultado} = 43{,}2$$

Interpretación Un SLO de 'tres nueves' (99,9 %) concede exactamente 43,2 minutos de presupuesto de error cada 30 días.
Comprobación de los pasos

- Paso 1: 43,2 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) [CH-04.1] Calcule (Presupuesto de error anual en minutos para un SLO de cuatro nueves 99,99 % (525600 / 10000)) expresado en min.
2 [PRB-0022] (Intermedio, Desarrollo por pasos) [CH-04.2] Calcule (Nodos totales mínimos en un clúster Raft para tolerar f = 3 fallos simultáneos (2 * 3 + 1)) expresado en nodos.

3 [PRB-0023] (Intermedio, Desarrollo por pasos) [CH-04.3] Calcule (Tope de espera en ms en el cuarto reintento (intento=4) con backoff exponencial de base 50 ms ($50 * 2^4$)) expresado en ms.
4 [PRB-0024] (Avanzado, Diseño) [CH-04.4] Calcule (Disponibilidad porcentual de un servicio con MTTF = 990 horas y MTTR = 10 horas (990 * 100 / (990 + 10))) expresado en %.

Capítulo 5

Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain

Ingeniería de seguridad defensiva desde el diseño (Secure by Design): modelado de amenazas STRIDE, criptografía moderna, identidad federada y seguridad de dependencias.

Consejo

El flujo Authorization Code con PKCE (RFC 7636) protege tanto a aplicaciones móviles y SPA como a servidores web frente a la interceptación del código de autorización.

Definición

Modelado de Amenazas (STRIDE) y Defensa en Profundidad ante OWASP Top 10: La seguridad no es una capa añadida al final del desarrollo, sino una propiedad emergente del diseño verificada mediante el principio de mínimo privilegio y fronteras de confianza explícitas.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Fijar hashes criptográficos en archivos de bloqueo (Cargo.lock, package-lock.json), generar inventarios SBOM y firmar artefactos en CI/CD protege contra ataques de confusión de dependencias.

5.1 Modelado de Amenazas (STRIDE) y Defensa en Profundidad ante OWASP Top 10 y Criptografía Aplicada Moderna: TLS 1.3, AEAD (AES-GCM / ChaCha20-Poly1305) y Argon2id

La metodología STRIDE clasifica las amenazas sobre cada flujo de datos en seis categorías: **S**poofing (suplantación de identidad), **T**ampering (manipulación de datos), **R**epudiation (repudio sin rastro auditable), **I**nformation Disclosure (fuga de información), **D**enial of Service (denegación de servicio) y **E**levation of Privilege (escalada de privilegios). Para neutralizar las vulnerabilidades más críticas del OWASP Top 10—como Broken Access Control (IDOR), Injection (SQLi, XSS, Command Injection) y Server-Side Request Forgery (SSRF)— el sistema debe verificar la autorización a nivel de objeto en cada petición, utilizar consultas parametrizadas, aplicar cabeceras `Content-Security-Policy` (CSP) y restringir las conexiones salientes del servidor hacia rangos IP privados o de metadatos cloud (`169.254.169.254`). Las comunicaciones en tránsito deben protegerse con **TLS 1.3** (RFC 8446), que completa el handshake en 1-RTT y exige intercambio de claves efímeras Diffie-Hellman sobre curvas elípticas (ECDHE con X25519) para garantizar Secreto Perfecto hacia Adelante (*Forward Secrecy*). Para datos en reposo, los esquemas **AEAD** (Authenticated Encryption with Associated Data) como `AES-256-GCM` o `ChaCha20-Poly1305` proporcionan simultáneamente confidencialidad e integridad mediante una etiqueta de autenticación de 128 bits. Para contraseñas, **Argon2id** exige una cantidad configurable de memoria RAM (p. ej., 64 MiB por hash) que neutraliza ataques paralelos con GPUs y ASICs.

5.2 Autenticación y Autorización Moderna: OAuth 2.1, OpenID Connect (OIDC), PKCE y Passkeys y Seguridad de la Cadena de Suministro (Supply Chain): Lockfiles, SBOM, Sigstore y SLSA

OAuth 2.1 delega autorización mientras que OpenID Connect (OIDC) añade una capa de identidad mediante un `ID Token` firmado en formato JWT (con algoritmos asimétricos `RS256` o `EdDSA`, verificando siempre la cabecera `alg`, el emisor `iss`, la audiencia `aud` y la expiración `exp`). En el flujo con **PKCE** (Proof Key for Code Exchange), el cliente genera un secreto efímero `code_verifier` y envía su hash SHA-256 (`code_challenge`) al iniciar el login, impidiendo que un atacante que intercepte el código de retorno pueda canjearlo por tokens. El software moderno depende de cientos de paquetes transitivos de código abierto. Asegurar la cadena de suministro conforme al marco **SLSA** (Supply-chain Levels for Software Artifacts) requiere compilar en entornos efímeros reproducibles, verificar sumas SHA-256 en lockfiles, emitir una lista de materiales de software (**SBOM** en formato CycloneDX o SPDX) y firmar imágenes y binarios sin claves de larga duración mediante certificados efímeros de **Sigstore/Cosign**.

Problema resuelto

Enunciado Un servidor configura Argon2id con un coste de memoria de 64 MiB por verificación de contraseña y 3 iteraciones. Si un atacante intenta evaluar en paralelo 512 candidatos simultáneos en una GPU, calcule la memoria VRAM total en GiB (1 GiB = 1024 MiB) necesaria para sostener esos 512 hilos sin penalización de ancho de banda.

Datos

- $M_{hash} = 64\,\text{MiB}$ — Memoria requerida por instancia de Argon2id
- $N_{par} = 512\,\text{hilos}$ — Candidatos evaluados en paralelo
- $K = 1024\,\text{MiB}\,\text{GiB}^{-1}$ — Mebibytes por gibibyte

Incógnita Memoria VRAM requerida V_gib en GiB

Modelo físico

Coste de memoria agregado de Argon2id frente a ataques de fuerza bruta en GPU

$$V_{\text{gib}} = \frac{512 \cdot 64}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 512 hilos por 64 MiB y dividimos entre 1024 MiB/GiB

$$V_{\text{gib}} = \frac{512 \cdot 64}{1024}$$

$$\frac{(512 \cdot 64)}{1024}$$

Resultado

$$\mathit{Resultado} = 32$$

Interpretación Evaluar tan solo 512 contraseñas en paralelo exige 32 GiB de VRAM dedicada, reduciendo en varios órdenes de magnitud la velocidad de fuerza bruta respecto a SHA-256.

Comprobación de los pasos

- Paso 1: 32 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) [CH-05.1] Calcule (Bytes totales de sobrecoste criptográfico en AES-256-GCM por mensaje (12 bytes de nonce IV + 16 bytes de tag)) expresado en bytes.
2 [PRB-0028] (Intermedio, Desarrollo por pasos) [CH-05.2] Calcule (Bits de entropía de una clave aleatoria de 32 bytes generada con un CSPRNG del sistema operativo (32 * 8)) expresado en bits.
3 [PRB-0029] (Intermedio, Desarrollo por pasos) [CH-05.3] Calcule (Reducción de latencia de establecimiento en ms al pasar de TLS 1.2 (2-RTT) a TLS 1.3 (1-RTT) con RTT = 45 ms) expresado en ms.
4 [PRB-0030] (Avanzado, Diseño) [CH-05.4] Calcule (Hilos concurrentes máximos de Argon2id (64 MiB) en un servidor de autenticación con 4096 MiB dedicados) expresado en hilos.

Capítulo 6

Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD

Ingeniería de plataformas, contenedores Linux (namespaces y cgroups v2), orquestación declarativa, infraestructura como código e integración y entrega continua.

Consejo

Declarar la infraestructura en código versionado con estado remoto bloqueable elimina la deriva de configuración (Configuration Drift) entre entornos de pruebas y producción.

Definición

Anatomía de los Contenedores Linux: Namespaces, cgroups v2 y OverlayFS: Un contenedor no es una máquina virtual con hipervisor propio, sino un grupo de procesos aislados en el mismo kernel Linux mediante namespaces y acotados en recursos mediante cgroups v2.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Desacoplar el despliegue técnico (Deployment) del lanzamiento funcional (Release) mediante despliegues Canary y Feature Flags reduce el radio de impacto de regresiones en producción.

6.1 Anatomía de los Contenedores Linux: Namespaces, cgroups v2 y OverlayFS y Orquestación con Kubernetes: Bucle de Reconciliación, Pods, Probes y Autoscaling

El kernel Linux proporciona tres mecanismos que sustentan el estándar OCI (Open Container Initiative): 1) **Namespaces** (`pid`, `net`, `mnt`, `uts`, `ipc`, `user`, `cgroup`), que restringen qué procesos, interfaces de red y puntos de montaje puede observar el contenedor; 2) **Control Groups (cgroups v2)**, que contabilizan y limitan el consumo de CPU, memoria RAM y E/S de disco; y 3) sistemas de archivos en unión (**OverlayFS**), que superponen capas inmutables direccionadas por hash SHA-256 con una capa superior efímera de escritura.

```
# Etapa 1: Compilación estática reproducible
FROM rust:1.80-alpine AS builder
WORKDIR /src
COPY Cargo.toml Cargo.lock ./
COPY src ./src
RUN cargo build --release --locked

# Etapa 2: Imagen mínima sin shell ni usuario root
FROM gcr.io/distroless/cc-debian12:nonroot
COPY --from=builder /src/target/release/motor-kv /usr/local/bin/motor-kv
USER nonroot:nonroot
ENTRYPOINT ["/usr/local/bin/motor-kv"]
```

En Kubernetes, la unidad mínima desplegable es el **Pod** (uno o más contenedores que comparten namespace de red y volúmenes). Los controladores (`Deployment`, `StatefulSet`, `DaemonSet`) supervisan la salud de los pods mediante tres sondas: `startupProbe`, `readinessProbe` (que retira el pod del balanceador `Service` cuando no puede recibir tráfico sin matarlo) y `livenessProbe` (que reinicia el contenedor

ante interbloqueos fatales). Todo contenedor de producción debe declarar `resources.requests` (usados por el `kube-scheduler`) y `limits`.

6.2 Infraestructura como Código (IaC) con Terraform y Gestión de Estado y Pipelines CI/CD y Estrategias de Despliegue Progresivo (Blue/Green, Canary, Feature Flags)

Terraform construye un grafo acíclico dirigido (DAG) de dependencias entre recursos cloud (redes VPC, clústeres EKS/GKE, bases de datos RDS, políticas IAM) y calcula un plan de ejecución determinista (`terraform plan`) antes de aplicar mutaciones (`terraform apply`). El archivo de estado (`terraform.tfstate`) debe almacenarse en un backend remoto cifrado con bloqueo transaccional y jamás incluir secretos en texto plano. Un pipeline de Integración Continua (CI) verifica automáticamente en cada Pull Request: formateo, análisis estático (linters y chequeo de tipos), pruebas unitarias y de integración, auditoría de vulnerabilidades (CVE) y compilación del artefacto firmado. En Entrega Continua (CD), un despliegue **Canary** encamina inicialmente el $1-5\,\%$ del tráfico real a la nueva versión y promueve automáticamente al $100\,\%$ sólo si los SLIs de tasa de error y latencia p99 permanecen dentro del umbral.

Problema resuelto

Enunciado Un Deployment de Kubernetes tiene actualmente 4 pods en ejecución con una utilización media de CPU del $90\,\%$. Si el Horizontal Pod Autoscaler (HPA) está configurado para mantener una utilización objetivo del $60\,\%$ según la fórmula $R_des = R_act*(U_act/U_obj)$, calcule el número de réplicas al que escalará el clúster.

Datos

- $R_{act} = 4\,\text{pods}$ — Réplicas actuales en ejecución
- $U_{act} = 90\ \%$ — Utilización media actual de CPU
- $U_{obj} = 60\ \%$ — Utilización objetivo configurada en el HPA

Incógnita Número deseado de réplicas R_des en pods

Modelo físico

Dimensionamiento de réplicas Kubernetes mediante Horizontal Pod Autoscaler (HPA)

$$R_{\text{des}} = \frac{4 \cdot 90}{60}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 4 réplicas actuales por la razón entre la utilización actual ($90\,\%$) y la objetivo ($60\,\%$)

$$R_{\text{des}} = \frac{4 \cdot 90}{60}$$

$$\frac{(4 \cdot 90)}{60}$$

Resultado

$$Resultado = 6$$

Interpretación El controlador HPA escala el Deployment a 6 pods para restablecer la utilización media de CPU en el $60\,\%$.

Comprobación de los pasos

- Paso 1: 6 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) [CH-06.1] Calcule (Reducción en MB de una imagen Docker al usar multi-stage distroless de 22 MB frente a la imagen de compilación de 840 MB) expresado en MB.
2 [PRB-0034] (Intermedio, Desarrollo por pasos) [CH-06.2] Calcule (Solicitudes de CPU totales en millicores reservadas por un Deployment de 6 pods con request de 250m cada uno) expresado en mCPU.
3 [PRB-0035] (Intermedio, Desarrollo por pasos) [CH-06.3] Calcule (Tráfico en req/s recibido por la versión Canary (5 %) en un servicio que recibe 12 000 req/s totales) expresado en req/s.
4 [PRB-0036] (Avanzado, Diseño) [CH-06.4] Calcule (Pods disponibles mínimos durante un RollingUpdate de 10 réplicas con maxUnavailable = 2) expresado en pods.

Capítulo 7

Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados

Síntesis de las 19 rutas formativas de la enciclopedia y análisis técnico de los 6 proyectos integradores verificados incluidos en el repositorio.

Consejo

Los proyectos de lenguajes dinámicos, tipado estructural y bases de datos demuestran compilación a bytecode, control de flujo reactivo con Token Bucket y contabilidad de doble partida en BCNF.

Definición

Arquitectura de los 19 Itinerarios de Aprendizaje Personalizados: El grafo acíclico dirigido (DAG) de prerrequisitos permite trazar rutas óptimas tanto para quien comienza desde cero absoluto como para ingenieros que se especializan en un nuevo dominio.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La evaluación de competencias combina corrección algorítmica, seguridad de memoria y tipos, claridad arquitectónica, cobertura de pruebas y observabilidad en producción.

7.1 Arquitectura de los 19 Itinerarios de Aprendizaje Personalizados y Proyectos Integradores de Sistemas: Motor KV en Rust, Arena Allocator en C17 y Planificador en C++20

La enciclopedia estructura 19 itinerarios completos documentados en `blueprint/learning-paths.json`: 1) **Cero Absoluto a Primer Empleo Profesional**, 2) **Grado Universitario Completo en Ciencias de la Computación (ACM/IEEE CS2023)**, 3) **Ingeniería de Sistemas y Bajo Nivel (C, C++, Rust, Zig, ASM)**, 4) **Desarrollo Web Full-Stack Moderno (HTML, CSS, TS, React/Next.js, SQL)**, 5) **Ingeniería Backend y Sistemas Distribuidos (Go, Java/Spring, C#/.NET, Rust/Axum)**, 6) **Ciencia de Datos, Computación Científica e IA (Python, R, Julia, SQL)**, 7) **Desarrollo Móvil Nativo y Multiplataforma (Swift, Kotlin, Dart/Flutter)**, 8) **Ingeniería de Bases de Datos y Plataformas de Datos**, 9) **Cloud, DevOps, SRE y Ciberseguridad**, 10) **Programación Funcional y Lenguajes Declarativos (Haskell, OCaml, Elixir, Erlang, Scala, F#, Clojure, Prolog)**, y 9 rutas adicionales de especialización directa por ecosistema. 1) En `projects/rust-kv-engine`, se desarrolla en Rust 2021 un motor de almacenamiento clave-valor persistente basado en Write-Ahead Log (WAL) binario con sumas de verificación CRC32 (polinomio IEEE `0xEDB88320`), compactación atómica y recuperación ante corrupción. 2) En `projects/c-arena-allocator`, se implementa en C17 estricto un asignador lineal de memoria (*Arena Allocator*) con alineación a potencias de dos y un analizador de paquetes binarios de red de copia cero (*zero-copy*) verificado con AddressSanitizer y UndefinedBehaviorSanitizer. 3) En `projects/cpp-task-scheduler`, se construye en C++20 un planificador de tareas por prioridad restringido estáticamente mediante `concept ExecutableTask` y `std::ranges`.

7.2 Proyectos Integradores de Alto Nivel y Datos: VM en Python 3.12, Router HTTP en TypeScript y Ledger SQL y Rúbricas de Evaluación por Niveles de Competencia (Principiante a Experto)

4) En `projects/python-bytecode-vm`, se implementa en Python 3.12 un compilador de expresiones aritméticas y variables a bytecode con plegado de constantes (*Constant Folding*) y su máquina virtual de pila, verificado con `pytest`, `ruff` y `pyright`. 5) En `projects/ts-reactive-http`, se construye en TypeScript 5 un enrutador HTTP declarativo con tubería de middlewares y limitador de tasa (*Token Bucket*) ejecutado nativamente en Node.js. 6) En `projects/sql-transactional-ledger`, se diseña en SQL un libro mayor contable de doble partida en BCNF con triggers de invariantes, CTEs recursivas de jerarquía de cuentas y funciones de ventana. Cada proyecto e itinerario se evalúa mediante cinco dimensiones objetivas: 1) **Exactitud funcional y manejo de casos límite**, 2) **Uso idiomático del lenguaje y de su sistema de tipos**, 3) **Complejidad temporal y espacial asintótica y constantes reales de caché/E/S**, 4) **Calidad de pruebas automatizadas (unitarias, propiedades y sanitizers)**, y 5) **Documentación técnica, reproducibilidad del build y ausencia total de advertencias del compilador**.

Problema resuelto

Enunciado Un estudiante autodidacta aborda el itinerario de Ingeniería Backend y Sistemas Distribuidos, que comprende 720 horas de estudio teórico y proyectos prácticos. Si dedica de forma constante 15 horas semanales, calcule el número exacto de semanas necesarias para completar la especialización.

Datos

- $H_{tot} = 720\,\text{horas}$ — Horas totales del itinerario formativo
- $h_{sem} = 15\,\text{horas}\,\text{semana}^{-1}$ — Dedicación semanal constante

Incógnita Duración del itinerario S_sem en semanas

Modelo físico

Planificación temporal de un itinerario formativo completo de 720 horas

$$S_{\text{sem}} = \frac{720}{15}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos las 720 horas totales del itinerario entre las 15 horas semanales

$$S_{\text{sem}} = \frac{720}{15}$$

$$\frac{720}{15}$$

Resultado

$$\mathit{Resultado} = 48$$

Interpretación Con una dedicación sostenida de 15 horas semanales, el estudiante completa el itinerario profesional en 48 semanas (menos de un año natural).

Comprobación de los pasos

- Paso 1: 48 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 **[PRB-0039] (Básico, Numérico)** [CH-07.1] Calcule (Horas dedicadas a práctica y proyectos si representan el 60 % de un itinerario de 720 horas) expresado en horas.

2 **[PRB-0040] (Intermedio, Desarrollo por pasos)** [CH-07.2] Calcule (Total de pruebas ejecutadas al correr las 6 suites de los proyectos integradores con un promedio de 5 aserciones por proyecto) expresado en aserciones.

3 **[PRB-0041] (Intermedio, Desarrollo por pasos)** [CH-07.3] Calcule (Bytes de cabecera por registro en el motor WAL de Rust (4 B CRC32 + 4 B $\text{key}_{l}en + 4B$ $\text{val}_{l}en + 1B$ op)) expresado en bytes.

4 **[PRB-0042] (Avanzado, Diseño)** [CH-07.4] Calcule (Saldo neto verificado en una partida contable doble donde la suma de cargos es 18 500 y la de abonos es 18 500) expresado en unidades.

Apéndice A

Notación

Símbolo	Significado	Unidades
$A = T_{\mathrm{MTTF}}/T_{\mathrm{tot}}$	Disponibilidad estacionaria de un sistema reparable	fracción
$I = \frac{C_e}{C_a + C_e}$	Métrica de inestabilidad de paquetes de Robert C. Martin	adimensional
$N \geq 2f + 1$	Nodos requeridos en consenso tolerante a f fallos de parada (Raft/Paxos)	nodos

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización

Problema resuelto [PRB-0001]
En una arquitectura modular, el paquete de facturación es importado por 18 módulos externos (acoplamiento aferente Ca = 18) e importa únicamente 2 bibliotecas estándar de tipos (acoplamiento eferente Ce = 2). Calcule el índice de inestabilidad I expresado en porcentaje.
Respuesta

$$Resultado = 10$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[CH-01.1] Calcule (Índice de inestabilidad porcentual de un adaptador HTTP con Ca = 1 y Ce = 9) expresado en %.
Respuesta 90
Ejercicio 2 [PRB-0004]
[CH-01.2] Calcule (Distancia a la secuencia principal en porcentaje si A = 40 % e I = 50 % (|40 + 50 - 100|)) expresado en %.
Respuesta 10
Ejercicio 3 [PRB-0005]
[CH-01.3] Calcule (Reducción de complejidad ciclomática al descomponer un método de complejidad 19 en 4 funciones de complejidad 2) expresado en puntos.
Respuesta 11
Ejercicio 4 [PRB-0006]
[CH-01.4] Calcule (Número de clases eliminadas al reemplazar 6 clases Strategy de un solo método por funciones de primera clase) expresado en clases.
Respuesta 6

Cálculos reproducidos

$\frac{(2 \cdot 100)}{(18+2)} = 10$

C.2 Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC)

Problema resuelto [PRB-0007]
Una consulta GraphQL solicita una lista de 50 pedidos junto con los datos del autor de cada pedido. Sin agrupación (N+1), el servidor ejecuta 1 consulta para la lista y 50 consultas individuales de 2 ms de latencia cada una. Con DataLoader, ejecuta solo 2 consultas (la lista y un único SELECT ... WHERE id IN (...)) de 2 ms cada una. Calcule el tiempo total de base de datos ahorrado en ms.
Respuesta

$$Resultado = 98$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]
[CH-02.1] Calcule (Consultas SQL ahorradas al resolver 120 elementos con DataLoader (2 consultas) frente a N+1 (121 consultas)) expresado en consultas.

Respuesta 119

Ejercicio 2 [PRB-0010]

[CH-02.2] Calcule (Latencia acumulada en ms de una cadena síncrona de 6 microservicios con 8 ms por salto) expresado en ms.

Respuesta 48

Ejercicio 3 [PRB-0011]

[CH-02.3] Calcule (Disponibilidad porcentual aproximada (en centésimas x100) de 3 servicios en serie al 99 % cada uno (10000 - 3*100)) expresado en centésimas de %.

Respuesta 9700

Ejercicio 4 [PRB-0012]

[CH-02.4] Calcule (Tamaño de página máximo devuelto si una API REST limita $\text{page}_size <= 100$ cuando el cliente solicita 5000) expresado en registros.

Respuesta 100

Cálculos reproducidos

$(((50 + 1) \cdot 2) - (2 \cdot 2)) = 98$

C.3 Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing

Problema resuelto [PRB-0013]

Una herramienta de Mutation Testing genera 250 mutantes sintácticos sobre el módulo de liquidación financiera. De ellos, 10 son mutantes equivalentes (no alteran la semántica) y la batería de pruebas detecta y hace fallar 216 de los mutantes restantes. Calcule el Mutation Score porcentual sobre los mutantes no equivalentes.

Respuesta

$$Resultado = 90$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]

[CH-03.1] Calcule (Mutantes supervivientes que requieren nuevos tests en el módulo auditado (250 - 10 - 216)) expresado en mutantes.

Respuesta 24

Ejercicio 2 [PRB-0016]

[CH-03.2] Calcule (Objetos nuevos creados en Git al modificar 1 archivo dentro de un subdirectorio de profundidad 2 (1 blob + 3 trees + 1 commit)) expresado en objetos.

Respuesta 5

Ejercicio 3 [PRB-0017]

[CH-03.3] Calcule (Casos generados en 5 segundos por un fuzzer libFuzzer que ejecuta 18 000 iteraciones por segundo) expresado en ejecuciones.

Respuesta 90000

Ejercicio 4 [PRB-0018]

[CH-03.4] Calcule (Tiempo total de CI en segundos para 1 200 tests unitarios de 2 ms y 30 tests de integración de 200 ms) expresado en s.

Respuesta 8.4

Cálculos reproducidos

$\frac{(216 \cdot 100)}{(250 - 10)} = 90$

C.4 Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE

Problema resuelto [PRB-0019]
Un servicio crítico de pagos define un SLO de disponibilidad del 99,9 % medido sobre una ventana de 30 días (43 200 minutos totales). Calcule el presupuesto de error mensual en minutos durante los cuales el servicio puede experimentar indisponibilidad sin violar su SLO.
Respuesta

$$Resultado = 43{,}2$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
[CH-04.1] Calcule (Presupuesto de error anual en minutos para un SLO de cuatro nueves 99,99 % (525600 / 10000)) expresado en min.
Respuesta 52.56
Ejercicio 2 [PRB-0022]
[CH-04.2] Calcule (Nodos totales mínimos en un clúster Raft para tolerar f = 3 fallos simultáneos (2 * 3 + 1)) expresado en nodos.
Respuesta 7
Ejercicio 3 [PRB-0023]
[CH-04.3] Calcule (Tope de espera en ms en el cuarto reintento (intento=4) con backoff exponencial de base 50 ms ($50 * 2^4$)) expresado en ms.
Respuesta 800
Ejercicio 4 [PRB-0024]
[CH-04.4] Calcule (Disponibilidad porcentual de un servicio con MTTF = 990 horas y MTTR = 10 horas (990 * 100 / (990 + 10))) expresado en %.
Respuesta 99

Cálculos reproducidos

$\frac{43200}{1000} = 43{,}2$

C.5 Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain

Problema resuelto [PRB-0025]
Un servidor configura Argon2id con un coste de memoria de 64 MiB por verificación de contraseña y 3 iteraciones. Si un atacante intenta evaluar en paralelo 512 candidatos simultáneos en una GPU, calcule la memoria VRAM total en GiB (1 GiB = 1024 MiB) necesaria para sostener esos 512 hilos sin penalización de ancho de banda.
Respuesta

$$Resultado = 32$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
[CH-05.1] Calcule (Bytes totales de sobrecoste criptográfico en AES-256-GCM por mensaje (12 bytes de nonce IV + 16 bytes de tag)) expresado en bytes.
Respuesta 28
Ejercicio 2 [PRB-0028]
[CH-05.2] Calcule (Bits de entropía de una clave aleatoria de 32 bytes generada con un CSPRNG del sistema operativo (32 * 8)) expresado en bits.
Respuesta 256
Ejercicio 3 [PRB-0029]

[CH-05.3] Calcule (Reducción de latencia de establecimiento en ms al pasar de TLS 1.2 (2-RTT) a TLS 1.3 (1-RTT) con RTT = 45 ms) expresado en ms.
Respuesta 45
Ejercicio 4 [PRB-0030]
[CH-05.4] Calcule (Hilos concurrentes máximos de Argon2id (64 MiB) en un servidor de autenticación con 4096 MiB dedicados) expresado en hilos.
Respuesta 64

Cálculos reproducidos

$\frac{(512 \cdot 64)}{1024} = 32$

C.6 Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD

Problema resuelto [PRB-0031]
Un Deployment de Kubernetes tiene actualmente 4 pods en ejecución con una utilización media de CPU del 90 %. Si el Horizontal Pod Autoscaler (HPA) está configurado para mantener una utilización objetivo del 60 % según la fórmula $R_des = R_act * (U_act/U_obj)$, calcule el número de réplicas al que escalará el clúster.
Respuesta

$$Resultado = 6$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
[CH-06.1] Calcule (Reducción en MB de una imagen Docker al usar multi-stage distroless de 22 MB frente a la imagen de compilación de 840 MB) expresado en MB.
Respuesta 818
Ejercicio 2 [PRB-0034]
[CH-06.2] Calcule (Solicitudes de CPU totales en millicores reservadas por un Deployment de 6 pods con request de 250m cada uno) expresado en mCPU.
Respuesta 1500
Ejercicio 3 [PRB-0035]
[CH-06.3] Calcule (Tráfico en req/s recibido por la versión Canary (5 %) en un servicio que recibe 12 000 req/s totales) expresado en req/s.
Respuesta 600
Ejercicio 4 [PRB-0036]
[CH-06.4] Calcule (Pods disponibles mínimos durante un RollingUpdate de 10 réplicas con maxUnavailable = 2) expresado en pods.
Respuesta 8

Cálculos reproducidos

$\frac{(4 \cdot 90)}{60} = 6$

C.7 Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados

Problema resuelto [PRB-0037]
Un estudiante autodidacta aborda el itinerario de Ingeniería Backend y Sistemas Distribuidos, que comprende 720 horas de estudio teórico y proyectos prácticos. Si dedica de forma constante 15 horas semanales, calcule el número exacto de semanas necesarias para completar la especialización.
Respuesta

$$Resultado = 48$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0039]

[CH-07.1] Calcule (Horas dedicadas a práctica y proyectos si representan el 60 % de un itinerario de 720 horas) expresado en horas.

Respuesta 432

Ejercicio 2 [PRB-0040]

[CH-07.2] Calcule (Total de pruebas ejecutadas al correr las 6 suites de los proyectos integradores con un promedio de 5 aserciones por proyecto) expresado en aserciones.

Respuesta 30

Ejercicio 3 [PRB-0041]

[CH-07.3] Calcule (Bytes de cabecera por registro en el motor WAL de Rust (4 B CRC32 + 4 B key$_l$$en$+4$B$ val$_l$$en$ + 1B op)) expresado en bytes.

Respuesta 13

Ejercicio 4 [PRB-0042]

[CH-07.4] Calcule (Saldo neto verificado en una partida contable doble donde la suma de cargos es 18 500 y la de abonos es 18 500) expresado en unidades.

Respuesta 0

Cálculos reproducidos

$\frac{720}{15} = 48$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 28 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 2 valor o condición.
¶21: dos testimonios. Ecuaciones: 1 valor o condición.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.

Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0) & Clean Architecture (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
In Search of an Understandable Consensus Algorithm (Raft) & Site Reliability Engineering (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10,

¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
OWASP Application Security Verification Standard (ASVS 4.0) & NIST SP 800-63B (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
REV-0001: ch-01 scientific
REV-0002: ch-02 scientific
REV-0003: ch-03 scientific
REV-0004: ch-04 scientific
REV-0005: ch-05 scientific
REV-0006: ch-06 scientific
REV-0007: ch-07 scientific

Apéndice E

Plan de estudio

Partes IX–XIII: Ingeniería Profesional, Cloud, Seguridad e Itinerarios

✓Principios de Diseño (SOLID, acoplamiento/cohesión), Patrones GoF y Refactorización

- Evaluar cuantitativamente el acoplamiento aferente y eferente, aplicar los principios SOLID sin sobreingeniería e implementar patrones de diseño GoF con tipos modernos.

2 conceptos planificados.

✓Arquitectura del Software: DDD, Arquitectura Hexagonal/Limpia, Microservicios y APIs (REST, GraphQL, gRPC)

- Diseñar sistemas bajo Arquitectura Hexagonal y Domain-Driven Design, elegir justificadamente entre monolitos modulares y microservicios, y especificar contratos de API REST, GraphQL y gRPC.

2 conceptos planificados.

✓Control de Versiones (Git Internals), Calidad, Testing, Property-Based Testing y Fuzzing

- Comprender el almacén direccionable por contenido de Git, diseñar estrategias de pruebas en pirámide/trofeo y verificar invariantes con Property-Based Testing, Fuzzing y Mutation Testing.

2 conceptos planificados.

✓Sistemas Distribuidos, Consenso (Raft), Resiliencia, Observabilidad (OpenTelemetry) y SRE

- Razonar sobre tiempo lógico y orden parcial, implementar patrones de resiliencia (Circuit Breaker, Backoff con Jitter, Idempotencia), instrumentar sistemas con OpenTelemetry y gestionar presupuestos de error SRE.

2 conceptos planificados.

✓Ciberseguridad Aplicada, Criptografía (TLS 1.3, Argon2id, AEAD), OAuth2/OIDC y Supply Chain

- Mitigar vulnerabilidades del OWASP Top 10, aplicar primitivas criptográficas estándar (Argon2id, AES-GCM, Ed25519), diseñar flujos OAuth 2.1 / OIDC con PKCE y asegurar la cadena de suministro de software.

2 conceptos planificados.

✓Infraestructura Cloud, Contenedores (Docker/OCI), Kubernetes, Terraform (IaC) y Pipelines CI/CD

- Construir imágenes OCI multi-stage mínimas y sin privilegios root, operar clústeres Kubernetes declarativos, aprovisionar infraestructura con Terraform y automatizar despliegues progresivos.

2 conceptos planificados.

✓Itinerarios de Especialización Profesional (19 Rutas) y Proyectos Integradores Verificados

- Planificar un itinerario de estudio personalizado desde nivel cero hasta nivel experto y ejecutar proyectos integradores completos con arquitectura de producción.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0) & Clean Architecture. https://www.computer.org/education/bodies-of-knowledge/software-engineering/vol-ref-1

[27] In Search of an Understandable Consensus Algorithm (Raft) & Site Reliability Engineering. https://raft.github.io/raft.pdf/vol-ref-2

[28] OWASP Application Security Verification Standard (ASVS 4.0) & NIST SP 800-63B. https://owasp.org/www-project-application-security-verification-standard/vol-ref-3